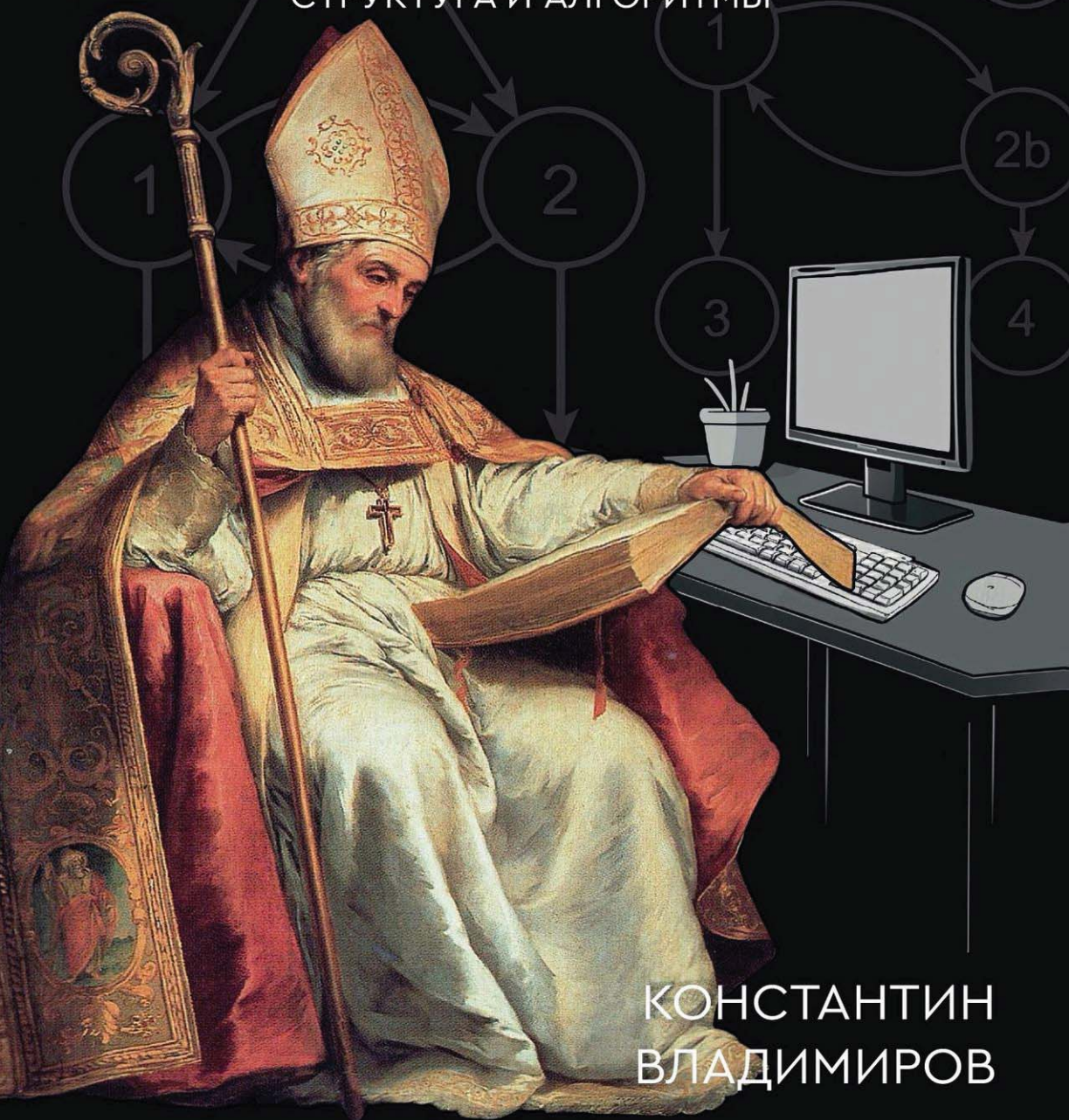


ОПТИМИЗИРУЮЩИЕ КОМПИЛЯТОРЫ

СТРУКТУРА И АЛГОРИТМЫ



КОНСТАНТИН
ВЛАДИМИРОВ

КОНСТАНТИН ВЛАДИМИРОВ

ОПТИМИЗИРУЮЩИЕ КОМПИЛЯТОРЫ

СТРУКТУРА И АЛГОРИТМЫ



Издательство АСТ
Москва

УДК 004.43
ББК 32.973
В57

Владимиров, Константин.

В57 Оптимизирующие компиляторы. Структура и алгоритмы /
Владимиров К.; — Москва: Издательство АСТ, 2024. — 272 с. —
(Программирование для всех)

ISBN 978–5–17–167965–1

«Оптимизирующие компиляторы» — настольная книга специалиста, который решил не просто укрепить свои знания, но и вывести навыки на новый уровень.

Вместе с Константином Владимировым вы разберете теорию оптимизирующей компиляции — все те сложные преобразования, которые происходят с текстом программы на его пути к исполняемому файлу, узнаете, что такое тулчейны и каким этапам трансформации подвергается программа до того, как будет впервые запущена, а также закрепите полученные знания, выполняя задания.

УДК 004.43
ББК 32.973

ISBN 978–5–17–167965–1

© Константин Владимиров, текст
© ООО Издательство «АСТ»

Оглавление

Введение	8
Глава 1. Тулчейны	13
1.1 Общий обзор	14
1.2 Компиляторы	15
1.2.1 Лексический анализ	18
1.2.2 Синтаксический анализ	21
1.2.3 Промежуточное представление	25
1.2.4 Машинно–независимые оптимизации	29
1.2.5 Кодогенерация	32
1.3 Ассемблеры	34
1.3.1 Ассемблирование функций и понятие ABI	37
1.3.2 Дизассемблирование и релокации	39
1.4 Линкеры	41
1.4.1 Оптимизации времени линковки	44
1.4.2 Релаксации	45
1.5 Задачи на самостоятельную проработку	48
Глава 2. Введение в оптимизации	50
2.1 Поток управления	51
2.2 Решётки и распространение констант	56

2.2.1 Бинарные отношения	56
2.2.2 Решётки	59
2.2.3 Продвижение констант.	61
2.3 Поток данных.	65
2.3.1 Достигающие определения	67
2.3.2 Доступные выражения	69
2.3.3 Активные переменные и обратный анализ.	71
2.3.4 Трансформации и анализ в HIR.	73
2.4 Нумерация значений.	75
2.5 Задачи на самостоятельную проработку	79
Глава 3. Построение SSA	81
3.1 SSA представление	82
3.1.1 Наивный способ построения SSA	85
3.2 Отношения и структуры доминирования.	87
3.2.1 Дерево доминаторов	89
3.2.2 Фронт доминирования	90
3.2.3 Графы схождения	93
3.3 Построение SSA	94
3.3.1 Свойства и виды SSA	95
3.3.2 Основной алгоритм	96
3.3.3 Построение SSA в LLVM	101
3.4 Задачи на самостоятельную проработку	104
Глава 4. Базовые оптимизации для SSA	106
4.1 Начинаем использовать SSA	107
4.1.1 Работа с SSA представлением	108
4.1.2 Граф зависимостей SSA	110
4.2 Продвижение констант и не только.	112
4.2.1 Простое продвижение констант	112

4.2.2 Добавляем информацию из CFG	115
4.2.3 Другие продвижения информации.	118
4.3 Глобальная нумерация значений.	120
4.3.1 Построение классов конгруэнтности	120
4.3.2 Улучшение эффективности разбиения	124
4.4 Устранение частичной избыточности	126
4.4.1 Разбиение критического ребра	127
4.4.2 Граф избыточных выражений	129
4.4.3 Алгоритм SSAPRE.	132
4.5 Задачи на самостоятельную проработку	137
Глава 5. Цикловые оптимизации	139
5.1 Естественные циклы	142
5.1.1 Обходы в графах и обращение дуг.	144
5.1.2 Сводимость	148
5.1.3 Канонический вид цикла	150
5.1.4 Замкнутое цикловое SSA.	155
5.2 Поиск и оптимизации индуктивностей	157
5.2.1 Индуктивные переменные	158
5.2.2 Поиск базовых индуктивностей.	161
5.2.3 Скалярная эволюция.	165
5.2.4 Упрощение вычислений в циклах.	168
5.2.5 Раскрутка циклов.	170
5.3 Инварианты и квази-инварианты	172
5.3.1 Вынос инвариантного кода из цикла	173
5.3.2 Пилинг циклов	175
5.4 Структурные оптимизации	178
5.4.1 Распределение и соединение циклов	179
5.4.2 Оптимизации для локальности данных	182
5.4.3 Разглаживание циклов	183

5.4.4 Вынос инвариантного управления	184
5.4.5 Управление цикловыми оптимизациями.	185
5.5 Задачи на самостоятельную проработку	188
Глава 6. Межпроцедурные оптимизации	189
6.1 Клонирование и трансформации функций	192
6.1.1 Продвижение констант и прочей информации	193
6.1.2 Эвристики и управление эвристиками.	196
6.1.3 Частичная девиртуализация.	199
6.1.4 Хвостовая рекурсия.	202
6.2 Подстановка и факторизация функций.	205
6.2.1 Подстановка функций	206
6.2.2 Вынос функций	208
6.3 Задачи на самостоятельную проработку	210
Глава 7. Разрушение SSA	212
7.1 Выбор инструкций	214
7.1.1 Перевод в представление SDHIR	217
7.1.2 Легализация и выбор инструкций.	220
7.1.3 Альтернативный подход: модификация LIR.	225
7.1.4 Конструкция Пробстинга.	227
7.1.5 Подход на основе PBQP	230
7.2 Распределение регистров	232
7.2.1 Разрушение SSA	233
7.2.2 Подходы, основанные на раскраске графа	237
7.2.3 Линейный подход к распределению регистров.	241
7.2.4 Подходы, основанные на PBQP и ILP.	244
7.3 Планирование инструкций	247

7.3.1 Планирование на виртуальных регистрах	250
7.3.2 Планирование на физических регистрах	252
7.4 Задачи на самостоятельную проработку	255
Заключение	257
Предметный указатель	260
Литература	264

Введение

Tolle numerum omnibus rebus et omnia pereunt.
– st. Isidore of Seville

В этой книге речь в основном пойдёт о компиляторах.

Изначально, когда программирование ещё не было так широко известно, как сейчас, термин «компилятор» относился исключительно к литературе. Так назывался человек, осуществлявший соединение (то есть компиляцию) нескольких, иногда разноязычных, текстов. Такой человек решал две основные задачи. Во-первых, ему надо было перевести (транслировать) те тексты, которые были написаны не на целевом языке. При переводе хороший компилятор вполне мог оптимизировать излагаемый материал, сделав его короче или упростив формулировки (но стараясь не потерять суть). Во-вторых, ему нужно было скомпоновать нужные части переведённых текстов в единый кодекс, свод или энциклопедию. Например, святой Исидор Севильский был первым компилятором мосарабского богослужебного обряда. И он же был его компоновщиком.

В программировании всё очень похоже. Есть разные модули, написанные на разных языках разными программистами. И должна быть некая программа, которая помогает собрать их воедино и перевести на язык, который поймёт машина. Эта программа и есть компилятор в широком смысле. Она не столько выполняет, сколько организует (драйвит) этот процесс, запуская много других программ, которые и делают всё на пути от исходного кода к исполняемому файлу — то есть и трансляцию, и оптимизацию, и ассемблирование, и линковку. И чаще всего люди, когда говорят про компилятор, имеют в виду именно драйвер компиляции (например программу `gcc`).

Компилятор, в узком смысле, это одна из программ, вызываемых драйвером компиляции. Эта программа переводит код других программ с высокоуровневого языка на низкоуровневый, то есть делает их оптимизирующую трансляцию. Далее, когда я буду говорить о компиляторе как о программе, я буду иметь в виду именно «компилятор в узком смысле».

На самом деле даже в узком смысле компилятор — это слишком широкая тема, которую будет сложно описать в рамках одной книги. Отмечу сразу, в этой работе мы не будем касаться динамической (just in time, JIT) компиляции, бинарной трансляции и всего, что с ними связано. JIT-компиляторы — это, несомненно, тоже компиляторы в узком смысле. Но речь дальше пойдет исключительно про статические компиляторы. Многие из тех алгоритмов, которые будут рассмотрены ниже, вполне применимы и используются в системах динамической компиляции, но там есть сложные детали и нюансы, которые мне хотелось бы опустить. Будем считать, что здесь и далее я буду иметь в виду исключительно статический компилятор и исключительно такой, который в итоге осуществляет трансляцию с языка программирования общего назначения в язык ассемблера целевой архитектуры.

Центральной идеей, вокруг которой в наше время строится практика оптимизаций в компиляторах, является идея промежуточного представления со статически единственными присвоениями (static single assignment, SSA). В этой книге мы пройдем путь от изобретения SSA к его построению, далее к использованию и далее к разрушению. Само это представление будет строго определено в разделе 3.1. Почему так поздно? Дело в том, что SSA исторически возникла очень поздно и есть гигантский пласт доисторической теории компиляторов, которые ей предшествовали и из которых она выросла. Необходимо минимальное знакомство с фундаментальными концепциями, предшествовавшими SSA, чтобы понимать откуда что растёт.

Структура книги довольно проста. Первая глава будет посвящена обзору современных систем компиляции (они также известны как тулчейны). В процессе я рассмотрю устройство компилятора и его окружение с высоты птичьего полёта. Эту главу можно считать общеобразовательной, и я рекомендую с ней ознакомиться даже если вы не будете далее читать эту книгу и вообще не собираетесь изучать оптимизации в компиляторах. Знание об устройстве и функциях современных компиляторов полезно для любого их пользователя, то есть для каждого

программиста. Вторая глава будет посвящена практикам оптимизации, подводящим к идее SSA. Я введу понятие решёток, расскажу про рекуррентные соотношения для анализа потока данных и приведу примеры такого рода оптимизаций. Третья глава будет посвящена построению SSA. В теоретическом смысле она будет самой интересной и математически нагруженной. Мы посмотрим на самые разные графы и научимся их трансформировать и строить над ними интересные структуры данных. Четвёртая глава будет посвящена базовым оптимизациям над SSA, пятая — цикловым (она по объёму будет вдвое больше четвёртой) и шестая — межпроцедурным оптимизациям. В седьмой главе мы научимся выбирать инструкции, разрушать SSA и поговорим о распределении регистров и о планировании исполнения. Седьмая глава будет тоже очень большой.

Автор этой книги является (в литературном смысле) в основном компилятором по отношению к её источникам. В этой книге будет мало нового и самостоятельного материала, и я почти не буду хвастаться своими исследованиями или исследованиями моих студентов. В основном я постараюсь аккуратно и точно изложить теорию оптимизирующей компиляции с учётом указанных выше ограничений. Книга написана от первого лица, но иногда я буду говорить во множественном числе — в основном, когда хочу подчеркнуть, что ожидаю вовлечения читателя в рассуждения. То есть во фразах «а теперь мы могли бы задаться вопросом», я пишу «мы», подразумевая нас с вами, уважаемый читатель. Когда я хочу сказать что-то отстранённое о себе, или сообщить какой-то факт, который от первого лица прозвучит странно, я буду использовать третье лицо как в начале этого абзаца.

Немного про используемые слова и термины. Такие слова как драйвер, тулчейн, трансляция и линковка являются англицизмами. В случае со словом «трансляция», этот англицизм к тому же неприятно пересекается с русским словом, обозначающим передачу звука и изображений на дальнейшее расстояние. Увы, в этой книге будет неизбежно много англицизмов. Автор является практикующим программистом и крутится в мире профессионального сленга. У меня не повернётся язык сказать «компоновщик» вместо «линкер» или «система компиляции» вместо «тулчейн».

Очень часто я буду вводить новые слова, образованные от англицизмов. Например, англицизм «спилл слот» (spill slot), обозначает временное хранилище в памяти для содержимого регистра. Образованное от него слово «спиллить» в свою очередь

означает процесс сохранения компилятором регистров в эти слоты. Вводя такого рода новообразования, я постараюсь каждый раз подробно объяснять, что я имею в виду. Иногда я буду пользоваться английскими аббревиатурами вроде SSA или CFG. Опять-таки они являются частью профессионального сленга, а соответствующие им русские эквиваленты СЕП (статическое единственное присвоение) или ГПУ (граф потока управления) будут смотреться непонятно и инородно для специалиста, а у новичка сформируют неправильные языковые привычки, которые помешают в дальнейшем профессиональном общении.

Автор имеет определённые языковые предпочтения: много лет используя и преподавая языки С и С++ (и разрабатывая компиляторы в основном на них и для них), я к ним в некотором роде привык. Поэтому, когда говорю о «каком-то» языке программирования, не уточняя, что это за язык, обычно я имею в виду что-то вроде С или С++: что-то статически типизированное, что-то компилируемое, что-то с явным управлением ресурсами, включая память и т.д. Увы, это исключает из этой книги интереснейшие алгоритмы оптимизирующей компиляции функциональных языков: не ожидайте здесь всякой деформации и прочего. Я считаю, что этому надо было бы посвящать отдельную книгу.

Тот язык псевдокода, на котором я буду описывать компиляторные алгоритмы, также будет подозрительно похож на современный высокоуровневый С++ (без некоторых технических деталей, выкинутых для краткости). Надеюсь, он окажется для читателя интуитивно понятен. Но будьте с этим псевдокодом аккуратны: он не компилировался, не тестировался и служит только для иллюстрации идей. Настоящие реализации в компиляторах сильно отличаются педантичностью рассмотрения краевых случаев, и краткость кода в них никто не считает достоинством. Десяток строчек псевдокода алгоритма, с учётом реальности, может стать тысячами строк обработки в конкретной реализации. Сам автор, читая литературу по компиляторам, иногда встречался с тем, что авторы отводили целую главу для описания сложного развитого псевдокода, а потом публиковали на нём процедуры на два-три разворота. При этом этот псевдокод также не исполнялся, не тестировался, так что взять как есть и просто использовать его всё равно было нельзя, а понять было куда сложнее. Поэтому я в псевдокоде стараюсь быть настолько кратким и ясным, насколько возможно, иногда сознательно опуская важные для практических реализаций оптимизации и проверки.

По опыту преподавания курса, лежащего в основе этой книги, оптимизации в компиляторах вызывают большой интерес и их довольно легко понять. Но даже лучший материал выветривается из головы без заданий и упражнений. Книга сопровождается двумя видами задач. Во-первых, заданиями непосредственно в тексте, которые позволяют что-то быстро попробовать и посмотреть и, во-вторых, несколько более сложными упражнениями в конце каждой главы.

Глава 1

Тулчейны

First we build the tools, then they build us.

– Marshall McLuhan

Так исторически сложилось, что компиляторы не работают в изоляции. На практике пользователь вообще редко запускает компилятор. И gcc и clang и даже cl это просто программы-оболочки (также говорят «драйверы»), которые запускают из-под себя не только компилятор, но и много что ещё. Конечно, пользователя интересует результат: превращение текста программы в исполняемый файл. Но именно для достижения такого результата одного только компилятора мало. Когда в обыденном языке мы говорим «компилятор gcc», мы обычно имеем в виду как раз ту самую программу-оболочку, под которой скрывается GNU Toolchain — громадный и разветвлённый набор компонентов и библиотек.

Если вы интересуетесь именно компилятором, то очень важно знать и понимать его место в тулчейне. И даже если рассматривать компилятор отдельно, он занимается не только оптимизациями. Поэтому не менее важно знать место оптимизатора в компиляторе. Общий ландшафт вокруг компиляторов сформирован самыми разными инструментами разработки — ассемблерами, линкерами, отладчиками, профилировщиками. Конечно, мне не хватит места поговорить о них всех. То небольшое, о чём я успею здесь рассказать, как мне кажется, важно знать не только если вы разрабатываете компиляторы, но даже если вы просто пользуетесь ими.

Со следующей главы мы углубимся в оптимизатор, а пока что давайте попробуем взлететь повыше и увидеть весь ландшафт и весь контекст, в котором будет происходить всё то, о чём мы поговорим позже в этой книге.

1.1 Общий обзор

Давайте начнём с простого вопроса. Вы написали какую-то программу, например, как на листинге 1.1, и запускаете компилятор (на самом деле — драйвер компиляции) `gcc`, чтобы получить исполняемый файл, как показано на том же листинге.

```
$ cat hello.c
#include <stdio.h>
int main() {
    printf("%s\n", "Hello, world!");
}
$ gcc hello.c -o hello.x
$ ./hello.x
Hello, world!
```

Листинг 1.1: Пример компиляции простейшего приложения

Прежде чем читать дальше, постарайтесь перечислить все программы, которые будут при этом вызваны.

Задание 1.1. *Какие программы будут вызваны при вызове `gcc`, показанном на листинге 1.1?*

Вы можете посмотреть в справочные материалы по GNU Toolchain, чтобы понять, какие из его опций предназначены для того, чтобы открыть вам детали происходящего под капотом.

```
$ gcc hello.c --verbose -save-temps
cc1 -E -quiet -v -imultiarch x86_64-linux-gnu hello.c \
    -mtune=generic -march=x86-64 -fpch-preprocess -o hello.i
cc1 -fpreprocessed hello.i -quiet -mtune=generic \
    -march=x86-64 -o hello.s
as -v --64 -o hello.o hello.s
collect2 crti.o hello.o -lgcc -lc -lgcc crtn.o -o a.out
```

Листинг 1.2: Детали компиляции приложения

Правильный ответ: будут вызваны компилятор `cc1`, ассемблер `as` и линкер `ld`. Убедиться в этом можно, модифицировав строчку вызова и добавив опции для детального отображения процесса компиляции и сохранения временных файлов. На листинге 1.2 приведена немного упрощённая картинка, в реальности вы увидите гораздо больше визуального мусора, установки переменных окружения и всего такого.

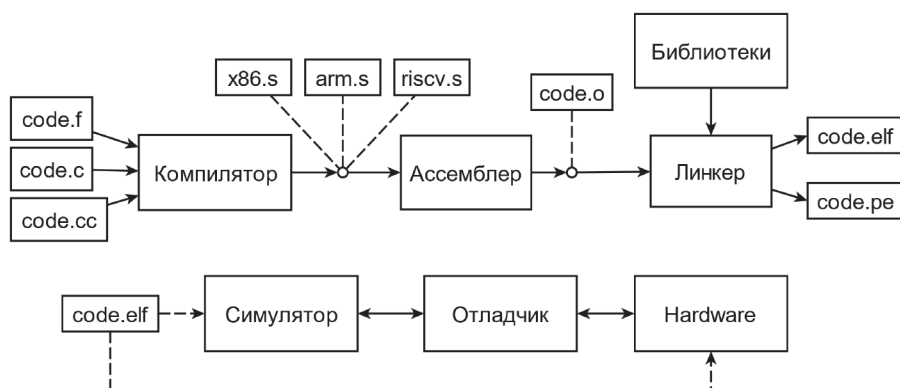


Рис. 1.1: Упрощённая схема тулчейна

Упрощённая схема тулчейна приведена на рис. 1.1. Основными её частями являются компилятор, ассемблер и линкер. Кроме того показано, что получившийся бинарник (binary file, сленговое выражение для файла, содержащего не текст, а двоичный код) можно отлаживать и запускать на железе или на симуляторе. Запуски на симуляторе могут быть полезны при кросс-компиляции, например, если вы на одной архитектуре компилируете код для исполнения на другой.

Задание 1.2. Скачайте кросс-тулчейн для любой архитектуры по вашему выбору и попробуйте скомпилировать программу и запустить её под симулятором.

Итак, под общей крышей программы-драйвера в основном живут компилятор, ассемблер и линкер. Настало время посмотреть на эти основные части поближе.

1.2 Компиляторы

If you lie to the compiler, it will get its revenge.
– Henry Spencer

Компилятор — это основа, главная и самая сложная часть тулчейна. Как можно судить по рис. 1.1, компилятор решает задачу, кажущуюся простой — он берёт программу на высокоуровневом языке и строит из неё ассемблер. Чтобы показать, что эта задача не тривиальна, давайте рассмотрим довольно простой код на листинге 1.3.

```
int sum(int n) {  
    if (n == 1)  
        return 1;  
    return n + sum(n - 1);  
}
```

Листинг 1.3: Сумма чисел от 1 до n

Как компилятор должен транслировать эту программу? Он может просто построить ассемблер как написано, но такой ассемблер будет не слишком хорош. Улучшая этот код, компилятор может заменить рекурсию на цикл. Тогда ему придётся ввести дополнительную переменную для счётчика цикла. Возможная трансформация такого рода показана на листинге 1.4.

```
int possible_sum1(int n) {  
    int i, s = 1;  
    for (i = n; i != 1; --i)  
        s += i;  
    return s;  
}
```

Листинг 1.4: Возможная трансформация
суммы чисел компилятором

Наконец компилятор может просто вернуть известное значение суммы, как на листинге 1.5.

```
int possible_sum2(int n) {  
    return n * (n + 1) / 2;  
}
```

Листинг 1.5: Ещё более агрессивная трансформация

Проблема в том, что этот вариант не эквивалентен листингу 1.3 для отрицательных аргументов. С другой стороны, если число изначально отрицательное, то и исходный код ведёт себя некорректно, переполняясь вниз. Вариант с циклом с листинга 1.4 сохраняет ошибочное поведение ошибочным причём точно таким же. Но он медленней, так как делает цикл там, где можно просто сразу вычислить значение для корректного случая. Наконец компилятор может выбрать гибридный вариант, оставив сокращённое вычисление при корректном поведении и оставив всё как есть при ошибочной семантике, как на листинге 1.4.

```
// compiler can rewrite things even this way
int possible_sum3(int n) {
    if (n > 0)
        return n * (n + 1) / 2;
    return n + possible_sum3(n - 1);
}
```

Листинг 1.6: Компромисс между оптимизацией
и сохранением семантики

Представьте объём анализа, который нужен для принятия такого решения. На момент написания этой книги два разных промышленных тулчейна — clang и gcc строят два разных варианта ассемблера для такого, казалось бы, простого примера. И даже просто глядя на сгенерированный ассемблер, пример которого приведён на листинге 1.7, мы иногда испытываем сложности с тем, чтобы соотнести его с исходным кодом.

```
possible_sum3(int):
    mov     eax, 1
    cmp     edi, 1
    je      .L1
    lea     eax, [rdi - 1]
    lea     ecx, [rdi - 2]
    imul    eax, ecx
    lea     edx, [rdi - 3]
    imul    rdx, rcx
    shr     rdx
    add     eax, edi
    sub     eax, edx
    inc     eax
.L1:
    ret
```

Листинг 1.7: Сумма чисел на ассемблере x86

Задание 1.3. Возможно, когда вы читаете эту книгу вы используете другую архитектуру или вам доступен более совершенный компилятор. Скомпилируйте все варианты суммы чисел, изучите их ассемблер.

Большинство компиляторов работают по правилу известному как «as if rule» — они оптимизируют программу так, чтобы последовательность побочных эффектов, порождаемых её корректным исполнением, была такой же, как если бы програм-

ма выполнялась написавшим её программистом в уме. Ключевым в этом правиле является именно то, что оно закладывается только на корректное исполнение. Исполнение, приводящее к неопределённому поведению (например, к знаковому целочисленному переполнению), считается невозможным и оптимизатором не учитывается.

Конструктивно компилятор состоит из фронтенда (compiler frontend), который занимается построением промежуточного представления из исходного кода, а также из бэкенда (compiler backend), который занимается его оптимизациями и кодогенерацией. Англицизмы фронтенд и бэкенд неизбежны: во-первых, они укрепились в технической традиции и не только в компиляторах — свои фронтенды есть и в вебе и в других областях (и там они значат другое); а во-вторых, адекватные переводы являются громоздкими и скорее запутывают. Иногда оптимизации выделяют в миддленд, оставляя в бэкенде только кодогенерацию, но в индустрии термин миддленд не распространён, и я буду объединять его с бэкендом, разделяя бэкенд на оптимизатор и кодогенератор.

1.2.1 Лексический анализ

Первая фаза фронтенда — это лексический анализ и препроцессинг. Ключевое понятие в этом процессе — это лексема или токен. Токен представляет собой атомарную единицу анализа, это что-то вроде отдельного слова или отдельного числа, или отдельной запятой, и так далее. Обычно его хранят как объединение его типа (ключевое слово, скобка, оператор, значение) с его значением (int, правая скобка, оператор больше, -1.5).

Главная задача лексического анализа — это нормализовать текст программы, в котором могут быть лишние переносы строк, пробелы, странные отступы и так далее, и получить из него массив токенов, над которым далее будет удобно работать синтаксическому анализатору. Если в языке есть препроцессор, то этот процесс становится двухфазным: сначала мы получаем представление программы в так называемых препроцессинговых токенах, чтобы над ним было удобно работать макроподстановке, а потом уже получаем его же в обычных токенах.

Типичная схема работы лексического анализатора представлена на рис. 1.2. Над препроцессинговыми токенами не только выполняются директивы препроцессора, а также раскрываются макросы, заменяются так называемые «escape»-последователь-

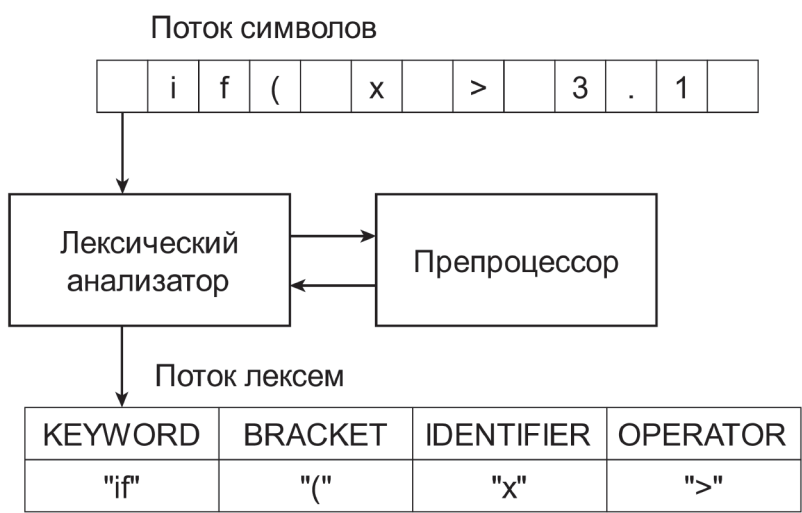


Рис. 1.2: Упрощённая схема лексического анализатора

ности (вы, вероятно, их неоднократно видели, например это `\n`, `\t` и т.д.) и объединяются литералы. В этот же момент происходит текстовое включение заголовочных файлов (если в языке оно поддержано).

С практической точки зрения иногда очень полезно посмотреть получившийся препроцессированный код. Представим ситуацию, когда дана программа на языке C, приведённая на листинге 1.8. Она содержит не вполне ясную ошибку: пытается проверить простое логическое условие и внезапно получает некорректный результат.

```
enum type_t {
    OP_IGNORED = 0x0, OP_SRC = 0x1000,
    OP_DST = 0x2000, OP_DSTSRC = 0x3000
};

bool is_op_src(enum type_t a) {
    return a & OP_SRC;
}

// ....

enum type_t x = OP_DSTSRC;
assert(is_op_src(x)); // FAILED!
```

Листинг 1.8: Программа с загадочной ошибкой

На последней строчке падает проверка `assert`, хотя, казалось бы, не должна. Ситуация выглядит совершенно мистической, но перестаёт такой быть, если заглянуть в препроцессированный файл.

```
$ gcc -E hello.c -o hello.i
$ cat hello.i
....
char is_op_src(enum type_t a) {
    return a & OP_SRC;
}
```

Листинг 1.9: Помощь препроцессирования

В этот момент видно, что случилось: кто-то определил `bool` в `char`. Это полезно в плане экономии места (в языке C `bool` это `int`), но в данном случае вызвало довольно тихое сужение типа.

Задание 1.4. *Попробуйте препроцессировать ваш код. В качестве исходного файла используйте обычную программу «Hello, world» с листинга 1.1. Сколько строк она содержит после препроцессора?*

Лексический анализ обычно сводится к разбору регулярных выражений. Анализатор может быть написан *ad hoc*, но может также быть автоматически сгенерированным такими программами, как `flex` из более высокоуровневого описания. Если вы не знаете, что такое регулярные выражения, то, чтобы разобраться в этом вопросе, настоятельно рекомендую обратиться к любой книге на эту тему, например к замечательной книге Джеффри Фридла [21], так как эти знания будут необходимы далее.

В некоторых языках, в частности в Python, лексические элементы (отступы) являются частью грамматики, что ведёт либо к контекстно-зависимой грамматике, либо к специальному лексическому разбору, изолирующему контекстно-зависимые пробельные конструкции. В этом случае написание лексического анализатора может стать интересной задачей.

Более подробно останавливаться на лексическом анализе сейчас не требуется, но я также могу рекомендовать хорошие книги по теме (см. [37] и прочие). Тема по большей части закрыта, интересных проблем там нет, и эта книга вряд ли к ней что-то добавит. В разделе задач вам будет предложено поупражняться с `flex` и, если вы ранее никогда этого не делали, я очень рекомендую не игнорировать это задание.

После лексического анализа и препроцессирования происходит синтаксический и семантический анализ.

1.2.2 Синтаксический анализ

*A Parser for Things is a function from Strings
to Lists of Pairs of Things and Strings!*

– Fritz Ruehr

Целью синтаксического анализа является (иногда неявное) построение синтаксического дерева (Abstract syntax tree, AST). Этот процесс называют синтаксическим разбором или парсингом. Синтаксическое дерево является представлением программы, связывающим операции с зависимостями по данным. При этом подвыражения являются вершинами этого дерева, дочерними относительно выражений, которые из них состоят.

Например, если дано присваивание $x = 2 * (y + z) + 5$, то его подвыражениями являются его левая и правая часть. При этом правая часть является суммой, один из операндов которой является произведением, и так далее.

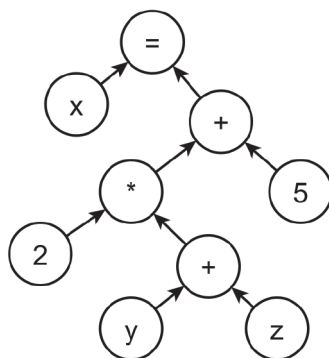


Рис. 1.3: Пример синтаксического дерева

Пример дерева для этого выражения приведён на рисунке 1.3. Видно, что синтаксическое дерево вскрывает структуру вычислений. Именно поэтому процесс его построения называется «разбором»: если нужно вычислить эту формулу, то достаточно подставить листья, а потом пройти по дереву, собирая подвыражения. Но используя деревья, формулы можно не только вычислять, но и делать многое другое.

Тут, конечно, встаёт вопрос — а как, собственно, научить компьютер выделять составные части, подвыражения и всё остальное, что необходимо для разбора такого рода? К счастью, теория синтаксического разбора отлично разработана.

Основой этой теории является понятие контекстно-свободной грамматики (context-free grammar). Контекстно-свободная

грамматика для простых арифметических выражений вроде того, разбор которого приведён на рисунке 1.3, показана на рисунке 1.4.

$$\langle \text{statement} \rangle ::= \langle \text{ident} \rangle '=' \langle \text{expr} \rangle$$

$$\langle \text{expr} \rangle ::= \langle \text{term} \rangle '+' \langle \text{expr} \rangle$$

$$| \langle \text{term} \rangle '-' \langle \text{expr} \rangle$$

$$\langle \text{term} \rangle ::= \langle \text{mult} \rangle '*' \langle \text{term} \rangle$$

$$| \langle \text{mult} \rangle '-' \langle \text{term} \rangle$$

$$\langle \text{mult} \rangle ::= '(' \langle \text{expr} \rangle ')'$$

$$| \langle \text{ident} \rangle$$

$$| \langle \text{number} \rangle$$

Рис. 1.4: Контекстно-свободная грамматика простых арифметических выражений

Любую грамматику можно рассматривать как порождающую грамматику. В треугольных скобках на рисунке 1.4 записаны нетерминалы, то есть такие символы, которые должны исчезнуть в процессе построения. Мы начинаем с нетерминала верхнего уровня — в данном случае это $\langle \text{statement} \rangle$ — выводим из него $\langle \text{ident} \rangle = \langle \text{expr} \rangle$ и далее $x = \langle \text{expr} \rangle$. Далее раскрываем $\langle \text{expr} \rangle$ и так далее, пока все нетерминалы не будут раскрыты. Терминальные символы называются так именно потому, что остаются в конце всех раскрытий.

Синтаксическое дерево программы (дерево грамматического разбора) следует из собственного контекстно-свободного представления и принятого нами правила порождения. Есть остроумные и сложные алгоритмы синтаксического анализа и разбора. Более подробно про это можно прочитать в прекрасной книге [37]. Все промышленные компиляторы, которые известны автору, пользуются самым простым рекурсивным спуском, который довольно интуитивен.

Задание 1.5. Постройте контекстно-свободную грамматику для правильных скобочных выражений. Постройте дерево вывода для выражения $(())(())(())$.

Реалистичное синтаксическое дерево с программы на языке С или С++ можно вывести на стандартный вывод или в файл (говорят также «сдампить» от английского «dump») разными об-

разами, в том числе используя clang. Листинг 1.10 показывает пример такого дампа.

```
$ clang -fsyntax-only -Xclang -ast-dump sumeven.c
....
| -ForStmt
| | -DeclStmt
| |   '-VarDecl used i 'int' cinit
| |   '-IntegerLiteral 'int' 0
| | -<<<NULL>>>
| | -BinaryOperator 'int' '<'
| |   | -ImplicitCastExpr 'int' <LValueToRValue>
| |   |   '-DeclRefExpr 'int' lvalue Var 'i' 'int'
....
```

Листинг 1.10: Дамп синтаксического дерева

Не все языки могут быть описаны контекстно-свободными грамматиками. Простейший пример: наличие у функции объявления до использования. Это контекстно-зависимая конструкция, так как программа может быть корректной или некорректной в точке использования функции в зависимости от того, что было написано существенно ранее в её объявлении.

Тем не менее, используемые нами грамматики в основном контекстно-свободны, потому что только для этого класса грамматик существуют эффективные методы синтаксического разбора. Самый общий алгоритм контекстно-свободного разбора, алгоритм Кока-Янгера-Касами [60] по сложности не превосходит $O(N^3)$, а для частных классов таких как $LL(1)$ возможны алгоритмы с линейной сложностью. В то же время парсинг даже сравнительно простых классов контекстно-зависимых грамматик может выливаться в задачи с решениями экспоненциальной сложности (см. [27]).

Задание 1.6. Разберитесь с алгоритмом Кока-Янгера-Касами для наиболее общей задачи синтаксического разбора. Он основан на дискретном динамическом программировании (*dynamic programming*). Если вы впервые слышите термин «динамическое программирование» (*dynamic programming*), постарайтесь разобраться с ним тоже, это будет нужно далее, см. например задачу 7.25, но не только в ней.

Чтобы иметь дело с контекстно-зависимыми конструкциями, в парсерах принято делать семантический анализатор (*semantic analyzer*), который оперирует сохранённым контекстом и на частично постро-

енном дереве принимает решение о том, как именно формируется следующий узел или выдаёт сообщение о семантических ошибках.

Пример семантической ошибки, специфичной для языка C++, приведён на листинге 1.11.

```
delete [] { return new int; }();
```

Листинг 1.11: Пример семантической ошибки

С точки зрения синтаксиса тут всё правильно и мало того, даже человек глазами может истрактовать этот код на C++ как правильный код: лямбда-функция, определённая и тут же вызванная на месте, возвращает выделенный указатель, после чего оператор `delete` его удаляет. Но по стандарту C++ квадратные скобки в данном случае не могут означать начало списка захвата лямбда-функции и всегда должны означать `delete[]`, и здесь такая конструкция не имеет смысла.

Семантический анализатор работает в паре с синтаксическим над построением синтаксического дерева, см. рис. 1.5. В обычной схеме (реализованной, например, во фронтенде clang), синтаксический анализатор при построении вершины AST, следующей из грамматики, обращается к семантическому анализатору за проверкой семантики порождаемой конструкции. Далее из синтаксического дерева порождается HIR, подробнее о котором см. в разделе 1.2.3.

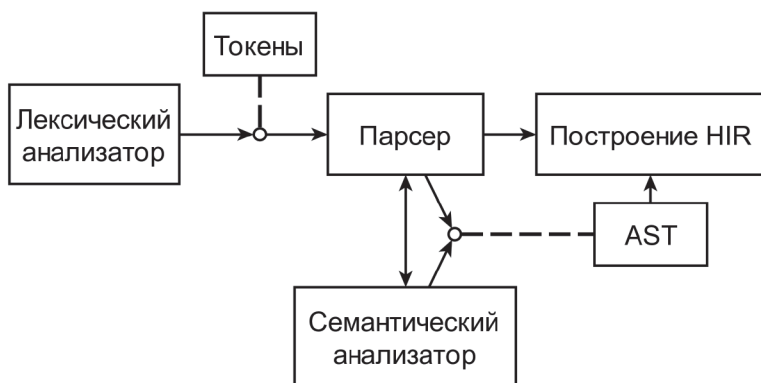


Рис. 1.5: Схема синтаксического и семантического анализа

В компиляторе clang, парсер при попытке построения выражения `delete`, выполняет функцию `ParseCxxDeleteExpression` и внутри неё обращается к методу `ActOnCXXDelete` семантического анализатора, который проверяет, что выражение удаления действительно соответствует семантике (то есть смыслу) удале-

ния. И если эта проверка успешна, именно семантический анализатор создаёт соответствующий узел и отдаёт обратно парсеру для подвешивания этого узла в дерево.

1.2.3 Промежуточное представление

После того, как синтаксическое дерево (AST) построено, фаза синтаксического и семантического разбора фактически завершена. Теперь компилятор имеет представление программы в удобном виде и над этим представлением могут быть даже проведены дополнительные проверки (например, такими инструментами как `clang-tidy`). Но, будучи удобным для представления и анализа программы, этот вид крайне неудобен для её трансформаций.

Чтобы облегчить оптимизации, AST переводится в промежуточное представление (IR, *intermediate representation*). Количество необходимых уровней промежуточного представления — это дискуссионный вопрос. Базовый GCC имеет два. Базовый LLVM имеет три, но за счёт идеи многоуровневого IR (MLIR) инфраструктура LLVM может поддерживать куда больше специализированных представлений. В теоретических целях, нам для этой книги понадобятся два.

1. Высокоуровневый IR (*high-level IR*, HIR) — это промежуточное представление удобное для машинно-независимых оптимизаций.
2. Низкоуровневый IR (*low-level IR*, LIR) — это промежуточное представление удобное для машинно-зависимых оптимизаций.

В этом разделе мы познакомимся с HIR, так как далее мы будем в основном работать с ним. Знакомство с LIR будет отложено до главы 7, где он нам пригодится. Обычно стараются делать так, чтобы HIR сохранял максимальное количество метаданных, оставшейся от конкретного языка программирования и наоборот, чтобы LIR был максимально близок к конструкциям конкретной машины, для которой происходит кодогенерация.

Общая схема работы оптимизатора показана на рис. 1.6. Фронтенд на ней визуально отделён от бэкенда и, собственно, HIR является интерфейсом между этими частями. Фронтенд обычно может быть заменён (для разных языков пишутся разные фронтенды и это нормально, пока все они генерируют один и тот же HIR). Так что мысленно дорисуйте этой картинке много разных фронтендов. А вот оптимизатор и бэкенд обычно остаются одной сущностью, в которой только заменяется описание целевой машины.

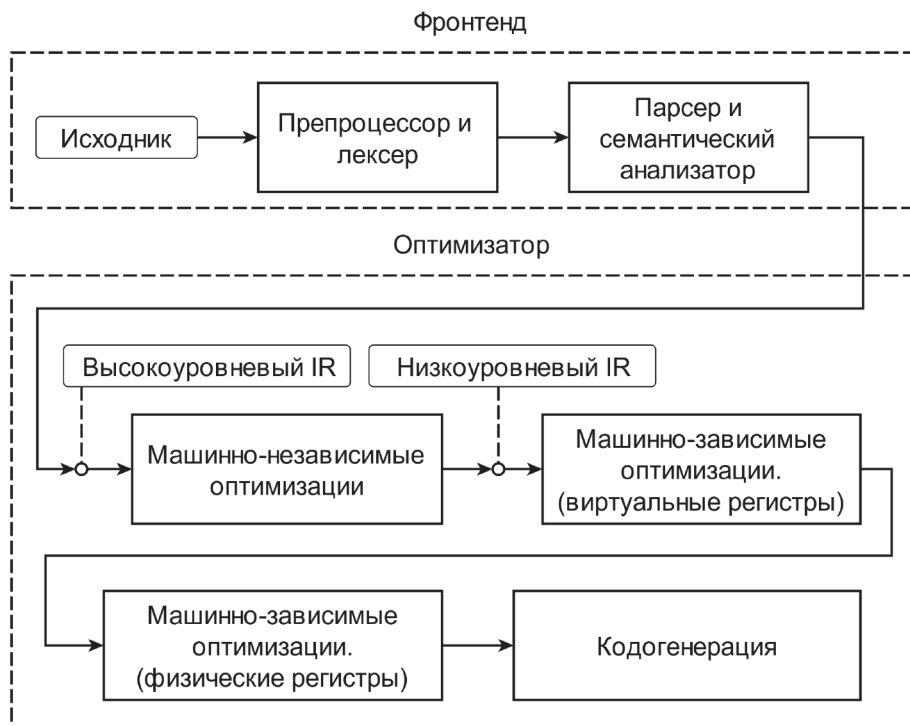


Рис. 1.6: Схема оптимизатора

В разных компиляторах промежуточное представление может очень сильно отличаться. В GCC в качестве HIR выступает Gimple, а в качестве LIR выступает RTL. В случае LLVM в качестве HIR выступает LLVM IR, а в качестве LIR выступает MachineIR. Различают также разные варианты MachineIR — с использованием физических либо виртуальных регистров.

Проектирование хорошего высокоуровневого IR, как и проектирование любого хорошего интерфейса — это сложная инженерная задача, в которой всегда надо найти баланс между стабильностью, расширяемостью и удобством. Лучшим из известных автору решений в этой области является SPIR-V, промежуточное представление для графической системы Vulkan, но увы, он слишком сложен, чтобы использоваться как учебный пример.

В этой книге в качестве HIR будет принят простой высокоуровневый IR, чем-то похожий и на Gimple и на LLVM IR. Я не предполагаю ни его бинарную стабильность, ни расширяемость, основная его задача — это быть удобным в работе и легко читаемым.

Структура этого промежуточного представления очень проста:

1. Каждое вычисление содержит не более одной бинарной или унарной операции. Это называется **трёхадресным** кодом.
2. Переменные типизированы, но тип может не быть указан в листинге, если он может быть выведен.
3. Единственная разрешённая конструкция перехода — это переходы на метки. Также разрешён возврат из функции в любом месте через `return`.
4. Разыменование `*x` означает обращение по адресу `x`. Остальные бинарные и унарные операции из языка C означают то же самое, что и в языке C, включая арифметику, побитовую арифметику, логику и сравнения.
5. Разделителем операций является точка с запятой. Несколько операций могут идти на одной строке (это сделано для экономии места в книге и не характерно для реалистичных высокоуровневых представлений). Исключением является условный оператор, где для читаемости я разделяю точкой запятой `if` от `else`.

Потренируемся на простейшей программе на языке C, представленной на листинге 1.12.

```
int foo(int *x, int y) {  
    int a = x[1] * (y + 5);  
    if (a > 0)  
        a = a - 5 * x[2];  
    return (a + a * y + a * x[3]);  
}
```

Листинг 1.12: Простейшая программа до перевода в NIR

Чтобы перевести эту программу в предлагаемую форму NIR, над кодом необходимо выполнить две операции:

1. Любые вычисления разбить так, чтобы получить трёхадресный код.
2. Избавиться от сложных управляющих конструкций, унифицировав их до переходов к меткам.

Обе операции делаются почти механически. Некоторая проблема может быть с индексацией массивов, но можно вспомнить, что по правилам языка C, `x[i]` это то же самое, что и `*(x + i)`.

Представление программы с листинга 1.12 после её трансформации в NIR показано на листинге 1.13.

```

int foo(int *x, int y) {
BB1: a1 = x + 1; t1 = *a1; t2 = y + 5;
      a = t1 * t2;
      if (a > 0) goto BB2; else goto BB3;
BB2: a2 = x + 2; t3 = *a2; t3 = 5 * t3;
      a = a - t3;
      goto BB3;
BB3: a3 = x + 3; t4 = *a3; t4 = a * t4;
      t5 = a * y; t5 = t5 + t4; a = a + t5;
      return a;
}

```

Листинг 1.13: HIR для программы с листинга 1.12

Для иллюстрации более осмысленной трансляции можно взять реалистичную программу на языке C, представляющую собой суммирование чётных элементов массива. Она приведена на листинге 1.14.

```

int sumeven(int *x, int n) {
    int sum = 0;
    for (int i = 0; i < n; ++i)
        if ((x[i] % 2) == 0)
            sum += x[i];
    return sum;
}

```

Листинг 1.14: Суммирование чётных элементов

Задание 1.7. Попробуйте перевести программу с листинга 1.14 в HIR самостоятельно.

Не читайте дальше пока не попробуете выполнить задание 1.7.

Вы должны были получить тот же результат, что и на листинге 1.15. Внимательно изучите этот листинг, посмотрите нет ли тут для вас чего-то неожиданного.

```

int sumeven(int *x, int n) {
BB1:  sum = 0; i = 0;
      goto LOOP;
LOOP: if (i < n) goto BB2; else goto BB5;
BB2:  tmp = x + i; elt = *tmp;
      odd = elt % 2;
      if (odd != 0) goto BB4; else goto BB3;
BB3:  tmp = x + i; elt = *tmp;

```

```
        sum = sum + elt;  
        goto BB4;  
BB4:    i = i + 1;  
        goto LOOP;  
BB5:    return sum;  
    }
```

Листинг 1.15: HIR для суммирования чётных элементов

В листинге 1.15 обратите внимание на формально вставленные `goto` там, где управление переходит к следующей метке, даже если оно «само» туда проваливается, как в случае перехода с B_3 на B_4 . Это нужно, чтобы чётко выделять так называемые **терминаторы**, и понадобится мне далее при введении понятия базового блока в разделе 2.1.

Кроме того, настоящий HIR в современных компиляторах уже на ранних этапах приводится к так называемому SSA представлению. Подробно мы его определим и научимся строить в разделе 3.1. Обычно в HIR делают визуальную разницу между изменяемыми переменными и SSA-значениями, которые не изменяются, будучи единожды присвоены. Их выделяют знаком процента или чем-нибудь в этом роде. Для целей изложения в этой книге я такого выделения делать не буду. Просто будем считать, что если мы видим хотя бы один ϕ -узел в дампе HIR, то весь HIR уже переведён в SSA.

Задание 1.8. *Получите для листинга 1.14 LLVM IR сразу после clang и Gimple сразу после GCC FE. Вы должны получить именно IR до построения SSA и начала любых оптимизаций. Сравните с тем упрощённым HIR, который мы будем далее использовать. Видите ли вы существенные особенности и различия?*

Задание 1.9. *Проанализируйте листинг 1.15. Можете ли вы найти какие-нибудь полезные трансформации, которые можно было бы провести над этим представлением?*

1.2.4 Машинно-независимые оптимизации

Верхнеуровневое промежуточное представление выражено в абстрактных операциях, а не в наборе конкретных операций целевой архитектуры. Это сделано не случайно. Существует довольно много трансформаций кода, которые в целом одинаковы для всех архитектур. Например, на ли-

стинге 1.15 мы можем заметить избыточные вычисления в B_3 : и `tmp` и `elt` уже подсчитаны в B_2 и могут быть переиспользованы.

Как пример я возьму компилятор LLVM и уже рассматривавшуюся функцию суммирования чётных чисел с листинга 1.14. Драйвер компиляции clang позволяет довольно легко получить HIR до и после верхнеуровневых оптимизаций.

```
$ clang -O2 -emit-llvm -S sumeven.c -Xclang\  
-disable-llvm-passes -o initial.ll  
$ clang -O2 -emit-llvm -S sumeven.c -o final.ll  
$ clang -O2 -emit-llvm -S\  
-mllvm --print-after-all sumeven.c >& all.log  
$ grep "\*\*\*" all.log
```

Листинг 1.16: Получение LLVM IR до и после оптимизаций

Метод через `grep` кажется немножко грубоватым и многие любят специализированные опции компилятора `-mllvm -debug-pass=Details`, но мне нравится простота `grep` и его настраиваемость на небольшие изменения дампов от версии к версии. Вы можете получить ту же информацию любым удобным для вас путем. На случай если вы не знаете, `grep` — это одна из распространённых утилит Unix Shell. Она ищет заданную подстроку в файле, при этом подстрока может быть задана регулярным выражением. Про все параметры и возможности этой утилиты можно прочесть в замечательной книге [45].

Слово «`grep`» изначально было аббревиатурой от «global regular expression print», но утилита приобрела такую популярность, что сейчас используется в качестве самостоятельного термина (to `grep` something и так далее). Мне нравится общеупотребительный в профессиональных кругах англицизм «грепнуть» означающий «провести поиск всех подстрок, удовлетворяющих регулярному выражению», и я далее буду его беззастенчиво использовать.

В выдаче команды `grep` мы видим несколько видов оптимизаций над HIR (в данном случае над LLVM IR). Каждая оптимизация представляет собой проход, или — что то же самое — пасс (optimization pass), или фазу оптимизации. Во время очередного прохода, компилятор может рассматривать контекст всего модуля, функции или даже отдельных частей функции (например, отдельный цикл).

Примеры оптимизаций над модулем можно получить (нагреть) из лога как показано на листинге 1.17.

```
$ grep "\*\*\*" all.log | grep "on \[module\]"
GlobalDCEPass on [module]
ConstantMergePass on [module]
```

Листинг 1.17: Выборка оптимизаций над модулем
из лога оптимизаций

Такие оптимизации рассматривают модуль целиком. Например, оптимизация GlobalDCE пытается убрать неиспользуемые статические функции из модуля, а ConstantMergePass — найти одинаковые глобальные константы и собрать их в одно опреледеление.

Примеры оптимизаций над функцией тоже легко гребаются из выдачи LLVM. Для этого надо немного изменить запрос, указав название желаемой функции.

```
$ grep "\*\*\*" all.log | grep "on sumeven"
InstCombinePass on sumeven
SimplifyCFGPass on sumeven
```

Листинг 1.18: Выборка оптимизаций над функцией из лога
оптимизаций

Оптимизациям такого типа для работы нужен контекст функции. InstCombine пытается объединять инструкции по правилам арифметики. SimplifyCfg упрощает граф потока управления. Подробнее мы поговорим о графе потока управления в разделе 2.1.

Примеры оптимизаций над циклами опять-таки легко получить гребом. Циклы, как и модули безымянны.

```
$ grep "\*\*\*" all.log | grep "on Loop"
LICMPass on Loop at depth 1
LoopFullUnrollPass on Loop at depth 1
```

Листинг 1.19: Выборка оптимизаций над циклами
из лога оптимизаций

Здесь LICM пытается вынести инварианты из цикла, а LoopFullUnroll пытается полностью линеаризовать (или «развернуть») цикл, расположив итерации одну за другой.

Собственно, почти вся эта книга (кроме текущей обзорной главы) будет посвящена изучению оптимизаций над HIR. Современные компиляторы широко используют для таких оптимизаций именно SSA представление (например, LLVM IR это SSA по определению), но мы изучим также альтернативы.

Задание 1.10. Будет отлично, если прямо сейчас вы поищете в справке на ваш компилятор, какие оптимизации он производит и посмотрите, что конкретно они делают. Попробуйте осознать предназначение хотя бы половины из них. Прежде чем изучать теорию компиляции очень полезно поиграть с компилятором и посмотреть, как разные оптимизации работают с HIR.

1.2.5 Кодогенерация

После того как машинно-независимые оптимизации над HIR выполнены, компилятор готов провести кодогенерацию. Кодогенерация — это сложный процесс, иногда кодогенератор имеет больше фаз чем оптимизатор. Но если смотреть на основные этапы, то кодогенерация по большому счёту состоит из выбора и планирования инструкций, распределения регистров и генерации ассемблерного кода.

Естественное начало этого процесса — это перевод HIR в LIR для машинно-зависимых оптимизаций. Этот процесс называется выбором или сопоставлением инструкций (instruction selection). Мы очень подробно его разберём в разделе 7.1. Пока что я предлагаю не вводить формальное описание LIR (там будут свои тонкости), а проиллюстрировать весь процесс на базовом LLVM IR. Очень простой код, который мы для этого используем приведён на листинге 1.20.

```
int fma(int x, int y, int z) {  
    return x + y * z;  
}
```

Листинг 1.20: Простейшая функция FMA

Функция вычисляет сумму одного операнда и произведения двух других. Если посмотреть LLVM IR, получающийся из этой функции, и приведённый на листинге 1.21, он очевиден и не вызывает удивления.

```
define i32 @fma(i32 %0, i32 %1, i32 %2) {  
    %4 = mul nsw i32 %2, %1  
    %5 = add nsw i32 %4, %0  
    ret i32 %5  
}
```

Листинг 1.21: LLVM IR для FMA

Задание 1.11. Переведите программу с листинга 1.20 в HIR.

Давайте посмотрим, как отработал выбор машинно-зависимых инструкций — LLVM IR (листинг 1.21) был трансформирован в MachineIR (листинг 1.22) при переходе от оптимизации к кодогенерации.

```
bb.0 (%ir-block.3):  
    liveins: $edi, $esi, $edx  
    %2:gr32 = COPY $edx  
    %1:gr32 = COPY $esi  
    %0:gr32 = COPY $edi  
    %3:gr32 = IMUL32rr %2:gr32, %1:gr32  
    %4:gr32 = ADD32rr %3:gr32, %0:gr32  
    $eax = COPY %4:gr32  
    RET 0, $eax
```

Листинг 1.22: Machine IR для FMA

На листинге 1.22, вместо машинно-независимых верхнеуровневых операций, мы видим такие инструкции как `IMUL32rr` и `ADD32rr`, которые подозрительно напоминают нам конкретные операции в ассемблере x86. Подробнее про алгоритмы выбора инструкций, приведшие к такому результату, мы поговорим в разделе 7.1. Обратите также внимание, что на листинге 1.22 физические регистры, определяемые позиционными операндами инструкции, пока что переключаются в виртуальные, обозначенные символом процента.

На приведённом представлении кодогенератор делает много чего — в частности планирует исполнение инструкций (производя *instruction scheduling*). Это большая и интересная тема, которую в этой книге мы обсудим в разделе 7.3.

Далее происходит распределение регистров. На этом этапе назначаются физические регистры, а там, где их не хватает, выделяются специальные области временного хранения значений на стеке, которые называются «спил слотами». Алгоритмы распределения регистров и связанную терминологию мы обсудим в разделе 7.2. Пример Machine IR после распределителя приведён на листинге 1.23.

```
bb.0 (%ir-block.3):  
    liveins: $edi, $edx, $esi  
    $esi = IMUL32rr $esi, $edx,  
            implicit-def dead $eflags  
    $eax = LEA64_32r $rsi, 1, $rdi, 0, $noreg  
    RET64 $eax
```

Листинг 1.23: Machine IR для FMA на физических регистрах

Мы видим, что на листинге 1.23 входом той же машинно-зависимой инструкции `IMUL32rr` являются уже физические регистры `%esi` и `%edx`, а также флаговый регистр в качестве неявного аргумента.

Задание 1.12. Низкоуровневый IR с листинга 1.23 очень похож на ассемблер. Запишите эквивалентный ему ассемблер x86.

После распределения регистров, кодогенератор делает последние преобразования, применяя так называемые правила точечных оптимизаций (*peephole rules*), ориентированные уже на детали целевой архитектуры. Пример такого преобразования — это *Load Value Injection Ret-Hardening* в x86: компилятор вставляет в код специальную поддержку для борьбы с атаками класса *Meltdown* см. [56]. Часто также происходит последнее перепланирование уже на физических регистрах.

1.3 Ассемблеры

Real programmers can write assembly code in any language.

– Larry Wall

Под словом «ассемблер» обычно понимается либо язык ассемблера для конкретной архитектуры, либо ассемблер как программа, которая транслирует программу на языке ассемблера в объектный код (*object code*). Объектным кодом называется специальное представление машинного кода, в котором инструкции уже закодированы, но который всё ещё предназначен не для исполнения, а для дальнейшей линковки (после которой он становится исполняемым кодом). В некоторых тулчейнах ассемблер как программа может быть (опционально) встроен в компилятор и тогда объектный код порождает сразу компилятор.

Здесь может возникнуть сразу несколько вопросов.

Например, зачем вообще нужен язык ассемблера, если можно сразу порождать объектный код? Наличие отдельного языка ассемблера мотивировано, во-первых, тем, что дизассемблер читать и отлаживать гораздо проще, чем бинарный код. Отладка по ассемблерному коду — это нормальный сценарий для отладки сложных проблем, плохо проявляющихся в отладочных сборках. Во-вторых, набор инструкций

для каждой архитектуры обычно куда больше, чем всё, что может породить компилятор. Существуют специальные инструкции для работы с системными регистрами и писать хотя бы небольшие программы на языке ассемблера в некоторых случаях приходится (например, при написании операционных систем).

Ещё один возможный вопрос это зачем нужен объектный код, почему бы сразу не порождать исполняемый файл, минуя промежуточные фазы? Здесь мотивация ещё проще. В большинстве известных мне языков программирования предусмотрена в том или ином виде отдельная трансляция. Как минимум она нужна для того, чтобы получить возможность иметь предварительно скомпилированный библиотечный код. Для такого кода необходим механизм, который позволял бы сохранять информацию для разрешения междоульных вызовов. Одновременно этот механизм не слишком нужен в итоговом исполняемом файле, где все кросс-модульные зависимости уже разрешены. Это и мотивирует наличие объектного кода.

Давайте посмотрим на конкретный пример программы на языке ассемблера, выводящей на экран «Hello, world!». Эта программа приведена на листинге 1.24.

```
.text
.globl main
.type main,@function
main:
    .cfi_startproc
    push rax
    .cfi_def_cfa_offset 16
    mov edi, offset .L.str.1
    call puts
    xor eax, eax
    pop rcx
    .cfi_def_cfa_offset 8
    ret
.Lfunc_end0:
    .cfi_endproc
.section .rodata.str1.1, «aMS», @progbits, 1
.L.str.1:
    .asciz «Hello, world!»
```

Листинг 1.24: x86 ассемблер для «Hello, world»

На листинге 1.24 показан ассемблер x86 (синтаксис Intel) для программы, печатающей на экран приветствие миру.

Что можно видеть в этом ассемблере?

- **Инструкции** — например, `push`, `mov`, `call`. Для каждой архитектуры есть свой, немного отличающийся набор инструкций, и он определяет её ISA.
- **Директивы** — например, директива `.text` открывает секцию кода, а директивы `.cfi_startproc` и `.cfi_endproc` сообщают важную отладочную информацию о начале и конце кода функции.
- **Метки** — например, метка `main:` определяет начало функции, а метка `.L.str.1:` определяет литерал в секции `rodata`.

На самом деле это не уникальные вещи для ассемблера x86. Все ассемблеры в мире очень похожи. Если вы знаете один, вы обычно знаете примерно все.

```
.text
.attribute    5, «rv64i2p1_m2p0_a2p1_f2p2_d2p2»
.globl main
.type  main,@function
main:
.cfi_startproc
addi    sp, sp, -16
.cfi_def_cfa_offset 16
sd      ra, 8(sp)
.cfi_offset ra, -8
auipc   a0, %pcrel_hi(.L.str.1)
addi    a0, a0, %pcrel_lo(.L.pcrel_hi0)
call    puts
li      a0, 0
ld      ra, 8(sp)          # 8-byte Folded Reload
addi    sp, sp, 16
ret
.Lfunc_end0:
.cfi_endproc
.section .rodata.str1.1,«aMS»,@progbits,1
.L.str.1:
.asciz «Hello, world!»
```

Листинг 1.25: rv64 ассемблер для «Hello, world»

Для примера на листинге 1.25 показан RISC-V ассемблер той же функции. По большому счёту вы видите там те же метки, те же директивы и очень узнаваемые инструкции. Всё что нужно, чтобы понимать любой ассемблер — это иметь доступ к спецификации ISA, документирующей что делает каждая инструкция.

Разумеется, «Hello, world» это не самая интересная программа, но кое-что интересное в ней есть. Она вызывает функцию `puts`, в которую вроде как должна передавать аргумент, собственно строковый литерал (или его адрес). Но в вызове `call puts` не видно никакого списка аргументов.

1.3.1 Ассемблирование функций и понятие ABI

Чтобы разобраться, как происходит передача аргументов в функции, давайте попробуем просто перечислить все возможные варианты. Первый вариант — это использовать для этого некоторые регистры.

```
auipc  a0, %pcrel_hi(.L.str.1)
addi   a0, a0, %pcrel_lo(.Lpcrel_hi0)
call   puts
```

Листинг 1.26: Формирование аргумента функции `puts`

И действительно, если мы рассмотрим кусок листинга «Hello world», вынесенного мной в отдельный листинг 1.26, то происходящее в ассемблере очень похоже на формирование в регистре `a0` (кстати, говорящее название — нулевой регистр аргументов) адреса строкового литерала, который функция `puts` и выводит на экран.

Но количество регистров ограничено, а аргументов может быть сколько угодно. Вряд ли их можно разместить в куче или в глобальной памяти. Поэтому второй вариант — это использовать стек.

```
int foo(int, int, int, int x, int y, int z, int v) {
    return x + y + z + v;
}
```

Листинг 1.27: Слишком много аргументов у функции

На листинге 1.27 показана функция, у которой последний аргумент не влезает в шесть аргументных регистров. На листинг—

ге 1.28 показан результирующий ассемблер. Обратите внимание как компилятор удачно приспособил инструкцию `lea` для сложения. По факту она должна формировать эффективный адрес (в данном случае в виде «база плюс индекс») но так как кодировка экономичная, а фактически это всё равно сложение, она использована для упрощения.

```
foo:
    lea    eax, [rcx + r8]
    add    eax, r9d
    add    eax, dword ptr [rsp + 8]
    ret
```

Листинг 1.28: Ассемблер для функции с листинга 1.27

Задание 1.13. *Перепишите ассемблерный код с листинга 1.28 не используя `lea`. Ограничьтесь мнемониками `add` и `mov`.*

Главным в листинге 1.28 является обращение к аргументу `v`, который не вместился в регистры. За этим аргументом мы обращаемся в стековый слот `[rsp + 8]`, потому что в слоте `[rsp]` лежит адрес возврата.

Задание 1.14. *Можете ли вы из этого простого примера догадаться куда растёт стек? Например, если вызвать из `foo` следующую функцию, где будет её адрес возврата?*

При использовании стека возникают следующие вопросы:

- В каком порядке закладывать в стек аргументы?
- Куда и как закладывать результаты?
- На стеке вызывающей или вызываемой функции они живут?

На эти и многие другие вопросы (например, на вопрос «сколько использовать регистров перед тем, как использовать стек») отвечает специальный документ, который называется Application Binary Interface или ABI.

Задание 1.15. *Прочитайте ассемблерный код с листинга 1.29 и попробуйте переписать его на C. Вам придётся обратиться к ABI для x86-64. Я специально изменил содержательное имя на `foo`, чтобы вам было интересней. Придумайте более разумное имя.*

```
foo:
    test esi, esi
    jle .L4
    mov rax, rdi
    lea edx, -1[rsi]
    lea rsi, 4[rdi+rdx*4]
    mov ecx, 0
.L3:
    add ecx, DWORD PTR [rax]
    mov DWORD PTR [rax], ecx
    add rax, 4
    cmp rax, rsi
    jne .L3
.L1:
    mov eax, ecx
    ret
.L4:
    mov ecx, 0
    jmp .L1
```

Листинг 1.29: Код к упражнению 1.15. Что делает эта функция?

Для облегчения выполнения задания 1.15 вам следует также разобраться с условными переходами в x86. Можете прочитать хорошую книгу по ассемблеру, например [61] или любую другую.

Лично автору этой книги, когда он изучал ассемблер, всегда казалось удивительным то, как примитивно все устроено на нижних уровнях бытия — и цикл, и ветвление, и функция для современных вычислительных систем всего лишь метки с разными правилами перехода между ними и сохранением/восстановлением контекста.

С точки зрения автора, человек не должен писать ассемблер. Кроме редчайших случаев писать ассемблер гораздо лучше получается у компиляторов. А вот что человек должен уметь делать очень хорошо, так это читать и понимать ассемблер. В процессе разговора об оптимизациях в компиляторе мы будем иногда заглядывать в получившийся ассемблер, будьте к этому готовы и не пугайтесь.

1.3.2 Дизассемблирование и релокации

Вернёмся теперь ещё раз к примеру «Hello, world» с листинга 1.24. Да, мы ещё не до конца его разобрали. Обратим внимание, что вызов функции `puts` является внешним вызовом в библи-

отеку языка C. После ассемблирования этой программы можно посмотреть получившийся объектный код в дизассемблере.

```
$ clang -c -g -O2 hello.c
$ objdump -dS -M intel hello.o
```

Листинг 1.30: Запуск дизассемблера objdump

Дизассемблер objdump, это специальная программа, которая позволяет восстановить информацию из бинарного представления в текстовый ассемблер. Некоторые опции этой программы можно использовать, чтобы аннотировать мнемоники использованной для них кодировкой.

```
Disassembly of section .text:
0000000000000000 <main>:
   0: 50                push    rax
   1: bf 00 00 00 00    mov     edi,0x0
   6: e8 00 00 00 00    call    b <main+0xb>
  b: 31 c0             xor     eax,eax
  d: 59                pop     rcx
  e: c3                ret
```

Листинг 1.31: Дизассемблированный объектный файл для «Hello, world»

На листинге 1.31 показан аннотированный кодировкой дизассемблер объектного кода, построенного для программы «Hello, world». Важно подчеркнуть — именно объектного кода до линковки. На этом листинге видно, что вызов call закодирован как `e8 00 00 00 00`, где нули — это дисплейсмент перехода.

Дисплейсмент (displacement) — это разновидность константы времени линковки. Обычная константа называется immediate value, например, в инструкции `mov rdi, 4` константа 4 — это собственно константа (immediate). Отличие дисплейсмента в том, что он считается относительно текущей инструкции. Грубо говоря, нулевой дисплейсмент в инструкции `call` означает переход непосредственно на следующую инструкцию.

Но, кажется, автор как раз хотел вызвать функцию из стандартной библиотеки, почему же дисплейсмент получился нулевым? Давайте подумаем об этом вместе: а что мог бы тут вставить ассемблер? На этапе трансляции в объектный код ему пока что неоткуда узнать, какое расстояние в итоговом слинкованном коде будет до функции `puts`. Поэтому ассемблер поступает хи-

тро: он забивает дисплейсмент нулями, но при этом информацию о том, что это внешний вызов и что линкер должен будет его разрешить, он записывает в отдельную секцию.

Именно это является ключом к разгадке того, почему ассемблер называется *assembler*, а не *encoder*. То, что он кодирует инструкции, это небольшая часть его работы. Главное, что делает ассемблер — это собирает секции и, в частности, — секции релокаций (собственно, «to assemble» — это и есть глагол «собирать»).

Релокации можно тоже дизассемблировать.

```
$ objdump -r hello.o
hello.o:      file format elf64-x86-64
RELOCATION RECORDS FOR [.text]:
OFFSET          TYPE          VALUE
0000000000000002 R_X86_64_32    .rodata.str1.1
0000000000000007 R_X86_64_PLT32 puts-0x0000000000000004
```

Листинг 1.32: Просмотр информации о релокациях

Тип релокации очень важен, так как именно он определяет, насколько у линкера будут развязаны руки для оптимизации. В следующей главе мы как раз об этом поговорим.

1.4 Линкеры

Статическая линковка запускается после ассемблирования. В самом простом случае можно просто отдать линкеру (например, GNU LD) ваши объектные файлы и стандартные библиотеки. Но пути к стандартным библиотекам обычно сильно зависят от опций сборки. Скажем у вас будет разная стандартная библиотека для 32-битного и 64-битного кода и разная для разных ABI в тех архитектурах, где это применимо. Поэтому обычно пути к стандартным библиотекам формирует драйвер компилятора на основании известных ему мистических сообщений. Их всегда можно подсмотреть, но никогда не рекомендуется задавать явно.

```
$ gcc hello.o -static --verbose
....
collect2 -m elf_x86_64 -static crtbeginT.o -L.
-L<other paths here> hello.o --start-group
-lgcc -lgcc_eh -lc --end-group crtend.o crtn.o
```

Листинг 1.33: Запуск collect2

Стандартная библиотека в этом примере состоит из одного файла `libc.a` и подключается неявной опцией компиляции `-lc`. В общем случае любая библиотека с именем `libxxx.a` подключается опцией `-lxxx`. Ничто не мешает вам собрать собственную библиотеку из ваших объектных файлов. Для этого используется утилита `ar`.

```
unsigned fact(unsigned x) {
    if (x < 2) return 1;
    return x * fact(x - 1);
}
```

Листинг 1.34: Файл `fact.c` реализует функцию факториала

```
unsigned fact(unsigned x);
int main() {
    unsigned f = fact(5);
    printf ("%u\n", f);
}
```

Листинг 1.35: Файл `main.c` использует функцию факториала

Собрать исходный код с листинга 1.34 в библиотеку и слинковать его с кодом с листинга 1.35 довольно просто.

```
$ gcc -O2 fact.c -c -o fact.o
$ ar cr libfact.a fact.o
$ gcc -static main.c -L. -lfact
```

Листинг 1.36: Создание статической библиотеки

Задание 1.16. *Распакуйте вашу стандартную библиотеку `libc.a` (предварительно попробуйте вычислить путь до неё из выдачи компилятора), соберите её обратно, посмотрите, работает ли она после этого.*

```
// a.c
extern int a;
int main() {
    return a;
}

// b.c: archived to libb.a
extern int b;
```

```
int a;
void assign() { a = b; }

// d.c: archived to libd.a
int b;
```

Листинг 1.37: Файлы a.c, b.c и d.c для примера
с порядком линковки

Поскольку большинство линкеров работают в один проход, они могут создавать проблемы с порядком включения. Скажем, для кода с листинга 1.37 из приведённых ниже четырёх строчек компиляции только одна будет верной:

```
$ gcc -L. -lb -ld a.c
$ gcc -L. -ld -lb a.c
$ gcc a.c -L. -ld -lb
$ gcc a.c -L. -lb -ld
```

Листинг 1.38: Порядок линковки имеет значение

Задание 1.17. *Поставьте эксперимент и определите какая из этих строчек работает.*

Первые две строчки исключаются так как a.c зависит от библиотек, а библиотеки точно должны быть указаны в конце. Это является также причиной того, что системы сборки разделяют CFLAGS и LDFLAGS — последние идут строго в конец строчки вызова драйвера. С третьей и четвёртой чуть сложнее, но в целом если логика понятна, то всё должно быть просто.

После линковки со стандартной библиотекой (то есть после того, как драйвер компилятора полностью отработал), тот же пример «Hello, world» с листинга 1.1, дизассемблируется с уже подставленными дисплейсменами.

```
00000000004016b0 <main>:
4016b0: 50                push   rax
4016b1: bf 04 80 49 00    mov    edi,0x498004
4016b6: e8 e5 a9 00 00    call   40c0a0 <_IO_puts>
4016bb: 31 c0             xor    eax,eax
4016bd: 59                pop    rcx
4016be: c3                ret
```

Листинг 1.39: Дизассемблированный исполняемый файл
для «Hello, world»

Это достигается тем, что во время работы линкер систематически для каждой входной секции применяет все релокации из соответствующей секции релокаций. Он ищет любые символы, на которые ссылается код, вычисляет результат в соответствии с типом релокации и пишет результат в выходную секцию. Если у релокации, как на листинге 1.32, тип `R_X86_64_32`, то вычисление для неё идёт по формуле $S + A$, где S — это значение релоцируемого символа (symbol value), и A — это добавляемое значение (addend).

Задание 1.18. Вторая релокация в листинге 1.32 имеет тип `R_X86_64_PLT32`. Найдите в документации, что это за тип и как считается итоговое значение этой релокации.

Время работы линкера также является тем моментом в программе, когда вся программа полностью известна и возможны максимально интересные межмодульные оптимизации (в их числе межмодульная инлайн-подстановка или межмодульное продвижение констант).

1.4.1 Оптимизации времени линковки

Существует очень простой, очень распространённый и очень болезненный, особенно для программ на языке C, случай, когда довольно короткая функция оказывается в теле одной единицы трансляции, а используется в другой.

```
int number() { return 42; }
```

Листинг 1.40: Файл `part.c` возвращает число

```
// compile: gcc -O2 part.c main.c
int number();
int main() { return number(); }
```

Листинг 1.41: Файл `main.c` использует это число

Если скомпилировать и слинковать эти два файла вместе в одной строке компиляции, то вызов функции `number` не будет подставлен и константа 42 не будет продвинута в функцию `main`.

Основная проблема тут это раздельная компиляция — на момент, когда линкер увидел тело функции из другого модуля, компилятор уже отработал над ним.

Спасительной идеей является идея оптимизаций времени линковки (link time optimizations, LTO). Основная работа начинается ещё на этапе ассемблирования: вместо того, чтобы делать

честные объектные файлы, ассемблер аннотирует их исходным кодом и далее использует плагин линкера, который на этапе линковки снова вызывает компилятор на эти псевдо-объектники и во время линковки ещё раз оптимизирует их.

```
$ gcc -O2 -flto part.c main.c
```

Листинг 1.42: Простой пример LTO компиляции

Неприятная особенность стандартного LTO заключается в существенном увеличении общего времени компиляции. Мало того, если отдельные файлы можно обрабатывать в параллель, то скомбинировав их в единый псевдообъектник, мы далее должны обрабатывать его весь и в одном потоке. Это может неприятно сериализовать компиляцию.

Попыткой выхода является подход параллельных оптимизаций (thin LTO) в инфраструктуре LLVM. При этом подходе компилятор до сброса псевдообъектного кода делает минимум оптимизаций, но при этом по графу вызовов устанавливает, какие функции выгоднее компилировать совместно с какими по всей программе. Далее уже на этапе линковки модули объединяются по наиболее связным функциям и межмодульные оптимизации над такими объединёнными компонентами связности также проходят в параллель.

Задание 1.19. Возьмите достаточно большую программу из своей кодовой базы и попробуйте на ней стандартный и параллельный подходы. Замерьте время компиляции и эффект, оказываемый на производительность.

1.4.2 Релаксации

После LTO объектные файлы окончательно становятся чисто объектными и далее линкер (сейчас я говорю исключительно про статический линкер) работает исключительно с собранными ассемблером секциями релокаций, обрабатывая входные и формируя выходные секции кода, данных и так далее. Проблема в том, что не только компилятор, но и ассемблер в своей работе вынуждены быть консервативными. По той же логике по какой на этапе линковки мы можем улучшить решения об инлайн подстановке, мы также можем улучшить решения о том, сколько места в кодировке занимает то или иное смещение. Это называется процессом релаксации релокаций (relaxation process) или попросту релаксациями.

Когда вы вызываете обычную функцию `foo` и делаете это в позиционно независимом коде (например, внутри разделяемой библиотеки и т.п.), вызов чаще всего будет выглядеть примерно как на листинге 1.43. Это вызов через таблицу адресов, которая заполняется динамически при загрузке библиотеки. Аналогично чтение глобальной переменной может выглядеть как чтение через промежуточный адрес в глобальной таблице.

```
call [rip + foo@GOTPCREL]
mov r8, dword ptr variable@GOTPCREL[rip]
mov rax, DWORD PTR tlsvar@TLSGD[rip]
```

Листинг 1.43: Вызов и чтение через GOTPCREL

Если линкер видит, что вызываемая функция или переменная, к которой вы обращаетесь, доступна статически, он может ослабить эти довольно дорогие обращения через лишнюю косвенность и заменить их (в чём, собственно, и состоит релаксация) на более дешёвые прямой вызов и прямое обращение. Точно также если линкер знает (а он обычно знает), где находится TLS (thread local storage, локальная память потока), то дорогие обращения к локальным для потока переменным через косвенность он постарается заменить на нечто простое, как на листинге 1.44.

```
call foo
mov r8, dword ptr variable[rip]
mov rax, DWORD PTR fs:tlsvar@tpoff
```

Листинг 1.44: После релаксации

Это даёт ощутимый выигрыш в производительности. Коварная особенность релаксаций заключается в том, что для некоторых архитектур для загрузки адреса требуется последовательность инструкций. Очень часто релокацией помещается только первая из них. Но если эти инструкции были как-то переупорядочены, то линкер уже не будет помнить их точного смыслового значения. Особенно это проявляется на архитектурах с сокращённым набором команд, таких как ARM и RISC-V. Там релокации несут ещё больше пользы, так как существенно сокращают количество инструкций, но при этом ещё более сложны и опасны. Интригующие подробности можно найти в [55].

Кроме этого, линкер может объединять идентичный код, мёрзить секции и проводить сборку мусора. Интересно, что код считается идентичным с точностью до релокаций: если в двух секциях мы видим те же байты кодировки, те же типы объектов, те же флаги секций, но при этом не одинаковые, а просто эквивалентные релокации, линкер может его свернуть.

Кроме статической линковки, есть также динамическая линковка, когда линкер работает и разрешает релокации, программа уже выполняется. Там открываются бездны, которых не хотелось бы касаться здесь, чтобы не раздувать и без того обширное изложение.

Хотелось бы отослать интересующихся к замечательной книге [39], если тема линковки заинтриговала и хочется узнать больше.

Нам же с вами, уважаемый читатель, пора завершать высокоуровневый обзор (надеюсь, это было интересно и поучительно) и переходить к рассмотрению главного вопроса — как же сделать порождаемый код в некотором смысле лучше.

То есть вернуться в компиляторы.

1.5 ЗАДАЧИ НА САМОСТОЯТЕЛЬНУЮ ПРОРАБОТКУ

*The enjoyment of one's tools
is an essential ingredient of successful work.*

– Donald Knuth

Задание 1.20. Исследуйте возможности препроцессора по сбору зависимостей для систем сборки. Если бы препроцессирование, включая замену выражений и включение кода, было сделано не частью лексического, а частью синтаксического анализа, насколько эта задача усложнилась бы?

Задание 1.21. Разберитесь с утилитами `flex` и `bison`. Используйте их, чтобы написать разбор простых арифметических выражений с возможными присваиваниями переменным как на листинге 1.45.

```
a = (5 + 3) / (7 - 2);  
b = (10 - a * 9 - 7 + 3 * a * (a - 17)) / 5;  
print b;
```

Листинг 1.45: Пример программы для лексического
и синтаксического анализа

Задание 1.22. Драйвера многих компиляторов строят AST уже после подстановки константно-вычисленных выражений и инстанцирования шаблонов. Оцените, насколько было бы сложно поднять такое представление обратно в код, чтобы оценить, как он выглядит после всех инстанцирований времени компиляции. Попробуйте написать такую утилиту.

Задание 1.23. Развивая задачу 1.21, постройте кодогенератор, который порождает бы из вашего синтаксического анализатора LLVM IR. Вам придётся подключить LLVM как библиотеку и разобраться в её интерфейсе. Можете взять не LLVM, а другую компиляторную инфраструктуру.

Задание 1.24. Добавьте в программы вашего языка программирования оператор считывания ввода со `stdin`, а также циклы и условные переходы. Добейтесь работы чисел Фибоначчи.

Задание 1.25. Сравните качество кодогенерации вашего простого фронтенда с качеством кодогенерации для той же программы, написанной на языке C.

Задание 1.26. Чтобы реализовать считывание и печать в вашем языке, вам нужно построить для него нечто вроде стандартной библиотеки с двумя этими функциями. Как вы обеспечите её совместимость с объектным файлом, полученным в результате кодогенерации?

Задание 1.27. Познакомьтесь с несколькими системами команд разных архитектур и ассемблерами для них на ваш выбор. Посмотрите одну и ту же программу на C, скомпилированную одним и тем же тулчейном, но в разных ассемблерах.

Задание 1.28. Представьте, что у вас есть две статические библиотеки, авторы которых придерживались разных конвенций о вызове. Как вы решите задачу использования обоих в своей программе.

Задание 1.29. Перечислите все секции, которые существуют в объектных, но не в бинарных файлах при работе вашего тулчейна. Выясните их предназначение.

Задание 1.30. В секциях с данными и даже с отладочной информацией в принципе тоже могут быть возможны релокации. Поэкспериментируйте сможете ли вы написать такой код, чтобы их вызвать. Если нет, то почему?

Задание 1.31. Большинство линкеров в скрипте линковки позволяют задавать дополнительные ограничения на размер и выравнивание секций. Прочитайте документацию про скрипты линковки. Подумайте, какой алгоритм размещения секций позволил бы учесть их изменения размера при релаксациях?

Глава 2

Введение в ОПТИМИЗАЦИИ

*Compiler design is full of beautiful examples
where complicated real-world problems are solved
by abstracting the essence of the problem mathematically.*
– Alfred Aho

Прошлая глава содержала общий обзор тулчейна. Далее я буду всё внимание уделять довольно узкому месту на этой общей картине, а конкретно машинно-независимым оптимизациям, описанным в разделе 1.2.4. В этом же разделе описана структура HIR, над которым, собственно, и будет происходить вся работа, все оптимизации и всё остальное.

Но что такое вообще оптимизация? И внезапно это приятно звучащий, но вводящий в заблуждение термин. Как уже было сказано, компилятор занимается трансляцией программы, её поэтапным переводом с высокоуровневого языка на язык ассемблера целевой архитектуры. В трансляции есть место, когда программа представлена как промежуточное представление. И вот в этом промежуточном представлении программу удобно трансформировать. Строго говоря, компилятор занимается не оптимизациями, а настраиваемыми трансформациями, нацеленными, разумеется, на улучшение тех или иных характеристик кода, но не обязательно к ним приводящим.

Однако использование слова «оптимизация» оправдано, во-первых, историческим контекстом и, во-вторых, тем, что современные оптимизирующие компиляторы действительно существенно улучшают код. Увы, по-настоящему оптимальные

решения многих задач трансформации, например минимизация регистров при распределении обычно очень сложны (иногда неполиномиально сложны), и поэтому компиляторы обычно оптимизируют по очень комплексному критерию, отчасти включающему асимптотическую сложность самого алгоритма как часть критерия. Грубо говоря, никакой алгоритм с чем более квадратичной сложностью в современный компилятор, вероятно, не примут. Он останется теоретической наработкой, а в практику пойдёт некое менее точное, но более эффективное приближение.

На первых этапах я не буду себя этим сковывать, но буду это постоянно иметь в виду.

В этом разделе и далее вам понадобятся базовые сведения из теории графов. Я надеюсь, они у вас есть. Если их нет, я рекомендую прекрасную и исключительно доступную книгу [46], где вводятся все основные понятия теории графов и обучение строится посредством вычислительных экспериментов, помогающих сформировать должную интуицию (или любую другую книгу, тысячи их).

Итак, давайте окунемся в мир оптимизирующих трансформаций. Когда речь идёт о трансформации программы, по-настоящему совместной или раздельной трансформации подвергаются две фундаментальные сущности: поток управления программы и её поток данных. И я начну с введения понятия потока управления.

2.1 Поток управления

Поток управления состоит из базовых блоков и связей между ними. Мы уже неформально познакомились с концепцией базового блока в разделе 1.2.3. Настало время ввести это определение формально.

Определение 2.1. *Базовым блоком называется последовательность инструкций HIR, в которую управление может войти только в начале последовательности и выйти только в конце, не считая вызова функций. Эта последовательность завершается одной или несколькими инструкциями перехода на другие базовые блоки.*

В русской традиции базовый блок также называется линейным участком, что хорошо его характеризует.

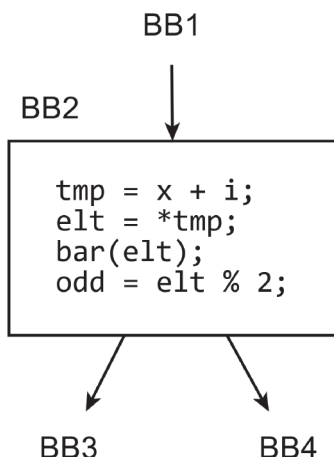


Рис. 2.1: Пример базового блока (см. листинг 2.1)

Инструкции перехода на другие базовые блоки имеют собственное красивое название.

Определение 2.2. *Терминатором называется инструкция, которая может находиться исключительно в конце базового блока и завершает его, передавая управление.*

В блоке на листинге 2.1 инструкция терминатора двойная: для перехода на В3 и В4. В принципе их могло бы быть и две, тогда `else` можно не писать.

```

BB2: tmp = x + i;
      elt = *tmp;
      bar(elt);
      odd = elt % 2;
      if (odd != 0) goto BB4; else goto BB3;
  
```

Листинг 2.1: Пример базового блока

Обратите внимание: здесь и далее в HIR я использую для второго блока метку вида `BB2`, в то время как в тексте и формулах я буду ссылаться на этот же блок как на V_2 . Надеюсь, это никого не запутает.

Из определения 2.1 понятно, что у базового блока всегда ровно один вход и сколько угодно выходов (по количеству инструкций терминаторов в конце блока). Обратим внимание: только обычный вызов функции не прерывает базовый блок. Например, на листинге 2.1 вызов функции `bar` терминатором не является. Он им являлся бы, если бы передавал управление на другой базовый блок в этом же графе по-

тока управления. Но обычная функция (хоть и путешествует по своим базовым блокам) всегда возвращается туда же, откуда была вызвана, то есть с точки зрения текущей функции не разрывает поток управления.

```
BB2: tmp = x + i;  
    elt = *tmp;  
    bar(elt) and goto BB6 on exception;  
    goto BB3;  
BB3: odd = elt % 2;  
    if (odd != 0) goto BB5; else goto BB4;
```

Листинг 2.2: Пример функции, бросающей исключения

Вызов функции, которая может бросить исключение, как на листинге 2.2, является терминатором, поскольку может передать исполнение как на следующую инструкцию, так и на обработчик исключения. Вообще почти всё связанное с обработкой исключений будет терминатором.

Также я буду использовать специальный терминатор `unreachable`, обозначающий, что управление просто не должно до него дойти.

```
BB2: tmp = x + i;  
    elt = *tmp;  
    return elt;  
    goto BB3;  
BB3: unreachable;
```

Листинг 2.3: Пример функции, содержащей недостижимый код

Из листинга 2.3 также должно быть ясно, что возврат из функции — это терминатор.

Задание 2.1. Вам может показаться интересным вопрос: «что же будет, если управление всё-таки дойдёт до недостижимого блока?». Это абсолютно не детерминировано. Наверняка у вас есть компилятор с возможностью использовать такого рода билтин. Посмотрите, к чему это приводит в вашей реализации.

Все базовые блоки вместе со специальным начальным блоком `entry` и конечным блоком `exit` являются вершинами графа потока управления функции (control flow graph, CFG). Ориентированными рёбрами этого графа являются возможные переходы между базовыми блоками.

Рассмотрим функцию, которая ищет старший установленный бит в числе, приведённую на листинге 2.4. Она имеет много на-

званий — count trailing zeroes, например. Я буду говорить о ней, как о функции bit scan reverse, или BSR, подчёркивая, что поиск идёт в обратном направлении.

```
// returns number of bit or one past size if not found
int bit_scan_reverse(unsigned n) {
    int bitsize = sizeof(n) * CHAR_BIT;
    for (int i = 0; i < bitsize; ++i) {
        int idx = bitsize - i - 1;
        if ((n >> idx) & 1) == 1)
            return idx;
    }
    return bitsize;
}
```

Листинг 2.4: Пример: старший установленный бит

Задание 2.2. Для равномерно распределённых чисел оцените среднее время работы этой программы в количестве пройденных итераций цикла. Оцените среднее время, если развернуть поиск и вести его в прямом порядке (например, каждый раз сдвигать число вправо на всё возрастающие сдвиги).

HIR для BSR можно построить простым способом — просто переписать эту функцию, построив exit-блок для каждого return.

```
int bit_scan_reverse(unsigned n) {
BB1: int bitsize = 64;
    int i = 0;
    if (i >= bitsize) goto BB4; else goto BB2;
BB2: ip1 = i + 1;
    idx = bitsize - ip1;
    ns = n >> idx;
    na = ns & 1;
    if (ns == 1) goto BB5; else goto BB3;
BB3: i = i + 1;
    if (i < bitsize) goto BB2; else goto BB4;
BB4: return bitsize;
BB5: return idx;
}
```

Листинг 2.5: HIR для BSR, первый вариант

Для HIR, показанного на листинге 2.5, мы получаем не очень приятный CFG с двумя выходными вершинами. Вместо этого при построении HIR можно попытаться создать CFG, у которого будет один вход и один выход как на листинге 2.6.

```

int bit_scan_reverse(unsigned n) {
BB1: int bitsize = 64;
    int retval = bitsize;
    int i = 0;
    if (i >= bitsize) goto BB5; else goto BB2;
BB2: ip1 = i + 1;
    idx = bitsize - ip1;
    ns = n >> idx;
    na = ns & 1;
    if (ns == 1) goto BB4; else goto BB3;
BB3: i = i + 1;
    if (i < bitsize) goto BB2; else goto BB5;
BB4: retval = idx;
    goto BB5;
BB5: return retval;
}

```

Листинг 2.6: HIR для BSR, второй вариант

Оба графа потока управления приведены на рис. 2.2. Слева приведён CFG, соответствующий листингу 2.5, справа граф для листинга 2.6. И тот и другой сохраняют семантику программы, которая делает буквально то же самое в обоих случаях.

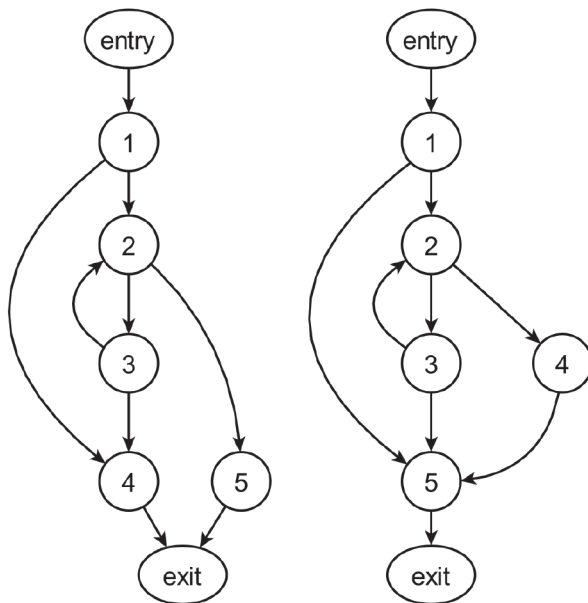


Рис. 2.2: Два возможных CFG для BSR

Задание 2.3. Нарисуйте граф потока управления для листинга 1.15.

Разговор о графах потока управления здесь не заканчивается. Мы ещё вернёмся к этим графам, когда будем вводить понятие цикла (например, см. раздел 5.1) и далее рассматривать цикловые оптимизации. Но пока что нам хватит информации для разговора о простых оптимизациях.

И конечно, какая оптимизация может быть проще, чем простое распространение констант?

2.2 Решётки и распространение констант

*Lattice theory is the whipping boy of mathematicians
in need of concealing their feelings of insecurity.*
– Gian-Carlo Rota

Давайте рассмотрим пример маленькой программы, приведённой на листинге 2.7. Она считает значения нескольких переменных, и пусть мы видим не всё её тело, будем считать, что делает она это не зря.

```
x = 10;  
z = read();  
y = 2 * x;
```

Листинг 2.7: Программа для распространения констант

Довольно очевидно, что если x — локальная переменная, то при любом исполнении этой программы y может принять только значение 20. Однако уже тут мы вынуждены накладывать ограничение на локальность. А что будет, если в программе есть ветвления или циклы и в общем случае при нетривиальном потоке управления?

Чтобы подойти к этой задаче, необходимо ввести или освежить в памяти такую математическую конструкцию, как **решётки** (lattices), а перед этим коротко разобраться с **бинарными отношениями**.

2.2.1 Бинарные отношения

В математике бинарное отношение (binary relation) ассоциирует элемент своей области определения с элементом своей области значений и является просто каким-то множеством пар

этих элементов. Мы говорим, что x соотносится с y как R (пишется xRy), если пара (x, y) принадлежит бинарному отношению R . Здесь и далее я буду под просто бинарным отношением иметь в виду гомогенное бинарное отношение, то есть такое, у которого эти области совпадают.

Итак, мы задаёмся областью определения X , скажем всеми натуральными числами. Тогда если стоит задача в качестве R задать отношение «меньше», то R будет множеством всех пар $(1,2), (1,3), (1,4), (2,3), (1,5), (2,4)$ и так далее.

Можно заметить, что гомогенные бинарные отношения имеют много общего с направленными графами. Так, например, мы можем нарисовать граф, у которого из вершины 1 будут идти стрелки во все вершины такие, что $1 < N$. Пример таких графов для некоторых отношений приведён на рисунке 2.3.

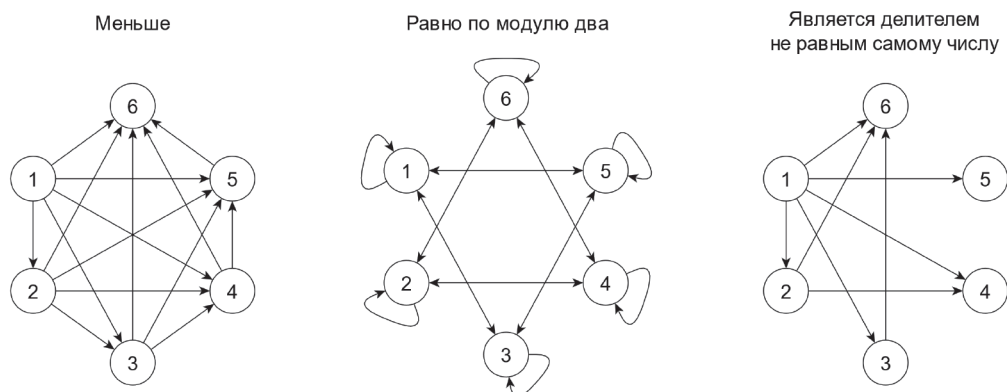


Рис. 2.3: Примеры графов для разных бинарных отношений

Можно также рассматривать бинарное отношение как предикат. Например, если R — это «меньше», то $1R2$ — это истина, а $2R1$ — это ложь, и в общем случае aRb (в смысле предиката) выполняется, если и только если aRb (в смысле введённого выше определения).

Я хочу обратить внимание читателя на важное отличие между этими двумя сторонами бинарного отношения: если мы рассматриваем его как множество, то упорядоченная пара ему принадлежит или не принадлежит; если мы рассматриваем его как предикат, то для пары своих аргументов он возвращает истину или возвращает ложь. Это кажется одним и тем же, но иногда глядя на отношение как множество, мы можем выделить его подмножества. А глядя как на предикат, мы можем установить его свойства.

Определение 2.3. Бинарное отношение R называется:

- **Рефлексивным**, если для любого a выполняется aRa .
- **Симметричным**, если для любых различных a и b , для которых выполняется aRb , также выполняется bRa .
- **Транзитивным**, если для любых различных a , b и c , для которых выполняются aRb и bRc , также выполняется aRs .

У всех терминов из определения 2.3 есть обратные. Например, отношение является антисимметричным, если для любых различных a и b , для которых выполняется aRb , выполняется отрицание для bRa (то есть bRa ложно, если aRb истинно).

Мне кажется интересным тот факт, что использованное мной в определении свойств отношений словосочетание «быть различным» — это тоже бинарное отношение.

Задание 2.4. Охарактеризуйте отношение «быть различным». Нарисуйте его граф и найдите у него хотя бы одно интересное свойство.

Точно так же я буду говорить о графах для рефлексивных, симметричных и транзитивных отношений. Хочется предупредить: в математике есть термин «транзитивный граф» и он используется совсем для другого. Я буду сокращать «граф транзитивного отношения» до немного некорректного «транзитивный граф» просто потому, что настоящих вершинно-транзитивных и рёберно-транзитивных в математическом смысле графов в этой книге встречаться не будет.

Визуально на рисунке 2.3 рефлексивным является граф для отношения «равно по модулю два», также он является и симметричным, а вот транзитивными являются и он, и граф для отношения «меньше». Транзитивность в ориентированных графах определяется так, что, если есть стрелка из a в b и стрелка из b в c , то есть стрелка из a в c .

Интересной операцией над графами, которая в будущем очень понадобится, является транзитивное замыкание: когда просто добавляются все такие стрелки, чтобы граф стал транзитивным.

Задание 2.5. Придумайте осмысленное бинарное отношение с не транзитивным графом. В какое отношение оно превращается после транзитивного замыкания?

2.2.2 Решётки

Представим, что некое отношение является рефлексивным, $a \leq b$ антисимметричным и транзитивным, например отношение таково для любых натуральных a и b . Такие отношения называются отношениями частичного порядка. Множество, на котором определено отношение частичного порядка, называется частично упорядоченным множеством (partial ordered set, poset).

Моё любимое частично упорядоченное множество это D_{12} : множество делителей числа 12 с бинарным отношением « a делит b ». Здесь между 2 и 3 порядка нет, но между 2 и 12 или 3 и 12 порядок есть — они делят 12, то есть предшествуют числу 12 в частичном порядке.

Ещё одним примером частично упорядоченного множества являются булевы строки с отношением порядка « a поразрядно меньше b ». Так, например строка 000 предшествует строке 001, а вот между строками 100 и 010 никакого порядка нет. Отношение «поразрядно меньше» можно трактовать как включение множеств, если 1 означает, что элемент на этой позиции включён в множество: очевидно, множество 110 включает 010 и 100 как подмножества (и оба поразрядно меньше его как булевы строки).

Можно рисовать графы частично упорядоченных множеств, как графы бинарных отношений вида « a предшествует b в частичном порядке отношения R ». Которые да, тоже являются бинарными отношением и внезапно тоже являются отношением частичного порядка, если таковым является исходное отношение R .

Три таких графа приведены на рисунке 2.4.

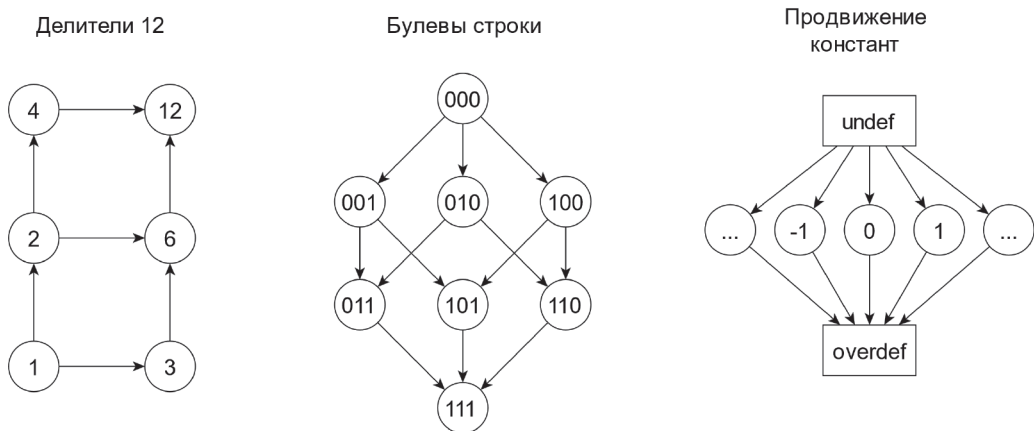


Рис. 2.4: Примеры графов для разных частично упорядоченных множеств

Определение 2.4. Частично упорядоченное множество называется ограниченной решёткой (lattice), если на нём можно ввести операции пересечения (meet) $a \wedge b$ и объединения (join) $a \vee b$, таким образом, что:

1. Их результаты тоже принадлежат тому же частично упорядоченному множеству.
2. $a \wedge (a \vee b) = a$
3. $a \vee (a \wedge b) = a$
4. При этом должны найтись наибольший и наименьший элементы $\top$ и $\perp$ такие, что $a \wedge \top = a$ и $a \vee \perp = a$.

Интуитивно $a \vee b$ — это ближайший элемент, которому предшествуют и a и b (least upper bound), а $a \wedge b$ — это ближайший элемент предшествующий a и b (greatest lower bound). В обоих случаях возможно совпадение результата операции с одним из аргументов. Также интуитивно $\top$ — это элемент, которому все предшествуют, а $\perp$ — это элемент, который предшествует всем.

Визуально очень легко запомнить: стрелочка в $\wedge$ (meet) глядит вверх, значит, это **нижняя** граница. Это контринтуитивно, но можно понять, почему это так, если подумать о булевых строках. Ну и, как обычно в математике, meet соотносится с умножением, а join — со сложением.

Здесь и далее я буду называть ограниченные решётки просто решётками.

Все частично упорядоченные множества на рисунке 2.4 являются решётками. Для решётки делителей двенадцати, meet — это gcd (наибольший общий делитель), а join — это lcm (наименьшее общее кратное). Для решётки булевых строк meet — это конъюнкция (логическое «и»), а join — дизъюнкция обозначенных строк.

Задание 2.6. Приведите пример конечного частично упорядоченного множества, не являющегося решёткой.

Самая интересная из решёток на рисунке 2.4 — это решётка продвижения констант. В ней $\top$ — это overdefined элемент, а $\perp$ — это undefined элемент. Операция join здесь — это операция схождения управления при продвижении констант, как показано на рисунке 2.5.

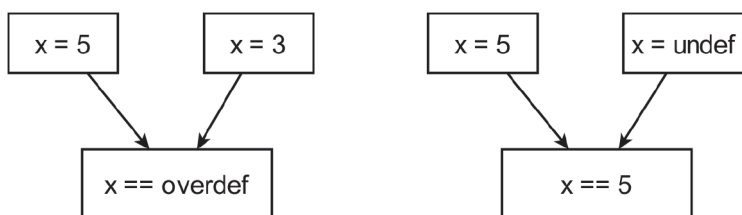


Рис. 2.5: Примеры схождения управления при продвижении константы

Опять-таки интуитивно, если в блоках предшественниках определены разные константы, то константа в точке схождения управления переопределена — она может прийти по любой дуге и, значит, мы уже не можем делать выводов о значении переменной. Но если определены одинаковые константы, или по одному из путей она вообще не определена, то всё отлично, и мы точно знаем, что за значение будет в точке схождения.

2.2.3 Продвижение констант

Если у меня есть вектор расширенных целочисленных значений, интересующих меня переменных, то в каждом базовом блоке их значения могут быть изменены. Некоторые меняются арифметически, некоторые переходят в состояние определённых или переопределённых.

Определение 2.5. *Передаточной функцией базового блока называется функция, определяющая изолированное действие этого базового блока на вектор переменных.*

Каждый базовый блок выполняет свою передачную функцию. По аналогии можно ввести передачные функции конкретных инструкций NIR или передачные функции подграфов потока управления.

```

BB1: i = 0;
      x = 1;
      y = 1;
      goto BB2;
BB2: x = x * i;
      i = i + 1;
      y = y * y;
      if (i < 10) goto BB2; else goto BB3;
BB3: p = x + y;

```

Листинг 2.8: Пример для продвижения констант

Например, на листинге 2.8 приведён пример программы, содержащей четыре переменных. Если мы зададимся вектором $[i, x, y, p]$, то его значение в точке entry это всегда $[\perp, \perp, \perp, \perp]$. Выпишем передаточные функции базовых блоков.

$$FOUT_{B_1}([i, x, y, p]) = [0, 1, 1, p], \quad (2.1)$$

$$FOUT_{B_2}([i, x, y, p]) = [i + 1, x \cdot i, y \cdot y, p],$$

$$FOUT_{B_3}([i, x, y, p]) = [i, x, y, x + y].$$

Кроме того, вектора на входе каждого блока комбинируются из входящих стрелок графа потока управления по простому правилу: мы вычисляем их комбинацию по решётке с помощью операции join.

$$FIN_{B_1}(C_{ENTRY}) = C_{ENTRY}, [0, 1, 1, p], \quad (2.2)$$

$$FIN_{B_2}(C_{B_1}, C_{B_2}) = C_{B_1} \vee C_{B_2}, \quad (2.3)$$

$$FIN_{B_3}(C_{B_2}) = C_{B_2}. \quad (2.4)$$

Тут же становится очевидным итеративный алгоритм: начиная с какого-то вектора состояния программы, можно исполнять передаточные функции блоков и схождения управления и раз за разом уточнять множества FIN и $FOUT$. Вопрос в том, сойдёмся ли мы при этом к некоему стабильному состоянию, когда следующие уточнения не будут в нём ничего менять?

Определение 2.6. *Неподвижной точкой для функции F на решётке является такая точка x , что $F(x) = x$.*

Рассмотрим простейший алгоритм продвижения констант, приведённый на листинге 2.9. Он совершенно не идеален, в дальнейшем он будет улучшен и переработан алгоритмически (см. раздел 4.2). Также в разделе 2.3.4 он критикуется с точки зрения принципа единственной ответственности, так как в нём смешаны анализ и трансформация. В этом алгоритме предполагается, что мы знаем передаточные функции всех базовых блоков, и `TransitionFunc(BB, MIn)` возвращает выходной вектор `MOut`.

```

void propagate_copies(HIRFunction F) {
    auto M = Map<BB, LatticeVector>;
    auto N = countof(F.all_vars);
    for (auto BB : F.all_blocks)
        M[BB] = LatticeVector{N, bottom};
    auto Changed = true;
    while (Changed) {
        Changed = false;
        for (auto BB : F.all_blocks) {
            auto MIn = LatticeVector{N, bottom};
            for (auto Pred : preds(BB))
                MIn = join(MIn, M[Pred]);
            auto MOut = TransitionFunc(BB, MIn);
            if (M[BB] != MOut)
                tie(M[BB], Changed) = (MOut, true);
        }
    }
    for (auto BB : F.all_blocks)
        for (auto [V, S] : zip(F.all_vars, M[BB]))
            if (S != bottom and S != top)
                F.replace_in_block(BB, V, S);
}

```

Листинг 2.9: Алгоритм CP:
простое продвижение констант

Алгоритм работает над векторами значений в решётках (тип `LatticeVector`). Можете думать о них, как об обычных целых числах, дополненных специальными значениями $\top$ и $\perp$ в смысле определения 2.4. Он работает «до победы», то есть изменяет функцию, пока может её изменить, до схождения процесса к неподвижной точке. На каждом шагу он для каждого блока вычисляет схождение значений в начале блока, применяет передаточную функцию блока и далее смотрит, изменилось ли что-нибудь.

Задание 2.7. В разделе заданий вам будет предложено доказать сходимость этого алгоритма. Пока что сделайте более простой анализ: попробуйте найти худший случай для его работы, то есть такой HIR, для которого он будет работать максимальное число итераций среди всех HIR того же размера.

Пример работы алгоритма CP для листинга 2.8 табулирован на рисунке 2.6.

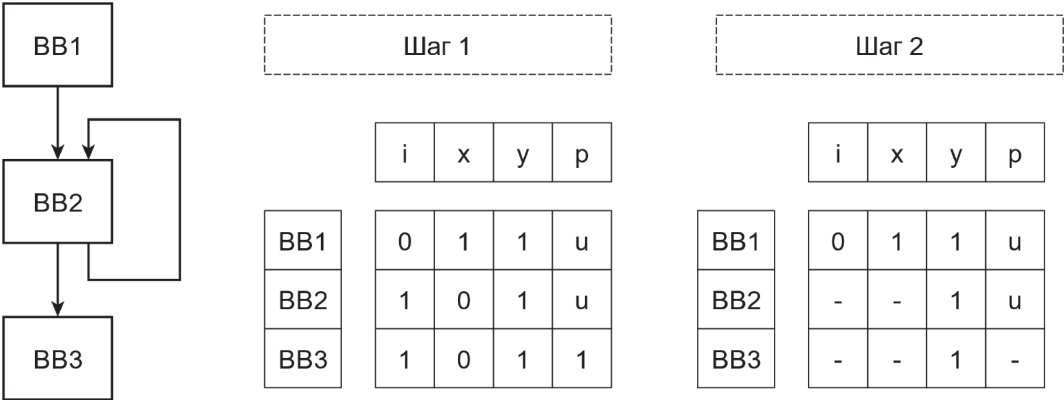


Рис. 2.6: Схождение к неподвижной точке

Схождение к неподвижной точке по алгоритму 2.9 происходит за два шага и далее ситуация не меняется при дальнейшем применении передаточных функций (2.1) и (2.2).

На первом шаге мы механически применяем передаточные функции всех базовых блоков. Первый базовый блок определяет переменные x , y и i , и мы записываем их в первую строку. Второй базовый блок их переопределяет. И, наконец, третий блок доопределяет p . Но на втором шаге нам уже надо иметь дело со схождениями управления. Во втором блоке сходящееся управление для i и x создаёт конфликты с их значениями в первом базовом блоке и делает их *overdefined* (обозначено знаком минус).

В итоге видно, что единственная константа, не принявшая значения *overdefined* в последнем блоке — это значение y (единица). Таким образом, можно заменить $p = x + y$ на $p = x + 1$, упростив (то есть оптимизировав) программу без потери семантики.

Эта простейшая оптимизация называется продвижением констант. У неё в таком виде есть проблемы, и тут есть, что улучшить. По сути, алгоритм СР — это разновидность алгоритма, предложенного Гэри Килдалом в 1972-м году (см. [32]). На самом деле Килдал сделал в этой статье гораздо больше: он создал анализ потока данных.

2.3 Поток данных

*It's fine to work on any problem,
so long as it generates interesting mathematics along the way
even if you don't solve it at the end of the day.*

– Andrew Wiles

Метод Килдала из раздела 2.2.3 был использован нами пока что только для продвижения констант. Но чем в принципе продвижение констант отличается от любого другого анализа данных, с которыми работает программа? Если его обобщить, не уточняя, что именно мы анализируем, этот метод сводится к простой идее.

$$\begin{aligned} out_B &= f_B(in_B), \\ in_B &= \bigvee_{p \in preds(B)} out_p. \end{aligned} \tag{2.5}$$

Для каждого блока B следует вычислить его передаточную функцию от входа к выходу. Там, где управление сходится, следует вычислить некую функцию схождения. Например, в случае продвижения констант — это будет `join` на решётке по всем входящим значениям. Здесь есть известная омонимия: функция схождения (*meet function*) в разных задачах анализа данных может быть как *lattice meet*, так и *lattice join*. Ниже мы увидим оба таких примера. Я буду далее использовать *meet* и *join* как названия операций на решётке, а функцию схождения называть по-русски.

Далее можно задаться некоторыми начальными значениями (опять-таки для разного типа анализов это может быть пустое множество или, наоборот, все элементы в каком-то значении) и запустить алгоритм, который, пока что-то изменяется, применяет передаточные функции и функции схождения.

Доказано (см. [32]), что для ограниченной решётки с монотонными передаточными функциями итеративный алгоритм, вычисляющий неподвижную точку, сходится к некоторому решению, увы, не всегда идеальному.

Ульман и Кам (см. [28]) попытались обобщить этот метод, чтобы обеспечить сходимость не к какому-то решению, а к так называемому МОР (*meet over all paths*) решению. Само название МОР не очень точное. В современных терминах точнее говорить про *join over all paths*, но термин прижился, и мне не хотелось бы его менять.

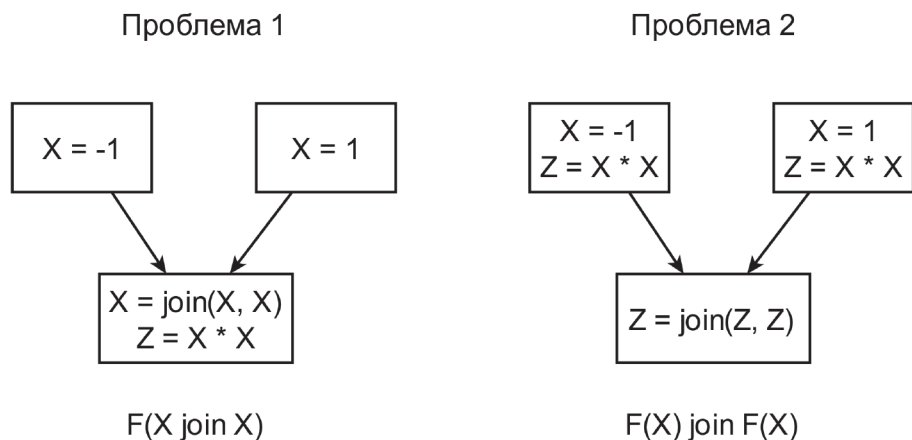


Рис. 2.7: Проблема метода Килдалла: сходимость к какой-то точке

Пример проблемы 1 на рисунке 2.7 показывает решение, для которого алгоритм СР найдёт решение $Z = \perp$, тогда как МОР-решение учитывает всю информацию по всем путям и даёт ответ $Z = 1$.

После тщательного анализа оказалось, что МОР-решение возможно только для дистрибутивных проблем, то есть только для таких, в которых $f(a \vee b) = f(a) \vee f(b)$. Если бы продвижение констант было дистрибутивной проблемой, то проблема 1 не отличалась бы от проблемы 2.

Как показали Ульман и Кам, в недистрибутивных задачах анализа потока данных, даже при условии монотонности, задача поиска МОР-решения является алгоритмически невычислимой (то есть эквивалентной так называемой «проблеме остановки» для машины Тьюринга). Этот неожиданно сильный и красивый результат я хотел бы оставить читателю на самостоятельное изучение.

Вместо дальнейшего погружения в теорию, я предлагаю посмотреть, как именно анализ данных распространяется на некоторые простые задачи. Обычно для таких задач в литературе (напр. [43], [37] и т.д.) выбирают три главных примера: анализ достигающих определений, доступных выражений и активных переменных. Делается это из простых соображений: достигающие определения и доступные выражения — это прямой анализ потока данных с разными функциями схождения (конъюнкцией и дизъюнкцией), в то время как активные переменные — это обратный анализ. Я, разумеется, не собираюсь отступать от традиции.

2.3.1 Достигающие определения

Уравнения анализа потока данных могут решать не только задачу продвижения констант. Как пример интересной задачи можно рассмотреть задачу нахождения множеств определений, потенциально достигающих базового блока в графе потока управления. Это называется достигающими определениями (reaching definitions). Термин звучит плохо, но он закрепился в литературе.

Достигающие определения важны для построения графов потока данных (data flow graphs). В современном мире эта методика мало используется напрямую, так как её вытеснило использование SSA-представления (которое будет рассмотрено в разделе 3.1), но она всё ещё интересна как подпрограмма многих важных машинно-зависимых оптимизаций.

Впервые метод вычисления потенциально достигающих определений и интервалов активности переменных был предложен в [3], но я в данном изложении несколько отойду от оригинала, чтобы показать единство методов анализа данных.

Сначала сформулируем, что мы, собственно, хотим: для каждого базового блока в программе мы хотели бы узнать все определения переменных, которые потенциально достигают данного блока. Здесь потенциальное достижение означает, что есть хотя бы один путь в графе потока управления от определения переменной до искомого базового блока, по которому эта переменная не переопределена.

```
// (x, undef), (y, undef)
BB0: x = 10;
     y = x + 2;
     goto BB1;
// (x, 0), (x, 1), (y, 0), (y, 1)
BB1: y = x + y;
     x = x - 1;
     if (x > 2) goto BB1; else goto BB2;
// (x, 1), (y, 1)
BB2: t = x + y
     return t;
```

Листинг 2.10: Пример достигающих определений

На листинге 2.10 аннотированы входящие достигающие определения для каждого базового блока $RIN(B)$. Для базового блока B_1 достигают точки входа, во-первых, определения x и y , сделанные

в B_0 , а во-вторых, определения x и y , сделанные в B_1 , в теле цикла. Точно также можно говорить о достигающих определениях на выходе из блока $ROUT(B)$.

Можно определить множество всех достигающих определений как множество пар (v, B) , где v — это имя переменной, а B — базовый блок, где она меняет значение. Например, для листинга 2.10 полное множество — это $S = (x, \perp), (x, B_0), (x, B_1), (y, \perp), (y, B_0), (y, B_1)$.

Если закодировать все подмножества битовым вектором из шести элементов, имеем:

$$\begin{aligned} RIN(B_1) &= 011011, \\ ROUT(B_1) &= 001001. \end{aligned} \quad (2.6)$$

Задание 2.8. Убедитесь, что $RIN(B_1)$ построено правильно. Для этого выпишите отдельно S и для каждого входящего элемента запишите 0, если он недостижим на входе B_1 , и 1, если достигим. Прodelайте то же с $ROUT(B_1)$.

С каждым базовым блоком B теперь можно связать две функции (или два множества), которые доступны к непосредственному вычислению один раз для каждого базового блока. Во-первых, это множество тех определений, которые этот блок вводит $GEN(B)$. Во-вторых, это те определения, которые он переопределяет $KILL(B)$.

Для случая, когда некоторая переменная внутри блока определяется несколько раз, её определение — это последнее внутри линейного участка. Например, для листинга 2.10, имеем $GEN(B_1)$ и $KILL(B_1)$ как в уравнении 2.7.

$$\begin{aligned} GEN(B_1) &= (x, B_1), (y, B_1), \\ KILL(B_1) &= (x, B_0), (y, B_0). \end{aligned} \quad (2.7)$$

При схождении управления на входе базового блока хочется оставить все переменные, достигающие данного блока, независимо от того, по каким путям они его достигают. Поэтому в качестве функции схождения здесь мы выбираем логическое или над битовыми векторами, где каждый бит означает, что соответствующее определение входит в число потенциально достигающих для рассматриваемого блока.

Комбинируя передаточные функции и функции схождения управления, можно записать простые уравнения для определения $RIN(B)$ и $ROUT(B)$.

$$\begin{aligned}
 RIN(B) &= \bigcup_{P \in \text{preds}(B)} ROUT(P), \\
 ROUT(B) &= GEN(B) \cup (RIN(B) - KILL(B)).
 \end{aligned}
 \tag{2.8}$$

Решение уравнений (2.8) также возможно напрямую, и они так же сходятся к неподвижной точке. Алгоритм получается на удивление похожим на алгоритм CP (см. листинг 2.9).

```

for (auto BB : all_blocks) {
    RIN[BB] = BitVector{N, 0};
    ROUT[BB] = BitVector{N, 0};
    Kill[BB] = killVector(BB);
    Gen[BB] = genVector(BB);
}
auto Changed = true;
while (Changed) {
    Changed = false;
    for (auto BB : all_blocks) {
        auto NR = BitVector{N, 0};
        for (auto Pred : preds(BB))
            NR = NR or ROUT[Pred];
        auto RO = GEN[BB] or (NR subtract KILL[BB])
        if (RIN[BB] != NR)
            tie(RIN[BB], Changed) = (NR, true);
        if (ROUT[BB] != RO)
            tie(ROUT[BB], Changed) = (RO, true);
    }
}

```

Листинг 2.11: Алгоритм RD: получение достигающих определений

Задание 2.9. Проанализируйте сходства и различия листинга 2.11 с листингом 2.9.

Теперь посмотрим пример с другой функцией схождения.

2.3.2 Доступные выражения

Назовём в данной точке программы (как обычно, на входе или на выходе базового блока) доступным выражением такое выражение, которое было предвычислено по всем путям, ведущим к этой точке, и в котором ни одна переменная не изменила своего значения.

```

// AVIN: {}
BB0: int y = x + 2;
      int x = a + b;
      x = x + y;
      goto BB1;
// AVIN: {a + b}
BB1: y = x + 2;
      int t = x - 1;
      if (y > t) goto BB3; else goto BB2;
// AVIN: {a + b, x + 2, x - 1}
BB2:
      y = x - 1;
      goto BB3;
// AVIN: {a + b, x + 2, x - 1}
BB3: t = x + 2;
      if (t > 0) goto BB1; else goto BB4;
// AVIN: {a + b, x + 2, x - 1}
BB4: t = a + b;
      return t;

```

Листинг 2.12: Пример доступных выражений

И снова проще начать с примера. На листинге 2.12 аннотированы входящие доступные выражения для каждого базового блока $AVIN(B)$. Например, выражение $a + b$ вычисленное в B_0 неизменно во всём цикле и, несомненно, доступно в начале B_1 . С другой стороны, выражение $x + 2$ разрушается в B_0 , и уже не доступно на входе B_1 .

Далее можно пройти все те же шаги, что и в *reaching definitions* (см. 2.3.1). Давайте сделаем это ещё раз, потому что по пути нас ждёт интересное отличие в определении функции схождения.

По определению нас интересуют все доступные выражения. На листинге 2.12 это выражения $x + 2$, $a + b$, $x + y$, $x - 1$. Из них можно составить битовый вектор из четырёх элементов. Будем говорить, что базовый блок порождает определяемые им выражения и убивает все те выражения, в которых задействованы переопределённые в блоке переменные.

Например, базовый блок B_1 порождает выражения $x + 2$ и $x - 1$. Также он убивает $x + y$, поскольку он переопределяет y . Полностью все множества, связанные с B_1 приведены в уравнении 2.9.

$$\begin{aligned}
 GEN(B_1) &= 1001, \\
 KILL(B_1) &= 0010, \\
 AVIN(B_1) &= 0100, \\
 AVOUT(B_1) &= 1101.
 \end{aligned}
 \tag{2.9}$$

Далее нужно определиться с функцией схождения. Если выражение доступно по одному пути, но не доступно по-другому, считать его доступным нельзя. Таким образом, естественная функция схождения здесь — это побитовая конъюнкция над битовыми векторами. То есть логическое «и», а не логическое «или» как в случае доступных определений.

$$\begin{aligned} AVIN(B) &= \bigcap_{P \in preds(B)} AVOUT(P), \\ AVOUT(B) &= GEN(B) \cup (AVIN(B) - KILL(B)). \end{aligned} \quad (2.10)$$

Итоговые уравнения 2.10 для анализа потока данных в случае доступных выражений очень похожи на уравнения 2.8 для доступных определений (с точностью до логики объединения результатов). Они снова по методу Килдалла сходятся к неподвижной точке.

Задание 2.10. *Выпишите алгоритм в стиле листинга 2.11, который работал бы для доступных выражений.*

Можем ли мы объединить или обобщить такого рода алгоритмы? Ответ, разумеется, утвердительный.

2.3.3 Активные переменные и обратный анализ

Итак, кажется, существует некий общий фреймворк, единый подход к проблемам анализа потока данных. Мы должны определиться с несколькими пунктами, после чего алгоритм анализа потока данных получается как бы сам собой.

- Множество интересующих нас сущностей. Например, множество доступных выражений или множество доступных определений.
- Множества GEN и KILL.
- Передаточные функции.
- Функция схождения управления. Это, как мы видели выше, может быть обычная конъюнкция или дизъюнкция над булевыми векторами, но в общем случае это функция meet или join в решётке (булева алгебра — это тоже решётка).
- Начальные условия.

В этом перечне не хватает одной главной вещи: описания того, куда направлен анализ потока данных — от начала к концу про-

граммы или от конца к началу. Чтобы это проиллюстрировать, рассмотрим задачу нахождения активных переменных.

Активной переменной (live variable) в точке p называется такая переменная, которая используется хотя бы на одном пути от p до выходного блока.

```
// (x, inact), (y, inact), (z, inact)
BB0: x = 10;
      y = x + 2;
      z = x * y;
      goto BB1;
// (x, act), (y, inact), (z, inact)
BB1: y = x - 5;
      x = x - 1;
      if (x > 2) goto BB1; else goto BB2;
// (x, act), (y, act), (z, inact)
BB2: t = x + y;
      return t;
// (x, inact), (y, inact), (z, inact)
```

Листинг 2.13: Пример активных переменных

В примере на листинге 2.13 вычисления y и z в B_0 излишни так как на выходе из этого блока они не активны: B_1 сразу перезаписывает y , а z вообще не используется. Таким образом анализ активных переменных раньше был полезен для удаления мертвого кода (в современных компиляторах, разумеется, всё удаление мёртвого кода основано на SSA и эта техника скорее историческая, как и большинство в этом разделе).

Множество $GEN(B)$ — это множество переменных, использованных в B до присвоения в том же блоке. Множество $KILL(B)$ — это множество переменных, которые в блоке получают значения. И поскольку нас волнует наличие использования хотя бы по одному пути, то функция схождения — это дизъюнкция.

Если расписать передаточные функции для задачи анализа активных переменных как в уравнении 2.11, то видно, что они идут в обратном направлении, то есть из конца программы в начало.

$$LVOUT(B) = \bigcup_{P \in succ(B)} LVIN(P), \quad (2.11)$$

$$LVIN(B) = GEN(B) \cup (LVOUT(B) - KILL(B)).$$

До некоторой степени, функция схождения управления тут становится функцией расхождения управления.

Задание 2.11. *Выпишите алгоритм в стиле листинга 2.11, который работал бы для активных переменных.*

2.3.4 Трансформации и анализ в NIR

В основном компиляторы заняты трансформациями кода. Но в некоторых случаях мы хотим пройти по коду не для того, чтобы его изменить, а для того, чтобы его проанализировать. Отсюда следуют два основных типа фаз компиляции — трансформации и анализы. Некоторая иерархия трансформаций в зависимости от их области применения (цикл, функция, модуль) была рассмотрена в разделе 1.2.4. Трансформация над NIR берёт на вход NIR и возвращает трансформированный NIR.

Примером трансформации является алгоритм CP с листинга 2.9. В нём производится итеративный анализ по решётке продвижения констант, после чего вычисленные константные значения переменных (если они не являются неопределёнными или переопределёнными) заменяют собой сами переменные во всех базовых блоках.

Но в этом же разделе, изучая анализ потока управления и анализ потока данных, мы также построили несколько чисто аналитических алгоритмов, например, анализ достигающих определений с листинга 2.11. Он, собственно, ничего не делает с кодом, а только строит для каждого базового блока список достигающих определений. В итоге, чтобы показать результаты его работы на листинге 2.10, были использованы аннотации посредством комментариев.

Считается хорошим тоном разделять аналитическую и трансформационную часть оптимизаций, вынося часто переиспользуемые анализы в отдельные аналитические проходы.

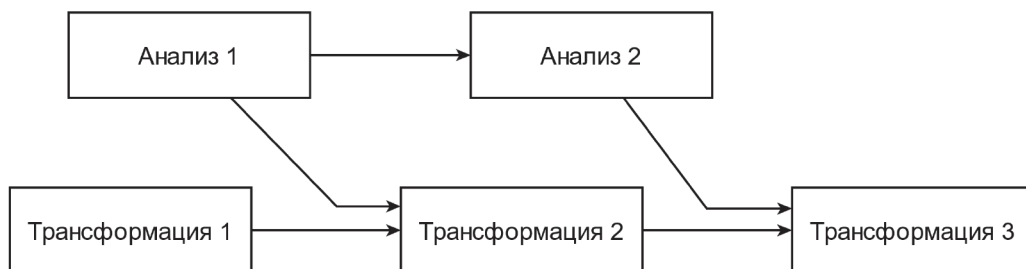


Рис. 2.8: Реалистичный оптимизационный конвейер

Оптимизации требуют анализа, чтобы знать, что и где изменять в HIR. Мы далее увидим ситуации, когда одни анализы требуют других анализов. Всё вместе это складывается в многоуровневый конвейер вроде изображённого на рисунке 2.8.

Иногда анализы нужно пересчитывать. Например, после выполнения продвижения констант может измениться набор доступных выражений. Поскольку аналитические пазы могут быть достаточно дорогими, в трансформационных пассах обычно можно указывать, какие анализы ими сохраняются неизменными. Например, то же самое продвижение констант по алгоритму СР оставляет неизменным поток управления (это может быть неверно для более продвинутых алгоритмов, см. раздел 4.2).

Есть две школы мысли относительно того, как должен выглядеть результат работы аналитического паса. Первая идея — это аннотация самого HIR метаданными. Это имеет свои преимущества: по сути, анализ превращается в трансформацию и все пазы унифицируются. Увы, неаккуратная работа с метаданными может привести к тому, что они останутся частью HIR, когда они там уже не нужны, а много аналитических пазов способны раздуть HIR метаданными в разы. Вторая идея — это хранить анализ в качестве отдельной структуры данных, которая может быть как изменяемой (если прямо в процессе трансформации анализ приходится постоянно кусочно пересчитывать), так и неизменяемой. В LLVM приняты оба подхода там, где они уместны. В этой книге из-за большей академичности изложения, я буду придерживаться второго подхода.

Таким образом, результат работы анализа *reaching definitions* над листингом 2.10 — это структура данных, приведённая на листинге 2.14.

```
BB0: RDIN = (x, undef), (y, undef)
BB1: RDIN = (x, 0), (x, 1), (y, 0), (y, 1)
BB2: RDIN = (x, 1), (y, 1)
```

Листинг 2.14: Результаты анализа *reaching definitions*

В итоге те трансформационные пазы, которые требуют этого анализа, берут на вход не только HIR, но и соответствующую аналитическую структуру, а компилятор можно рассматривать как набор трансформационно-аналитических фаз, каждая из которых берёт HIR плюс некоторые структуры данных и возвращает (возможно трансформированный) HIR плюс некоторые другие структуры данных.

Я, как вы может быть обратили внимание, использую слова «пасс», «проход» и «фаза» (в смысле фазы оптимизации) взаимозаменяемо и намерен придерживаться этого и впредь.

Задание 2.12. *Разделите продвижение констант на фазы анализа и трансформации. Как бы вы спроектировали структуру данных для хранения анализа?*

2.4 Нумерация значений

Одна из основных проблем в программах, написанных людьми, это избыточные вычисления. Человек, когда он пишет код, вообще не особо думает (и не должен особо думать) про оптимизацию промежуточных вычислений. Он думает о том, как максимально ясно и полно донести суть программы.

В итоге следующий код вполне нормален для каких-нибудь научных расчётов.

```
k = (a + b) * j;  
m = (a + b) * (j + 1);
```

Листинг 2.15: Пример кода для нумерации значений

Разумеется, вычисление $(a + b) * j$ здесь фактически происходит дважды. Но уже на самых ранних этапах развития программирования было понятно, что эту избыточность должен убирать не сам человек, а компилятор при планировании исполнения программы.

Также избыточность может появиться естественным образом в процессе раскрытия высокоуровневых конструкций.

```
int arr[10][10];  
arr[2][3] = 5; // *(arr + 2 * 10 + 3) = 5  
arr[2][6] = 7; // *(arr + 2 * 10 + 6) = 7
```

Листинг 2.16: Пример порождения избыточности компилятором

Избыточность также может проявляться в наличии мёртвого кода и в наличии ненужных копий, используемых в разных местах программы. В общем случае источник избыточности — это несоответствие использованных в программе **переменных** вычисляемым в процессе исполнения программы **значениям**.

В 1958-м году академик Ершов, работая над первыми советскими компиляторами, опубликовал основополагающую статью на тему

нумерации значений (value numbering, см. [20]) для удаления общих подвыражений. Будучи далее многократно переработана исследователями в этой области, эта статья до сих пор является наиболее цитируемой советской статьёй на тему компиляторных технологий.

Суть алгоритма нумерации значений очень проста, если использовать его внутри базового блока. Обсуждение глобальной нумерации значений, работающей по всему графу потока управления, мы отложим до введения SSA-представления и вернёмся к этому в разделе 4.3.

```
t = a + b;  
k = t * c;  
s = a + b;  
m = s * c;  
m = m + s;  
r = k + m;  
return r;
```

Листинг 2.17: Пример для локальной нумерации значений

Пример на листинге 2.17 воспроизводит первый пример этого раздела после перевода в трёхадресный HIR. Входными переменными являются a , b и c и их значения сразу получают номера 1, 2 и 3.

Далее следует построить таблицу из трёх столбцов: номер строки, фрагмент вычисления и присвоенное значение. Присвоенное значение вычисляется исходя из хеша фрагмента вычислений. Фрагмент вычислений, например $a + b$ записывается как $(+,1,2)$. Если операция коммутативна, то мы всегда будем упорядочивать номера значений по возрастанию, таким образом $b + a$ на этой строке тоже раскрылось бы в $(+,1,2)$. В итоге это значение получает номер 4 и далее $(*,3,4)$ получает номер 5. Далее опять встречается $(+,1,2)$, но его номер уже присвоен и его можно проигнорировать, выдав на этой строчке переменной s тоже номер 4. Далее снова возникает $(*,3,4)$, в итоге чего m получает также номер 5. Далее возникает $(+,4,5)$, и t принимает номер 6, завершаясь на $(+,5,6)$ для возврата с номером 7.

Результат приведён на листинге 2.18, имена переменных изменены на номера значений кроме имён параметров.

```
v4 = v1 + v2; // v1 = a, v2 = b  
v5 = v3 * v4; // v3 = c  
v6 = v4 + v5;  
v7 = v5 + v6;  
return v7;
```

Листинг 2.18: Пример после локальной нумерации значений

Можно заметить, что локальная нумерация значений трансформировала вычисления, построив из них направленный ациклический граф (directed acyclic graph, DAG) с минимальным необходимым количеством вершин, показанный на рисунке 2.9.

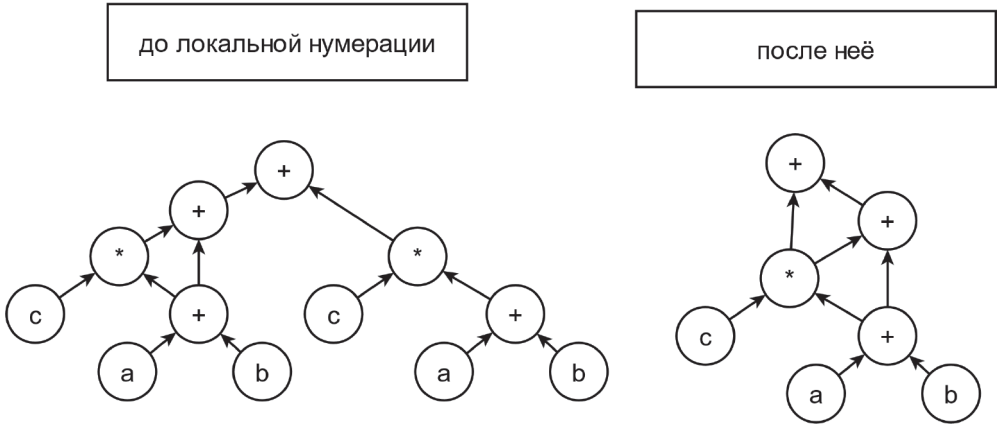


Рис. 2.9: DAG до и после локальной нумерации значений

Чтобы записать алгоритм, необходимо построить основную структуру данных Dag: отображение из фрагмента вычислений в значение. Вспомогательная структура данных VarTable описывает отображение переменной в значение и нужна для переименования переменных. Поскольку с этими структурами придётся работать одновременно, добавим вспомогательную функцию, обновляющую их синхронно, показанную на листинге 2.19.

```
int AddP(auto& Dag, auto& VarTable, auto P, int VN) {
    Dag.add(hash(P), VN);
    VarTable.add(P, VN);
    return VN + 1;
}
```

Листинг 2.19: Вспомогательная функция: добавляем P в Dag и в отображение

Сам алгоритм довольно прост и выглядит довольно естественно, он приведён на листинге 2.20. Здесь `params` — это список параметров функции. Каждый параметр является фрагментом вычислений и при этом имеет имя. Они являются листовыми узлами в 2.9.

```

void lvn(Function F) {
    auto Dag = HashMap<Int, Int>;
    auto VarTable = HashMap<Variable, Int>;
    auto VN = 1;
    for (auto P : F.params) {
        VN = AddP(Dag, VarTable, P, VN);
    }
    for (auto I : F.statements) {
        renameargs(I.rhs, VarTable);
        sortargs(I.rhs);
        HashVal = hash(I.rhs);
        if (Dag.exists(HashVal)) {
            VN = Dag.getNum(HashVal);
            VarTable.add(I.lhs, VN);
        }
        else
            VN = AddP(Dag, VarTable, I.lhs, VN);
    }
}

```

Листинг 2.20: Алгоритм LVN: локальная нумерация значений

Алгоритм LVN работает именно так, как и было описано в примере 2.17. Сначала в набор DAG-фрагментов вычислений запоминается для входных параметров функции. Они же добавляются в таблицу переменных. Далее по всем выражениям, аргументы переименовываются по таблице значений, сортируются, чтобы избежать разного хеширования коммутативных операций, и если такой фрагмент вычислений находится, то номер этого вычисления уже известен.

Задание 2.13. *Напишите самостоятельно вспомогательные функции `sortargs` и `renameargs`.*

Разумеется, в таком виде, как он написан, алгоритм LVN имеет существенную проблему: сейчас он может работать только если функция состоит из единственного базового блока. В разделе заданий на самостоятельную проработку вам будет предложено переделать этот алгоритм чтобы он работал для каждого базового блока, а некий внешний код вычислял что же для базового блока является входными параметрами.

Самое интересное в представлении после локальной нумерации — это тот факт, что ни одна переменная не определена дважды. И действительно, каждая переменная — это новый узел в DAG и требует лишь одного определения.

Это наблюдение послужило основой для возникновения SSA-представления, обладающего, по сути, тем же фундаментальным свойством. Вот к нему-то мы и перейдём в следующем разделе.

2.5 ЗАДАЧИ НА САМОСТОЯТЕЛЬНУЮ ПРОРАБОТКУ

Задание 2.14. Докажите, что если бинарное отношение R является отношением эквивалентности над множеством A , то любой элемент $a \in A$ принадлежит некоему классу эквивалентности и только ему.

Задание 2.15. Дистрибутивной решёткой является такая, в которой для любых x, y и z выполняется соотношение 2.12. Приведите разумный пример для дистрибутивной и для не дистрибутивной решётки.

$$x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z) \quad (2.12)$$

Задание 2.16. Нарисуйте решётку всех монотонных булевых функций, берущих на вход и возвращающих двухбитовый вектор.

Задание 2.17. Постройте HIR и граф потока управления для алгоритма сортировки вставками, приведённого на листинге 2.21.

```
for (int i = 0; i < len; ++i) {  
    int key = arr[i];  
    for (int j = i - 1; j >= 0; --j) {  
        if (arr[j] < key)  
            break;  
        arr[j + 1] = arr[j];  
    }  
    unsigned pos = j + 1;  
    arr[pos] = key;  
}
```

Листинг 2.21: Функция на C. Входом являются `arr` и `len`

Задание 2.18. Продолжая решение проблемы 2.17, для алгоритма сортировки вставками постройте множества доступных выражений.

Задание 2.19. Напишите программу на вашем любимом языке программирования, реализующую алгоритм CP (см. листинг 2.9).

Задание 2.20. Докажите сходимость алгоритма CP к неподвижной точке, если передаточные функции монотонны.

Задание 2.21. Прочитайте статью [28] и попробуйте осознать, почему МОР-решение возможно только для дистрибутивных проблем и что именно ломается в не дистрибутивном случае.

Задание 2.22. Напишите программу на вашем любимом языке программирования, реализующую алгоритм RD (см. листинг 2.11).

Задание 2.23. Обобщите алгоритм LVN. Как написан на листинге 2.20, он может работать только для функций, состоящих из единственного базового блока. Ваша задача сделать так, чтобы он работал для каждого базового блока, а некий внешний код вычислял, что же для базового блока является входными параметрами. Напишите этот псевдокод.

Задание 2.24. Алгоритм LVN делает только один проход по базовому блоку. Подумайте, может быть есть смысл его замкнуть, чтобы он изменял базовый блок до победы, как это делает алгоритм CR?

Задание 2.25. Предположим, вам надо скопировать поток управления, сделав по образцу текущей функции другую функцию, в котором он имеет такую же структуру. Напишите программу для этого.

Глава 3

Построение SSA

*There is nothing more difficult to take in hand, more perilous to conduct,
or more uncertain in its success,
than to take the lead in the introduction of a new order of things.*
– Niccolo Machiavelli

Уравнения для анализа потока данных, про которые я рассказывал в разделе 2.3, являются мощным и универсальным инструментом, но оставляют впечатление, что мы должны снова и снова пересчитывать их целиком при даже небольших трансформациях кода и потока управления. Кроме того, что это очень дорого, это неэффективно — компилятор раз за разом забывает результаты предыдущих расчётов и заново запускает сложный итеративный процесс. Хотелось бы найти некое представление программы, которое делало бы анализ потока данных таким же простым и естественным делом, как анализ потока управления.

Это представление называется SSA (static single assignment, статическое единственное присвоение) и первые публикации начали появляться в 80-х и связаны с именами Кена Задека и Марка Вегмана. Было сделано несколько подходов к снаряду — в частности, в ранних публикациях вместо ϕ -функций упоминались так называемые «точки порождения» и «точки переименования». Итогом почти десятилетнего исследования стала фундаментальная работа [17], заслуженно набравшая за последние тридцать лет более трёх с половиной тысяч цитирований.

Дальнейшее шествие SSA по миру компиляторов было действительно триумфальным. Сейчас в основных популярных компиляторах, например GCC и LLVM, большая часть выполняющихся над кодом трансформаций так или иначе выполняются на SSA представлении. Выходит также отличная современная

литература, практически полностью посвящённая SSA представлению. Я могу очень порекомендовать всем интересующимся отличный сборник [49], которым автор неоднократно вдохновлялся при написании этой книги. Он не рассчитан на новичков, но время, затраченное на его прочтение, многократно окупится, если вы собираетесь серьёзно работать в области оптимизирующей компиляции.

3.1 SSA представление

Рассмотрим программу, приведённую на листинге 3.1. Она довольно проста, но даже в таком простом коде можно запутаться относительно того, чему равно значение x в каждой точке программы.

```
x = 1;  
y = x + 1;  
x = 2;  
z = x + 1;
```

Листинг 3.1: Программа не в SSA

Мы могли бы переписать эту программу так, чтобы каждая переменная определялась только один раз, как на листинге 3.2.

```
x1 = 1;  
y = x1 + 1;  
x2 = 2;  
z = x2 + 1;
```

Листинг 3.2: Программа в SSA

Формальное определение 3.1 добавляет к такого рода интуиции не слишком много.

Определение 3.1. *Представление с единственным статическим присваиванием (static single assignment, SSA) — это представление кода, в котором у каждой переменной есть единственное определение, сопровождающееся инициализацией.*

Настоящие проблемы начинаются, когда управление должно сойтись, например, если переменная получает новое значение по обоим веткам условного оператора, как на листинге 3.3.

```

x = input();
y = 0;
if (x == 42)
    y = 1;
else
    y = x + 2;
return y;

```

Листинг 3.3: Программа не в SSA

Задание 3.1. *Прежде чем читать дальше, подумайте, как бы вы привели эту программу в соответствие определению 3.1.*

В классической формулировке в месте схождения управления мы ставим новое определение переменной, которое задаётся ϕ -узлом.

Определение 3.2. $\phi(x_1, x_2, \dots, x_n)$ задаёт схождение управления от переменных x_i , определённых в разных базовых блоках. В программе мы пишем $\text{PHI}(x1:\text{BB1}, x2:\text{BB2}, \dots, xn:\text{BBn})$, чтобы дополнительно указать базовый блок предшественника, дающий информацию о том, по какому пути приходит определение переменной. Иногда мы будем опускать блоки, если они ясны из контекста.

Небольшое терминологическое отступление. Английское слово *node* (использованное, в частности, в оригинальном термине « ϕ -node») часто обозначает также вершину графа. И да, мы увидим такие графы, у которых ϕ -узлы будут вершинами (см. раздел 4.1.2). Но, если я имею в виду выражение, определяющее присваивание из ϕ , я буду систематически говорить именно про узлы. Мне кажется, это методически правильно: ϕ -узел как бы связывает поток данных. Листинг 3.4 демонстрирует результат переписывания листинга 3.3 в SSA представление и в форму HIR (см. раздел 1.2.4).

```

BB0: x1 = input();
      y1 = 0;
      if (x1 != 42) goto BB2; else goto BB1;
BB1: y2 = 1;
      goto BB3;
BB2: y3 = x1 + 2;
      goto BB3;
BB3: y4 = PHI(y2:BB1, y3:BB2);
      return y4;

```

Листинг 3.4: Программа в SSA

Обратите внимание: поскольку SSA представление используется для машинно-независимых оптимизаций, это в любом случае HIR. Таким образом, при построении SSA я следую базовым правилам принятого нами высокоуровневого IR из раздела 1.2.3. То есть я разбиваю все высокоуровневые конструкции, такие как ветвления и циклы на базовые блоки и указываю явные переходы между базовыми блоками. Подробнее про базовые блоки можно прочитать в разделе 2.1.

Напомню, что по введённым там определениям, в частности, по определениям 2.1 и 2.2, каждый базовый блок заканчивается одной или несколькими специальными инструкциями, так называемыми терминаторами. В качестве терминаторов в HIR выступают `goto` и `return`. Опущенная инструкция-терминатор в последнем базовом блоке означает «и так далее» — я подразумеваю, что там может быть произвольное количество кода, который не слишком важен для иллюстрируемой мысли.

Кроме того, как было указано в разделе 1.2.3, код трёхадресный, то есть сложные инструкции разбиваются на более простые так, чтобы в каждой инструкции было не более одного присваивания и не более одной операции или вызова функции. Выражение $\varphi(x_1, \dots, x_n)$ считается неделимой n -арной операцией.

Основные правила, действующие относительно φ -узлов, следующие:

1. Все φ -узлы размещаются исключительно в начале базового блока.
2. Все φ -определения обрабатываются одновременно.

Второй пункт очень важен, и мы неоднократно вернёмся к нему впоследствии. Первый пункт следует из второго — попытки смещения линейного исполнения базового блока с параллельным исполнением φ -узлов могут порождать совершенно не нужные сложности.

```
BB0: x1 = input();
      y1 = input();
      goto BB1;
BB1: x2 = PHI(y1:BB0, y2:BB1);
      x3 = x2 + y2; // offending linear execution
      y2 = PHI(x1:BB0, x3:BB1); // problem
      if (x2 != 42) goto BB1;
```

Листинг 3.5: Пример сложностей при смешивании кода и phi

Проблемы смешивания очевидны на листинге 3.5. Если бы φ -узлы для x_2 и y_2 стояли рядом, это был бы просто параллельный обмен значениями переменных x и y на каждой итерации цикла. Но вмешивается линейное определение x_3 , и теперь не очень ясно, как считать y_2 параллельно с x_2 , то есть φ -узлы сериализуются. Это не очень хорошо, потому что многие алгоритмы закладываются на возможность свободно переставлять вычисления φ -узлов, и это разумно отражает тот факт, что значения в них приходят с рёбер, входящих в блок и, значит, делают это независимо.

Задание 3.2. Попробуйте сделать из листинга 3.5 корректный SSA с соблюдением обоих пунктов.

3.1.1 Наивный способ построения SSA

Существует простой и достаточно прямолинейный способ построения SSA представления, основываясь на уравнениях анализа данных. В качестве примера рассмотрим листинг 3.6, аннотированный информацией про достигающие определения.

```
BB0: x = 0;
      y = 0;
      if (x >= 10) goto BB2;
      goto BB1;
// RDIN: (x, 0), (y, 0),
//        (x, 1), (y, 1)
BB1: y = y + x;
      x = x + 1;
      if (x < 10) goto BB1;
      goto BB2;
// RDIN: (x, 0), (y, 0),
//        (x, 1), (y, 1)
BB2: return y;
```

Листинг 3.6: Достигающие определения

Задание 3.3. Прежде чем читать дальше, попробуйте сами придумать, как именно использовать эту информацию для максимально наивного построения SSA. Алгоритм совсем не сложен.

Итоговый мета-алгоритм построения состоит из нескольких простых и очевидных шагов.

- Получаем решение проблемы достигающих определений для всех определений в анализируемом коде. Этот алгоритм описан в разделе 2.3.1.
- Для всех достигающих определений вставляем ϕ -узлы.
- Переименовываем переменные с помощью вычисления достигающих определений ещё раз, потому что новые вставленные ϕ -узлы — это определения (этот остроумный шаг предложен в [16]).

На листинге 3.7 показан пример таким образом построенного SSA для цикла. Я по традиции строю SSA исключительно на базе HIR и надеюсь, что это не вызывает трудностей при прочтении. Чтобы посмотреть точное соответствие, вам нужно сделать дополнительный мысленный шаг трансформации в HIR для исходного кода с листинга 3.6.

```

BB0: x1 = 0; y1 = 0;
      if (x >= 10) goto BB2;
      goto BB1;
BB1: x2 = PHI(x1:BB0, x3:BB1);
      y2 = PHI(y1:BB0, y3:BB1);
      y3 = y2 + x2;
      x3 = x2 + 1;
      if (x3 < 10) goto BB1;
      goto BB2;
BB2: x4 = PHI(x1:BB0, x3:BB1);
      y4 = PHI(y1:BB0, y3:BB1);
      return y4;

```

Листинг 3.7: Итоговое SSA представление

Видно, что такой подход приводит к появлению лишних ϕ -узлов, например переменная x_4 абсолютно не нужна и не используется. Далее надо запускать какой-нибудь нетривиальный проход, чтобы оптимизировать полученное SSA-представление и убирать лишние узлы, что само по себе дорого и гораздо лучше иметь оптимальный SSA по построению.

Кроме того, дважды решить уравнения для анализа потока данных может быть накладно с точки зрения времени работы алгоритма: речь идёт о дорогих итеративных процедурах.

Далее я объясню, как строить SSA представление более эффективно в разделе 3.3.2. Но для этого сначала надо ввести понятие отношений доминирования и построить специальную структуру данных, известную как фронт доминирования.

3.2 Отношения и структуры доминирования

Dominance. Control. These things the unjust seek most of all.

– Robert Fanney

Отношения доминирования вводятся на графе потока управления и определяются очень просто.

Определение 3.3. B_i доминирует B_j , если и только если B_i располагается на любом пути от entry к B_j .

Также для инфиксного обозначения этого бинарного отношения я буду использовать операцию $\preceq$, скажем $B_j \preceq B_i$ означает, что B_i доминирует B_j . Впрочем, гораздо чаще я буду использовать просто слово «доминирует».

Из определения 3.3 следует, во-первых, что каждая вершина доминирует сама себя. А во-вторых, мы можем вывести рекуррентное соотношение, анализируя множества доминирующих вершин для их предшественников в графе, чтобы установить такое множество для исследуемой вершины.

$$Dom(n) = n \cup \left(\bigcap_{m \in pred(n)} Dom(m) \right) \quad (3.1)$$

Из уравнения 3.1 сразу же следует рекурсивный алгоритм поиска всех доминаторов. Его псевдокод показан на листинге 3.8.

```
for (auto block : blocks)
    Dom[block] = all;
Dom[entry] = Set<Block>{entry};
Changed = true;
while (Changed) {
    Changed = false;
    for (auto block : blocks) {
        set Temp = all;
        for (auto j : preds(block))
            Temp = intersect(Temp, Dom[j]);
        append(Temp, block);
        if (Dom[i] != Temp) {
            Changed = true;
            Dom[i] = Temp;
        }
    }
}
```

Листинг 3.8: Algorithm D

Работа этого алгоритма на простом графе проиллюстрирована на рис. 3.1. На каждой итерации множества доминаторов подсчитываются с учётом состояния множеств, подсчитанных для предков на предыдущих шагах. Например, на третьем шаге множество для 4 считается как пересечение множеств для 3 и 6, оцененных на втором шаге. Проверьте, что после третьего шага подсчитанные множества доминаторов перестают меняться, и алгоритм сходится к неподвижной точке.

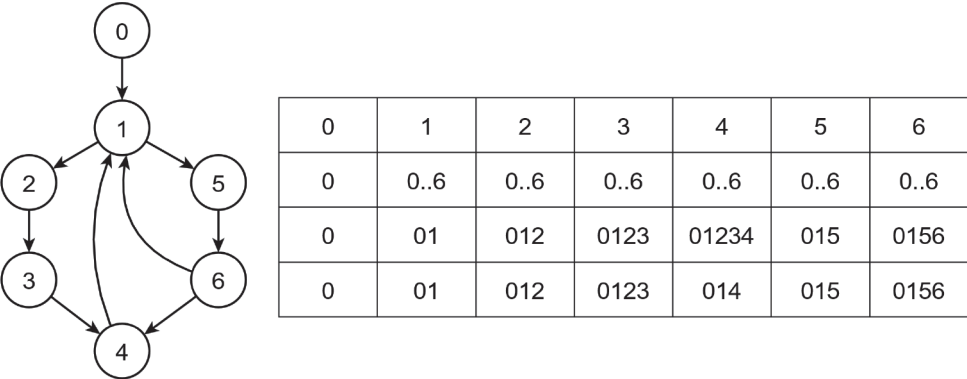


Рис. 3.1: Пример работы алгоритма D

Всегда ли этот алгоритм будет сходиться? Да. Доказательство этого факта довольно просто вытекает из наблюдения, что на каждом шаге оценённое множество доминаторов для каждого базового блока может или остаться прежним, или сократиться. Вам будет предложено провести это доказательство самостоятельно в упражнениях.

Можно ли улучшить среднее время работы этого алгоритма? Да, если использовать обход в одном из возможных порядков поиска в глубину (а конкретно в reverse post order, можно также сказать «в обращённом постордере»). К этому порядку обхода мы ещё вернёмся, когда мы будем говорить о циклах, см. раздел 5.1.1, где мы существенно улучшим алгоритм D.

Прекрасная работа Тарьяна на эту тему является классической и несомненно стоит того, чтобы её изучить, см. [38].

Определение 3.4. Вершина B_i строго доминирует B_j , если она доминирует B_i и не совпадает с ним.

Я буду также обозначать строгое доминирование как операцию $\prec$, например, можно записать, что $B_j \prec B_i$.

Определение 3.5. Вершина B_i непосредственно доминирует B_j , если $B_i \prec B_j$ и при этом B_i является ближайшим к B_j доминатором по всем путям от entry.

Понятие ближайшего для доминаторов можно ввести обычным образом: чтобы B_i была ближайшим доминатором, не должно существовать такого B_k , что $B_k \prec B_j$, и при этом $B_i \prec B_k$.

К определению 3.5 можно добавить, что непосредственный доминатор у каждой вершины является единственным.

3.2.1 Дерево доминаторов

*Trees sprout up just about everywhere
in computer science.
— Donald Knuth*

Построим граф, у которого ребро от m к n есть, если m непосредственно доминирует n . Поскольку у любой вершины в таком графе не более одного родителя, это направленное дерево, и оно называется деревом доминаторов (dominator tree).

Определение 3.6. Дерево доминаторов для графа G — это ориентированное дерево $Dom(G)$, у которого ребра соответствуют отношениям непосредственного доминирования в G .

Алгоритм построения дерева доминаторов по подсчитанным множествам доминаторов приведён на листинге 3.9.

```
DomTree = Graph{};
for (auto [I, Domset] : Dom) {
    Domset = exclude(Domset, I)
    if (empty(Domset))
        continue;
    if (size(Domset) == 1) {
        add(DomTree, tie(head(Domset), I))
        continue;
    }
    auto J = closest(Domset, I);
    add(DomTree, tie(J, I))
}
```

Листинг 3.9: Algorithm DT

В этом алгоритме использована вспомогательная процедура `closest`, которую вам будет предложено реализовать в качестве упражнения (см. задание 3.13).

Дерево доминаторов для графа рис. 3.1 приведено на рис. 3.2.

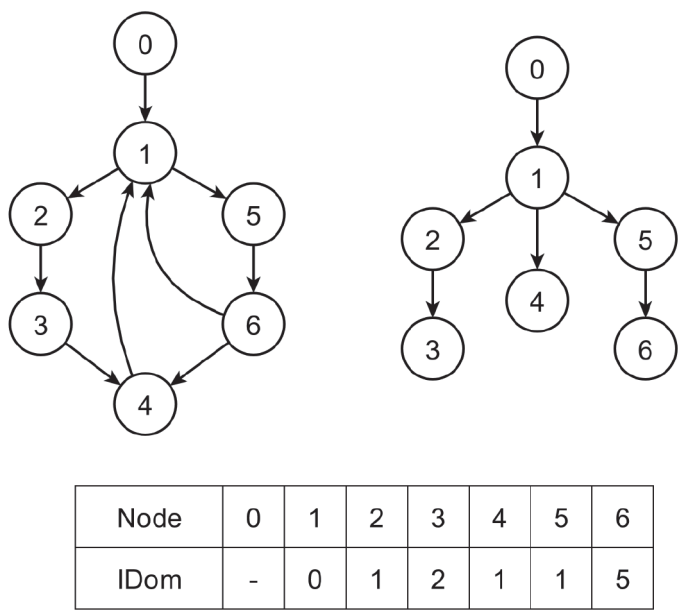


Рис. 3.2: Дерево доминаторов построенное алгоритмом DT

Само по себе дерево доминаторов является важной структурой для многих оптимизаций. Например, общее подвыражение из двух блоков логично выносить в их общий доминатор, который как раз проще всего находится как общий предок в дереве. Но для нас сейчас важную роль играет эта структура данных для того, чтобы построить следующую структуру, отвечающую за то, куда нам всё-таки вставлять φ -узлы. А именно — построить фронт доминирования (также говорят «границу доминирования», в оригинале — *dominance frontier*) и его итерированную версию.

3.2.2 Фронт доминирования

Представьте, у нас есть некоторый базовый блок B_n который доминирует блок B_m . Представим, что в B_m используется переменная, которая определена в B_n . Простой вопрос: нужно ли в B_m

вставлять ϕ -узел для этой переменной? И ответ, конечно, отрицательный: из доминирующего блока переменная и так туда попадёт по всем путям.

Но теперь представим, что у базового блока B_m есть ещё один потомок, скажем B_k , в который входят ещё какие-то дуги, то есть B_n , возможно, доминирует его предка, а не его (ситуация, когда все пути к B_k проходят через B_n тоже возможна, но сейчас менее интересна).

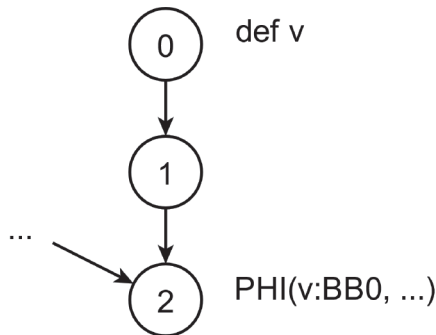


Рис. 3.3: Объяснение фронта доминирования

Подобная ситуация показана на рисунке 3.3 — здесь базовый блок B_0 доминирует предков B_2 , но не сам B_2 . Тогда мы говорим, что B_2 входит во фронт доминирования B_0 .

Определение 3.7. *Фронтом доминирования (dominance frontier) для блока B_n называется множество всех блоков B_m , таких, что B_n доминирует хотя бы одного из предков B_m , и B_n не является строгим доминатором B_m .*

Только в условиях, задаваемых определением 3.7, вставка ϕ -узла становится возможной (и мы дальше увидим, что при итерировании фронта она становится необходимой). О фронте доминирования всех вообще блоков заданного графа потока управления несколько проще думать в терминах отдельного графа для фронта доминирования. Для небольших графов такая визуализация очень помогает и делает сопутствующие задачи особенно простыми.

Определение 3.8. *Графом фронта доминирования (DF-графом) для графа G называется граф, в котором вершины совпадают с вершинами G , а направленное ребро от V_i к V_j существует только в случае, когда V_j входит во фронт доминирования для V_i .*

Пример интуитивно вычисленного фронта доминирования показан на рисунке 3.4. Слева показан исходный граф, справа граф фронта доминирования для него, в соответствии с определением 3.8.

Можно проследить, что B_4 доминирует B_7 , но не B_3 , так что во фронт доминирования B_4 входит B_3 . Точно так же, B_4 доминирует B_6 , но не B_1 и, таким образом, во фронт доминирования B_4 входит B_1 .

Посмотрите ещё раз на рисунок 3.4 и задайтесь вопросом — а какие вообще блоки вошли во фронты доминирования других блоков? Куда входят стрелки?

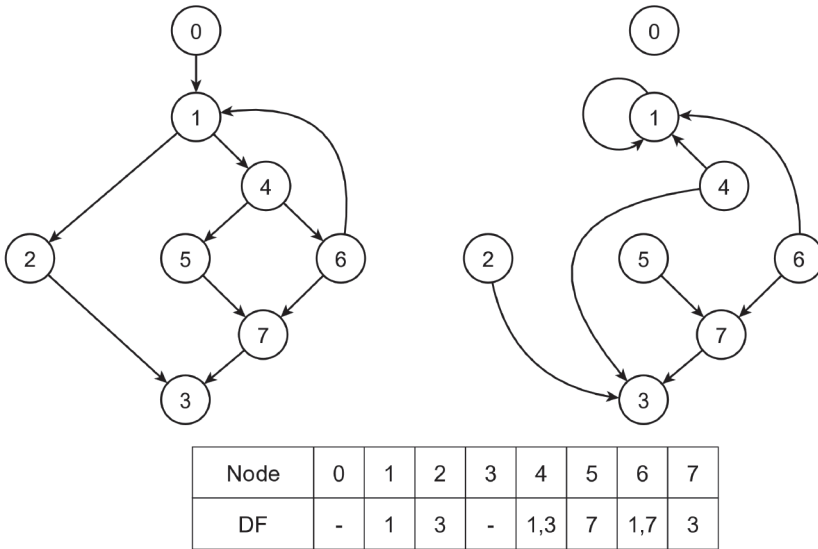


Рис. 3.4: Пример фронта доминирования

Задание 3.4. Прежде чем читать дальше, постарайтесь как-то охарактеризовать такие блоки.

Немного наблюдения позволяет заметить интересный факт: в графе для DF, стрелки идут только к базовым блокам B_1 , B_3 и B_7 , которые в оригинальном графе являются узлами схождения управления.

Тогда, чтобы найти все фронты доминирования, в которые входит данный базовый блок, начиная от него, можно предложить алгоритм, показанный на листинге 3.10. По всем предшественникам базового блока нужно найти такие, которые не являются его непосредственными доминаторами. И тогда они и все их непосредственные доминаторы, также не являющиеся доминаторами данного блока, содержат этот блок в своём фронте доминирования.

```

auto analyseDF(Function F) {
    auto IDom = F.immediateDominatorSet();
    auto DF = Map<Block, Set<Block>>{};
    for (auto B : F.blocks)
        for (auto P : F.preds(B)) {
            auto R = P;
            while (R != IDom[B]) {
                DF[R] = DF[R] join B;
                R = IDom[R];
            }
        }
    return DF;
}

```

Листинг 3.10: Алгоритм DF — фронт доминирования

В этом алгоритме строится множество DF для каждого базового блока B , но ничего не стоит использовать эту информацию для построения DF -графа. Вам будет предложено это сделать в разделе упражнений.

3.2.3 Графы схождения

Можно думать о фронте доминирования ещё одним способом, который несколько более интуитивен и несколько более просветляет, если изложенное в разделе 3.2.2 показалось вам излишне формальным.

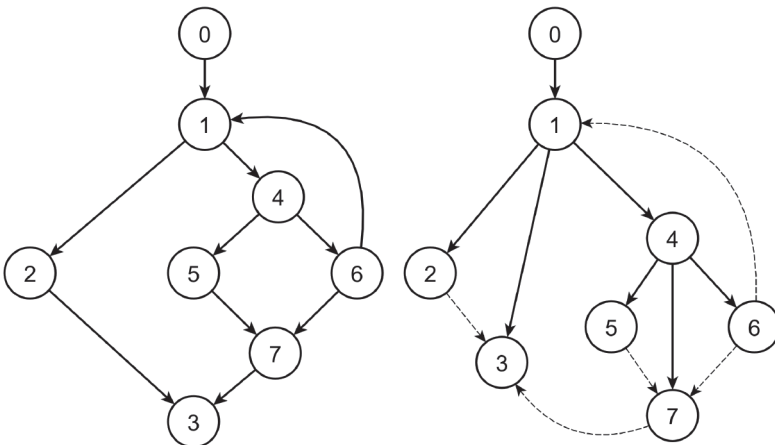


Рис. 3.5: Пример дерева доминаторов со связями схождения управления

На рисунке 3.5 слева приведён граф, а справа показано дерево доминаторов для этого графа с добавленными (пунктиром) стрелками, соответствующими рёбрам соединения (join edges, J-рёбрам), то есть таким рёбрам графа потока управления, у которых их начальный блок не является строгим доминатором их конечного блока.

Определение 3.9. *Графом схождения (Def-Join Graph или DJ-графом) называется граф с двумя видами рёбер. Первый вид соответствует рёбрам дерева доминаторов, а второй вид соответствует добавленным J-рёбрам.*

По этому дереву с дополнительными связями очень легко понять, как строится DF-граф. Например, мы видим в дереве связь $7 \rightarrow 3$ и при этом связь пунктирная, то есть 7 не доминирует 3. Значит в DF будут $7 \rightarrow 3$, далее поднимаясь по сплошным стрелкам $4 \rightarrow 3$, но не $1 \rightarrow 3$, потому что 1 доминирует 3.

В разделе задач вам будет предложено последовательно построить DJ-граф, а потом DF-граф. Вы можете сравнить себя с алгоритмом DF с листинга 3.10 и проверить, получилось ли у вас проще.

3.3 Построение SSA

*Give me six hours to chop down a tree
and I will spend the first four sharpening the axe.*
– Abraham Lincoln

Существует достаточно много алгоритмов построения SSA. Все они так или иначе включают расстановку ϕ -узлов и переименование переменных. В этом разделе я сначала расскажу про основной алгоритм, являющийся простым, практичным и эффективным (но оставляющим место для улучшения), и покажу примеры его работы на HIR. Он, по сути, не менялся начиная от фундаментальной публикации 1991-го года (см. [17]).

Но чтобы эта книга не была слишком теоретической, я также покажу, как решают ту же проблему современные компиляторы на примере LLVM. Там используется на удивление отличающийся подход к представлению IR, но в итоге алгоритмы имеют много общего (и много общего с основным алгоритмом).

И первый вопрос: какое именно SSA представление мы строим? Хорошо, оно является SSA по определению. Должно ли оно удовлетворять ещё каким-то свойствам?

3.3.1 Свойства и виды SSA

Современная литература, например [49], описывает несколько разновидностей SSA, определяя их через наличие и отсутствие некоторых свойств.

- 1. Свойство минимальности** — означает, что вставлено минимально необходимое количество ϕ -узлов при условии выполнения свойства единичной достижимости.
- 2. Свойство строгости** — означает, что каждая переменная объявлена до использования. В некоторых языках, например в C, строгость не гарантируется на языковом уровне. Тем не менее, строгое SSA представление можно строить, вставляя в ϕ -узел неопределённые значения по соответствующим веткам.
- 3. Свойство доминирования** — означает, что каждое использование переменной доминируется точкой её определения.

Для того, чтобы построить минимальную SSA форму, необходимо вставить наименьшее количество ϕ -узлов так, чтобы результирующая программа удовлетворяла свойству единой точки достижимости, которое означает, что каждой точке программы достигает не более одного определения одной и той же переменной. В этом случае мы допускаем вставку ϕ -узлов (то есть определений переменной) в такие места в программе, где ранее эта переменная вообще не употреблялась.

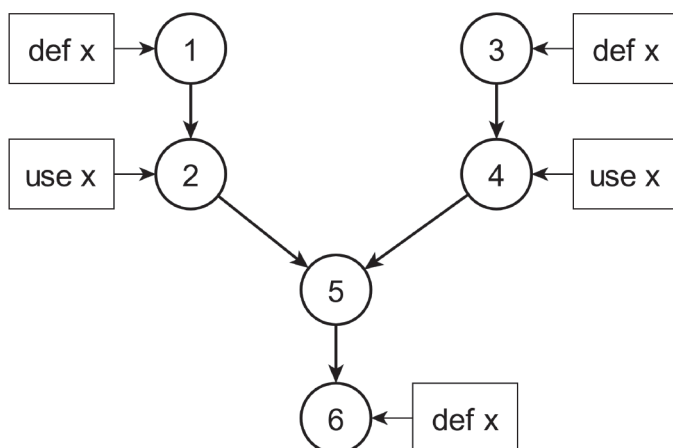


Рис. 3.6: Пример до переименования для срезанной SSA-формы

В качестве примера посмотрите на рисунок 3.6. В нормальном случае мы автоматически поставим φ -узел в блок 5, и именно так будет действовать основной алгоритм. Но вообще-то он там не нужен, потому что переменная x в блоке 5 не является активной. Про анализ активных переменных мы уже говорили в разделе 2.3.3.

Чтобы избежать вставки лишнего определения в блоке 5, можно ослабить свойство единичной достижимости и сделать вместо него свойство, что каждой точки использования переменной (а не каждой точки в программе вообще) достигает не более одного её определения. Тогда мы можем не ставить φ -узел в точку схождения если переменная не активна (в указанном выше смысле) в этой точке. Такая SSA форма называется срезанной формой (pruned SSA). В этой книге мы не будем рассматривать эту форму и её применения, хотя, скажем компилятор LLVM строит именно срезанную SSA (подробнее см. задачу 3.10). В практическом смысле алгоритм построения мало отличается и дальнейшее оперирование такой формой не отличается вообще. Заинтересовавшиеся могут ознакомиться со статьёй [13].

Теперь настало время воспользоваться фронтом доминирования для построения минимального SSA, сохраняющего свойства доминирования (в дальнейшем я буду подразумевать свойство доминирования по умолчанию если не указано обратное).

3.3.2 Основной алгоритм

При построении минимального SSA во всех случаях используется концепция семантической сети. Интуитивно это очень просто понять: одна и та же переменная может «путешествовать» по программе: определяться, переопределяться, использоваться в различных контекстах и так далее. Все такие определения и использования составляют на самом деле одну и ту же переменную. Мы будем многократно сталкиваться с такого рода сетями далее, поэтому следует дать формальное определение.

Определение 3.10. *Семантической сетью для переменной является совокупность её индивидуальных значений, связанных отношениями взаимного использования. В эту сеть входят все места, где переменная была определена, использована, переименована и так далее.*

С учётом этого определения, последовательность действий для построения SSA выглядит довольно просто.

- Для каждой переменной, которая определяется и используется в разных блоках, составляется семантическая сеть.
- Для каждой такой сети итерируется фронт доминирования от блоков с определениями и вставляются ϕ -узлы.
- Далее определения внутри сети переименовываются так, чтобы гарантировать инварианты SSA формы.

Также во всех случаях я буду использовать немного синтетический код с листинга 3.11 в качестве примера, над которым будет работать алгоритм построения SSA.

```
int bar(int);
int baz(int, int);

int foo(int c) {
    int x = 0, y = 0;
    do {
        if (c) {
            x = 1; y = 2;
        }
        else {
            x = bar(x); y = bar(x); x = y;
            if (x > -2)
                break;
        }
        x = baz(x, y);
    } while(x > 0);
    return x;
}
```

Листинг 3.11: Пример для построения SSA

Задание 3.5. *Переведите этот код в NIR и вставьте для него ϕ -узлы наивным методом*

Задача основного алгоритма — это вставить ϕ -узлы и переименовать переменные. Будем вставлять узлы для одной переменной за один раз. Выберем любую переменную в программе, например, x , и заведём для неё счётчик $iter$, который инкрементируется каждый раз, когда ϕ -узел вставляется в блок.

Для каждого блока заведём также атрибут `inserted(B, Var)`, который устанавливается в 1, если для этой переменной φ -узел в этом блоке уже есть. Можно не хранить это как атрибут, а реализовать как функцию, анализирующую заголовок блока. Точно так же можно реализовать функцию `defined(B, Var)`, означающую, что переменная объявлена в блоке. И, разумеется, мы в этой точке можем считать, что фронт доминирования каждого блока известен. Получившийся алгоритм приведён на листинге 3.12.

```
auto WL = Stack{};
auto Iter = 0;
for (auto V : all_variables) {
    for (auto B : all_blocks | defined(B, V))
        WL.push(B);

    while (!WL.empty()) {
        auto DefBlock = WL.pop();
        for (auto B : DF(DefBlock)) {
            if (inserted(B, V))
                continue;
            // iterate
            Iter = Iter + 1;
            B.insert_phi(V);
            WL.push(B);
        }
    }
}
```

Листинг 3.12: Алгоритм SI — вставка φ -узлов

Очень важным является итерирование фронта доминирования, помеченное на листинге 3.12 комментарием «iterate». Поскольку фронт доминирования распространяется дальше, необходимо добавить в рабочий список новые блоки, что и происходит.

Собственно, граф итерированного фронта доминирования тоже можно нарисовать: он будет транзитивным замыканием графа DF, то есть везде, где есть рёбра $a \rightarrow b$ и $b \rightarrow c$, следует добавить ребро $a \rightarrow c$. Но поскольку мы этот граф в явном виде не используем, мы его и не строим, он строится неявно в процессе работы алгоритма SI.

Чтобы отслеживать, что и где этот алгоритм вставил, переведём основной пример с листинга 3.11 в HIR.

```

BB0: c = input(); x = 0; y = 0;
      goto BB1;
BB1: if (!c) goto BB3; else goto BB2;
BB2: x = 1; y = 2;
      goto BB4;
BB3: x = bar(x); y = bar(x); x = y;
      if (x > -2) goto BB5; else goto BB4;
BB4: x = baz(x, y);
      if (x <= 0) goto BB5; else goto BB1;
BB5: return x;

```

Листинг 3.13: Пример HIR до построения SSA

Задание 3.6. Прежде чем читать дальше, самостоятельно примените алгоритм SI к листингу 3.13. Для этого сначала постройте дерево доминаторов и DF-граф. Какие ϕ -узлы были вставлены?

После отработки алгоритма SI, листинг 3.13 выглядит следующим образом:

```

BB0: c = input(); x = 0; y = 0;
      goto BB1;
BB1: x = PHI(x, x); y = PHI(y, y);
      if (!c) goto BB3; else goto BB2;
BB2: x = 1; y = 2;
      goto BB4;
BB3: x = bar(x); y = bar(x); x = y;
      if (x > -2) goto BB5; else goto BB4;
BB4: x = PHI(x, x); y = PHI(y, y);
      x = baz(x, y);
      if (x <= 0) goto BB1; else goto BB5;
BB5: x = PHI(x, x);
      return x;

```

Листинг 3.14: Пример HIR после вставки ϕ -узлов

Задание 3.7. Убедитесь, что вставка ϕ -узлов таким образом выполняет свойства минимального SSA. Оцените, сколько определений мы сэкономили по сравнению с наивным методом.

После того как мы определились с позициями ϕ -узлов, остаётся только переименовать переменные.

Переименование переменных происходит в порядке обхода в глубину дерева доминаторов (чтобы сохранить свойство доминирования в минимальной SSA). Для каждой переменной

поддерживается стек переименований $S(Var)$. В каждом базовом блоке анализируется каждая инструкция. Сначала все операнды в правой части переименовываются в соответствии с верхушкой стека. Далее для переменной в левой части создаётся и помещается на стек новая версия. После этого исследуются ϕ -узлы потомков по всем рёбрам, и там тоже делается переименование. Но идти в сами потомки торопиться не надо: мы заглядываем только в доминируемые блоки и устраиваем переименование там по такому же алгоритму.

Сам алгоритм приведён на листинге 3.15. Самое важное, что следует проследить — это работа со стеками переименований и с деревом доминаторов. Обратите внимание на функцию `domchild` — она возвращает всех потомков в дереве доминаторов (то есть все непосредственно доминируемые узлы).

```
// initially for all V, S[V] = {V0};

// this is a recursive function
void rename(BB& Current) {
    for (auto I : instrs(Current)) {
        for (auto V : rhs(I))
            replace(V, S[V].top());
        for (auto V : lhs(I)) {
            Cnt = counter(V);
            S[V].push(version(V, Cnt));
            replace(V, S[V].top());
        }
    }
    for (auto B : succ(Current))
        replace_this(Current, B);
    for (auto B : domchild(Current))
        rename(B);
    // pop every version pushed in Current
}
```

Листинг 3.15: Алгоритм SR — переименование переменных

Задание 3.8. *Примените алгоритм SR к листингу 3.14.*

В задачах для самостоятельной работы вам будет предложено реализовать эти алгоритмы и даже оптимизировать алгоритм SR, который, конечно, вовсе не обязан быть рекурсивным.

Давайте теперь на секунду вернёмся из идеального мира HIR и посмотрим, как то же самое работает в реалистичном LLVM IR.

3.3.3 Построение SSA в LLVM

Переведём основной пример с листинга 3.11 в LLVM IR до построения SSA. Это можно сделать, скомпилировав его, как описано в разделе 1.2.4.

```
define dso_local i32 @foo(i32 noundef %0) {
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    store i32 %0, ptr %2
    store i32 0, ptr %3
    store i32 0, ptr %4
    br label %7
7:
    %8 = load i32, i32* %2
    %9 = icmp ne i32 %8, 0
    br i1 %9, label %10, label %11
10:
    store i32 1, i32* %3
    store i32 2, i32* %4
    br label %19
.....
```

Листинг 3.16: Пример LLVM IR до построения SSA

Листинг 3.16 должен сразу вызывать вопросы. Это уже LLVM IR, то есть уже SSA представление. Не собрались ли мы переводить из SSA в SSA?

Если к нему приглядеться, то видно, что все доступы к локальным переменным, используемым внутри программы, делаются через память, то есть через записи в аллоса в начале модуля. Формально являясь SSA, фактически он оперирует изменяемыми значениями.

На самом деле, поскольку единицей операции над агрегатной структурой в LLVM является GEP, для агрегатов тут всё немного сложнее: структуры и массивы на стеке, рассматриваются как единая аллоса и доступы к их элементам выглядят как несвязные обращения в память.

```

%a = alloca [8 x i8]
%gep1 = getelementptr i8, ptr %a, i32 0
%gep2 = getelementptr i8, ptr %a, i32 4
store float 0.0, ptr %gep1
store float 1.0, ptr %gep2
.....
%f1 = load float, ptr %gep1
%f2 = load float, ptr %gep2

```

Листинг 3.17: Пример LLVM IR для упрощения агрегатов

Разумеется, перед построением SSA для максимального подъёма семантической сети (см. определение 3.10) желательно распутать все эти обращения, заменив операции над частью агрегата на операции над скаляром. Этим в инфраструктуре LLVM занимается фаза с логичным названием скалярная замена агрегатов (Scalar Replacement Of Aggregates, SROA). Во многом эта фаза может быть улучшена. Например, автор этой книги руководил интересной дипломной работой, в которой удалось ускорить важные алгоритмы в графическом компиляторе, предварив эту фазу более умной раскладкой структур (см. [57]).

Не удивительно, что в LLVM именно SROA занимается построением SSA. Используя подпрограмму `PromoteMemoryToRegister`, фаза SROA делает ровно то, что ранее было сделано в основном алгоритме в разделе 3.3.2. Она расставляет ϕ -узлы, переименовывает переменные, создаёт SSA-значения и убирает ставшие ненужными выделения памяти. На самом деле она делает даже больше. К примеру, она вставляет ϕ -узлы с `undef` и `poison` значениями там, где значение переменной либо неизвестно, потому что не нужно, либо неизвестно из-за ошибки программиста (если программа использует неинициализированную переменную и тому подобные случаи). То, что получается в итоге, приведено на листинге 3.18.

```

define dso_local i32 @foo(i32 noundef %0) {
  br label %2
2:
  %3 = phi i32 [ 0, %1 ], [ %13, %10 ]
  %4 = icmp ne i32 %0, 0
  br i1 %4, label %5, label %6
5:
  br label %10
.....

```

Листинг 3.18: Пример LLVM IR после построения SSA

Задание 3.9. *Самостоятельно получите полный листинг. Сравните положение ϕ -узлов в исполнении LLVM и те их позиции, которые нашёл алгоритм SI.*

Задание 3.10. *Сможете ли вы найти такую программу, где разница будет заметна?*

Задачу 3.10 можно считать задачей повышенной сложности. Ответ (возможно) следует искать в тонкой разнице между обычной и срезанной формами. Возможно, алгоритм, строящий SSA в LLVM, получит в каких-то случаях немного другие позиции для вставки ϕ -узлов. Тем не менее, найти программу, которая продемонстрировала бы минимальное отличие будет непросто (а, может быть, и невозможно вовсе, давайте избегать спойлеров).

Из интересного мы видим, что в LLVM переменные сразу теряют идентичность. Если в HIR они всё ещё будут называться x_1, x_2, y_1 и так далее, то в LLVM IR это будет просто %2, %3, %4.

Можно подискутировать о том, хорошо это в целом или плохо.

3.4 ЗАДАЧИ НА САМОСТОЯТЕЛЬНУЮ ПРОРАБОТКУ

If you can solve it, it is an exercise; otherwise it's a research problem.

– Richard Bellman

Задание 3.11. Напишите программу, которая генерирует случайный граф. Попробуйте добавить возможность ввести ограничения, чтобы граф был похож на реально встречающиеся в программировании графы потока управления.

Задание 3.12. Докажите сходимость алгоритма D (см. листинг 3.8). Проанализируйте его среднее время работы.

Задание 3.13. Для алгоритма DT (см. листинг 3.9) реализуйте функцию *closest*.

Задание 3.14. Проанализируйте граф, приведённый на рисунке 3.7. Определите для него *dominator sets*.

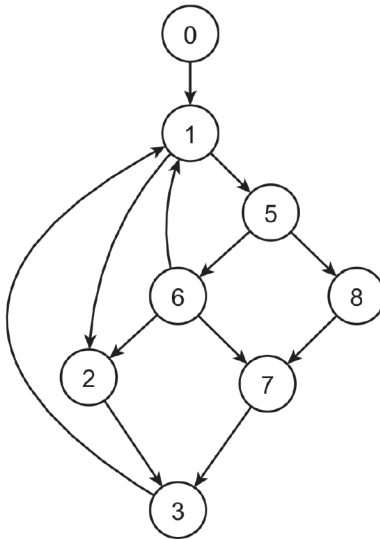


Рис. 3.7: Пример для задания 3.14

Задание 3.15. Для графа из задания 3.14 нарисуйте дерево доминаторов. Пунктиром нарисуйте соединительные стрелки.

Задание 3.16. Продолжая задание 3.15, постройте для того же графа соответствующий ему граф фронт доминирования, используя алгоритм DF с листинга 3.10.

Задание 3.17. Проанализируйте алгоритм SI (см. листинг 3.12), докажите сходимость итерирования фронта доминирования для любой переменной.

Задание 3.18. Допишите недостающие части алгоритма SR (см. листинг 3.15) и попробуйте сделать его итеративным, убрав рекурсию. В общем случае рекурсию из компиляторных алгоритмов стараются убирать из-за угрозы переполнения стека.

Задание 3.19. Продолжая задачу 3.11, напишите для сгенерированных случайных графов построение DJ-графа и далее, используя его, напишите построение DF-графа. Напишите также построение итерированного фронта доминирования для произвольного графа.

Глава 4

Базовые оптимизации для SSA

If you optimize everything, you will always be unhappy.
– Donald Knuth

Как уже было сказано (см. раздел 1.2.4), современные компиляторы предпочитают SSA представление для проведения оптимизаций над HIR. В этом разделе я разберу несколько основных трансформаций и постараюсь сделать очевидным почему информация, которую предоставляет SSA, настолько удобна для анализа. Но сначала я бы хотел немного разобраться с терминологией.

На ранних стадиях развития теории, для алгоритма, работающего над SSA, было принято добавлять в название слово «sparse» (то есть «работающий в разреженной форме»). SSA представление действительно вводит большое количество дополнительных переменных и ϕ -узлов. В итоге пространство кода выглядит более разреженным. В нём, если так можно сказать, легче дышится.

Приведём пример использования sparse в названиях. Обычное продвижение констант на английском языке называется «constant propagation». Продвижение, основанное на SSA, — это уже «sparse constant propagation». Если же учитывать статически берущиеся бранчи и использовать информацию о потоке управления, название алгоритма усложняется до «sparse conditional constant propagation» и так далее.

Я не собираюсь ни использовать такого рода терминологию, ни тем более переводить её. Для меня все алгоритмы, делающие продвижение констант, будут просто алгоритмами, делающими

продвижение констант. Да, они пользуются разными принципами, но, как писал один испанский поэт, «Достаточно, чтобы нож был ножом». Когда дело доходит до реализации компилятора, в ход обычно идёт один, лучший из тех, что есть. Я, конечно, внесу соответствующие буквы в аббревиатуры алгоритмов как дань традиции.

А вот что я собираюсь переводить очень точно и очень близко к тем работам, которые буду цитировать, так это различие между «глобальными» и «локальными» подходами. Локальный алгоритм (например, локальная нумерация значений, локальное распределение регистров) работает только внутри базового блока. Напротив, «глобальным» называется алгоритм (глобальная нумерация значений, глобальное удаление общих подвыражений и т.п.), работающий на всей функции. Если алгоритм способен работать между функциями (или даже между модулями) он называется «межпроцедурным». Таким образом, межпроцедурный алгоритм превосходит глобальный, а глобальный превосходит локальный в том смысле, что, работая для сложного потока управления, он работает также для всех базовых блоков в нём.

На практике в компиляторах самые полезные алгоритмы — это алгоритмы глобальные. Локальные обычно почти ничего не могут сделать, а межпроцедурные бывают сложны и сомнительны. Поэтому очень часто я буду опускать термин «глобальный». Например, когда ниже мы будем говорить просто о продвижении констант или устранении частичной избыточности, имеется в виду именно глобальное продвижение констант и глобальное устранение частичной избыточности. Исключение — глобальная нумерация значений (*global value numbering*), где тот факт, что она глобальная, никогда не опускается в названии по историческим причинам.

Итак, теперь, когда мы определились с терминологией, давайте приступим. Начнём мы с простого и постепенно движемся к сложному. Интересно, что даже самые простые вещи, например, продвижение констант, в итоге таят в себе сюрпризы и неожиданные глубины.

4.1 Начинаем использовать SSA

Чтобы сразу продемонстрировать силу SSA представления, давайте посмотрим алгоритм, который на этом представлении становится исключительно простым, а именно алгоритм

удаления мёртвого, то есть неиспользуемого кода (dead code elimination, DCE), представленный на листинге 4.1.

```
auto VCnt = Bimap{Value, Int};
for (auto V : ssa_values) {
    VCnt[V] = 0;
    for (auto U : ssa_users(V))
        VCnt[V] += 1;
}
while (VCnt.count_with_value(0) != 0) {
    for (auto V : VCnt.with_value(0)) {
        for (auto U : ssa_uses(V))
            VCnt[U] -= 1;
        remove_def(V);
    }
}
```

Листинг 4.1: Алгоритм DCE: удаление мёртвого кода

Алгоритм DCE работает очень просто. Изначально для каждого SSA значения он устанавливает счётчик, содержащий информацию о том, сколько других значений его используют. Далее для всех значений, в которых этот счётчик равен нулю, определение этого значения удаляется, и для всех значений, которые оно использовало, счётчик уменьшается.

Несмотря на такую простоту, уже здесь нужно понимать нетривиальную разницу между теми значениями, которые по отношению к данному являются пользователями (Users), и теми, которые в нём являются использованиями (Uses). Мало того, чтобы удалить значение нужно как-то перейти от него самого к определяющей его инструкции.

4.1.1 Работа с SSA представлением

Для того, чтобы записывать и читать алгоритмы этого раздела, нам потребуется определённая в том, как манипулировать представлением в SSA форме. Это представление было введено в главе 3, но одно дело — ввести, а другое дело — использовать и трансформировать, сохраняя свойства. Нет ничего легче, чем разрушить свойства SSA при неумелой трансформации. Восстанавливать их потом обычно не проще, чем заново построить представление.

Поэтому мы уделим немного времени тонкостям практической работы с SSA, прежде чем продолжим.

```
BB0: y1 = 6;
      goto BB1;
BB1: x1 = y1 * 2;
      x2 = y1 + 5;
```

Листинг 4.2: HIR для базовой работы с SSA

Базовым термином для того, с чем мы всё-таки манипулируем, является SSA-значение (SSA value). Например, на листинге 4.2 значениями являются y_1 , x_1 и x_2 . При этом каждое значение может также являться пользователем для других значений. Например, множество `ssa_users` для y_1 включает x_1 и x_2 .

Очень хотелось бы, чтобы читатель здесь остановился на этой концепции. Каждое SSA значение по своему определению (или даже — для своего определения) использует другие значения. Таким образом, значение является пользователем (SSA User), и оно же является используемым (SSA Use) для своих пользователей. Все вместе SSA-значения, объединённые отношениями взаимного использования, составляют семантическую сеть (см. определение 3.10). Их можно рассматривать как своего рода серию модификаций одной и той же логической переменной. Это не всегда корректно, если они пересекаются по времени жизни, но это интересная умозрительная концепция.

```
Instruction InsnY1 = basic_blocks[0].front();
Value ValY1 = InsnY1;
auto N = InsnY1.operands[0].as_scalar_int();
if (is_numeral(N))
    for (auto Val : ssa_users(ValY1))
        replace_uses_with(Val, ValY1, N);
```

Листинг 4.3: Проход по uses

Чтобы посмотреть, как это используется на практике, давайте разберём пример кода на листинге 4.3. Этот код модифицирует HIR с листинга 4.2, продвигая значение первой переменной блока B_0 , то есть переменной y_1 во всех её пользователей.

Присвоение `ValY1 = InsnY1` во второй строчке работает потому, что любая инструкция — это значение. Опять-таки в этот дуализм надо вдуматься: по свойствам SSA каждая инструкция определяет единственное и при этом уникальное значение. Что и даёт нам возможность их логически отождествить и свободно переходить от определяемого значения к определяющей его инструкции.

В третьей строчке мы хотим получить что-то, что есть в инструкции, в частности, выражение в правой части присваивания. Для таких нужд логично использовать именно инструкцию, хотя в силу свойств представления, написать `valY1.rhs()` — тоже не ошибка, я нахожу это хуже читаемым и запутывающим.

Функция `replace_uses_with(I, V, N)` — это ещё одна важная встроенная функция, которая пытается в инструкции *I* заменить все вхождения *V* на *N*. Я передал туда первым аргументом просто значение, поскольку переход к инструкции делается неявно. Встроенная она в том смысле, что я не собираюсь её определять в терминах принятого в этой книге псевдокода. При практической реализации HIR, её определение зависит от реализации.

Этот прекрасный дуализм, заключающийся в том, что значение это и есть инструкция, а инструкция это и есть значение — это одна из главных особенностей SSA, и я буду ей пользоваться раз за разом. Разумеется, тут есть свои тонкости: существуют аргументы функции, константы и прочие такого рода SSA-значения, которые инструкциями не являются и разрушают общую симметрию и красоту картины. Но тут всегда можно вставить проверку.

4.1.2 Граф зависимостей SSA

Граф зависимостей SSA (SSA Graph) — это граф, вершинами которого являются определения SSA-переменных, а ориентированное ребро присутствует, если определение переменной использует другую переменную.

Задание 4.1. *Исходя из свойств SSA, будет ли этот ориентированный граф ациклическим?*

Полезность этого графа станет очевидна позже, сейчас достаточно просто научиться его строить для простых случаев.

```
BB0: y1 = 6;
      if (y1 < 3) goto BB1; else goto BB2;
BB1: x1 = 4;
      goto BB3;
BB2: x2 = 5;
      goto BB3;
BB3: x3 = PHI(x1, x2);
      z1 = x3 + y1
```

Листинг 4.4: HIR для примера продвижения констант

Граф зависимостей SSA для листинга 4.4 приведён на рисунке 4.1.

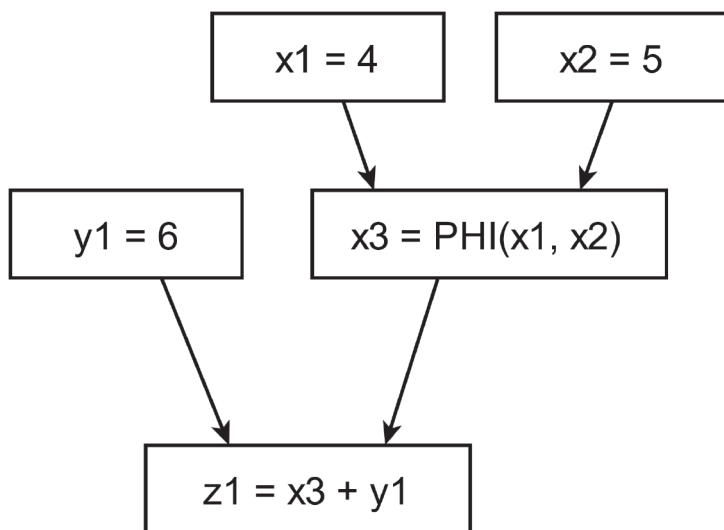


Рис. 4.1: Пример графа зависимостей SSA

Вершинами нулевой степени входа в этом ориентированном графе будут либо константы, либо вызовы каких-то внешних функций без аргументов, либо аргументы функции. Такие вершины формируют новые SSA-значения, независимо от уже сформированных значений.

Каждое направленное ребро такого графа имеет вид $def(E) \rightarrow use(E)$. Для ребра E запись $def(E)$ означает, что мы из ребра SSA графа получаем определение переменной. Все узлы и листья графа входят в множество `ssa_nodes`.

Задание 4.2. Предложите максимально простой алгоритм построения графа зависимостей SSA, используя простейшие операции над SSA из раздела 4.1.1.

Как можно легко видеть, SSA представление само по себе задаёт граф зависимостей в неявной форме. Однако в некоторых случаях он нам будет нужен именно в форме явного графа. Например, в разделе 4.2.1 мы будем добавлять в рабочий список его рёбра. Ещё более интересный пример: при анализе индуктивных переменных в циклах мы будем искать в нём сильно связанные компоненты (см. раздел 5.2).

Отныне и далее я просто буду считать, что если SSA представление построено, то и его граф зависимостей доступен.

4.2 Продвижение констант и не только

В разделе 2.2.3 мы обсудили простое продвижение констант, основанное на итеративном решении уравнений потока данных на решётке продвижения констант. Алгоритм CP (см. листинг 2.9), использованный для этого, делает массу работы, которая теперь кажется лишней. Естественной идеей является использовать SSA-представление для достижения того же (а вскоре и лучшего) результата.

4.2.1 Простое продвижение констант

Первой публикацией на эту тему была работа Рейфа и Льюиса [50], где был предложен базовый алгоритм. Он находит те же константы, что и ранее приведённый алгоритм Килдалла, но делает при этом гораздо меньше работы (и чем меньше вы используете переменных, тем меньше работы он выполняет). Интересно, что авторы не оперировали именно SSA-формой, у них была собственная идея глобального графа значений. Здесь я изложу более современную версию этого простого алгоритма, основанную на более эффективных структурах данных. А именно — на графе зависимостей SSA (см. подраздел 4.1.2).

Его основной идеей является продвижение констант сразу по рёбрам зависимостей по данным. В общем случае такой подход можно предложить не только для продвижения констант, но и для продвижения копирований и вообще для любого продвижения по данным. Алгоритм SCP, приведённый на листинге 4.5, является частным случаем именно этого подхода. В разделе 4.2.3 будет продемонстрировано обобщение на примере несколько более сложного алгоритма.

```
auto LVs = LatticeSet{};
auto WL = Stack{};

void scp() {
    initlattice();
```

```

for (auto E : ssa_edges)
    if (LVs[def(E)] != top)
        WL.push(E);
while(!WL.empty()) {
    auto E = WL.pop();
    auto D = def(E), U = use(E);
    auto M = meet(LVs[D], LVs[U]);
    if (M == LVs[U])
        continue;
    LVs[U] = M;
    for (Instruction I : ssa_users(U)) {
        WL.push(ssa_edge(U, I));
        auto OldVal = LVs[I.lhs()];
        auto NewVal = ssa_recompute_def(I.rhs(), U, M);
        LVs[I.lhs()] = NewVal;
        propagate(I, NewVal);
    }
}
}

```

Листинг 4.5: Algorithm SCP

Алгоритм SCP работает довольно прямолинейно. Каждый лист анализируется на предмет того, является ли он константой и тогда его значение — это значение константы, иначе он *overdefined* (то есть принимает значение $\top$). Всем узлам графа зависимостей, не являющимся листьями, изначально присваивается значение *undefined*. Далее строится начальный список интересующих нас рёбер. Решёточная операция выполняется на ребре над значениями его концов и записывается в значение точки использования. Если она привела к изменению в этой точке, то обновляются все инструкции, использующие результат изменившейся инструкции (на листинге 4.5 это внутренний цикл по *ssa_users*).

Обработка списка вершин достаточно проста, проблемы может вызвать только функция *ssa_recompute_def*, которую вам будет предложено реализовать в упражнениях. Эта функция перевычисляет значение выражения, в которое входит *U*, и берёт на вход кроме *U*, также старое и новое значение. Перед продвижением проверка нового значения на то, что это именно число, а не $\perp$ и не $\top$, необходима, чтобы не построить абсурдную правую часть.

Задание 4.3. *Прежде чем читать дальше, попробуйте самостоятельно восстановить подпрограммы `initlattice` и `propagate`.*

```
void initlattice() {
    for (auto N : ssa_nodes) {
        if (!is_leaf(N)) { LVs[N] = bottom; continue; }
        if (is_const(N)) { LVs[N] = N; continue; }
        LVs[N] = top;
    }
}
```

Листинг 4.6: Подпрограмма инициализации

Проще всего написать подпрограмму инициализации решётки. Как показано на листинге 4.6, мы должны поставить в $\perp$ все нелистовые узлы и в определённые значения все листья, являющиеся константами (от которых алгоритм и начнёт продвижение). Мы переиспользуем эту функцию в более совершенных подходах к продвижению констант.

```
void propagate(Instruction I, Value NewVal) {
    if (NewVal is numeral)
        for (auto Val : ssa_users(I))
            replace_uses_with(Val, I, NewVal);
}
```

Листинг 4.7: Подпрограмма распространения

Также несложна подпрограмма распространения (см. листинг 4.7). Она заменяет в зависимых инструкциях результат более ненужной инструкции на целочисленное значение, которым эта инструкция может быть заменена.

При чтении алгоритма SCP возможна критика, что мы не разделили его на анализ и трансформацию. Но здесь это не так просто: как и в любом продвижении информации по потоку данных, построенные заранее аналитические структуры быстро устаревают, так что эффективней делать и то и другое одновременно. Самой близкой к чисто трансформационной части как раз и является подпрограмма распространения.

Задание 4.4. *Оцените асимптотическую сложность алгоритма SCP с листинга 4.5 и сравните со сложностью алгоритма CP с листинга 2.9.*

Алгоритм SCP хорош и работает достаточно быстро, но он, как и алгоритм CP, пропускает важный случай: на листинге 4.4 мы видим, что значение x_3 на самом деле может быть определено, если мы заметим, что переход из блока B_0 в блок B_1 просто не может произойти. Впервые этот факт был учтён в древнем (и также основанном на уравнениях анализа данных, но чрезвычайно полезном) алгоритме, опубликованном Вебрейтом в его работе [58].

Вебрейт предложил пытаться вычислять константные условия перехода и хранить для блоков признак: исполняемые они в принципе или нет. Если блок не исполняемый, то и его передаточная функция не должна учитываться. Таким образом, его алгоритм проводил одновременно и продвижение констант, и удаление мёртвого кода.

Зададимся вопросом о том, можем ли мы учесть состояние потока управления вместе с графом зависимостей SSA и построить алгоритм не хуже? И ответ, конечно, утвердительный. Не просто можем, а прямо сейчас и займёмся.

4.2.2 Добавляем информацию из CFG

Интересной особенностью SSA-графа является тот факт, что он ничего не знает о потоке управления. Это чистый поток данных. Если мы попытаемся воспользоваться идеями, предложенными выше, мы довольно быстро придём к вопросу о том, как сопоставить эти потоки друг с другом. Допустим в программе точно известно, что некое SSA значение сравнения истинно или ложно. Как понять какой из блоков недостижим? И наоборот, если некий блок недостижим — как это повлияет на граф значений?

Алгоритм SCCP (см. листинг 4.8) решает эти проблемы поддержкой не одного, а двух рабочих листов: FlowWorkList и SSAWorkList. Изначально FlowWorkList инициализируется всеми рёбрами, исходящими из entry. SSAWorkList изначально пуст. У каждого ребра графа потока управления есть связанный флаг, F (на листинге это `CfgF`). Он контролирует активное участие ребра в φ -функции блока, в который оно входит. Изначально этот флаг равен 0 для всех рёбер. Каждому SSA значению, про которое не известно, что оно константа или $\perp$, изначально устанавливается значение T .

```

auto LVs = LatticeSet{};
auto SSAWL = Stack{};
auto CfgWL = Stack{entry_edges};
auto CfgF = Map<Edge, Int>{};

void sccp(Function F) {
    initlattice();
    for (auto E : all_edges(F)) CfgF[E] = 0;
    while (!SSAWL.empty() || !CfgWL.empty()) {
        if (!SSAWL.empty()) {
            auto E = SSAWL.pop();
            auto EOUT = {BB(def(E)), BB(use(E))};
            if (CfgF[EOUT] == 0) visitop(use(E));
        }
        else {
            auto E = CFGWL.pop();
            auto B = tip(E);
            CfgF[E] = 1;
            for (auto Op : all_instructions(B))
                visitop(Op);
            auto Es = out_edges(B);
            if (Es.size() == 1 && CfgF[Es.front()] == 0)
                CfgWL.push(Es.front());
        }
    }
}

```

Листинг 4.8: Algorithm SCCP

Алгоритм SCCP работает, пока оба рабочих списка не станут пустыми, причём каждый раз он берёт элемент из любого из рабочих списков. Это довольно слабое условие, и в листинге 4.8 я его немного усилил: если SSA граф не пуст, ребро всегда берётся оттуда. В заданиях вам будет предложено поэкспериментировать с разными стратегиями.

Итак, теперь есть несколько альтернатив. Если выбрано ребро SSA-графа, которое ведёт в другое SSA-выражение, то всё как раньше, а вот если оно ведёт в φ -узел, то посещается φ -функция. Если же выбрано ребро графа потока управления, то, если F для него уже установлен, оно просто пропускается, если нет, то его F устанавливается и посещаются все инструкции блока, в который ведёт это ребро (включая φ -функции).

Сердцем этого алгоритма является подпрограмма `visitor`, которую я выделил на листинг 4.9.

```

void visitop(Instruction Op) {
    if (Op is Phi) {
        auto OldVal = LVs[Op.lhs()];
        auto M = top;
        for (auto U : uses(Op))
            M = meet(M, LV[U]);
        if (M != OldVal) LVs[Op.lhs()] = M;
        propagate(Op, M);
    }
    else if (Op is Terminator) {
        for (auto [E, C] : outgoing(Op))
            if (satisfiable(C))
                CfgWL.push(E);
    }
    else {
        changed = update(Op);
        if (changed)
            for (auto U : users(Op))
                SSAWL.push({Op, U});
    }
}

```

Листинг 4.9: Подпрограмма посещения инструкции для SCCP

Она распадается на три самостоятельных случая. Отдельно обрабатывается ситуация терминатора: тогда мы должны обновить рабочий список новыми разведанными рёбрами потока управления. Мы это делаем так поздно как раз потому, что в процессе продвижения констант вычисленное условие может стать ложным (и при этом инвариантно ложным). В этом случае мы по этой ветке не ходим и как исполняемую её не помечаем. Также отдельно обрабатываются ϕ -узлы, и их обработка в целом похожа на то, что мы уже видели в алгоритме SCP.

Выше переиспользованы также функции `initlattice` и `propagate`, разработанные ранее. В упражнении вам будет предложено восстановить функцию `update`, которая просто обновляет информацию в узле (и тоже пользуется подпрограммой распространения).

Задание 4.5. В алгоритме SCCP на листинге 4.8 есть отдельная обработка случая безусловного перехода. На самом деле достаточно пометить целевой блок как достижимый и передать оставшуюся часть работы в процедуру посещения операции с ли-

стинга 4.9. Это позволит несколько упростить алгоритм. Попробуйте продумать это упрощение и найти какие-то ещё возможности.

Задание 4.6. При посещении операции на листинге 4.9 возможно допущена ошибка и следует добавлять в SSA Users потенциально изменившихся SSA Users, если исследуемая операция — это ϕ -узел. Продумайте этот момент, используя карандаш и бумагу.

Может показаться, что алгоритм SCCP излишне сложен, и можно было бы просто сначала провести удаление мёртвого кода, а потом уже продвижение констант более простым алгоритмом. Проблема в том, что тогда этот процесс пришлось бы зациклить до сходимости, так как при продвижении констант появляется новый мёртвый код, а при удалении нового мёртвого кода улучшается продвижение констант. Процесс сошёлся бы, но пришлось бы раз за разом платить за инициализацию всех структур данных ради весьма небольших изменений. В то же время алгоритм SCCP делает это за один проход.

4.2.3 Другие продвижения информации

Кроме продвижения констант, продвигать можно много другой информации. Рассмотрим несколько искусственный пример промежуточного представления, в котором возможно довольно интересное продвижение копирований (copy propagation).

```
BB0: x0 = arg0;  
      y0 = arg1;  
      goto BB1  
BB1: x1 = PHI(x0, x2)  
      y1 = PHI(y0, y2)  
      if (x1 > y1) goto BB2; else goto BB3;  
BB2: tmp = x1;  
      x2 = y1;  
      y2 = tmp;  
      goto BB1;  
BB3: return x1;
```

Листинг 4.10: NIR для примера продвижения копирований

Эта программа возвращает минимум из двух аргументов. Первым шагом можно продвинуть значение x_1 вместо *tmp* и убрать присваивание *tmp*. Далее копирование может быть продвинуто в ϕ -узел, и базовый блок B_2 оказывается пустым. Тогда его становится довольно просто убрать вообще. Впечатляющий результат приведён на листинге 4.11.

```
BB0: x0 = arg0;  
      y0 = arg1;  
      goto BB1  
BB1: x1 = PHI(x0, y1)  
      y1 = PHI(y0, x1)  
      if (x1 > y1) goto BB1; else goto BB3;  
BB3: return x1;
```

Листинг 4.11: Результат продвижения копирований

В листинге 4.11 использовано то интересное свойство SSA-представления, что все ϕ -узлы исполняются параллельно. Иначе x_1 был бы перетёрт первым присваиванием.

Кроме продвижения копирований, ещё одним интересным продвижением информации по графу является продвижение значений переменных. Это можно рассматривать как своего рода обобщения продвижения констант. При продвижении констант мы точно знаем значения SSA-values на всём пути продвижения. Здесь же мы обходимся частичной информацией.

Говоря более формально, продвижение диапазонов значений (value range propagation, VRP) — это оптимизация, которая пытается оценить диапазон возможных значений переменных в каждой точке их времени жизни и убрать избыточные операции на основании этой информации.

```
if (a < 100 && b < 100) {  
    c = a + b;  
    if (c < 200)  
        foo(c);  
}
```

Листинг 4.12: Пример для продвижения диапазонов

Посмотрите на пример листинга 4.12. В данном случае совершенно очевидно, что проверка для *c* избыточна и код можно упростить, выкинув её.

Следующий вопрос: как же это всё-таки сделать? Но тут как раз нет ничего сложного. Одновременное продвижение любой

информации по SSA графу и по CFG делается практически так же, как продвижение констант в алгоритме SCCP. В заданиях в конце этого раздела вам будет предложено модифицировать алгоритм SCCP для продвижения копирований и для продвижения диапазонов значений.

4.3 Глобальная нумерация значений

SSA-based optimization algorithms do not need to distinguish between global and local optimizations.

– Kenneth Chow

Интересным наблюдением относительно алгоритма SCCP (см. листинг 4.8) является тот факт, что он начинает работу с предположения, что все рёбра не исполняемы, и далее ставит флаги исполняемости на рёбра, которые оказались исполняемыми. Глобальная нумерация значений начинается со столь же нереалистичного предположения, что все инструкции с одним и тем же оператором имеют тот же выходной результат.

В случае обычной нумерации значений в классическом варианте таких предположений не делалось и классы эквивалентности внутри базового блока искались как в листинге 2.20 прямым построением DAG по операциям. Разумеется, это не может работать внутри цикла или при наличии ветвлений, и нужен некий итеративный процесс уточнения значений. И заметим, что вообще-то для такого разбиения требуется SSA-представление, иначе одно и то же значение может попасть в разные классы эквивалентности.

4.3.1 Построение классов конгруэнтности

Итак, все инструкции (а значит и все SSA-значения) разбиваются на пакеты P_i , изначально состоящие из инструкций с общим оператором. Например, все сложения попадают в один пакет, все умножения в другой, все ϕ -функции в третий и т.д. Для каждого неизвестного SSA-значения x (например, аргумента исследуемой функции или результата вызова внешней функции) формируется отдельный пакет P_x , состоящий только из него.

Задание 4.7. Прежде чем читать дальше, попробуйте построить исходное разбиение для листинга 4.13 самостоятельно.

```
int foo(int x, int y) {
    a = x + y;
    b = x - y;
    c = a - b;
    d = x + y;
    return d + c;
}
```

Листинг 4.13: Пример линейного участка до GVN

Две инструкции считаются одинаковыми, если они принадлежат одному пакету, и все их операнды попарно принадлежат одинаковым пакетам. Дополнительно ϕ -функции считаются одинаковыми, если выполнено то, что написано выше, и если они принадлежат к одному базовому блоку.

Никто не мешает применить глобальную нумерацию к линейному участку в SSA представлении. Пакетами на листинге 4.13 изначально будут: $P_+ = (a, d)$, $P_- = (b, c)$, $P_x = (x)$, $P_y = (y)$. При этом у инструкции, порождающей значение c , один операнд лежит в пакете P_+ , а другой в пакете P_- . Для значения b один операнд лежит в P_x а другой в P_y . Если мы обнаружили что две инструкции, лежащие в одном пакете, не эквивалентны, пакет надо разбить по классам эквивалентности. После первой итерации имеем $P_+ = (a, d)$, $P_-^1 = (b)$, $P_-^2 = (c)$, $P_x = (x)$, $P_y = (y)$. Точно по такой же логике значения a и d эквивалентны. Это заканчивает разбиение.

Если в конце разбиения две операции эквивалентны, и одна доминирует над другой, то второе значение избыточно и может быть заменено на первое.

```
int foo(int x, int y) {
    a = x + y;
    b = x - y;
    c = a - b;
    return a + c;
}
```

Листинг 4.14: Пример линейного участка после GVN

На листинге 4.14 замена d на a произведена в том числе во всех инструкциях, которые используют d , а инструкция, соответствующая d , убрана.

Задание 4.8. *Примените к тому же линейному участку обычную нумерацию значений (см. листинг 2.20).*

Единственная тонкость здесь — как именно делать разбиение пакета. Первый, не слишком эффективный, алгоритм представлен на листинге 4.15. В нём делается очевидная вещь: мы берём инструкцию и сразу создаём для неё новый пакет. Далее сравниваем её со всеми инструкциями в её пакете. Если какая-то эквивалентна ей, то мы помещаем её в новый пакет, если нет — оставляем в старом. Если старый пакет в итоге не опустел, значит нам нужно будет сделать ещё одну итерацию.

```
Partition split(Partition P) {
    auto Changed = false;
    do {
        auto PN = Partition{};
        for (auto Packet : P) {
            auto I = Packet.pop();
            auto PacketI = Packet{}, PacketNonI = Packet{};
            PacketI.put(I);
            while(!Packet.empty()) {
                auto J = Packet.pop();
                match(I, J) ? PacketI.put(J) :
                           PacketNonI.put(J);
            }
            PN.push(PacketI);
            if (!PacketNonI.empty()) {
                PN.push(PacketNonI);
                Changed = true;
            }
        }
        P = PN;
    } while (Changed);
    return P;
}
```

Листинг 4.15: Простое неэффективное разбиение

Проследим работу этого алгоритма на более сложной программе, в которой есть нетривиальный поток управления (см. листинг 4.16).

Задание 4.9. *Перед тем как читать дальше, попробуйте сделать это самостоятельно!*

```
int foo(int x1, int y1) {
    BB0: a1 = 1;
        b1 = 1;
        goto BB1;
```

```

BB1: x2 = PHI(x1, x3);
      a2 = PHI(a1, a3);
      b2 = PHI(b1, b3);
      x3 = x2 + y1;
      a3 = a2 + x3;
      b3 = b2 + x3;
      if (x3 > 0) goto BB1; else goto BB2;
BB2: x4 = x3 + b3;
      return x4;
}

```

Листинг 4.16: Пример работы GVN над циклом

Сформируем пакеты $P_1 = (x_1)$, $P_2 = (y_1)$, $P_3 = (x_3, a_3, b_3, x_4)$, $P_4 = (x_2, a_2, b_2)$, $P_5 = (a_1, b_1)$. Напомним: в пакеты для простоты включаются только SSA values, потому что они однозначно определяют инструкции.

Далее проверим пакет P_3 . В нём a_3 и b_3 имеют первый аргумент из пакета P_4 , а второй одинаковый, и поэтому они конгруэнтны. Переменные x_3 и x_4 отделяем в $P_6 = (x_3)$ и $P_7 = (x_4)$. По тем же соображениям в пакете P_4 остаются только две инструкции $P_4 = (a_2, b_2)$, тогда как x_2 отделяется в пакет $P_8 = (x_2)$.

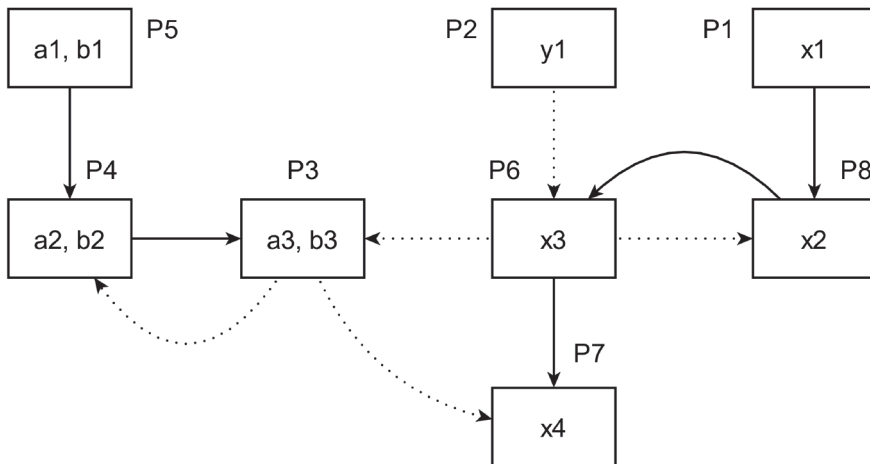


Рис. 4.2: Граф конгруэнций для цикла

Итоговые конгруэнтности программы с листинга 4.16 изображены на рисунке 4.2. На этом рисунке сплошная стрелка означает источник первого аргумента, пунктирная — источник второго. Схлопывая конгруэнтные инструкции и заменяя их на доминирующие, мы получаем упрощённый код на листинге 4.17.

```

int foo(int x1, int y1) {
BB0: a1 = 1;
    goto BB1;
BB1: x2 = PHI(x1, x3);
    a2 = PHI(a1, a3);
    x3 = x2 + y1;
    a3 = a2 + x3;
    if (x3 > 0) goto BB1; else goto BB2;
BB2: x4 = x3 + a3;
    return x4;
}

```

Листинг 4.17: Цикл после GVN

Задание 4.10. *Оцените асимптотическую эффективность алгоритма, предложенного на листинге 4.15. Можете ли вы предложить лучшие?*

4.3.2 Улучшение эффективности разбиения

Более эффективный алгоритм был предложен Вегманом и прочими [4] на основании работы Ахо, Ульмана и Хопкрофта [1]. Этот алгоритм был реализован в компиляторе GCC в 2004-м году (см. замеры и детали реализации в диссертации [36]). Позже эта реализация также была перенесена в LLVM, и она работает без существенных доработок по сей день.

Идея (восходящая к Хопкрофту) заключается в том, чтобы не разбивать базовый блок по одной инструкции, а, наоборот, разбивать одним пакетом другие пакеты. Рассмотрим в каждой инструкции аргумент с индексом i и построим множество $INV_i(P_k)$, состоящее из всех инструкций с i -м операндом из k -го пакета. Далее, для каждого пакета, если он пересекается с обратным пакетом данного блока по определённому аргументу и не является его подмножеством, то его следует разбивать на подмножество пакета и разность с ним.

Эта изящная идея реализована в листинге 4.18.

```

Partition split(Partition P) {
    auto WL = indexes(P); // 0, 1, ..., N
    while (!WL.empty()) {
        auto I = WL.pop();
        auto Packet = P[I];
        for (K : [1, 2]) { // assume 3-address code

```

```

auto INV = inverse(Packet, K);
for (auto PX : P) {
    if (!intersect(PX, INV) || is_subset(PX, INV))
        continue;
    auto P1 = set_intersection(PX, INV);
    auto P2 = set_difference(PX, INV);
    P.remove(PX);
    P.insert(P1, P2);
    WL.push(index(P, smallest(P1, P2)));
}
}
}
return P;
}

```

Листинг 4.18: Алгоритм GVN

Особый теоретический интерес вызывает вопрос: «почему из двух получившихся блоков мы добавляем в рабочий список индекс наименьшего?». Обоснование этого можно почитать в оригинальном алгоритме разбиения Хопкрофта. Интересно, что Хопкрофт использовал его для минимизации конечных автоматов — удивительные сближения из, казалось бы, совсем не связанных областей.

Задание 4.11. Реализуйте функцию `inverse`, которая подсчитывает для k -го аргумента пакета P множество $INV_k(P)$.

В упражнениях вам будет предложено доказать корректность алгоритма GVN и проанализировать его алгоритмическую сложность. Но уже сейчас можно сказать, что этот алгоритм получается достаточно хорош для практических применений. К сожалению, у него есть некоторые естественные ограничения, которые мешают объявить его универсальным средством борьбы с избыточностью в том виде, в котором я пока что его сформулировал.

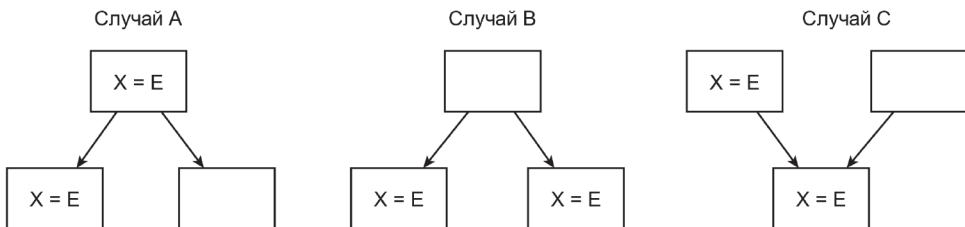


Рис. 4.3: Естественные ограничения GVN

Глобальная нумерация значений эффективно справляется с ситуацией А на рисунке 4.3 и со многими другими аналогичными ситуациями, когда избыточные вычисления выносятся в инструкцию-доминатор. Увы, она не справляется с ситуацией Б с того же рисунка, то есть не способна вынести общее подвыражение в общий доминатор. Для этого нам придётся изучить совершенно иной алгоритм.

4.4 Устранение частичной избыточности

Несмотря на то, что в названии этого раздела речь идёт про частичную избыточность, конечно, если мы умеем устранять частичную, мы устраним и полную. При этом надо понимать, что на пути устранения именно частичной избыточности мы входим на зыбкую почву, когда трансформация теоретически может как улучшить, так и несколько ухудшить результат. Тем не менее классический алгоритм, который мы здесь рассмотрим, настолько красив и полезен, что во всех известных мне компиляторах он используется по умолчанию на должном уровне оптимизации.

Основная задача, как вы уже поняли, состоит в том, чтобы сделать что-то со случаем Б с рисунка 4.3, где показана полная избыточность и с аналогичными случаями частичной избыточности, когда вычисление избыточно не по всем путям. Второй класс случаев представляет собой гораздо более опасную ситуацию — будучи вынесено в общий доминатор вычисление может попасть на те пути, где его раньше не было. И если один из этих путей окажется горячим, то мы проиграли.

Очень долгое время лучшим алгоритмом для решения задач такого рода считался алгоритм, основанный на уравнениях анализа потока данных. Его исходную версию предложили Морел и Ренвойз [42], и он далее улучшался многими исследователями. Неплохой обзор литературы можно найти в работе Бриггса [6]. Даже после возникновения SSA формы очень долгое время удаление частичной избыточности (PRE, partial redundancy elimination) держалось как скала — считалось, что SSA тут ничего не может дать.

Только в 1999-м году появилась публикация, предлагающая исключительно эффективный алгоритм удаления частичной избыточности в SSA представлении (см. [31]). Именно предложенный там алгоритм ниже будет рассмотрен в качестве алгоритма SSAPRE.

Понимать этот алгоритм лучше по частям. Для начала мне придётся ввести понятие критического ребра и разбиения критического ребра. Это поможет не только здесь, но и при разрушении SSA и много где ещё. Далее мы должны будем ввести понятие синтаксической избыточности. В отличие от алгоритма GVN (см. листинг 4.18), который удалял избыточность, основываясь на значениях, SSAPRE будет удалять избыточность, основываясь на выражениях (см. раздел 4.4.2). По синтаксической избыточности придётся построить факторизованный граф и на нём — нечто вроде отдельного SSA представления для избыточности. А далее над этим представлением провести ещё ряд анализов чтобы решить в какие блоки безопасно двигать код.

Итак, давайте начнём с критических рёбер. К слову, это довольно частый вопрос на всякого рода собеседованиях в компиляторные команды.

4.4.1 Разбиение критического ребра

Термин «критическое ребро» в математике и программировании неприятно перегружен значениями. Критическими называют рёбра в сетях, такие, что уменьшение ёмкости этого ребра уменьшает ёмкость всей сети. Критическими называют рёбра в графах, если удаление такого ребра уменьшает хроматическое число графа. Во всех случаях речь идёт о чём-то, избавление от чего создаёт новую ситуацию или даёт новые возможности.

Определение критического ребра, которое интересует нас с точки зрения компилятора и которое будет использоваться везде далее, дано ниже.

Определение 4.1. *Критическим называется ребро, которое идёт из блока с более чем одним потомком в блок с более чем одним предком.*

Интуитивно речь тут идёт о таком ребре, которое мешает нам перемещать вычисления между базовыми блоками. Сама идея прослеживается ещё в ранней работе Розена и Вегмана о GVN [51], но там не идёт речь о критических рёбрах — они просто предлагают создавать базовые блоки, чтобы безопасно выносить туда код. Судя по всему, впервые сам термин появил-

ся в таком виде в работе Бриггса [7] в связи с проблемой потери копий при распространении копирований по SSA-графу.

Проще всего понять это определение, посмотрев на рисунок 4.4.

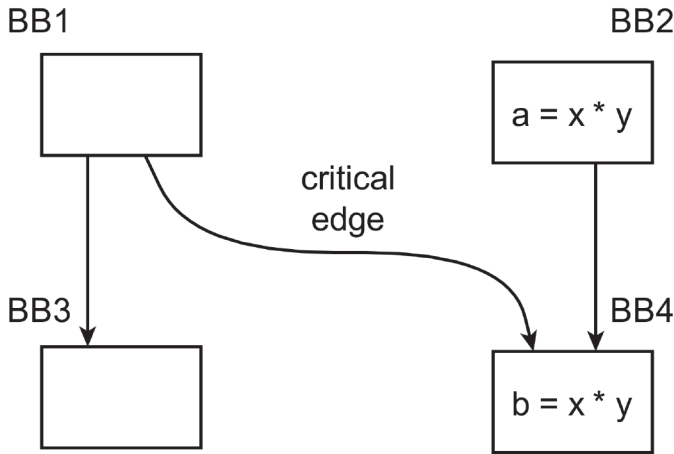


Рис. 4.4: Критическое ребро в графе

Что мы видим? Вычисление b в блоке B_4 является частично избыточным. Но попытка вынести это вычисление в блок B_1 создаст это вычисление на пути к B_3 , а его раньше там не было. В этом смысле критическое ребро является помехой не только для удаления избыточности, но и для того, чтобы просто вставить какие-нибудь вычисления вдоль этого ребра. Это нам очень понадобится, когда мы будем разрушать SSA-представление в разделе 7.2.1, и я даже там попрошу вас вернуться и перечитать этот раздел, если вы сейчас читаете его недостаточно внимательно.

Почему критическое ребро нам так сильно мешает? Потому что вставка вычислений в его начало и в его конец совершенно не симметрично себя ведёт. Вставка в начало распространяется на лишние базовые блоки и ухудшает код.

Чтобы избавиться от критических рёбер, их принято **разбивать**, то есть вставлять в такие рёбра дополнительные базовые блоки, как показано на рисунке 4.5.

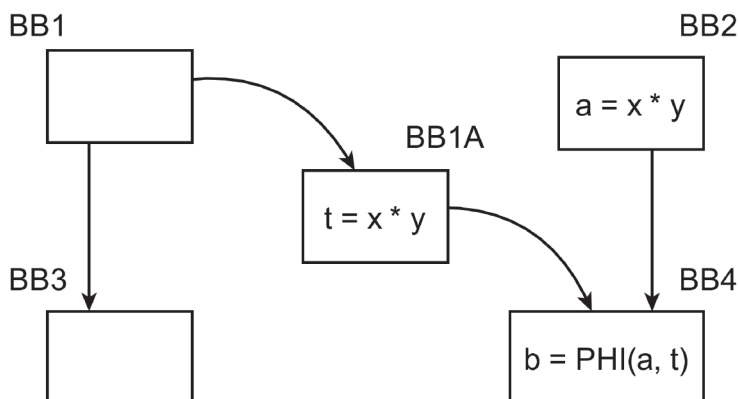


Рис. 4.5: Разбиение критического ребра

Эти дополнительные базовые блоки — это и есть те самые базовые блоки, в которые Розен предлагал выносить код. Здесь мы дополнительно вставляем блок B_{1A} , и теперь у нас нет критических рёбер, и ничто не мешает удалять частичную избыточность.

Задание 4.12. Предложите алгоритм поиска и разбиения критических рёбер по графу потока управления. Можете обратиться к SSA Book [49] или любой другой статье, где он описан. Но он действительно несложен, так что можете попробовать переизобрести его самостоятельно.

Предположим, все критические рёбра разбиты. Что происходит дальше?

4.4.2 Граф избыточных выражений

Как уже было сказано, алгоритм SSAPRE ориентируется в первую очередь на удаление синтаксически-определённой избыточности. Эта фраза буквально означает, что для программы в SSA представлении мы считаем кандидатами на избыточность все вычисления, которые синтаксически выглядят одинаково.

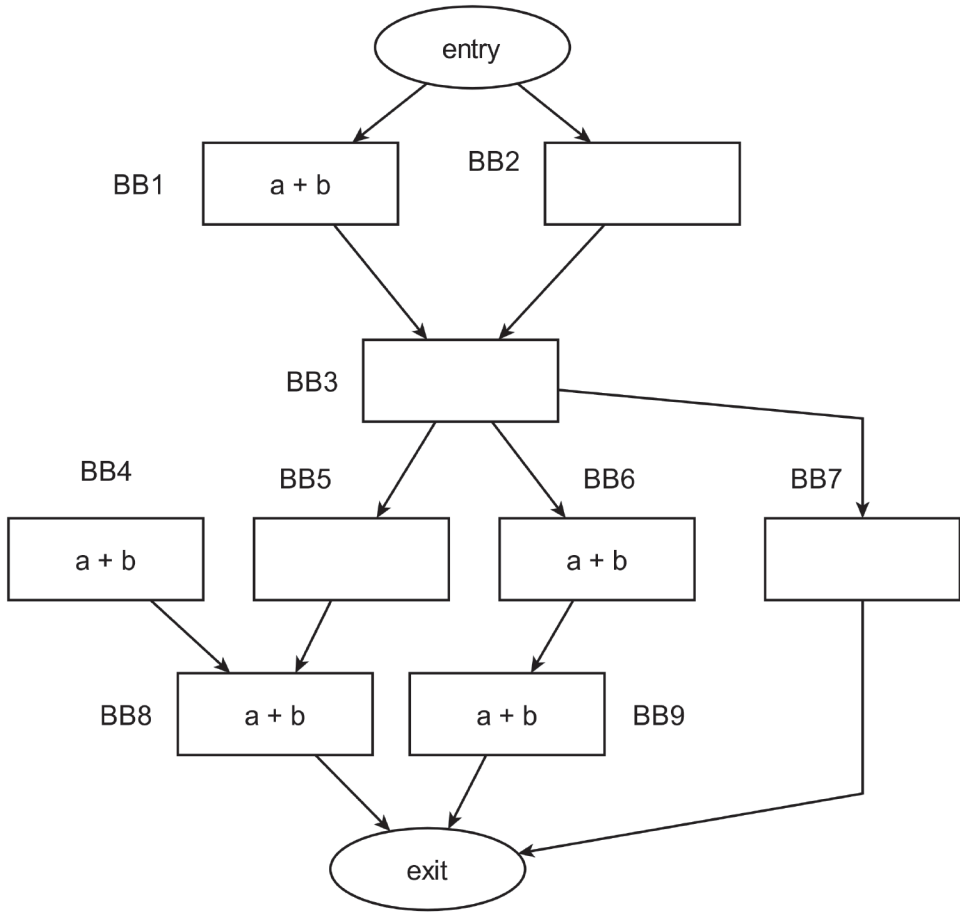


Рис. 4.6: Пример синтаксической избыточности

Чтобы понять смысл термина «синтаксическая избыточность» давайте внимательно посмотрим на рисунок 4.6. Это поток управления, который уже находится в SSA представлении. В блоках B_1 , B_4 , B_6 , B_8 , и B_9 вычисляется синтаксически одинаковое выражение вида $a + b$, которое в силу свойств SSA везде является одним и тем же вычислением, и является замечательным кандидатом на то, чтобы где-то быть избыточным. Обратите внимание на то, что критических рёбер в этом графе уже нет. Такие блоки как B_5 наводят на мысль, что они как раз и были вставлены при разбиении критических рёбер.

В оригинальном алгоритме в статье [31] авторы предлагают считать кандидатами на избыточность выражения, независимо от их SSA версий. Например, $a_1 + b_1$ и $a_3 + b_2$, которые по внеш-

нему виду разные, вполне могут значить одно и то же (обычно это бывает если из программы не убраны лишние копирования). В заданиях в конце главы вам будет предложено распространить SSAPRE на такие случаи и проанализировать полезность такого распространения по сравнению с простым сору propagation до PRE.

Задание 4.13. *Посмотрите действительно внимательно на рисунок 4.6 и попробуйте вручную максимально убрать частичную избыточность и не создать никакой новой. Это не так просто даже для такого простого графа. Как вы будете подходить к этой задаче?*

Заметьте: я вообще не задавался вопросом как именно и сколько раз выражение вычислено внутри базового блока. Меня интересует сам факт, что оно как-то вычислено. Базовый блок B_1 может выглядеть как на листинге 4.19 или как угодно иначе.

```
BB1: x1 = a * b  
      x2 = a + b;  
      x3 = x2 - x1;  
      x5 = x3 / x2;  
      goto BB2;
```

Листинг 4.19: Пример возможного линейного участка BB1

Теперь давайте я проделаю над рисунком 4.6 следующее: оставлю в графе только блоки с интересующим нас избыточным выражением и введу дополнительные переменные для этой избыточности.

Граф, изображённый на рисунке 4.7, называется приведённым графом избыточности (FRG, factored redundancy graph). Чтобы не тратить места на страницах этой не такой уж и толстой книги, я нарисовал его сразу с построенной SSA. Конечно, в реальности это делается в два этапа: сначала вводится модифицируемая переменная h , которая назначается всюду как результат интересующего нас избыточного выражения, а потом для неё строится SSA со вставкой ϕ -узлов по алгоритму 3.12.

Далее тезис такой: только те базовые блоки, в которых оказались ϕ -узлы, будут нами рассматриваться для возможной вставки по входящим рёбрам вычисления, которое и позволит далее удалить избыточность. При этом мы не можем сделать вставку во

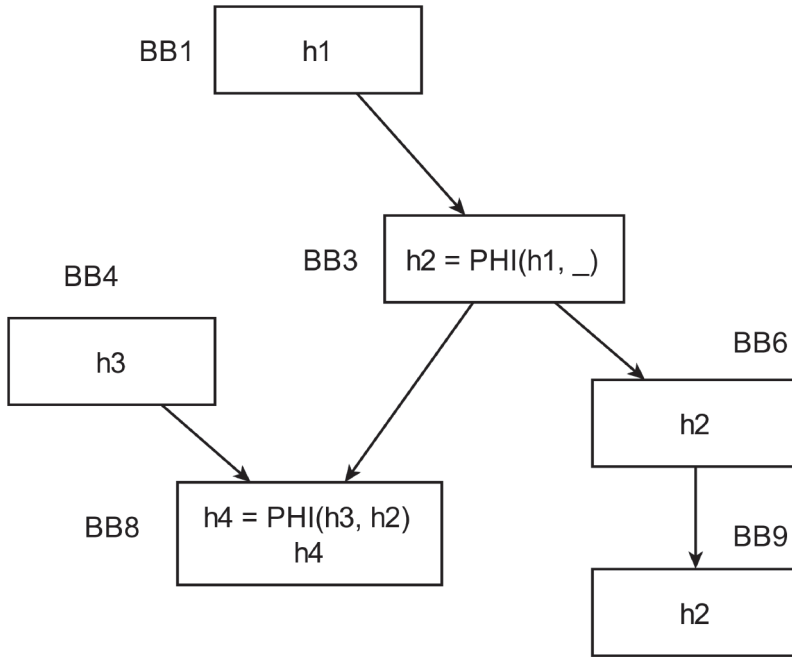


Рис. 4.7: Приведённый граф избыточности

все такие блоки: некоторые из них не безопасны, а в некоторые вставка не требуется. И вот собственно это решение и составляет сердце алгоритма.

4.4.3 Алгоритм SSAPRE

Итак, вставки нужно рассматривать только для φ -узлов. Определим φ как небезопасный (не полностью предсказуемый), если существует путь управления от данного φ , вдоль которого выражение не вычисляется до завершения программы. Это определение транзитивно: либо существует прямой путь к завершению программы, вдоль которого результат φ не используется, либо его значение может стать операндом другого φ , который также не является безопасным.

Таким образом, задача анализа безопасности вставки сводится к типичной задаче обратного распространения по потоку данных на графе избыточности. Алгоритм DS, осуществляющий этот анализ, приведён на листинге 4.20.

```

auto DownSafe = Map<PHI, bool>;

void reset_downsafe(PHI Def) {
    if (!DownSafe[Def]) return;
    DownSafe[Def] = false;
    for (Op : Operands(Def))
        if (no_real_use(Op) && def(Op) is Phi)
            reset_downsafe(def(Op))
}

void downsafe(Graph FRG) {
    for (auto F : all_phis(FRG)) {
        DownSafe.insert(F, true);
        auto BB = basic_block(F);
        if (path_no_use_exist(BB, exit, F))
            DownSafe[F] = false;
    }
    for (auto F : all_phis(FRG)) {
        if (!DownSafe[F])
            for (Op : Operands(F))
                if (no_real_use(Op) && def(Op) is Phi)
                    reset_downsafe(def(Op))
    }
}

```

Листинг 4.20: Алгоритм DS: анализ безопасности вставки

Функция `path_no_use_exist` проверяет существует ли от данного базового блока до завершения программы хотя бы один путь, по которому нет ни одного использования данного SSA значения. На самом деле путей может быть очень много, но во-все не обязательно перебирать их все — достаточно выкинуть из подграфа от текущего блока до выхода все те блоки, в которых значение используется, и посмотреть, остался ли там один любой путь. Если остался, значит, он существует и блокирует безопасность вставки.

Задание 4.14. *Напишите реализацию функции `path_no_use_exist` в простом псевдокоде. Можете ли вы предложить, как сделать её более оптимальной?*

Функция `no_real_use` в этом алгоритме также представляет определённый интерес. Она возвращает истину в том случае, если у значения нет никаких использований в одном из рассма-

тривиальных избыточных выражений, то есть все пути в факторизованном графе избыточности ведут в ϕ -узлы. На самом деле все значения этой функции могут быть простым способом закешированы ещё в момент построения дерева доминаторов.

Задание 4.15. Напишите реализацию функции `no_real_use`. К слову, это хорошее упражнение в оперировании SSA представлением.

В конце алгоритма DS мы имеем разметку всех узлов, в которые вставка не безопасна. Во все остальные узлы её можно производить. Но довольно-таки бессмысленно проводить её по тем веткам, по которым эта вставка, наоборот, создаст избыточность.

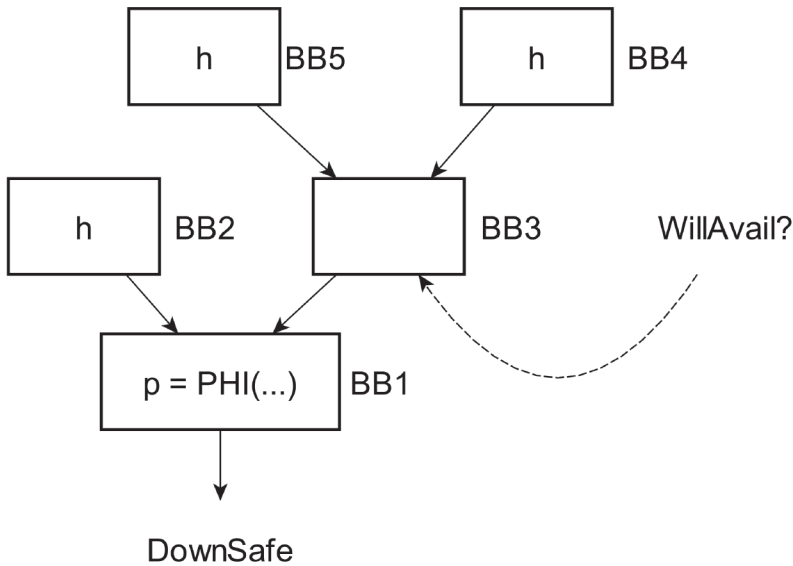


Рис. 4.8: Демонстрация возможно излишней вставки

В качестве примера можно рассмотреть рисунок 4.8. Допустим, что вставка для B_1 безопасна. В блок B_2 вставка не нужна. Нужна ли она в B_3 ? И, разумеется, с учётом приведённого графа, на этот вопрос следует ответить отрицательно.

Как мы могли бы подойти к задаче написания функции проверки доступности (её можно назвать `willavail`), которая размечает блоки, в которых вставка не нужна, так как искомая переменная и так будет в них доступна?

Здесь можно заметить, что эта функция сводится к продвижению информации по FRG. Мы начинаем с признака доступности, который устанавливается в истину на всех φ -узлах, кроме тех, которые помечены как не являющиеся безопасными, и тех, у которых в φ -функции отсутствует один из аргументов (примерно как в функции для h_2 в блоке B_3 на рис. 4.7). На таких узлах он выставляется в ложь. Далее ложное значение распространяется по рёбрам FRG так, что в итоге признак доступности становится ложным у всех φ -узлов, для которых ни одна безопасная вставка не может сделать выражение доступным.

Позиции, помеченные как доступные для вставки, обозначают места в программе, где вставка возможна сейчас или станет доступна после этапа финализации.

Задание 4.16. *Напишите псевдокод для функции `willavail`.*

Алгоритм SSAPRE целиком показан на листинге 4.21. Благодаря разделению на функции, он выглядит очень просто, но каждая функция таит в себе массу возможностей к исследованию и улучшению. Например, функцию `willavail` можно оптимизировать, постаравшись разметить позиции доступности таким образом, чтобы итоговые интервалы жизни тех позиций, где мы всё-таки решили делать вставки, были несколько короче.

```
split_critical_edges();  
auto FRG = build_factored_redundancy_graph();  
insert_phis(FRG);  
downsafe(FRG);  
willavail(FRG);  
finalize(FRG);
```

Листинг 4.21: Алгоритм SSAPRE: сводим всё воедино

Функция `finalize` единственная, которая ещё не была мной рассмотрена. Этот шаг алгоритма удаляет лишние φ -узлы и зависящие от них переменные, вынося частичную избыточность по всем веткам, которые не помечены как те, в которых она и так будет доступна.

Этап финализации является идеологически довольно простым, но технически довольно сложным и деликатным, там можно легко сделать серьёзные ошибки. Поэтому я здесь удержусь от приведения псевдокода, который будет скорее вводить

в заблуждение. Но основная идея проста и понятна: в итоге работы алгоритма SSAPRE граф с рисунка 4.6 должен трансформироваться в показанный на рисунке 4.9, и весь анализ для этого готов.

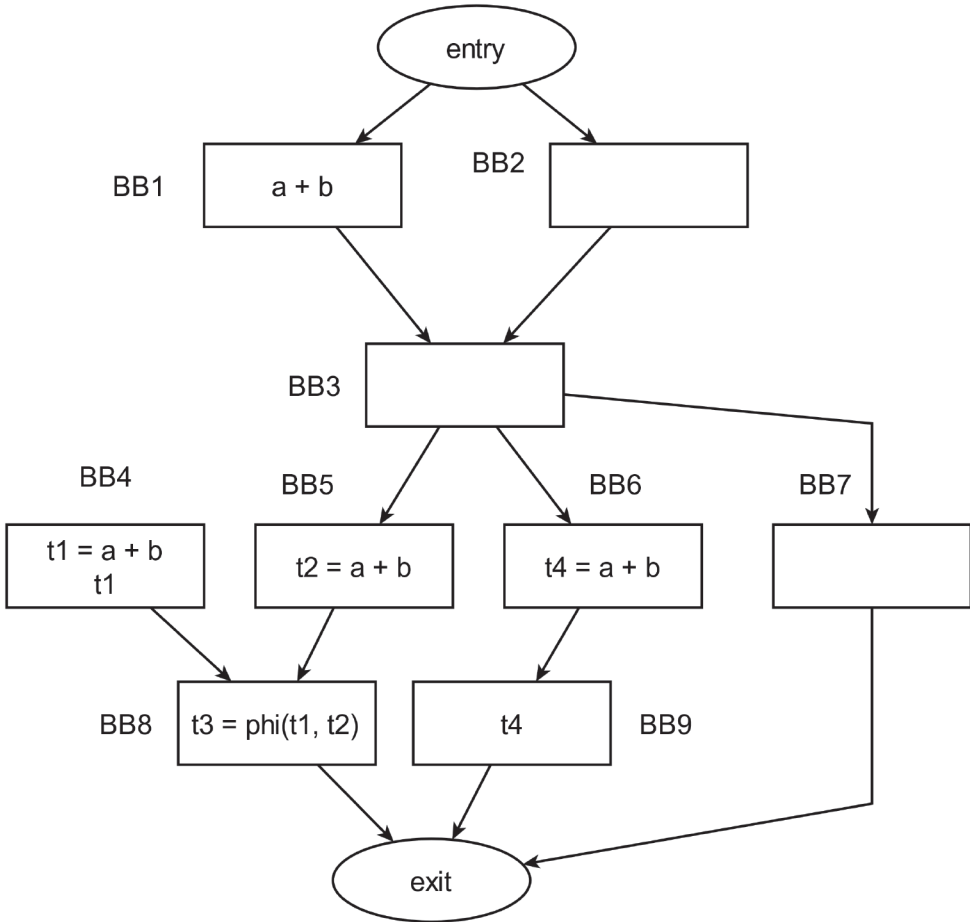


Рис. 4.9: После устранения частичной избыточности

На первом этапе финализации в те узлы, где вставка безопасна, она должна быть сделана по всем входящим рёбрам, кроме тех, по которым значения уже доступны. Этот шаг сопровождается удалением ненужных более φ -узлов для избыточности и возможной вставкой настоящих φ -узлов для схождения временных переменных (см. t_2 в блоке B_8). Далее все избыточности, которые могут быть заменены на использование предвычисленной временной переменной, заменяются на него.

4.5 Задачи на самостоятельную проработку

Great opportunities almost never fit your schedule.
Reid Hoffman

Задание 4.17. Продолжая задачу 2.25, добавьте в разработанный там алгоритм копирование графа зависимостей SSA вашей функции.

Задание 4.18. Оцените асимптотическую сложность алгоритма SCP с листинга 4.5 и сравните со сложностью алгоритма CP с листинга 2.9.

Задание 4.19. Реализуйте функцию `update` для алгоритма SCCP с листинга 4.8.

Задание 4.20. Оцените асимптотическую сложность алгоритма SCCP.

Задание 4.21. Для алгоритма SCCP не хватает финального штриха — обработки и удаления из ф-узлов неисполняемых рёбер (а возможно и удаления целых неисполняемых блоков). Он только помечает их как не исполняемые, но не удаляет. Напишите алгоритм DCE (*dead code elimination*) для удаления по проведённому анализу.

Задание 4.22. В реализации алгоритма SCCP я несколько усилил условие выбора ребра. В оригинальной формулировке всё равно из какого из двух рабочих листов брать ребро. Проанализируйте — правильно ли я поступил, и можете ли вы предложить лучшую и более эффективную стратегию. Нарисуйте худшие случаи для каждой возможной краевой стратегии: из одного, из другого, по очереди и т.п.

Задание 4.23. Исследуя идеи алгоритма SCCP, попробуйте написать похожий на него алгоритм *copy propagation* (см. пример на листинге 4.10).

Задание 4.24. Обобщая SCCP, напишите реализацию алгоритма *value range propagation* см. пример на листинге 4.12.

Задание 4.25. Оцените асимптотическую сложность алгоритма GVN с листинга 4.18. Докажите сходимость этого алгоритма. Какой случай входных данных вы бы назвали для него наихудшим?

Задание 4.26. Сгенерируйте всевозможные случайные направленные графы для N вершин (выберите для начала $N = 5$) и проанализируйте, у какого процента из них появляются критические рёбра (и сколько таких рёбер). Понемногу увеличивайте N и посмотрите динамику. Попробуйте объяснить результаты математически.

Глава 5

ЦИКЛОВЫЕ ОПТИМИЗАЦИИ

*Programmers waste enormous amounts of time
thinking about, or worrying about,
the speed of noncritical parts of their programs.*
– Donald Knuth

В основном все компьютеры в мире заняты исполнением циклов. Три составных части вычислительной полноты любого языка — это циклы, ветвления и массивы. Можно сказать, что массивы занимают место, циклы занимают время, а ветвления делают программу нетривиальной. Именно на циклах мы концентрируемся, пытаясь оценить вычислительную сложность программы. Ну и наконец именно в них мы зависаем навсегда, если уж где-нибудь зависаем.

Даже незначительные улучшения в очень часто исполняемом цикле способны сэкономить буквально часы и годы машинного времени. При этом такого рода улучшениями, разумеется, не должен заниматься программист. Циклы возникают в программах естественно и выражают простую идею «сделать что-то несколько раз». Задача программиста — написать их достаточно ясно и просто. Всё остальное делают компиляторы.

В качестве примера рассмотрим простой цикл, приведённый на листинге 5.1. Это функция, которая ищет позицию элемента в массиве. На вход приходят два указателя (на начало массива и на позицию за его концом) и значение искомого элемента. В случае неудачи возвращается численное значение позиции сразу за концом массива.

```
int find_pos(int *start, int *fin, int elt) {
    for (int *p = start; p != fin; ++p)
        if (*p == elt)
            return (p - start);
    return (fin - start);
}
```

Листинг 5.1: Поиск позиции элемента: исходный код

Если достаточно наивно переложить эту программу на HIR, получится код, приведённый на листинге 5.2.

```
int find_pos(int *start, int *fin, int elt) {
    BB0: p1 = start;
        if (p1 != fin) goto BB1; else goto BB4;
    BB1: p2 = PHI(p3, p1);
        if (*p2 == elt) goto BB3 else goto BB2;
    BB2: p3 = p2 + 1;
        if (p3 != fin) goto BB1; else goto BB4;
    BB3: t1 = (p2 - start);
        return t1;
    BB4: t2 = (fin - start);
        return t2;
}
```

Листинг 5.2: Поиск позиции элемента: HIR

Казалось бы, что тут можно изменить или оптимизировать?

Задание 5.1. *Перед тем как читать дальше, подумайте, как бы вы изменили этот цикл и почему?*

Тем интересней, что на достаточно высоком уровне оптимизаций, разные компиляторы (я взял GCC 13 и clang 17) здесь порождают структурно совершенно разное промежуточное представление (и, конечно, разный ассемблер). На рисунке 5.1 я показываю получившийся поток управления для разных компиляторов. Чтобы происходящее было легче осознать, я аннотирую его ассемблером.

В листингах ассемблера, приведённых рядом с графами потока управления, в `rdi` хранится `start`, в `rsi` хранится `fin`, в `rdx` хранится `elt`. Вы можете освежить в памяти раздел 1.3.1, если концепция ABI немного забылась.

Глядя на получившийся ассемблер, можно увидеть, что компиляторы переписали цикл очень существенно. Если записать то,

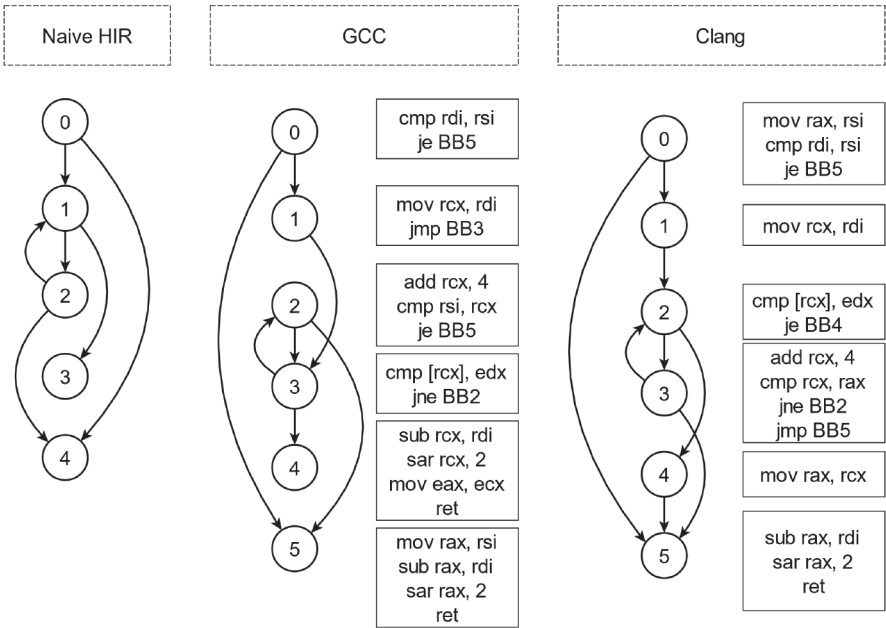


Рис. 5.1: Работа компиляторов над простым циклом

что получилось у GCC, обратно на C, то получится нечто вроде кода, приведённого на листинге 5.3.

```
int find_pos(int *start, int *fin, int elt) {
    if (start != fin) {
        int *p = start;
        do {
            if (*p++ == elt)
                return (p - start);
        } while (p != fin);
    }
    return (fin - start);
}
```

Листинг 5.3: Поиск позиции элемента: версия компилятора

Компилятор считает, что код на листинге 5.3 будет работать эффективнее, чем код на листинге 5.1. С высокой вероятностью он в этом прав. Но никакой программист всерьёз не будет такое писать в промышленном коде. Это нечеловеческий код: в нём неясная логика, он сложно читается, в него сложно вносить изменения. Поэтому цикловые оптимизации очень полезны — они

позволяют делать из простого и понятного кода сложный и эффективный незаметно для разработчика кода.

Задание 5.2. *Посмотрите, что выдаёт ваш компилятор.*

В этой главе мы пройдем через цикловые оптимизации и посмотрим, что делают компиляторы с циклами. Если вы думаете, что пример на рисунке 5.1 сильно модифицирован, то вас ждёт много нового — мы увидим такой код, который вообще перестаёт быть узнаваемым после оптимизаций.

Также не стоит думать, что мы закрываем глаза, поднимаем руки и позволяем компилятору делать вещи, как ему заблагорассудится. В разделе 5.4.5 мы, изучив основные оптимизации над циклами, изучим также инструментарий прикладного программиста, с помощью которого он может повлиять на их качество и эффективность.

5.1 Естественные циклы

В русском языке существует неприятная омонимия: слова цикл (cycle, в смысле цикла в ориентированном графе, например, петля в связном списке) и цикл (loop, в смысле цикла в программе) звучат одинаково. По-английски всё проще: cycle и loop — это просто разные термины, и размежевание между ними происходит вполне естественно.

К сожалению, попытка употреблять англицизмы, например вместо «циклы» говорить «лупы», во-первых, приводит к конфликтам с увеличительными стёклами, а во-вторых, к неблагозвучию, если, например, мы за них всерьёз возьмёмся.

Поэтому здесь и везде далее, когда я говорю про циклы, я буду иметь в виду циклы в программе. Если мне придётся говорить про циклы в графах, я буду выкручиваться, например, называя их замкнутыми путями. «Граф называется двудольным, если не имеет замкнутых путей нечётной длины» — звучит неплохо и мы спасли слово «цикл» для программирования. С другой стороны, для (направленных) ациклических графов я буду продолжать использовать слово «ациклический», благо оно в программировании не встречается, а «граф без замкнутых путей» звучит странно и слишком вычурно.

Но спасти слово для программирования — это полдела. Посмотрите на рисунок 5.2, показывающий несколько возможных графов потока управления в программе. На нём сразу видна

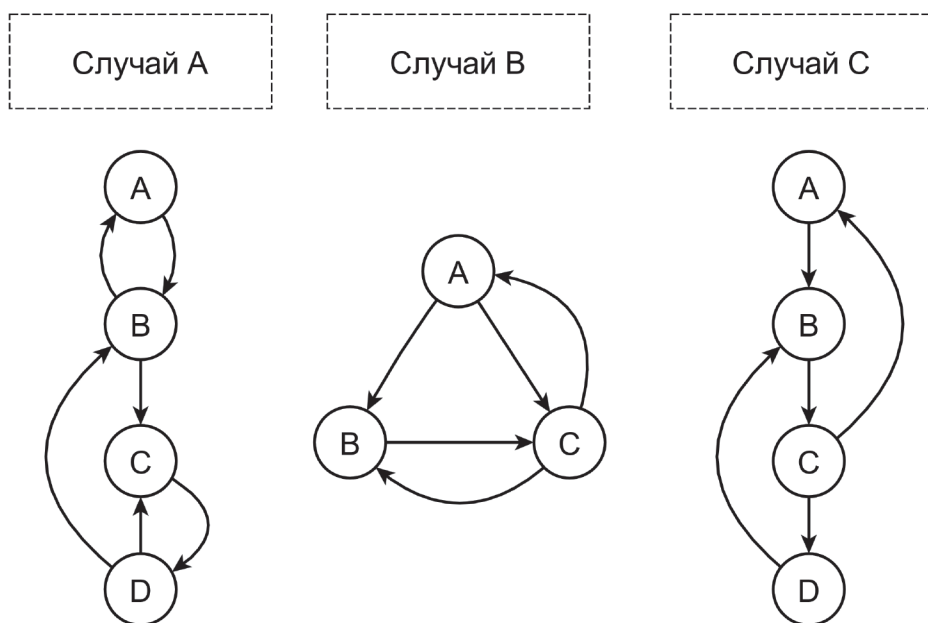


Рис. 5.2: Три случая для поиска циклов

проблема. Пожалуйста, подумайте над задачей 5.3, прежде чем читать далее.

Задание 5.3. *Какие компоненты в графах с рисунка 5.2 вы назвали бы циклом в вашем любимом языке программирования? Можете ли вы написать соответствующие программы?*

Интуитивно для нас цикл — это возвращение назад по потоку управления программы, и это в чём-то верно. Но за годы торжества структурного программирования, все привыкли, что это возвращение само по себе структурно. То есть циклы и ветвления правильно вложены, нет внезапных прыжков изнутри одного цикла внутрь другого и так далее. На уровне написания кода это, как правило, действительно так. Но кто может знать, что будет с вашей программой после её реструктурирования оптимизирующим компилятором?

Можно было бы рубануть с плеча и назвать обобщённым циклом вообще любой сильно связанный компонент в графе. Это допустимо, но, говоря так, мы говорим слишком много. Подумайте, например, о сложности алгоритма выноса инварианта из произвольного сильно связного компонента. Или о попытке определить в произвольном сильно связном компоненте индуктивную переменную. Кажется, для цикла нам нужно какое-то

понятие, немного лучше бьющееся с программистской интуицией на тему цикла.

Чтобы формально ввести такое не слишком общее, но более естественное понятие цикла, придётся обратиться к информации о доминировании из раздела 3.2 и строго определить понятие обратной дуги.

Определение 5.1. *Обратной дугой (back edge) называется такая дуга $m \rightarrow n$, что $n \in \text{Dom}(m)$.*

Задание 5.4. *Перечислите все обратные дуги в смысле определения 5.1 на рисунке 5.2.*

Теперь можно ввести понятие естественного цикла.

Определение 5.2. *Естественным циклом (natural loop) называется такой подграф потока управления, в котором $m \rightarrow n$ — это обратная дуга и который состоит из всех узлов, из которых достижима вершина m , если стереть n .*

Задание 5.5. *Найдите на рисунке 5.2 все естественные циклы.*

Если все сильно связанные компоненты в графе потока управления являются естественными циклами, такой граф потока управления называется сводимым. Сводимые графы обладают рядом приятных свойств, и в некоторых случаях мы можем даже применять специальные техники, чтобы создавать сводимые графы из несводимых. Но, прежде чем перейти к анализу сводимости, придётся немного глубже погрузиться в теорию графов.

5.1.1 Обходы в графах и обращение дуг

Два основных способа обойти ориентированный граф — это поиск в ширину и поиск в глубину. При поиске в ширину мы обходим всех потомков текущей вершины раньше, чем остальные вершины. При поиске в глубину мы, наоборот, откладываем обход прочих потомков, пока не обойдём потомков первого произвольно выбранного потомка. Поиск в ширину нам далее в этой книге не понадобится, все алгоритмы для компиляторов будут использовать поиск в глубину.

Поиск в глубину и различные упорядочения вершин при таком обходе давайте проиллюстрируем на рисунке 5.3.

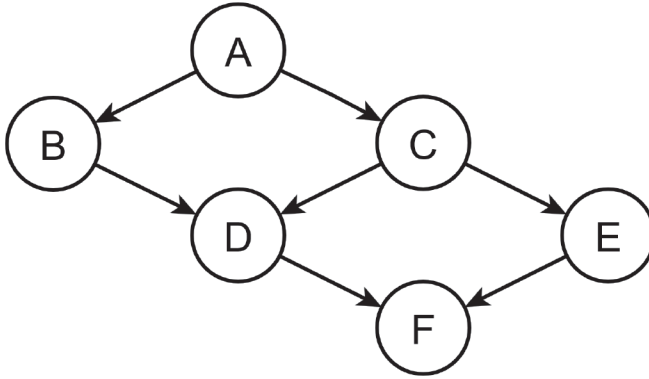


Рис. 5.3: Граф для иллюстрации RPO

Начиная с вершины A, у нас есть выбор её ребёнка. Например, мы можем пойти в B и далее в D и F. Вершина F не имеет детей, значит, нам надо вернуться назад в D, но у вершины D тоже нет других детей, мы возвращаемся в B, и далее возвращаемся в A. Теперь мы снова в A, и у неё есть непосещённый ребёнок C, у которого есть E. Итого, один возможный обход в глубину — это ABDFDDBACE. Ещё один возможный обход в глубину — это ACDFDCECAB.

Задание 5.6. Выпишите для графа на рисунке 5.3 все возможные обходы в глубину.

Обход в глубину даёт возможность упорядочить вершины, приписав им номера. Одним из самых естественных способов упорядочивания является прямой порядок (часто используют англицизм «преордеринг»). В этом порядке номер в обходе присваивается вершине, как только она в нём встретилась. Например, преордерингом обхода ABDFDDBACE будет ABDDFCE, а преордерингом для ACDFDCECAB будет ACDFEVB. Вторым возможным порядком является обратный порядок или постордеринг. При этом мы присваиваем первые номера узлам, когда посещаем их в последний раз. Обратный порядок ABDFDDBACE будет начинаться с F, потому что это первый узел, из которого мы ушли и более не вернулись. Далее обратный порядок будет продолжаться D, потом B и т.д. В итоге обратным порядком будет FDBECBA. Заметьте, что обратный порядок не является обращённым прямым порядком.

Задание 5.7. *Выпишите для графа на рисунке 5.3 все возможные прямые и обратные порядки.*

И внезапно самым полезным оказывается обращённый обратный порядок (reverse post order, RPO), называемый также топологическим порядком для ациклических графов. Давайте просто механически обратим для обхода ABDFDVACE его обратный порядок FDBECA. Мы получим топологический порядок ACEBDF, обладающий интересным свойством: все вершины графа в нём обходятся раньше их потомков. Разумеется, таких порядков тоже больше одного (а именно столько же, сколько и просто обратных порядков). Когда мы будем говорить фразы вроде «обойдём этот граф в топологическом порядке», мы будем иметь в виду, во-первых, любой топологический порядок, а во-вторых, подразумевать, что граф ациклический, иначе никакого топологического порядка нет.

Обход в обращённом постордере позволяет существенно улучшить построение доминаторов. Алгоритм, улучшающий наивный подход листинга 3.8 за счёт использования обратного постордера, приведён на листинге 5.4. Он в целом следует изложению в [15].

```

auto PO = Cfg.getPO;
auto POMap = Map<Block, int>;
for (auto B : blocks)
    Dom[B] = all;
Dom[entry] = entry;
auto Counter = 0;
for (auto B : PO)
    POMap[B] = Counter++;

auto Changed = true;
while (Changed) {
    Changed = false;
    for (auto B : PO.reverse()) {
        auto Q = B.pred(0); // 0 is arbitrary, any will do
        for (auto P : B.preds()) {
            while(Q != P) {
                (POMap[Q] < POMap[P]) ? Q = Dom[P] : P = Dom[P];
            }
        }
    }
}

```

```

    if (Dom[B] != Q) {
        Dom[B] = Q;
        Changed = true;
    }
}
}

```

Листинг 5.4: Algorithm DPO — доминаторы в RPO

Алгоритм DPO крайне интересен и стоит вдумчивого изучения. Он построен вокруг простой идеи: зная непосредственных доминаторов всех предшественников узла, мы можем найти общий доминатор, взбираясь вверх по логическому дереву доминаторов (как будто мы неявно его строим). Он и будет непосредственным доминатором узла. Это и происходит в самом вложенном цикле на листинге 5.4. Но это будет работать только в том случае, если, придя в эту точку, мы гарантируем, что подсчитали всех предшественников. Для этого нам и нужен проход в обратном постордере.

Задание 5.8. Попробуйте самостоятельно построить доминаторы узлов какого-нибудь реалистичного графа с помощью алгоритмов *D* и *DPO*, сравните время работы.

Теперь, когда у нас есть понимание RPO и его полезности, вернёмся к обратным дугам и циклам. Очевидно, что, если в графе есть циклы, это ничего не меняет: у него есть один или несколько постордерингов. Это значит, что даже в направленном графе общего вида, можно применить обращённый постордеринг как порядок его обхода. Вернёмся к рисунку 5.2 и для простоты рассмотрим случай *B*. Возможные обходы — это ABC или ACAB (не подумайте плохого). Возможные постордеринги — это CBA или BCА, и, значит, возможные обращённые постордеринги — это ACB и ABC.

Введём функцию $RPO(V)$, которая для вершины *V* возвращает её номер в заранее заданном обращённом постордеринге. Тогда можно ввести два определения. Ниже слова «ребро» и «дуга» для графа потока управления используются взаимозаменяемо.

Определение 5.3. Обращённой дугой (*retreating edge*) называется такая дуга $m \rightarrow n$, что для некоторого обращённого постордера $RPO(m) \geq RPO(n)$.

В случае В на рисунке 5.2 ребро $B \rightarrow C$ для обращённого постордеринга АСВ является обращённой дугой. Но она, конечно, не является обратной дугой, поскольку вершина С не доминирует вершину В.

5.1.2 Сводимость

Можно дать несколько независимых эквивалентных друг другу определений сводимости. Исторически само слово «сводимый» (reducible) происходит от анализа регионов графа через так называемые T_1 и T_2 преобразования. Оба преобразования упрощают поток управления. Преобразование T_1 убирает цикл с обратной дугой на себя, заменяя его на базовый блок. Преобразование T_2 сливает в один блок два блока идущих подряд. Оба преобразования показаны на рисунке 5.4.

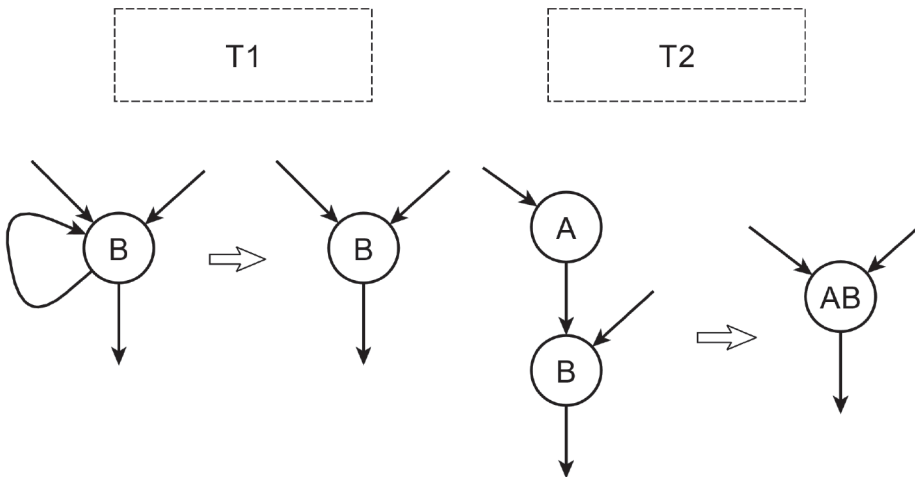


Рис. 5.4: Преобразования T_1 и T_2

Если после конечного числа применения этих двух преобразований граф потока управления «сводится» в один базовый блок, то он сводимый. Про эту теорию можно почитать в [43] и других источниках.

Но мы же не зря говорили про обращённые дуги и порядки обхода, поэтому я дам то определение, к которому я подводил, а остальные мы получим как следствия из него.

Определение 5.4. Сводимым (reducible) называется такой граф, в котором каждая обращённая дуга является обратной дугой.

В этом смысле случай В на рисунке 5.2 является несводимым.

Чем плоха несводимость? Всё дело в важном следствии из определения 5.4, а именно: сводимый граф можно через нумерацию в RPO разложить на ациклический граф и совокупность обратных дуг. Действительно, если в каждом RPO обращёнными могут быть только обратные дуги, то, убрав их все из графа, мы получаем оставшийся ациклический направленный граф (directed acyclic graph, DAG).

Если же граф несводимый, то даже если убрать из него все обратные дуги, в нём останутся замкнутые пути. В итоге мы не можем применить над таким графом все те богатые оптимизации, которые работают над естественными циклами, поскольку простыми средствами в несводимом регионе нельзя определить индуктивности, инварианты и так далее.

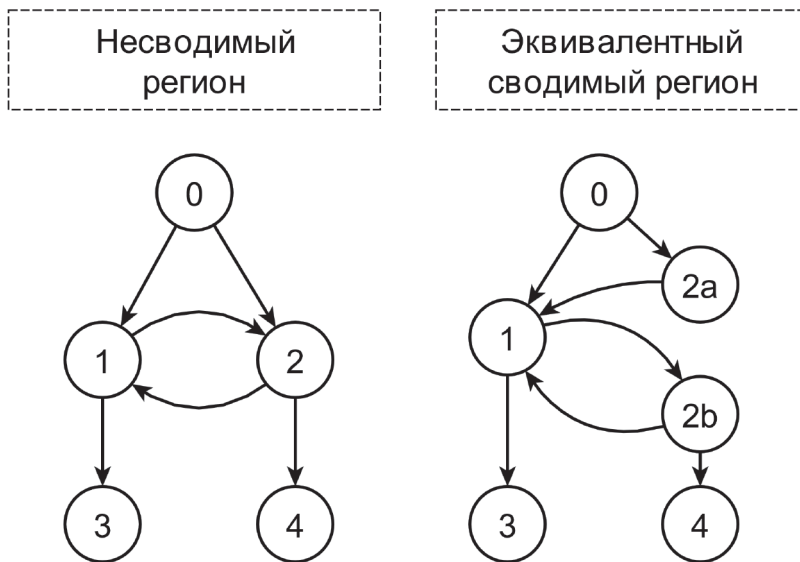


Рис. 5.5: От несводимого региона к естественному циклу

Несводимый граф можно иногда сделать сводимым, разбив некоторые базовые блоки. Пример такой трансформации показан на рисунке 5.5.

Чтобы лучше понять, что это за трансформация, давайте взглянем на неё с другой стороны — как на трансформацию исходного кода на языке C.

```
BB0: if (cond(x, y)) goto BB2;  
BB1: x = expr(x, y);  
BB2: y = otherexpr(x, y);  
      if (othercond(x, y)) goto BB1;
```

Листинг 5.5: Пример несводимого региона в коде на C

Код, показанный на листинге 5.5 выглядит ужасно, но очень точно соответствует несводимому циклу на рисунке 5.5, если в нём оставить только блоки B_0 , B_1 и B_2 . От него можно перейти к гораздо более простому структурному коду, представленному на листинге 5.6.

```
if (cond(x, y))  
    y = otherexpr(x, y);  
else {  
    x = expr(x, y);  
    y = otherexpr(x, y);  
}  
while(othercond(x, y)) {  
    x = expr(x, y);  
    y = otherexpr(x, y);  
}
```

Листинг 5.6: Переход к сводимому региону на языке C

Увы, мы не всегда можем сделать это так красиво. В работе [11] показано, что из кода, соответствующего полному графу с n узлами можно получить сводимый граф не меньше, чем с 2^{n-1} узлом. Иными словами, попытка сделать граф сводимым может привести к его экспоненциальному росту. К счастью, в практике обычного программирования мы редко видим такой код (и слабо его можем даже представить).

Итак, будем далее считать, что все графы потока управления являются сводимыми. Следующим шагом давайте разберёмся, как мы могли бы ещё больше облегчить себе трансформации циклов.

5.1.3 Канонический вид цикла

В естественных циклах, которые мы будем далее изучать, важными понятиями являются хедер (от английского «header», заголовок) и прехедер (соответственно, предзаголовок). Начнём со вполне простого определения.

Определение 5.5. Хедером или заголовком естественного цикла, построенного на обратной дуге $V_m \rightarrow V_n$ является блок V_n .

Например, на рисунке 5.6 для обоих случаев хедером цикла будет базовый блок V_3 .

Чтобы мотивировать концепцию прехедера, рассмотрим программу, содержащую некоторое общее вычисление, в частности, вызов функции без побочных эффектов, не зависящей от индуктивных значений цикла. Это общее вычисление является избыточным, и его хочется вынести из цикла.

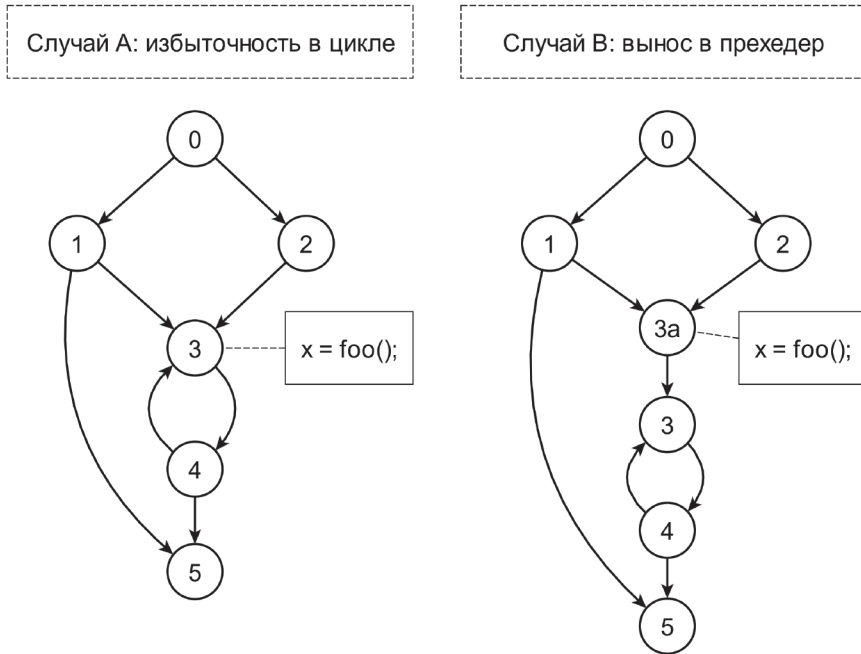


Рис. 5.6: Вынос избыточности в прехедер

Для случая А на рисунке 5.6 вынос избыточности в блоки V_1 и V_2 создаст на пути $V_0 \rightarrow V_1 \rightarrow V_5$ избыточность, которой там ранее не было. И если этот путь окажется горячим, то увы, мы ухудшили вещи, а не улучшили их. А вот для случая В всё обстоит куда более позитивно: вынос избыточности в блок V_{3a} совершенно безопасен.

Определение 5.6. Прехедером V_p естественного цикла, построенного на обратной дуге $V_m \rightarrow V_n$ является блок, предшествующий блоку V_n . При этом в блок V_n разрешено входить только двум дугам: $V_m \rightarrow V_n$ и $V_p \rightarrow V_n$. Все остальные пути в цикл идут через прехедер.

Задание 5.9. Подумайте, как отработал бы на графе с рисунка 5.6 алгоритм SSAPRE? Можете ли вы провести параллель между выносом прехедера и разбиением критического ребра?

Таким образом, в любом цикле частью его канонического вида должен быть прехедер. Даже если это изначально будет пустой прехедер, его вынесение имеет смысл для того, чтобы появилось место, куда из цикла можно вынести избыточность.

Теперь рассмотрим пример довольно простого цикла, показанного на листинге 5.7. В этом цикле опять-таки не случайно введена некоторая избыточность вокруг переменной *y*. Она выглядит искусственной, но, вообще избыточность в реалистичных циклах — это скорее норма, чем экзотика: подумайте про вычисление итератора конца каждый раз при сравнении с ним и так далее. Тут я просто всё упростил и сделал более явным.

```
while (x > 5) {  
    x = foo(x);  
    y = bar();  
}
```

Листинг 5.7: Мотивация канонической формы на C

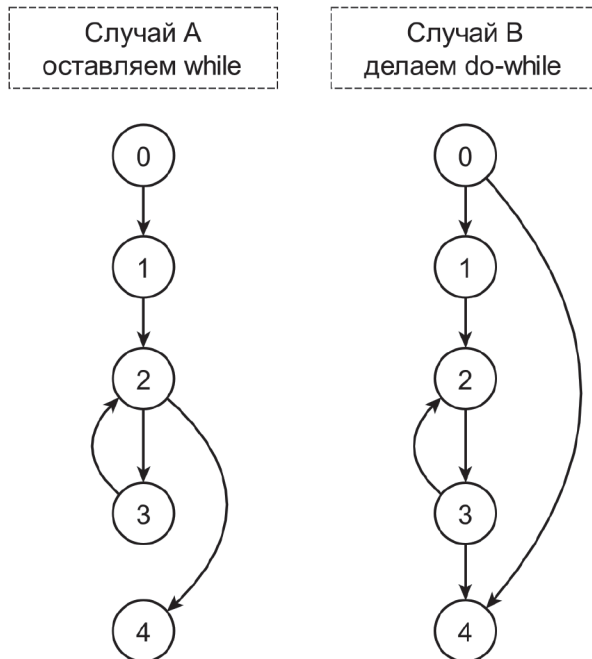


Рис. 5.7: Две возможных формы while

На рисунке 5.7 показаны два варианта потока управления, в которые этот цикл можно скомпилировать. Случай А в точности соответствует листингу 5.7, переписанному в HIR как есть (см. листинг 5.8). Разделение блоков B_0 и B_4 здесь немного искусственное, но оно нужно для симметрии двух случаев.

```
BB0: goto BB1;
BB1: // preheader
    goto BB2;
BB2: if (x < 5) goto BB3; else goto BB4;
BB3: x = foo(x);
    y = bar();
    goto BB2;
BB4: // exit block
```

Листинг 5.8: Случай А в HIR

Случай В немного интересней. Если записать его на языке С, то это что-то вроде переписывания while в do-while листинга 5.9.

```
BB0: if (x >= 5) goto BB4; else goto BB1;
BB1: // preheader
    goto BB2;
BB2: x = foo(x);
    y = bar();
    goto BB3;
BB3: if (x < 5) goto BB2; else goto BB4;
BB4: // exit block
```

Листинг 5.9: Случай В в HIR

Разделение блоков BB2 и BB3 немного искусственное. Обойтись без него можно, но тогда картинка была бы менее наглядной.

Задание 5.10. Попробуйте, прежде чем читать дальше, как-то аргументировать — какую форму вы предпочтёте?

Разумеется, правильный ответ здесь подсказывает специально введённая мной избыточность. В варианте А у нас вполне может быть так, что цикл не исполнится ни разу. Но тогда вынесение этой избыточности в B_1 как раз создаёт избыточные вычисления, а не устраняет их.

Правильная структура HIR после выноса избыточности в пре-хедер показана на листинге 5.10.

```
BB0: if (x >= 5) goto BB4; else goto BB1;
BB1: y = bar();
      goto BB2;
BB2: x = foo(x);
      goto BB3;
BB3: if (x < 5) goto BB2; else goto BB4;
BB4: // exit block
```

Листинг 5.10: Случай В после оптимизаций

Таким образом, канонической формой цикла выгодно делать случай В, как на листинге 5.9.

Определение 5.7. *Защёлкой (latch) естественного цикла в канонической форме, построенного на обратной дуге $B_m \rightarrow B_n$ является блок B_m . Именно в этот блок в канонической форме цикла выносятся вычисления его условия.*

Теперь зададимся вопросом, обязан ли блок-защёлка совпадать с тем блоком, из которого действительно происходит выход? Мы убедились, что стоит избегать выходных дуг в хедере, но довольно легко представить случай, когда выход может быть из середины цикла.

Определение 5.8. *Выходным блоком цикла является блок, дуга из которого ведёт в любой блок, не принадлежащий циклу.*

В нашей канонической форме у цикла может быть несколько защёлок и несколько выходных блоков, в том числе совпадающих с защёлками.

Задание 5.11. *Нарисуйте такой цикл в каноническом виде.*

Дополнительно введём следующее определение для того, чтобы характеризовать блоки, в которые мы выходим из цикла.

Определение 5.9. *Пограничным блоком для цикла является блок, в который ведёт дуга из выходного блока.*

В английском языке выходной и пограничный блоки называются «exiting block» и «exit block» соответственно. Слово по-

граничный выбрано мной чтобы показать, что это уже не блок, входящий в цикл, а всего лишь пограничный для него. Конечно же один и тот же блок может быть пограничным для нескольких разных циклов.

Итак, далее все циклы мы будем считать представленными в каноническом виде. Составляющие канонического вида — это вынесенный прехедер, защёлка в начале обратной дуги, идущей на хедер (то есть как бы в конце цикла). При этом эта защёлка должна совпадать с выходным блоком. Кроме того, канонический вид позволяет произвольное количество выходных блоков, не совпадающих с хедером.

Определение 5.10. *Канонической формой цикла является форма, в которой в нём есть прехедер и единственная обратная дуга. При этом все блоки, принадлежащие циклу, строго доминируются его хедером, т.е. ни один выходной блок не имеет предшественников вне цикла.*

Такая форма возможна, разумеется, только для сводимого цикла и именно она позволяет нам наиболее эффективно работать с избыточностями в теле цикла.

Определение 5.10 привлекательно в теоретических работах вроде этой книги, но настоящая каноническая форма цикла может несколько отличаться для разных компиляторов.

Задание 5.12. *Проанализируйте канонический вид цикла в том компиляторе, который вы лично используете. Есть ли там особенности?*

Я ещё дополню определение 5.10 в разделе 5.2.2, но пока что будем работать с базовым вариантом.

5.1.4 Замкнутое цикловое SSA

Важной частью облегчения работы над циклами является их трансформация в специальное модифицированное SSA-представление. Оно мотивируется не столько выносом избыточностей, сколько возможностью структурных преобразований.

```

BB0: x0 = init();
      c = cond();
      goto BB1;
BB1: x1 = PHI(x0, x2, x3);
      if (c) goto BB2; else goto BB3;
BB2: x2 = bar(x1);
      goto BB4;
BB3: x3 = buz(x1);
      goto BB4;
BB4: x4 = PHI(x2, x3);
      if (x4 < N) goto BB1; else goto BB5;
BB5: x5 = PHI(x4);
      use(x5);

```

Листинг 5.11: Цикл для мотивации LCSSA

Определение 5.11. *Представление цикла в виде замкнутого циклового SSA (loop closed SSA) возможно в том случае, если все SSA-значения, определённые в цикле, используются только внутри этого цикла. Допустимо также использование этих значений в пограничных блоках цикла в качестве аргументов ϕ -узлов.*

Чтобы соблюсти это свойство, все значения, которые должны использоваться вне цикла, такие как переменная x_5 на листинге 5.11 должны быть введены отдельными ϕ -узлами по выходным рёбрам. Далее, если мы меняем местами цикл и ветвление (это будет позднее называться loop unswitching), то у нас получается HIR, приведённый на листинге 5.12. Мы просто меняем методику вычисления x_5 , и внешний по отношению к циклу код ничего не замечает.

```

BB0: x0 = init();
      c = cond();
      if (c) goto BB1; else goto BB2;
BB1: x1 = PHI(x0, x2);
      x2 = bar(x1);
      if (x2 < N) goto BB1; else goto BB5;
BB2: x4 = PHI(x0, x3);
      x3 = buz(x4);
      if (x3 < N) goto BB2; else goto BB5;
BB5: x5 = PHI(x2, x3);
      use(x5);

```

Листинг 5.12: Тот же цикл после трансформации

Разумеется, без loop-closed SSA принципиальных проблем бы тоже не было. Но мы были бы вынуждены идти и за пределами цикла обновлять потенциально очень большое количество пользователей соответствующих значений. Все было бы локальнее и удобнее, если бы все, что случалось в цикле, там и оставалось.

В официальной документации на LLVM также упомянуто, что LCSSA несколько упрощает реализацию и использование подпрограмм для поиска индуктивностей.

Отсюда и далее мы будем считать, что граф потока управления — сводимый, все циклы — каноникализированы и представлены в LCSSA даже при условии, что это вызывает появление странных ϕ -узлов с одним аргументом, как на листинге 5.11.

5.2 Поиск и оптимизации индуктивностей

Индуктивности или индуктивные переменные циклов — это ещё один термин, который довольно просто понять интуитивно, но при этом неожиданно сложно формализовать. Представьте, что вы в своей программе написали три цикла, показанных на листинге 5.13.

```
// case A
for (int i = 0; i < 3; ++i)
    use(i + 7);

// case B
for (int i = 7; i * i < 100; ++i)
    use(i);

// case C
for (int i = 0, j = 0; i * j < 9; ++i, ++j)
    use((i + j) / 2 + 7);
```

Листинг 5.13: Найдите отличия в этих трёх циклах

Для вас, как для программиста, это разные циклы, которыми вы, возможно, хотели сказать разные вещи. Но с точки зрения компилятора они генерируют одинаковую последовательность побочных эффектов, и логично ожидать, что при идеальной работе компилятора они будут сооптимизированы в одинаковый код.

Интуитивно в первых двух циклах (случаи А и В) одна индуктивная переменная i . В случае В выходное условие зависит от

её квадрата. Для случая С у нас интуитивно две индуктивности, но вторая выглядит избыточным вычислением и, вероятно, будет убрана после нумерации значений.

5.2.1 Индуктивные переменные

Переведём все три цикла в HIR и в LCSSA представление. Я переведу один и оставляю два других как упражнения.

```

BB0: i1 = 0;
      if (i1 >= 3) goto BB2; else goto BB1;
BB1: i2 = PHI(i1, i3);
      t = i2 + 7;
      use(t);
      i3 = i2 + 1;
      if (i3 < 3) goto BB1; else goto BB2;
BB2: i4 = PHI(i1, i3);

```

Листинг 5.14: HIR для случая А

Задание 5.13. Напишите HIR для случаев В и С, он будет довольно похож.

На рисунке 5.8 представлен граф зависимостей SSA для листинга 5.14. Видно, что индуктивная переменная образует в этом графе замкнутый путь (вообще-то цикл, но мы помним, что слово «цикл» мы оставили для циклов в программе). В общем случае если некие переменные как-то взаимно изменяются в цикле, то речь может идти о сильно связанном компоненте в SSA графе.

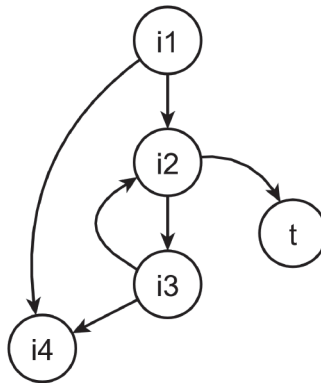


Рис. 5.8: Индуктивность — это замкнутый путь в SSA графе

Формальное определение индуктивности общего вида дано ниже (см. определение 5.12).

Определение 5.12. *Индуктивной переменной общего вида (обобщённой индуктивностью, *generalized induction variable, GIV*) является любой замкнутый путь в SSA представлении, все вершины которого находятся внутри естественного цикла.*

Обобщённая индуктивная переменная может быть очень неудобна для операций с ней. Она может затрагивать много вычислений, в том числе очень сложных, проходить через ϕ -узлы и внешние функции. В самом общем случае с ней ничего не сделать. К счастью, большая часть программистов пишут обычные циклы, как на листинге 5.13. Обычными эти циклы делает именно тот факт, что в них используются обычные (или **базовые**) индуктивности.

Определение 5.13. *Базовой индуктивной переменной (*basic induction variable, BIV*) является переменная, которая внутри цикла изменяется на константу, единожды за итерацию цикла. Можно записать: $b(n) = b(n - 1) + K$.*

Определение 5.13 можно расширить, добавив кроме констант любые инвариантные значения. Это вносит некоторые технические сложности, но такой анализ часто ведёт к лучшим оптимизациям. На дальнейшее изложение это не повлияет, так что давайте останемся с более простым определением.

Обратим внимание, что циклы, в которых удаётся найти базовую индуктивную переменную, довольно просто распараллелить. Потому что для них верно также $b(n) = b(0) + K * n$. В таком случае у нас всегда есть выражение в замкнутой форме для каждого номера итерации.

```
BB0: i1 = 0; w1 = 0;
      goto BB1;
BB1: i2 = PHI(i1, i3);
      w2 = PHI(w1, w3);
      w3 = w2 + i2;
      i3 = i2 + 1;
      if (i3 < N) goto BB1; else goto BB2;
BB2: w4 = PHI(w3);
      use(w4);
```

Листинг 5.15: Базовые и обобщённые индуктивности

На листинге 5.15 приведён пример. Базовую индуктивность образует сеть i_1, i_2, i_3 . Обобщённую индуктивность образует сеть w_1, w_2, w_3, w_4 . Тут может возникнуть законный вопрос: формально i_2 тоже входит в сеть для w_3 и так далее. Что же считать собственно индуктивной переменной? Можем ли мы говорить вообще о какой-то одной индуктивности?

Довольно стандартный ответ на этот вопрос очень прост: индуктивная переменная — это ϕ -узел в голове цикла. То есть на листинге 5.15 базовой индуктивной переменной является i_2 , а обобщённой индуктивной переменной является w_2 .

Задание 5.14. *Напишите эволюцию индуктивности w_2 от итерации к итерации этого цикла.*

В одном цикле может быть несколько базовых индуктивностей. Мало того, у нас всегда есть выбор какую из них считать «главной». В качестве примера на листинге 5.16 приведён случай, когда в цикле три индуктивных переменных.

```

BB0: t1 = 202; p1 = &a[0]; i1 = 0;
      goto BB1;
BB1: i2 = PHI(i1, i3);
      p2 = PHI(p1, p3);
      t2 = PHI(t1, t3);
      *p2 = t2;
      i3 = i2 + 1;
      p3 = p2 + 4;
      t3 = t2 - 2;
      if (i3 < N) goto BB1; else goto BB2;
BB2: p4 = PHI(p3);
      use(p4);

```

Листинг 5.16: Несколько базовых индуктивностей

Здесь из трёх базовых индуктивностей i_2, p_2 и t_2 интуиция подсказывает, что написан всё-таки цикл по переменной i_2 . Поэтому что именно эта переменная в защёлке цикла сравнивается с условием выхода. В принципе, можно переписать его так, чтобы сравнивалась другая.

Задание 5.15. *Напишите этот цикл на языке C тремя способами, делая каждую из трёх индуктивностей визуально главной.*

Найти в цикле базовую индуктивность (и перейти к ней, о чём далее) может быть очень полезно. Правильно выбранная индуктивность существенно сокращает циклы и даёт дорогу многим цикловым оптимизациям.

5.2.2 Поиск базовых индуктивностей

Поиском базовых индуктивностей люди занимались до изобретения SSA представления, см. [14], но это не было ни просто, ни быстро, ни надёжно. Я не хотел бы углубляться в историю, а предлагаю сразу пользоваться преимуществами SSA представления и искать индуктивности довольно простым алгоритмом, который сопоставляет паттерны потока данных. Такие алгоритмы предлагались несколько раз (напр. [23]), и отличаются друг от друга, в основном, различным уровнем внимания к краевым случаям.

Пример возможной проблемы при поиске базовых индуктивностей простым анализом паттернов по SSA графу, связанный с наличием вложенных циклов, приведён на листинге 5.17.

```
i = 0;
do {
  k = 0;
  do {
    k = k + i * 7;
  } while(i * i > k);
  i = i + 1;
} while (i < 10);
```

Листинг 5.17: Пример вложенных циклов

Здесь во вложенном цикле переменная i участвует в довольно сложном вычислении. Простой алгоритм вполне способен её на этом основании забраковать. Но все эти сложные вычисления происходят во вложенном цикле, а для внешнего цикла i вполне является базовой индуктивностью.

Чтобы в таких условиях корректно обрабатывать вложенные циклы, нужно иметь информацию, достаточную, чтобы отличать блоки, принадлежащие циклам разных уровней. К счастью, в сводимом потоке управления все циклы являются либо непересекающимися, либо вложенными. Это позволяет, анализируя естественные циклы, построить древовидную структуру, которая соотносит циклы с вложенными циклами и теми базовыми блоками, которые эти циклы занимают.

Эта структура называется деревом циклов (loop tree) и проиллюстрирована на рисунке 5.9. Она не всегда так называется и не всегда в явном виде существует в промышленных компиляторах. Например, в LLVM вместо дерева циклов используется инцидентная структура LoopInfo. Тем не менее, дальнейшее изложение будет несколько проще если мы предположим, что она существует, и мы можем на неё ссылаться.

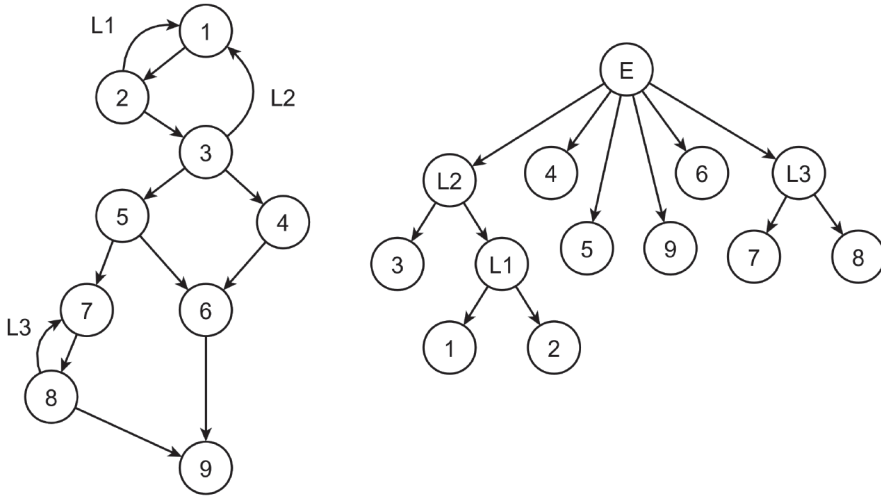


Рис. 5.9: CFG и его дерево циклов

Задание 5.16. Предложите простой алгоритм для построения дерева циклов, если поток управления сводимый.

Построить дерево циклов с учётом возможных несводимых регионов, оставив информацию об этих регионах, вам будет предложено в заданиях к этому разделу.

После предварительной подготовки структур данных можно обнаружить базовые индуктивные переменные в цикле, проанализировав в SSA графе связанные компоненты, которые лежат внутри цикла. Детали приведены на листинге 5.18. Каждый цикл на вход этого алгоритма подаётся в соответствии с построенным деревом циклов, движение идёт изнутри наружу.

```

auto biv_lookup(Loop L) {
    auto BIVs = Map<SSAVal, int>{};
    auto Increments = Map<SSAVal, int>{};
    IVCOUNT = 0;
    for (auto Phi : L.header().phis()) {

```

```

auto Inc = 0;
auto IVTmp = Map<SSAVal, int>{};
for (auto AddSub : ssa_users(Phi)) {
    if (!L.contain(AddSub) || BIVs.contain(AddSub))
        continue;
    auto I = add_sub_increment_orzero(AddSub);
    if (I == 0)
        continue;
    auto Cmp = ssa_single_user(AddSub);
    if (!Cmp || !check_compare(Cmp) ||
        ssa_single_user(Cmp) != Phi)
        continue;
    if (Inc != 0 && Inc != I) {
        Inc = 0;
        break;
    }
    Inc = I;
    IVTmp[Phi] = IVTmp[AddSub] = IVTmp[Cmp] = IVCount;
}
BIVs.insert(IVTmp.begin(), IVTmp.end());
Increments[Phi] = Inc;
IVCount++;
}
return {BIVs, Increments};
}

```

Листинг 5.18: Алгоритм BIV — поиск базовых индуктивностей

Здесь проверка `L.contain(AddSub)` проверяет принадлежность определения текущему циклу с учётом дерева циклов.

Задание 5.17. *Можно ли упростить алгоритм 5.18, используя тот факт, что все циклы у нас в замкнутой форме?*

Само дерево тут не упоминается, так как это глобальная структура (как CFG или SSA граф), и ей можно, по нашим соглашениям пользоваться неявно и свободно из любой точки программы.

Структура данных `BIVs` — это отображение из SSA value в номер индуктивной переменной. Она позволяет запомнить все переменные, входящие в траекторию, а не только ϕ -узел. Далее нам это понадобится для анализа скалярной эволюции. Алгоритм гарантирует, что один номер будет соответствовать одному ϕ -узлу в хедере.

Также в алгоритме BIV структура данных `Increments` хранит отображение заглавных φ -узлов в их инкременты.

Проговорим алгоритм BIV словами: для каждого φ -узла в хедере мы анализируем все его выходные дуги. Если они идут за пределы цикла, пропускаем. Если они идут куда-то, откуда нельзя установить инкремент (в неправильные операции и так далее), то пропускаем. Если из инкремента они не идут в сравнение, а из сравнения обратно в анализируемый φ -узел — снова пропускаем.

Если в итоге нашлась такая переменная, которая нам нужна, то ей можно дополнить список правильных исходящих определений.

Заметьте, как изящно алгоритм BIV не считает фатальной ошибкой ситуацию, когда найдены два пути из одного φ -узла с разными инкрементами. Пример такого кода можно довольно легко написать:

```
do {  
    if (cond())  
        i += 3;  
    else  
        i += 5;  
} while (i < 10);
```

Листинг 5.19: Пример сложной индуктивности

Здесь, конечно, у нас нет базовой индуктивности (но создать её поможет вынос из цикла инвариантного управления, см. раздел 5.4.4). Поэтому алгоритм останавливается, если зависимость по данным от инкремента идёт не в сравнение, а куда угодно ещё, в частности, в другой φ -узел не в хедере.

Задание 5.18. Можете ли вы предложить дальнейшие улучшения для алгоритма BIV?

Дополним наше определение канонической формы цикла (см. 5.10). Будем считать, что цикл находится в канонической форме, если в нём с условием выхода сравнивается именно базовая индуктивность. Например, в цикле В на листинге 5.13 это исходно не так, и значит, это надо нормализовать. Следующий раздел будет посвящён формализации такого рода переходов.

5.2.3 Скалярная эволюция

Представим себе цикл с базовой индуктивностью i , в котором единственное её использование это $i * 4$. В принципе, мы могли бы ввести зависимую индуктивность $j = i * 4$ и перейти к ней, убрав умножение. Но чтобы это сделать, мы должны знать про i довольно много всего. Мы должны знать её начальное значение, её инкремент на каждом шаге и, если есть конечное значение, то с чем она сравнивается в конце. Говоря коротко — для смены индуктивности мы должны знать её скалярную эволюцию.

Определение 5.14. *Скалярной эволюцией (SCEV, scalar evolution) переменной называется зависимость её значения от номера итерации.*

Начнём с задачи, которая взята мной из [48]. В интернете часто встречаются такие задачи с припиской вроде «только люди с IQ больше 150 могут решить эту задачу». Вам дана последовательность чисел, как в 5.1, и нужно найти формулу, порождающую все эти числа.

$$a_n = 96, 726, 2556, 6216, 12336, 21546 \quad (5.1)$$

Разумеется, тут есть много разных подходов к решению. Один из самых интересных, (а именно он и будет нам нужен в этой главе) — это подход через конечные разности. Обозначим исходную последовательность как нулевые разности $\Delta_n^0 = a_n$ и введём первые разности как в уравнении 5.2.

$$b_n = \Delta_n^1 = a_{n+1} - a_n = 630, 1830, 3660, 6120, 9210 \quad (5.2)$$

Далее вторые разности и т.д.

$$\Delta_n^2 = b_{n+1} - b_n = 1200, 1830, 2460, 3090 \quad (5.3)$$

$$\Delta_n^3 = 630, 630, 630 \quad (5.4)$$

Внезапно уже четвёртые разности стали нулевыми, то есть мы на верном пути.

Далее воспользуемся формулой 5.5, которая восстанавливает общее выражение полинома по первым членам конечных разностей.

$$f(n) = \sum_{i=0}^p \Delta_n^i \binom{n}{i} \quad (5.5)$$

Откуда далее имеем:

$$f(n) = 96 + 630 * n + 1200 * \frac{n * (n - 1)}{2} + 630 * \frac{n * (n - 1) * (n - 2)}{2 * 3} \quad (5.6)$$

И вот здесь мы сделаем небольшой финт. Вместо того, чтобы честно раскрывать скобки в 5.6 (это довольно просто и можно оставить как упражнение), мы скажем, что оно соответствует цепочке рекуррентностей $\square 96, +, 630, +, 1200, +, 630 \square$.

```
int f(int n) {
    int a = 96, b = 630, c = 1200, d = 630;
    for (int i = 0; i < n; ++i) {
        a = a + b;
        b = b + c;
        c = c + d;
    }
    return a;
}
```

Листинг 5.20: Интуиция цепочки рекуррентностей на языке C

Это мотивируется листингом 5.20. Действительно, записанная в таком виде функция возвращает все числа из 5.1 и способна предсказать последующие.

Задание 5.19. Раскройте скобки в 5.6 и найдите замкнутый вид той же формулы. Кажется, он несколько страшнее, и его вычисление требует возведений в степень.

Разумеется, любая базовая индуктивность b по определению может быть записана как $\square b_0, +, i \square$, где b_0 — это её начальное значение, а i — это шаг инкремента. Чем на самом деле хороши цепочки рекуррентностей для анализа скалярной эволюции, так это тем, что они образуют алгебру.

Например, если добавить к процессу аддитивной скалярной эволюции константу, она будет добавлена к начальному значению, а если умножить, то умножены будут обе части.

$$c + \langle a_1 + b \rangle = \langle c + a_1, +, b \rangle \quad (5.7)$$

$$c * \langle a, +, b \rangle = \langle c * a, +, c * b \rangle \quad (5.8)$$

$$\langle a, +, b \rangle + \langle c, +, d \rangle = \langle a + c, +, b + d \rangle \quad (5.9)$$

Задание 5.20. Подсчитайте чему равно $\langle a, +, b \rangle * \langle c, +, d \rangle$. Очевидно, это будет аддитивная скалярная эволюция теперь уже на три компонента.

Точно так же можно обходиться с мультипликативной эволюцией.

$$c * \langle a, *, b \rangle = \langle c * a, *, b \rangle \quad (5.10)$$

Конечно, мультипликативная эволюция для практических задач не так интересна.

Теперь надо ввести очередное определение.

Определение 5.15. Зависимой индуктивной переменной (*DIV, dependent induction variable*) называется индуктивная переменная, связанная с базовой индуктивностью через скалярную эволюцию.

Заметим: зависимая индуктивная переменная вообще не обязана быть замкнутым путём в SSA графе. Например, на листинге 5.21 зависимой индуктивной переменной является j .

```
BB0: i1 = 0;
      if (i1 < n) goto BB1; else goto BB2;
BB1: i2 = PHI(i1, i3);
      j1 = i2 * 4;
      use(j1);
      i3 = i2 + 1;
      if (i3 < n) goto BB1; else goto BB2;
BB2: i4 = PHI(i1, i3);
```

Листинг 5.21: Зависимая индуктивность

Естественной оптимизацией этого цикла будет переход от $i = \langle 0, +, 1 \rangle$ к $j = \langle 0, +, 4 \rangle$ как это показано на листинге 5.22

```

BB0: i1 = 0;
      if (i1 < n) goto BB1; else goto BB2;
BB1: i2 = PHI(i1, i3);
      use(i2);
      i3 = i2 + 4;
      if (i3 < n) goto BB1; else goto BB2;
BB2: i4 = PHI(i1, i3);

```

Листинг 5.22: Переход к зависимой индуктивности

В заданиях к разделу вам будет предложено спроектировать алгоритм для поиска зависимых индуктивностей и их скалярной эволюции. Дальнейший переход от индуктивности к индуктивности — это тривиальная (хотя и несколько утомительная) техническая работа над SSA-представлением. В современных компиляторах такого рода переходами занимается фаза оптимизаций индуктивных переменных (Induction Variable Optimizations). Она не так уж и сложна. Но её детали я хотел бы оставить за скобками, чтобы объём изложения не вышел из-под контроля.

Задание 5.21. Изучите самостоятельно, как происходит оптимизация индуктивных переменных в вашем компиляторе.

5.2.4 Упрощение вычислений в циклах

Завершая разговор об индуктивностях, рассмотрим важную прикладную задачу перехода к новой индуктивности. Трансформация, решающая эту задачу, называется упрощением сложности цикловых вычислений (LSR, Loop Strength Reduction).

Базовый пример для этой оптимизации приведён на листинге 5.23.

```

void foo(int *a, int n, int k) {
    for (i = 0; i < n; ++i)
        a[i] = i * k; // *(a + i * sizeof(int))
}

```

Листинг 5.23: Пример для упрощения вычислений

Давайте для анализа переведём это в HIR. Можете в качестве упражнения сделать это самостоятельно. Итог показан на листинге 5.24.

```

foo(a1, n1, k1):
BB0: i1 = 0;
      if (i1 < n1) goto BB1; else goto BB2;
BB1: i2 = PHI(i1, i3);
      j1 = i2 * 4;
      a2 = a1 + j1;
      t1 = i2 * k1;
      *a2 = t1;
      i3 = i2 + 1;
      if (i3 < n1) goto BB1; else goto BB2;
BB2: return;

```

Листинг 5.24: Пример листинга 5.23 в HIR

Задание 5.22. *Прежде чем читать дальше, определите на этом листинге скалярную эволюцию для всех базовых и зависимых переменных.*

Результат анализа скалярной эволюции:

- Базовая индуктивность: $i = \langle 0, +, 1 \rangle$
- Зависимая индуктивность: $j = \langle 0, +, 4 * i \rangle$
- Зависимая индуктивность: $a = \langle a_1, +, 4 * i \rangle$
- Зависимая индуктивность: $t = \langle 0, +, k * i \rangle$

Главная идея для сокращения сложности вычислений в цикле — это осмысленный выбор индуктивности, к которой планируется переходить. Этот выбор может исходить, в частности, из нашего знания о той платформе, для которой мы будем компилировать. Например, если на платформе умножение гораздо дороже сдвига, имеет смысл избавиться от умножения на k , так как умножение на 4 это и есть сдвиг. А вот если всё плохо с режимами адресации памяти, и адресация с индексом будет невозможна, тогда есть смысл перейти к индуктивности a , вынеся формирование адреса в накопление в цикле, как это сделано на листинге 5.25.

```

foo(a1, n1, k1):
BB0: t1 = 4 * n1; b1 = a1 + t1;
      if (a1 < b1) goto BB1; else goto BB2;
BB1: a2 = PHI(a1, a3);
      k2 = PHI(0, k3);
      *a2 = k2;
      k3 = k2 + k1;
      a3 = a2 + 4;
      if (a3 < b1) goto BB1; else goto BB2;
BB2: return;

```

Листинг 5.25: HIR после LSR

Задание 5.23. *Посмотрите, что делает ваш компилятор. Попробуйте его на нескольких разных архитектурах.*

Чтобы реализовать такого рода платформенно-зависимый выбор внутри общей части компилятора, обычно делают специальную функцию, так называемый хук (target hook). Он динамически вызывается из кодогенератора конкретно той архитектуры, под которую происходит компиляция. В итоге мы можем написать платформенно-независимый код, который вызывает этот хук и на основании него принимает решение. Мы ещё увидим эту технику дальше в этом разделе: точно так же платформенно-специфичные решения будут приниматься во многих цикловых трансформациях.

5.2.5 Раскрутка циклов

Частным случаем перехода к другой индуктивности является полезное и популярное преобразование, которое называется раскруткой цикла (loop unroll). При такой трансформации мы не столько меняем индуктивность, сколько увеличиваем шаг текущей индуктивности.

Имея цикл с листинга 5.26, можно перейти от него к циклу с листинга 5.27. Формально в SSA это будет сменой индуктивности, но интуитивно мы итерируемся по той же переменной просто в четыре раза реже.

```
for (int i = 0; i < 4 * n; ++i) {  
    a[i] += i;  
}
```

Листинг 5.26: Цикл до раскрутки

```
for (int i = 0; i < n; ++i) {  
    int j = 4 * i;  
    a[j] += j;  
    a[j + 1] += j + 1;  
    a[j + 2] += j + 2;  
    a[j + 3] += j + 3;  
}
```

Листинг 5.27: Цикл после раскрутки

В чём основные выгоды такого преобразования? Во-первых, лёгким движением руки мы в четыре раза сократили количество условных переходов и сэкономили кучу истории их предсказания в аппаратуре. Во-вторых, очень часто после развёртки появляются дополнительные возможности для оптимизаций. Например, операции с памятью для листинга 5.27 напрашиваются на векторизацию, если она поддержана в аппаратуре.

Разумеется, лучше всего, если мы из скалярной эволюции заранее знаем шаг цикла, и он кратен тому фактору, на который мы раскручиваем. Если мы не уверены, но по каким-то причинам всё-таки хотим раскрутить, нам придётся вставлять дополнительные выходные дуги.

```
for (int i = 0; i < n; ++i) {  
    a[i] += i;  
}
```

Листинг 5.28: Мы не уверены в границе цикла

```
for (int i = 0; i < n / 4; ++i) {  
    int j = 4 * i;  
    a[j] += j;  
    if (j + 1 == n) break;  
    a[j + 1] += j + 1;  
    if (j + 2 == n) break;  
    a[j + 2] += j + 2;  
    if (j + 3 == n) break;  
    a[j + 3] += j + 3;  
}
```

Листинг 5.29: Раскрутка с добавлением дуг

Такой код, как на листинге 5.29, вызывает обоснованный вопрос: зачем мы вообще это сделали? Должны быть очень убедительные замеры, чтобы такое мотивировать. Лучше, чтобы они дополнялись какими-то конкретными соображениями в аппаратуре.

Впрочем, компиляторы умеют делать очень неглупые вещи даже в условиях неопределённости. Листинг 5.30 выглядит гораздо более убедительно: мы несомненно улучшили цикл, заплатив за это однократным и неприятным хвостом.

```
if (n > 0) {
    for (int i = 0; i + 3 < n; i += 4) {
        a[i] += i;
        a[i + 1] += i + 1;
        a[i + 2] += i + 2;
        a[i + 3] += i + 3;
    }
    switch(n % 4) {
        case 3:
            a[n - 3] += n - 3; break;
        case 2:
            a[n - 2] += n - 2; break;
        case 1:
            a[n - 1] += n - 1; break;
    }
}
```

Листинг 5.30: Раскрутка с обработкой хвоста

Всё это были случаи частичной раскрутки цикла. Когда я говорю просто о раскрутке, здесь и далее я имею в виду частичную раскрутку, как наиболее часто встречающийся вариант.

Полной раскруткой (full unroll) называется такая раскрутка цикла, когда от него ничего не остаётся. Например, случай А с листинга 5.13 скорее всего любым разумным компилятором будет раскручен полностью — в нём всего три итерации по базовой индуктивности. Случаи В и С с того же листинга могут быть полностью раскручены или нет в зависимости от того, насколько удачно компилятор меняет индуктивность. На момент написания этой книги, gcc с ними не справлялся.

Обратной оптимизацией к раскрутке является свёртка (reroll) цикла обратно. Свёртка может быть применима для оптимизации по размеру. Иногда компиляторы специально делают раскрутку, потом оптимизации на созданном раскруткой контексте, а потом при возможности свёртку. Это помогает найти в цикле неочевидные возможности к оптимизации, скрытые между итерациями.

5.3 Инварианты и квази-инварианты

Самым большим выигрышем, который мы можем получить при работе с циклами в оптимизирующем компиляторе, оказывается вынос из цикла практически любого избы-

точного вычисления. До некоторой степени мы уже говорили о ликвидации избыточности, когда обсуждали алгоритмы нумерации значений и частичного удаления избыточности. Но волшебством специализированных алгоритмов на циклах является то, что они оперируют над подграфами куда более частного вида. Следовательно, они могут делать гораздо более интересные вещи.

В разделе 5.2.3 мы поговорили о том, как в циклах определяются индуктивности. Я не зря начал именно с них: дело в том, что инварианты в цикле частично определяются через индуктивности и более того, через их скалярную эволюцию. Так что, если вы по каким-то причинам пропустили объяснение цепочек рекуррентностей, освежите его в памяти, прежде чем читать дальше.

Определение 5.16. *Инвариантом в цикле является либо (1) выражение, не имеющее зависимостей от обобщённых индуктивностей цикла либо (2) индуктивная переменная с тривиальной скалярной эволюцией. То есть $\langle N, +, 0 \rangle$, $\langle M, *, 1 \rangle$ и т.п.*

Далее мы сначала обсудим вынос простых инвариантов, а потом в разделе 5.3.2 предложим интересную оптимизацию, которую компиляторы часто не любят делать.

5.3.1 Вынос инвариантного кода из цикла

Как мы уже обсуждали в 5.1.3, вынесение перехедера цикла обязательно для того, чтобы вынесенные инварианты не создавали избыточность по лишним веткам. Там это дополнительно сопоставлено с разбиением критического ребра. В дальнейшем мы будем считать, что все циклы имеют канонический вид, то есть включают в себя прехедер, выход из защёлки и базовую индуктивность, которая сравнивается с условием выхода в защёлке.

Начнём с того, что вынесем из цикла очевидные инварианты. Для этого можно использовать простой (возможно, излишне простой) алгоритм, приведённый на листинге 5.31.

```

void simple_invariant_cleanup(Loop L) {
    auto Changed = true;
    while (Changed) {
        Changed = false;
        for (auto Def : L.instructions()) {
            if (Def.loads())
                continue;
            bool Move = true;
            for (auto Use : ssa_uses(Def)) {
                auto D = ssa_def(Use);
                if (L.contain(D)) {
                    Move = false; break;
                }
            }
            if (!Move) continue;
            L.move_to_preheader(Def);
            Changed = true;
        }
    }
}

```

Листинг 5.31: Алгоритм LICM — вынос простых инвариантов

Алгоритм LICM ищет среди всех инструкций в цикле те, которые не пользуются для формирования своих операций другими инструкциями из этого цикла. Поскольку вынос каждой такой инструкции в прехедед потенциально создаёт новые возможности для выноса инвариантов, он, как и многие алгоритмы в этой книге, заиклен, пока вносит изменения.

Алгоритм назван LICM по названию трансформации выноса инварианта из цикла (loop invariant code motion, LICM).

Обратите внимание на главную неприятную особенность этого алгоритма — он, к сожалению, не работает с памятью. Сохранение не образует SSA value, а загрузки фильтруются на уровне алгоритма, так как их вынос вверх немного осложнён возможным алиасингом с сохранениями в память. Увы, вынос операций с памятью — одна из самых полезных оптимизаций. То же самое относится к вызову функций, и это ещё обиднее.

Задание 5.24. *Попробуйте поддержать в алгоритме LICM вынос в прехедед хотя бы самых простых загрузок из памяти.*

Обратите также внимание на то, как LICM работает над гнездом циклов.

```
for (i = 0; i < N; ++i)
  for (j = 0; j < M; ++j)
    use(i + j + 2 * M - 2 * N);
```

Листинг 5.32: Инвариант во вложенном цикле

В примере с листинга 5.32 инвариантное вычисление $2 * M - 2 * N$ сначала из вложенного цикла выносится в его прехедер, являющийся частью охватывающего цикла. И далее, из охватывающего цикла выносится в его прехедер. Это ещё раз подчёркивает, как важно обходить дерево циклов изнутри наружу, от самых вложенных циклов к самым верхним: при таком обходе инварианты всплывают за одну итерацию и их вынос не приходится зацикливать.

5.3.2 Пилинг циклов

Рассмотрим цикл, представленный на листинге 5.33.

```
for (i = 1; i < n; ++i)
  a[i] = a[1] + i;
```

Листинг 5.33: Квази-инвариант

Можно ли из него вынести инвариантную загрузку $a[1]$? В чистом виде этого сделать нельзя так как этот элемент меняется в процессе исполнения цикла. Но мы могли бы создать этот инвариант, если бы можно было вынести перед циклом его первую итерацию. Я немного конвертирую цикл в каноническую форму и сделаю этот вынос на листинге 5.34.

```
if (n > 1) {
  a1 = a[1] + 1; a[1] = a1;
  if (n > 2) {
    i = 2;
    do {
      a[i] = a1 + i;
      i += 1;
    } while (i < n);
  }
}
```

Листинг 5.34: После каноникализации и пилинга

Мы снова видим нечеловеческий код, который получается из простого и понятного. Но он, конечно, гораздо эффективнее, если

только n достаточно велико, чтобы скомпенсировать лишние условные переходы. В нём нет постоянной загрузки $a[1]$, у нас не сбивается кеш, обращения к памяти идут строго по линейке. Эта трансформация называется пилингом цикла (loop peeling, открутка итераций) и проиллюстрирована на рисунке 5.10.

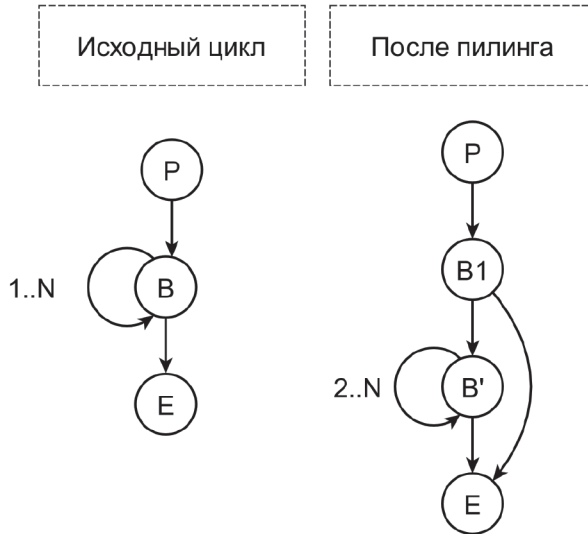


Рис. 5.10: Пилинг цикла

От слова пилинг (с ударением на первую и) происходит прекрасный квази-англицизм «отпилить». Если поставить в этом слове ударение на вторую и, то можно уже совсем на русском сказать, что мы отпилили от цикла первую итерацию. Разумеется, отпилить можно не только первую итерацию как на рисунке 5.10, а сколько угодно. Есть даже варианты пилинга, которые откручивают итерации с конца.

Можно сказать, что пилинг — это разбиение цикла пополам с полной раскруткой одной из его частей (про раскрутку см. раздел 5.2.5).

Основная проблема тут заключается в том, как именно можно обнаружить возможность для такого рода трансформации. Для этого в программе необходимо найти квази-инварианты.

Определение 5.17. *Квази-инвариантом цикла называется переменная, становящаяся инвариантной если убрать из цикла первые (или последние) несколько итераций.*

Интересная техника поиска квази-инвариантов изложена в [29]. Она основана на построении графа зависимостей переменных. Введём ещё несколько определений.

Определение 5.18. Истинной зависимостью по данным между SSA переменными x и y является использование x в определении y , если при этом в коде программы x доминирует y .

Определение 5.19. Анти-зависимость по данным между SSA переменными x и y является использование x в определении y , если при этом в коде программы y доминирует x .

В качестве очень упрощённого примера для этих определений можно рассмотреть код с листинга 5.35. В этом коде довольно очевидна истинная зависимость между x_2 и x_2 и анти-зависимость между y_2 и y_1 .

```
BB1: x1 = PHI(x0, x2);
     y1 = PHI(y0, y2);
     x2 = x1 + y1;
     y2 = 2;
     if (x2 < n) goto BB1;
```

Листинг 5.35: Пример простых зависимостей

Задание 5.25. Прежде чем читать дальше, найдите все остальные зависимости на листинге 5.35.

Теперь настало время построить очередной довольно удобный граф, работа с которым позволит просто определять квази-инварианты. Обычно главные вопросы к любому графу — это какие у него вершины и какие у него рёбра.

Определение 5.20. Графом зависимости переменных (*variable dependency graph*, VDG) является граф, вершинами которого являются SSA значения, а рёбрами — истинные зависимости и анти-зависимости.

VDG для листинга 5.35 проиллюстрирован на рисунке 5.11. Сплошными стрелками показаны истинные зависимости, прерывистыми — антизависимости.

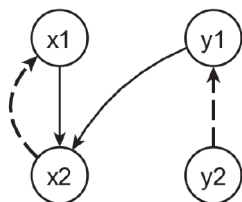


Рис. 5.11: VDG для простого примера

Если по всем путям, кончающимся в некоей вершине VDG, соответствующей φ -узлу, нет путей с вершинами из его связанных компонентов, то эта вершина входит в квази-инвариант. Так, например, для рисунка 5.11, совершенно точно этому условию с единичной длиной максимального пути удовлетворяет вершина u_1 .

Задание 5.26. *Предложите простой алгоритм для определения квази-инвариантов.*

В разделе заданий на самостоятельную работу вам будет предложено построить такого рода граф для HIR-версии листинга 5.33.

В циклах могут быть не только квази-инварианты, но ещё и квази-индуктивности, то есть после пилинга цикла могут появляться возможности для оптимизации новых индуктивностей, возникших в результате пилинга. Теорию этого процесса мы тут не рассматриваем, но вы можете изучить работу [29], где она изложена.

5.4 Структурные оптимизации

Фактически мы уже встречались со структурными преобразованиями циклов, рассматривая раскрутку циклов (см. 5.2.5). Но раскрутка относится скорее к работе с индуктивностями и на самом деле не сильно затрагивает структуру циклового гнезда. В этом разделе мы познакомимся с преобразованиями циклов, которые вносят в их поток управления существенные изменения.

Все эти преобразования технически несложно производить, имея в своём распоряжении средства скалярной эволюции для индуктивностей и дерево циклов. В разделе заданий вам будет предложено написать псевдокод для некоторых из них.

Самое простое и базовое структурное преобразование — это разбиение цикла (loop splitting).

```
for (i = 0; i < n; i++) {  
    if (i < m)  
        smth();  
    other();  
}
```

Листинг 5.36: Пример до разбиения

Такой цикл, как на листинге 5.36, содержит двойную проверку условия и может быть разбит на тот, где условие выполнено, и оставшуюся часть. Получившаяся в итоге пара циклов на листинге 5.37 напрашивается на дальнейшие преобразования.

```
for (i = 0; i < min(m, n); i++) {  
    smth();  
    other();  
}  
for (i = m; i < n; i++) {  
    other();  
}
```

Листинг 5.37: Пример после разбиения

Например, первый из двух получившихся циклов можно распределить. А во втором мы не выполним ни одной итерации, если $n < m$ и можно убрать минимум из инициализатора.

Как уже было отмечено выше, пилинг цикла можно представить как его разбиение по итогам анализа квази-инвариантов и далее, как полную раскрутку верхней части. Очень многие вещи можно свести к разбиению цикла и какому-то остатку работы после разбиения. Впрочем, и само разбиение может выглядеть иначе.

5.4.1 Распределение и соединение циклов

Продолжая тему разбиения, давайте подумаем вот о чём. Бить цикл мы можем не только по его итерационному пространству, но и по телу цикла. На листинге 5.38 приведена следующая распространённая ситуация: один цикл изменяет несколько массивов в памяти.

```
for (i = 0; i < n; i++) {  
    a[i] = i;  
    b[i] = i;  
}
```

Листинг 5.38: Пример с плохой локальностью кеша

При таком подходе, если a и b лежат достаточно далеко друг от друга, наблюдаются неприятные эффекты, связанные с работой кешей процессора: каждый раз мы обращаемся к далеко расположенным данным и это может фатально сказаться на скорости.

Оптимизация, которая здесь способна исправить ситуацию называется распределением цикла (loop distribution). Она делает ровно то самое разбиение по телу цикла, распределяя работу одного цикла между несколькими.

```
for (i = 0; i < n; i++)  
    a[i] = i;  
for (i = 0; i < n; i++)  
    b[i] = i;
```

Листинг 5.39: После распределения цикла

Обратная к ней оптимизация называется соединением циклов (loop fusion). Она берёт несколько разрозненных циклов и объединяет их в один, переходя от листинга 5.39 обратно к 5.38.

На листинге 5.40 проиллюстрирована очень интересная ситуация, когда выгодны обе оптимизации: и обратная, и прямая. В этом коде у компилятора практически нет шансов без объединения циклов понять, что для массива *b* все его элементы нулевые.

```
for (i = 0; i < n; i++)  
    a[i] = 0;  
for (i = 0; i < n; i++)  
    b[i] = a[i];
```

Листинг 5.40: Пример с выгодным объединением циклов

В общем случае такая задача называется «соединением для переиспользования». Мы соединяем циклы для того, чтобы далее через GVN или через CSE убрать избыточность в присвоении $b[i] = a[i]$, заменив это на присвоение $b[i] = 0$. Далее получившийся цикл можно снова распределить и выиграть локальность кеша, сведя всё к двум вызовам `memset`.

В интереснейшей работе [30] показано, что в общем случае соединение циклов для переиспользования является NP-сложной задачей. Это внезапная жемчужина среди цикловых алгоритмов. Очевидно, что распределение циклов (так же как их раскрутка и многие другие из рассмотренных здесь алгоритмов) работают на довольно простых эвристических основах, а сами по себе алгоритмически не сложны. В то же время найти оптимальное соединение циклов — неполиномиально сложно. Разумеется, есть приближённые, в том числе, жадные алгоритмы для этой задачи, имеющие разумную сложность (см. ту же работу).

Задание 5.27. Попробуйте разобраться в алгоритме сведения соединения циклов для переиспользования к известной NP-сложной проблеме наименьшего k -разреза. Для этого вам, вероятно, придётся прочитать упомянутую статью. Но это время не будет потрачено зря.

Больше про NP-сложные задачи и сведение их друг к другу можно прочитать в прекрасной работе [22].

На момент написания этой книги ни GCC ни LLVM не справляются с задачей оптимизации листинга 5.40. В обоих компиляторах слишком рано срабатывает распознавание идиомы цикла с заполнением нулями, и он заменяется на вызов функции `memset`, а попытка соединить циклы даже не предпринимается.

Но есть один частный случай объединения циклов, который работает почти всегда. Это случай их объединения при раскрутке циклового гнезда. Представьте себе рассмотренную ранее оптимизацию раскрутки (см. 5.2.5) для листинга 5.41.

```
for (i = 0; i < 2 * n; i++)  
    for (j = 0; j < m; j++)  
        use(i, j);
```

Листинг 5.41: Пример с раскруткой гнезда

Формально, раскручивая по i , мы получим дублирование тела внешнего цикла, то есть дважды внутренний цикл. Что, конечно, может быть неоптимально, поэтому сразу при проведении раскрутки внешнего цикла, производится соединение внутренних циклов.

```
for (i = 0; i < n; i++) {  
    for (j = 0; j < m; j++) {  
        use(2 * i, j);  
        use(2 * i + 1, j);  
    }  
}
```

Листинг 5.42: После раскрутки с соединением

Эта оптимизация присутствует в большинстве компиляторов и называется «unroll and jam». Интересно, что именно в английском языке очень сложен выбор между `fusion` и `jam` (это одно и то же) и между `distribution` и `fission` (тоже одно и то же). К счастью, на русском слова объединение и распределение не вызывают путаницы.

5.4.2 Оптимизации для локальности данных

Идеи локальности данных, которыми было, в частности, обосновано распределение циклов в разделе 5.4.1, можно развивать. Поскольку на момент написания этой книги все распространённые архитектуры компьютеров общего назначения существенно зависят от эффективности кешей, оптимизации для локальности данных принято выносить в машинно-независимую часть и также делать на SSA представлении. Хотя, конечно, по своей сути они крайне машинно-зависимы.

Одной из таких оптимизаций является оптимизация изменения порядка циклов. На листинге 5.43 приведён пример зануления матрицы с одним из возможных порядков цикла. Понятно, что можно поменять циклы по i и j местами. Эта оптимизация называется сменой порядка циклов (loop interchange).

```
for (j = 0; j < m; j++)  
  for (i = 0; i < n; i++)  
    a[i][j] = 0;
```

Листинг 5.43: Пример для перестановки циклов

Задание 5.28. *Попробуйте замерить производительность с одним и другим порядком циклов. Результаты могут отличаться в разы. Самостоятельно установите наилучший порядок. Определите, сможет ли ваш компилятор здесь провести замену циклов.*

В некоторых случаях, таких, как умножение матриц, никакой порядок циклов не сообщает данным локальности. Можно делать ручные оптимизации вроде предварительного транспонирования одной из матриц, но такое сложно ожидать от компилятора. Зато в некоторых случаях компилятор может произвести дробление циклов (loop tiling). Дробление основано на наблюдении, что всегда возможен переход от гнезда циклов, приведённого на листинге 5.44, к гнезду, приведённому на листинге 5.45.

```
for (i = 0; i < N; ++i)  
  for (j = 0; j < M; ++j)  
    access(a[i], b[j]);
```

Листинг 5.44: Исходный цикл для дробления

```

for (i = 0; i < N; i += B)
  for (j = 0; j < M; j += B)
    for (x = i; x < min(N, i + B); ++x)
      for (y = j; y < min(N, j + B); ++y)
        access(a[x, y]);

```

Листинг 5.45: Блочный цикл

В случае, когда доступы сконцентрированы внутри блока, данные выбираются довольно близко, и ускорение получается очень неплохим. Дробление циклов довольно сложно обосновать в компиляторе, потому что компиляторы консервативны. Обычно такую трансформацию делают люди руками. Компилятор тоже может справиться, но его об этом надо специально попросить с помощью механизмов из раздела 5.4.5.

Ещё более интересными являются полиэдральные оптимизации и аффинные трансформации циклов. В этой книге для них не хватит места, но вы можете почитать про них в [25]. Неожиданно хороший обзор аффинных трансформаций и трансформаций, основанных на цикловых зависимостях, также приведён в [37].

5.4.3 Разглаживание циклов

We swallow greedily every lie that flatters us.

– Denis Diderot

Обратной оптимизацией к дроблению является разглаживание циклов (loop flattening). При этом мы уменьшаем количество индуктивных, собирая гнездо из нескольких циклов в один. Исходное гнездо циклов с листинга 5.46 как бы разглаживается до единого цикла, представленного на листинге 5.47.

```

for (int x = 0; x < 8; ++x)
  for (int y = 0; y < 4; ++y)
    for (int z = 0; z < 16; ++z)
      use(arr[x * 64 + y * 16 + z]);

```

Листинг 5.46: Цикл до разглаживания

```

for (int xyz = 0; xyz < 512; ++xyz)
  use(arr[xyz]);

```

Листинг 5.47: Цикл после разглаживания

На момент написания книги с такой оптимизацией не справляется ни один из известных автору компиляторов, так что это чистая теория. Некоторые компиляторы делают полную раскрутку внутреннего цикла и на этом успокаиваются, некоторые не делают с этим гнездом вообще ничего, особенно при оптимизациях по размеру.

Задание 5.29. *Предложите эффективный алгоритм разглаживания циклов.*

Автор долго сомневался насчёт адекватного перевода термина «flatten». Приходящие первыми на ум слова «упрощение» и «выравнивание» выглядят слишком обобщёнными, а «выравнивание» к тому же создаёт неприятную аллюзию на машинно-зависимую оптимизацию, которая выравнивает цикл по границам линии кеша инструкций. Мы не будем рассматривать тут такие оптимизации, но я не хотел бы вводить осведомлённого о них читателя в заблуждение. Поэтому я остановился на разглаживании. Действительно, в этом случае поток управления получается более гладким.

5.4.4 Вынос инвариантного управления

В разделе 5.3 мы познакомились с инвариантными и квази-инвариантными переменными в циклах. Но инвариантным может быть и управление.

```
for (i = 0; i < n; ++i) {  
    a[i] += b[i];  
    if (condition)  
        b[i] = 0;  
}
```

Листинг 5.48: Пример инвариантного управления
до выноса

Обратим внимание, что такого рода ветвление внутри цикла очень мешает общим и афинным оптимизациям итерационного пространства, которые я упоминал в разделе 5.4.2. Если условие оказывается инвариантом цикла, то оно может быть вынесено, и цикл может быть перестроен, как показано на листинге 5.49.

```
if (condition) {  
    for (i = 0; i < n; ++i) {  
        a[i] += b[i];  
        b[i] = 0;  
    }  
} else {  
    for (i = 0; i < n; ++i)  
        a[i] += b[i];  
}
```

Листинг 5.49: Пример вынесенного инвариантного управления

Оба получившихся цикла гораздо проще, цикл по первой ветке может быть распределён, и получившаяся конструкция становится идеальна для векторизации и прочих полезных преобразований.

Такое преобразование цикла называется выносом инвариантного управления (loop unswitching). В реальной жизни контекст для него встречается не так часто, но, когда встречается, его применение позволяет сэкономить достаточно много. Также при его применении должен тонко учитываться баланс между ускорением более простого цикла и существенным увеличением размера кода. В работе [41] предложен интересный алгоритм выноса управления, цель которого — в первую очередь сократить время работы программы в наихудшем случае. В отсутствие профиля программы, это консервативная, но очень разумная стратегия.

5.4.5 Управление цикловыми оптимизациями

В большинстве структурных цикловых оптимизаций формальный анализ, необходимый для того, чтобы понять, стоит их применять или нет, довольно сложен. Он опирается на так называемые эвристики (compiler heuristics), то есть конкретные и зачастую специфичные для архитектур пороговые значения, подсказываемые опытом и здравым смыслом. Использование эвристик выходит за пределы компиляторной теории, но часто выручает в сложных ситуациях на практике. В целом компилятор ведёт свою часть анализа чуть лучше, когда у него есть информация о профиле приложения или когда он оптимизирует весь модуль в процессе ЛТО оптимизаций, но даже там

информации часто не хватает. Если же компилятор не уверен, то он ведёт себя консервативно и не пытается.

Вместо того, чтобы руками исправлять код, внося те же трансформации и делая его нечитаемым, цикловыми оптимизациями можно управлять через опции и через прагмы (`pragma directive`). Прагма это директива препроцессора с поведением, зависящим от реализации. Пример простой прагмы приведён на листинге 5.50.

```
#pragma clang loop unroll_count(4)
for (int i = 0; i < n; i++)
    a[i] = bar();
```

Листинг 5.50: Пример ручного контроля
развертки циклов

Поскольку механизм прагм зависит от реализации, в примере на листинге 5.50 указана прагма аннотированная именем конкретного компилятора. Такая прагма будет скорее всего проигнорирована другими компиляторами. Но раскрытие цикла — это довольно полезно, поэтому некоторые прагмы являются почти стандартными — то есть будут подхвачены почти всеми разумными компиляторами. Пример приведён на листинге 5.51.

```
#pragma unroll(4)
for (int i = 0; i < n; i++)
    a[i] = bar();
```

Листинг 5.51: Пример ручного контроля
в компиляторно-независимой манере

Я далее буду придерживаться хорошего тона и точно указывать компилятор, когда это возможно. Разумеется, даже указанный компилятор ничем вам не обязан. В примере на листинге 5.52 мы попросили распределить цикл.

```
#pragma clang loop distribute(enable)
for (int i = 0; i < n; i++) {
    a[i] = bar();
    b[i] = buz();
}
```

Листинг 5.52: Пример ручного управления
распределение циклов

Увы, компилятор на момент написания этой книги не смог понять из прагмы чего именно мы хотим и единственное, что он тут может, это выдать в консоль извинения вида «loop not distributed: the optimizer was unable to perform the requested transformation» или что-то вроде этого в зависимости от версии. В этот момент вы тяжело вздыхаете и идёте распределять цикл руками, не раньше.

Гораздо более хрупкий и опасный механизм управления — это опции компилятора. Например, для clang вы можете подать опцию «enable-loop-flatten» и разрешить разглаживание циклов. Но вообще-то оно на момент написания этой книги выключено по умолчанию не просто так: пасс недостаточно надёжен, недостаточно продуман и недостаточно оттестирован, и поэтому выключен по умолчанию. Включая нечто такое, вы включаете это на свой страх и риск.

5.5 ЗАДАЧИ НА САМОСТОЯТЕЛЬНУЮ ПРОРАБОТКУ

```
for (int i = 0; i < n; i++) {  
    if (foo(i)) continue;  
    if (bar(i)) break;  
    buz(i);  
}
```

Листинг 5.53: Пример цикла для каноникализации

Задание 5.30. *Представьте цикл с листинга 5.53 в каноническом виде.*

Задание 5.31. *Предложите алгоритм для построения дерева циклов, если поток управления несводимый.*

Задание 5.32. *Подумайте, что делать со скалярной эволюцией последовательностей, у которых чередуется знак разностей? Например, если переменная поочерёдно принимает значения 1 и -1, можно ли записать её цепочкой рекуррентностей?*

Задание 5.33. *Предложите алгоритм для определения скалярной эволюции распознанных алгоритмом BIV базовых индуктивностей.*

Задание 5.34. *Предложите алгоритм для определения зависимых индуктивностей и их скалярной эволюции. При необходимости посмотрите его в вашем компиляторе.*

Задание 5.35. *Переведите код с листинга 5.33 в HIR, канонизируйте цикл, при необходимости перейдите к индуктивности в виде указателя и постройте VDG для определения квази-инвариантов, который мотивировал бы применение пилинга.*

Глава 6

Межпроцедурные оптимизации

*Alone we can do so little;
together we can do so much.*
– Helen Keller

До сих пор наш оптимизатор делал не так уж и много. Все трансформации, вносимые и обычными и цикловыми оптимизациями, были в первую очередь локальными относительно функций. Но этого явно мало, ведь есть столько прекрасных вещей, которые можно сделать между функциями. По сути, функция — это универсальный строительный блок программирования, и мы пока даже не погружались в то, какими они бывают и зачем нужны. Кажется, для этого настало время.

Большинство языков программирования (все известные автору и наверняка многие другие) поддерживают режим раздельной трансляции. В этом режиме программист транслирует (компилирует, возможно частично линкует) часть программы и делает из получившегося бинарного кода статическую или динамическую библиотеку. Пример, как это сделать на С, я приводил ранее в разделе 1.4. Для того, чтобы получившаяся библиотека была полезна, она должна предоставлять некий интерфейс. Функции, входящие в такой внешний интерфейс, называются функциями, имеющими внешнее связывание. С ними связываются (собственно, линкуются) пользователи библиотеки. Чтобы такие функции были полезны, они должны иметь фиксированный ABI (если вы не помните, что это такое — вернитесь к разделу 1.3.1).

В противоположность функциям с внешним связыванием, функции с внутренним связыванием являются деталью реализа-

ции библиотеки и не видны её внешним пользователям. Также бывают сущности вообще без связывания, например, это методы классов. Крайний пример здесь — это виртуальные методы, связывание которых происходит через динамическую диспетчеризацию по таблице. И, наконец, внутреннее связывание может быть разных видов: в C++ начиная с 2020-го года существует модульное связывание. Но в любом случае все эти функции — и с модульным связыванием, и с внутренним, и вообще без него — законная жертва. Первое, что с ними можно сделать, это просто удалить, если их никто не использует. Но их также можно изуродовать: изменить им ABI, подменить тела, расклонировать, склеить, подставить и вынести. И всем этим мы займёмся, не ведая слёз сострадания.

Чтобы провести тривиальную оптимизацию удаления неиспользуемых статических функций, а также для удобства, стоит уже сейчас ввести понятие графа вызовов (call graph). Он чем-то похож на граф потока управления из раздела 2.1, но его узлами являются функции, а не отдельные базовые блоки.

Определение 6.1. *Графом вызовов функции называется ориентированный граф, вершинами которого являются функции, а ребро присутствует в случае, если одна функция вызывает другую. Рёбра бывают двух типов. Основные рёбра соответствуют прямым вызовам, когда вызываемая функция прямо указана. Вспомогательные рёбра — это рёбра косвенных вызовов.*

Как пример можно рассмотреть граф вызовов для программы, приведённой на листинге 6.1. Здесь функции `bar` и `buz` имеют одинаковую сигнатуру, и, значит, обе являются потенциальными кандидатами на вызов по косвенности.

```
using pred_t = int(int);
int foo(int n, pred_t p);
int bar(int n);
int buz(int n);

int foo(int n, pred_t p) { return bar(n) + p(n); }
int bar(int n) { return (n > 3) ? bar(n - 1) : buz(n); }
int buz(int n) { return 42; }
```

Листинг 6.1: Пример простого графа вызовов

Визуально граф вызовов для кода с листинга 6.1 показан на рисунке 6.1. Прямыми стрелками показаны прямые и пунктирными косвенные вызовы.

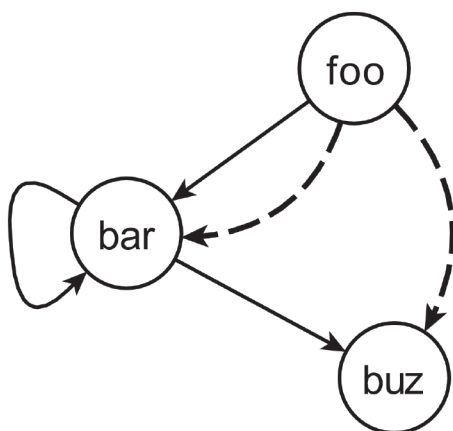


Рис. 6.1: Граф вызовов для примера с листинга 6.1

Что делать алгоритму удаления неиспользуемых функций, если в коде встретился вызов по косвенности? В лучшем случае можно попробовать доказать на этапе компиляции, что направление может быть только одним. Увы, это часто так не работает. Межпроцедурные оптимизации стараются ставить довольно рано (тот же инлайн разблокирует многие цикловые оптимизации), и код там ещё не очень почищен. Если же направление косвенного вызова, как это часто бывает, не известно на этапе компиляции, можно консервативно предположить, что целью вызова может быть любая функция с правильным количеством аргументов.

Наивный алгоритм приведён на листинге 6.2. Он довольно прост и относится к разновидности уже полюбившихся нам алгоритмов из серии «делай, пока можешь».

```

void dfe(CallGraph CG) {
    auto Changed = true;
    while (Changed) {
        Changed = false;
        for (auto F : CG) {
            if (extern(F)) continue;
            if (!CG.neighbours(F).empty())
                continue;
            CG.remove(F);
            Changed = true;
        }
    }
}

```

Листинг 6.2: Алгоритм DFEN: удаление недостижимых функций

В алгоритме DFEN метод `neighbours` в классе `CG`, как легко догадаться, возвращает количество соседей вершины по обоим классам рёбер (учитываются только входящие рёбра).

Задание 6.1. *Напишите алгоритм построения графа вызовов. Учтите косвенные вызовы и консервативно проведите рёбра.*

Алгоритм удаляет изолированные вершины из графа, а далее обходит граф ещё раз чтобы выяснить не привело ли это к образованию новых изолированных вершин и т.д. Его можно улучшить, если единожды запустить анализ сильно связанных компонентов, а далее выкинуть все компоненты, в которых нет ни одной внешней функции.

Задание 6.2. *Напишите улучшенный алгоритм удаления, основанный на анализе сильно связанных компонентов.*

Итак, граф вызовов построен, важные атрибуты функций определены и настало время приступить к празднику.

6.1 Клонирование и трансформации функций

Сложно ли в SSA представлении полностью клонировать функцию, назначив другое имя для получившейся копии? Вовсе нет. С этим справится довольно простой алгоритм, который в первую очередь воспроизводит граф потока управления, далее накладывает на него SSA граф, заполняя блоки выражениями и восстанавливая ϕ -узлы. Воспроизведение потока управления было задачей 2.25, а воспроизведение SSA-графа было задачей 4.17. Я надеюсь, они обе у вас решены. Из этих решений можно собрать алгоритм (назовём его алгоритмом CLN), осуществляющий простое клонирование функций, как показано в листинге 6.3.

```
int foo(int x) {
    if (x <= 2)
        return x;
    return bar(x);
}

int foo.clone(int x) {
    if (x <= 2)
        return x;
    return bar(x);
}
```

Листинг 6.3: Пример работы алгоритма CLN

Функция полностью копируется, и, если в ней были вызовы каких-то функций, они сохраняются.

Задание 6.3. *Напишите алгоритм CLN.*

К сожалению, в этом максимально простом алгоритме есть одна существенная проблема. Она заключается в том, что, если склонировать этим алгоритмом рекурсивную функцию, он сделает точную копию себя, то есть оставит рекурсивный вызов в старого себя.

```
int fact(int x) {
    if (x <= 2)
        return x;
    return x * fact(x);
}

int fact.clone(int x) {
    if (x <= 2)
        return x;
    return x * fact(x);
}
```

Листинг 6.4: Алгоритм CLN с рекурсией

Задание 6.4. *Подумайте, как бы вы решили эту проблему?*

В разделе задач вам будет предложено усовершенствовать алгоритм CLN, чтобы он лучше работал с рекурсивными контекстами.

Итак, можно считать, что клонирование мы освоили. Но какие техники могут быть основаны на такого рода клонировании и требовать его? Очевидно те, где сами функции могут изменяться.

6.1.1 Продвижение констант и прочей информации

Представим несколько вырожденный случай, когда статическая функция имеет единственное использование, и в точке использования известен один из её аргументов.

```
static int bar(int x) {
    if (x > 0)
        return x * 2;
    // more code here
}

int foo(int x) {
    // .....
    int z = x * bar(1);
    // .....
}
```

Листинг 6.5: Пример случая для межпроцедурного продвижения констант

В данном случае функция `foo` вызывает `bar` с константным аргументом. Если это единственный вызов, то мы можем сделать двухэтапное межпроцедурное продвижение констант. Первый этап: клонировать функцию `bar`, спропагировав в неё единицу и заменив её вызов в функции `foo` на вызов копии.

```
static int bar(int x) {
    if (x > 0)
        return x * 2;
    // more code here
}

static int bar.constprop.1() {
    return 2;
}

int foo(int x) {
    // .....
    int z = x * bar.constprop.1();
    // .....
}
```

Листинг 6.6: Межпроцедурное продвижение констант: клонирование

Алгоритмически пропация единицы сводится к убиранию аргумента, подстановке всех его значений в единицу и вызова алгоритма SCCP для получившегося кода.

```

void propagate_arg(Function F, int ArgNo, auto Val) {
    auto Def = F.get(ArgNo);
    if (!static(F))
        return;
    for (auto U : ssa_users(Def))
        replace_uses_with(U, Def, Val);
    if (F.cguses.empty())
        F.remove_arg(ArgNo);
    sccp(F);
}

```

Листинг 6.7: Алгоритм AP: пропaгация аргумента

Далее узел прошлой функции остаётся изолированным в графе и, если функция была статической, он может быть удалён. Заметим, если функция была не статической, мы всё равно можем создать её статическую копию.

Алгоритм обработки всего графа вызовов довольно интуитивен. Он изложен, например, в [10], но, в целом, там простая идея, основывающаяся на знакомой уже идее рабочего списка (worklist), так как в общем случае продвигать константы можно по взаимно рекурсивным вызовам, сложным и запутанным. Я уверен, читатель к этому моменту должен быть вполне способен восстановить этот алгоритм самостоятельно.

По тому же принципу можно делать пропaгацию диапазона значений (value range propagation, см. раздел 4.2.3 где описан базовый случай).

```

static int bar(int x) {
    if (x > 0)
        return x * 2;
    // more code here
}

int foo(int x) {
    // .....
    if (y > 3)
        z = x * bar(y);
    // .....
}

```

Листинг 6.8: Пример случая для межпроцедурного VRP

В случае, приведённом на листинге 6.8 может быть создана копия функции `bar`, в которой уже не будет проверки аргумента, так как она уже проделана в точке вызова.

В алгоритме AP на листинге 6.7 я просто так взял и удалил аргумент из списка аргументов функции. Это изменяет публичный интерфейс функции, поэтому я сделал это под двумя условиями: во-первых, функция статическая, а, во-вторых, изолированная. Статическая изолированная вершина в графе вызовов может появиться только по одной причине: она только что была скопирована.

Принципиально ничего не мешало бы алгоритму AP работать над неизолрованными функциями, достаточно того, что они статические. Проблема тут в следующем: если мы изменяем ABI в некоторой вершине графа вызовов, то мы должны также изменить все точки, в которых эта функция вызывалась, и построить там для неё новый вызов. Но внезапно такой трюк удивительно сложен в инфраструктуре любого современного компилятора. В разделе задач вам будет предложено попробовать сделать это для какого-либо промышленного фреймворка.

6.1.2 Эвристики и управление эвристиками

Каждый раз, когда мы клонируем функцию (и в будущем, когда мы будем делать инлайн-подстановку и так далее), мы немало рискуем. Например, представьте себе ситуацию, когда вам в четырёх точках вызова некоей функции известны четыре разных, но неизменно константных значения её третьего аргумента. Будем ли мы в данном случае делать четыре копии? А что, если сорок четыре? Понятно, что где-то пора остановиться.

Чтобы продемонстрировать, насколько реальная работа над компилятором отличается от изложения компиляторной теории, я позволю себе рассмотреть задачу межпроцедурного продвижения констант и клонирования функций в её реальной сложности.

Какие независимые параметры нам тут стоит рассмотреть? Немного подумав, я могу предложить довольно длинный список. И, вероятно, я при этом многое упустил.

1. Возможный выигрыш при продвижении константы или значения от уменьшения времени исполнения функции.
2. Добавка к возможному выигрышу, если клонирование и продвижение делает известными границы цикла.
3. Цена раздувания кода от клонирования функции.

4. Функция для эвристической оценки потенциальной полезности клонирования.
5. Порог для клонирования (клонировются все функции, набравшие оценку полезности выше пороговой).
6. Максимальное количество значений, которые мы рассматриваем для подстановки в каждый аргумент функции.
7. Максимальная глубина клонирования.
8. Максимальное количество вызывающих друг друга клонов, сделанных при клонировании рекурсивной функции.
9. Пороговое значение вероятности вызова, когда мы вообще решаем начать клонировать рекурсивные функции.

Некоторые из этих пунктов сами по себе требуют разветвлённых пояснений. Например, из чего складывается возможный выигрыш от уменьшения времени исполнения функции? Играет ли важную роль то, что она стала лучше попадать в кеш инструкций? Играет ли роль сокращение её цикломатической сложности? Получаем ли мы новые возможности для оптимизации, которые открываются из-за ставшими более простыми управляющих конструкций? И если мы отвечаем да по всем этим пунктам, то исчерпывают ли они картину?

Задание 6.5. *Стоит ли дать пользователю компилятора руками управлять тем, насколько ему важна добавка к выигрышу, если продвижение информации улучшает цикловые оптимизации? Обоснуйте ответ.*

Цена раздувания кода от клонирования функции складывается из размера клона, отнесённого к размеру модуля. Если модуль в целом уже большой и содержит много функций, то относительное раздувание кода при клонировании будет куда менее заметно, чем если модуль невелик.

Задание 6.6. *Сформулируйте обоснованную систему ограничений, показывающую, когда клонирование надо остановить. Предложите, что из этих ограничений могло бы управляться опциями, доступными пользователю. Сравните с реализацией какого-либо промышленного компилятора.*

Заметим, что, если вызов достаточно глубоко закопан внутри серии условных операторов, его клонирование может быть менее полезным, так как его исполнение менее вероятно, а код раздувается больше.

```
static int bar(int x);
static void buz(int x);

int foo(int x) {
    if (cond()) {
        // .....
        if (cond2())
            z = x * bar(y);
        // .....
    }
    while(x-- > 0)
        buz(x);
}
```

Листинг 6.9: Пример того, что не всё клонирование одинаково

Пример для этого показан на листинге 6.9. Здесь клонирование `bar` является куда более сомнительным, чем клонирование `buz`, и это надо как-то учесть в оценке выигрыша.

Функция для эвристической оценки потенциальной полезности клонирования должна каким-то образом поставить функции оценку как кандидату на клонирование и продвижение. Эта оценка может быть, например, неким коэффициентом, умноженным на выигрыш и делённым на проигрыш или что-то в этом духе.

Эти рассуждения можно продолжать. Обычно это заканчивается тем, что у пользователя появляется десяток верньеров, каждый из которых имеет удивительно смутное значение. В дальнейшем появляются различные проекты, которые пытаются подобрать лучшие значения этих ручек параметрическим поиском или с помощью машинного обучения, или с помощью ещё более сомнительных методов.

Задание 6.7. Известно довольно много проектов по подбору оптимальных опций компилятора с помощью методов машинного обучения. Найдите любой такой и попробуйте его на каком-нибудь большом проекте (например, на компиляторе). Попробуйте заметить эффект.

Всё это разнообразие и сложность начинают цвести по банальной причине. В отличие от теории, в реальности у нас нет совершенно чёткого функционала качества, по которому мы оптимизируем программу. Мы всего лишь прикладываем какие-то трансформации, которые могут делать лучше или хуже. И, по-

сколько их много, картина запутывается. Далее у прагматичного программиста не остаётся выбора, кроме как сделать две вещи:

1. Зафиксировать все эвристические параметры так, чтобы они давали лучший результат на некотором наборе бенчмарков.
2. Дать пользователю ручки, чтобы, если его набор отличается, скрутить применяемые оптимизации в нужную сторону. Так и живём.

Я постараюсь больше не пускать такого рода рассуждения на страницы этой книги, так как она всё-таки посвящена теории, а не прагматике. Но, в любом случае, когда мы думаем о применении тех или иных оптимизаций, всегда полезно иметь в виду реальность.

Задание 6.8. Вернитесь к любой из рассмотренных нами ранее оптимизаций, например, к раскрутке циклов. Оцените количество ручек, ползунков и верньеров, которые компилятор по-хорошему должен дать пользователю. Сначала сделайте это умозрительно, а потом загляните в скрытые опции вашего компилятора.

6.1.3 Частичная девиртуализация

В языках высокого уровня (таких, как SmallTalk, SELF или C++) обычным механизмом абстракции являются виртуальные функции, т.е. функции, которые диспетчеризуются динамически. Это даёт важный механизм абстракции — полиморфное поведение, то есть разное поведение при разных динамических типах аргументов. В общем контейнере или по границам общего интерфейса статический тип может быть стёрт, и при вызове мы полагаемся на динамический тип (то есть на сохранённый где-то указатель на функцию). В SmallTalk вообще все вызовы идут через динамическую диспетчеризацию. C++ устроен более консервативно, и программист должен явно попросить динамическую диспетчеризацию через ключевое слово `virtual`.

Принося очевидные выгоды с точки зрения проектирования сложных систем, динамическая диспетчеризация имеет негативное влияние на производительность. Наибольшие проблемы возникают из-за неё при инлайн-подстановке функций. Про инлайн мы подробно поговорим позднее в разделе 6.2.1, а сейчас достаточно базового понимания: если функция вызывается по указателю, её подстановка бывает тривиально невозможна, так как одному указателю может соответствовать

несколько разных функций, и конкретная может быть не известна на этапе компиляции.

Кроме того, такие вызовы добавляют в граф вызовов вспомогательные рёбра в терминах определения 6.1. Это мешает удалению недостижимых функций и отрицательно влияет на размер кода. Поэтому практически сразу после появления соответствующих механизмов абстракции началось исследование возможностей компиляторов по девиртуализации. Подробный обзор и алгоритм RTA, который мы будем здесь рассматривать, можно найти в работе [5], а его реализация в GCC подробно рассмотрена в [44].

Алгоритм RTA, приведённый на листинге 6.10, предполагает наличие метаданных относительно иерархии классов, в частности — исчерпывающего описания отношений наследования между классами. За это отвечает функция `GetFEClassesInfo`. Аргументами для RTA являются граф вызовов, в котором виртуальные вызовы не разрешены, и корневая функция, с которой начинается работа. На выходе алгоритм RTA вычисляет возможные вызываемые функции для каждой точки виртуального вызова в векторе списков VCT. Далее те точки, в которых оказывается только одна возможная вызываемая функция, могут быть девиртуализованы.

```
build_virtual_calls_targets(CallGraph CG,
                           Function Root) {
    Stack<Function> Methods;
    List<Function> CallSites;
    Vector<List<Function>> VCT(CG.size());
    auto Classes = GetFEClassesInfo();
    for (auto M : CG) Visited[M] = false;
    for (auto C : Classes) Live[C] = false;
    Methods.push(Root);

    while (!Methods.empty()) {
        auto M = Methods.pop();
        if (Visited[M]) continue;

        for (auto CS : CG.callsites(M)) {
            if (CS.virtual()) {
                VCT[M].add(CS.static_type());
                CallSites.add(CS);
                continue;
            }
        }
    }
}
```

```

        for (auto TM : CG.targets(CS))
            Methods.push(TM);
    }

    for (auto C : GetFEInstantiations(M)) {
        if (Live[C]) continue;
        Live[C] = true;
        for (auto CS : CallSites) {
            TM = C.dynamic_target(CS);
            if (!TM) continue;
            VCT[M].add(TM);
            Methods.push(TM);
        }
    }
    Visited[M] = true;
}
return VCT;
}

```

Листинг 6.10: Алгоритм RTA: построение virtual call targets

Для каждого класса *C* в алгоритме RTA поддерживается метка активности *Live[C]*. Изначально ни один класс в программе не считается активным. Алгоритм также поддерживает список уже посещенных вызовов *CallSites*. Список методов изначально пуст, и анализ начинается с корневого метода, который помещается в этот список.

Каждый метод в списке анализируется по очереди. В процессе анализа метода *M* для каждого виртуального вызова используются статический класс объекта, для которого вызывается метод, и его активные подклассы. Это позволяет определить возможные функции для виртуального вызова и добавить соответствующую информацию в *VCT*.

Поскольку другие подклассы могут впоследствии быть помечены активными, алгоритм сохраняет информацию о посещённых вызовах. Он помечает как активные все классы, созданные в этом методе (их список возвращает функция *GetFEInstantiations*). Когда новый класс становится активным, новые методы могут стать достижимыми через виртуальные вызовы, сделанные на базе нового созданного класса. Информация о виртуальных вызовах используется для нахождения этих методов и добавления информации в *VCT*.

Алгоритм заканчивает работу и возвращает построенную структуру *VCT*, которая содержит всю информацию о возмож-

ных виртуальных вызовах из каждого метода. Такого рода непрямые вызовы функций можно рассматривать как дополнительные рёбра графа вызовов, поэтому в работе [44] алгоритм RTA работает сразу в терминах добавления рёбер в граф.

Задание 6.9. *Напишите на основе vst простую реализацию девиртуализации.*

Важно провести девиртуализацию достаточно рано так как она разблокирует много важных межпроцедурных оптимизаций, включая рассмотренное ранее межпроцедурное продвижение констант, а также инлайн-подстановку, к рассмотрению которой мы перейдём довольно скоро.

Задание 6.10. *Как вы думаете, надо ли вносить возможную выгоду от девиртуализации как дополнительный параметр в эвристику для продвижения информации и клонирования, рассмотренную в разделе 6.1.2?*

Алгоритм RTA, приведённый на листинге 6.10, предполагает, что на этапе его применения мы обладаем полной информацией о программе. В разделе задач вам будет предложено модифицировать его так, можно было воспользоваться частичной информацией.

6.1.4 Хвостовая рекурсия

В задаче клонирования функций, рассмотренной в разделе 6.1, мы столкнулись с очевидной проблемой клонирования рекурсивной функции. По определению, клонирование оставляет тело неизменным, но тогда у функции меняется смысл, поскольку технически она перестаёт вызывать саму себя.

Рекурсия мешает не только в этом смысле. Она попросту делает программы менее безопасными, поскольку рекурсия может приводить к исчерпанию стека. В отличие от кучи, исчерпание которой может быть легко проконтролировано программистом, исчерпание стека — это неконтролируемый взрыв и разрушение программы, которое никак нельзя обработать.

Поскольку с рекурсией столько проблем, хотелось бы максимально удалить рекурсию, где это возможно. И самым простым

случаем, где удаление точно возможно и часто даже очень просто, является случай хвостовой рекурсии (tail recursion).

Классическое определение хвостовой рекурсии можно найти, например, в [43]. Ниже оно довольно точно воспроизведено в определении 6.2.

Определение 6.2. *Вызов процедуры f из процедуры g является хвостовым вызовом (tail recursive call), если единственная вещь, которую g делает после того, как f возвращает управление — это тоже возвращает управление.*

Для используемого в этой книге HIR можно немного ослабить это определение: мы допускаем между вызовом функции и возвратом произвольное количество переходов и ϕ -узлов (если более ничего не происходит по всем путям).

```
int fact(int x) {  
    if (x < 2) return 1;  
    return x * fact(x - 1);  
}
```

Листинг 6.11: Рекурсивный факториал

Программа с листинга 6.11 представляет собой рекурсивную функцию, вычисляющую факториал.

Задание 6.11. *Является ли программа на листинге 6.11 примером хвостовой рекурсии?*

Интуитивно это пример хвостовой рекурсии. Но в смысле приведённого выше определения 6.2 — ничего подобного. Мы ведь должны ещё домножить на x . Это куда более очевидно в HIR на листинге 6.12.

```
int fact(int x) {  
BB0: if (x > 1) goto BB1; else goto BB2;  
BB1: x0 = fact(x - 1);  
    x1 = x0 * x;  
    goto BB2;  
BB2: x2 = PHI(x1, 1);  
    return x2;  
}
```

Листинг 6.12: HIR для наивного факториала

Чтобы сделать факториал представляющим действительно хвостовую рекурсию, его следует трансформировать в следующий вид, приведённый на листинге 6.13.

```
int fact_prime(int r, int x) {
BB0: if (x > 1) goto BB1; else goto BB2;
BB1: r0 = r * x;
      x0 = x - 1;
      r1 = fact_prime(r0, x0);
      goto BB2;
BB2:
      r2 = PHI(r, r1);
      return r2;
}
int fact(int x) {
      return fact_prime(1, x);
}
```

Листинг 6.13: HIR для хвостовой рекурсии

Рисунок 6.2 служит иллюстрацией для двух вариантов потока управления: в случае, если хвостовая рекурсия есть, и в случае, если её нет.

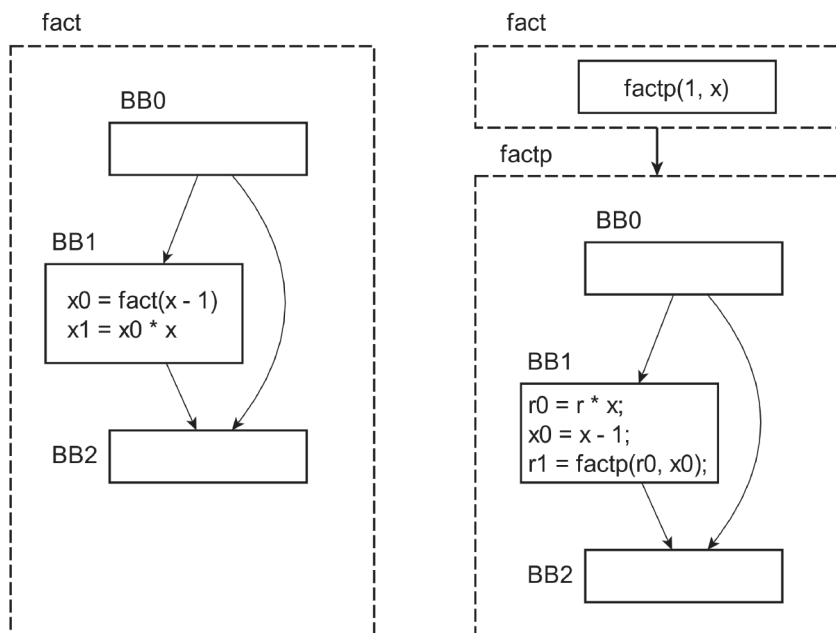


Рис. 6.2: Хвостовая и обычная рекурсия

На рисунке 6.2 не обозначены некоторые операции в блоках, сводящиеся только к переключиванию данных (такие, как φ -узлы и прочее). После того как хвостовая рекурсия схлопывается в цикл, вспомогательную функцию можно заинлайнить.

Трансформация листинга 6.13 теперь очевидна. Фаза анализа состоит из обнаружения того, что между рекурсивным выходом и возвратом ничего нет, кроме переименований переменных и φ -узлов. Фаза трансформации меняет вызов на обратную дугу к началу функции.

Задание 6.12. *Напишите анализ и трансформацию для настоящей хвостовой рекурсии и проверьте его с помощью карандаша и бумаги на листинге 6.13.*

Основной проблемой в компиляторе теперь является трансформация процесса из рекурсивного в процесс с хвостовой рекурсией с помощью введения дополнительных счётчиков. Это интересная задача, но для её решения нет общего алгоритма. Современные компиляторы справляются с ней в основном эвристически.

В разделе заданий вам будет предложено разобраться с тем, как ваш компилятор трансформирует хвостовую рекурсию в случае вычисления чисел Фибоначчи.

6.2 Подстановка и факторизация функций

*We believe that the user is responsible
for the logical structure of a program, not the physical structure.*

– Robert W. Scheifler

Функция — это важный механизм абстракции, значение которого для структурирования кода исключительно велико. Обычно программисты принимают решение о том, что вынести в функцию, руководствуясь исключительно высокоуровневыми соображениями, такими как удобство понимания и поддержки кода, не особенно задумываясь какой эффект это произведёт на производительность. Многие, включая автора этой книги, согласны, что это и есть правильный подход. Человек должен думать о человеческих проблемах, тогда как проблемы производительности должен (в меру сил) решать компилятор.

Иногда функции в прикладной программе оказываются полностью переработаны. С человеческой точки зрения, код выглядит как стройная последовательность небольших функций, логично вызывающих друг друга. Однако в финальном ассемблере всё может

оказаться слеplено в единый ком с торчащими из него ошметками и странными вызовами, которых в исходном коде никто не делал.

Для такого рода переразбивки у компилятора есть (несколько упрощенно) две ручки: подстановка (inline) тела вызываемой функции в точку вызова с удалением инструкции вызова и вынос (outline), то есть собственно помещение части функции в отдельную функцию и вставка инструкции вызова. Увы, разумного способа сохранить англоязычную симметрию слов inline и outline тут нет (не называть же подстановку вносом).

Для производительности программ подстановка, как правило, более важна, чем вынос, поэтому начнем с неё.

6.2.1 Подстановка функций

За счёт чего происходит выигрыш в подстановке и что нас ограничивает в том, чтобы делать её всегда? В первую очередь при подстановке мы экономим время, которого требует вызов функции, а также передача аргументов, исполнение её пролога и эпилога и т.д. Это прямая выгода от инлайна. Непрямая, но очень существенная выгода заключается в тех оптимизациях, которые инлайн-подстановка разблокирует.

Первые попытки поставить задачу инлайн-подстановки формально и решить её в общем случае были предприняты довольно рано (см., например, прекрасную работу [52]).

Определение 6.3. *Дана программа, в которой известны все вызовы и все функции. Для неё задан максимальный допустимый размер всей программы и максимальный допустимый размер каждой функции. **Задача подстановки** состоит в том, чтобы найти для неё последовательность подстановок, которые минимизируют время исполнения.*

Задача подстановки в определении 6.3 поставлена как задача минимизации. Но у неё также есть форма с ответом да или нет: для заданного времени исполнения ответить существует ли такая последовательность подстановок, которая сократит время исполнения до не более чем заданного.

Оказывается, что в общем случае проблема подстановки неполиномиально сложна. Можно довольно легко свести к ней известную NP-сложную проблему упаковки рюкзака: для чисел $i_1, \dots, i_k$ найти такую подпоследовательность, сумма которой равна k . Для этого сконструируем для каждого числа i_j две процедуры P_j и R_j , таким образом, чтобы каждая P_j содержала два вызова R_j , только один из которых

является кандидатом на подстановку. При этом каждая подстановка, если случается, стоит ровно i_j места и экономит ровно i_j времени. Я думаю, читатель способен закончить редукцию самостоятельно.

Задание 6.13. Завершите сведение задачи о рюкзаке к обобщённой инлайн-подстановке.

Больше про NP-сложные задачи и сведение их друг к другу можно прочитать в прекрасной работе [22].

Впрочем, тот факт, что у задачи подстановки нет оптимального решения за полиномиальное время, не означает, что у неё нет разумного приближённого решения.

Одно из таких решений (чем-то похожее на используемое в современных автору компиляторах) основано на жадном алгоритме и представлено на листинге 6.14. Предположим, что заданы константы `FSIZE_LIMIT` и `MSIZE_LIMIT`, означающие максимальные размеры для одной функции и для всего модуля. Также предполагается заданным предикат `is_candidate`, который определяет, является ли функция в принципе кандидатом на инлайн. И, наконец, задана функция `priority`, которая возвращает относительный приоритет функции.

```
greedy_inline(CallGraph CG) {
    PriorityQueue<Function> PQ;

    for (auto M : CG)
        if (is_candidate(M))
            PQ.push(M, priority(M));

    while (!PQ.empty() && CG.total_size < MSIZE_LIMIT) {
        auto M = PQ.pop();

        for (auto CS : CG.callsites(M)) {
            if (CS.virtual() || CS.indirect())
                continue;
            if (CS.function().size > FSIZE_LIMIT)
                continue;
            do_inline(CS, M);
            if (CG.total_size >= MSIZE_LIMIT)
                break;
        }
    }
}
```

Листинг 6.14: Алгоритм GI: жадный инлайнинг

Сердцем алгоритма GI является очередь с приоритетами и идея его работы довольно проста: все функции складываются в эту очередь и каждой назначается приоритет. Далее функция в соответствии с приоритетом вынимается из очереди и пытается подставиться во все места, куда её можно подставить.

Задание 6.14. *Наличие одной константы `FSIZE_LIMIT` — это немного ограничивающее предположение. В реальности имеет смысл следить не за абсолютным размером функции, а накладывать ограничение на её относительный рост. Продумайте эту идею и модифицируйте алгоритм GI соответствующим образом.*

В профессиональной карьере автора была смешная история, связанная с таким поведением настоящего компилятора. В системе, состоящей из двух несвязных компонент, но компилирующейся совместно в режиме LTO, очередь на инлайн оказалась общей. В итоге при внесении изменений в одну часть, сменились приоритеты и поломался инлайн в другой части системы, что привело к серьёзной деградации производительности.

Представьте себе удивление невезучего программиста, с которым такое произошло.

Задание 6.15. *Подумайте, как бы вы доработали алгоритм для устойчивости к таким случаям, так, чтобы небольшое изменение приоритетов не приводило к кардинальной смене плана подстановок. Простое жадное решение тут уже не годится, возможно вам поможет линейное программирование.*

6.2.2 Вынос функций

Если вдуматься, инлайн-подстановка создаёт в машинном коде то, чего все стараются избежать в коде, написанном людьми, а именно — она создаёт дублирование кода. Увы, такое дублирование кода, во-первых, создаёт не только инлайн-подстановка, но и, например, макросы или средства кодогенерации. А во-вторых, оно всегда очень неприятно влияет на размер программ.

Поэтому одной рукой пытаясь заинлайнить как можно больше для оптимальности, компиляторы другой рукой пытаются не просто не заинлайнить слишком много, а местами даже факторизовать общий код и вынести его в функцию там, где это

разумно, а программист не справился. Техника называется *code factoring* и её реализация в GCC неплохо изложена в [40]. Также у автора этой книги был студент, который сделал довольно успешный диплом по *code factoring* в LLVM. Но здесь мы, как обычно, отвлечёмся от прикладной конкретики и сконцентрируемся на алгоритмах.

Обычный вынос состоит из трёх этапов. Далее производится анализ: можно ли его вынести? И если нельзя, то код переупорядочивается так, чтобы сделать вынос возможным. Сам вынос делается в конце.

Чтобы это проиллюстрировать, на рисунке 6.3 приведён пример ситуации, когда вынос некоторых участков довольно прост. В частности, легко можно вынести блоки B_1 и B_2 . В то же время, если для выноса выбраны блоки B_3 и B_4 , то их вынос куда сложнее и скорее всего требует нетривиальной реорганизации кода.

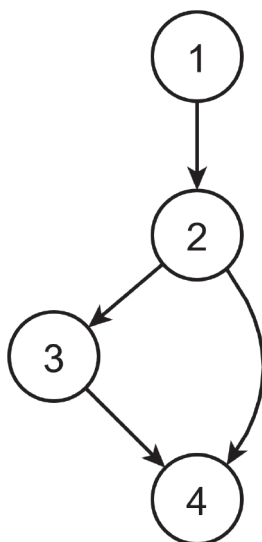


Рис. 6.3: Пример блоков для выноса в функцию

Я не буду сейчас касаться сложной теории переупорядочивания IR с целью создать такую последовательность, которая может быть извлечена. Желающие могут заглянуть, например, в [34].

6.3 ЗАДАЧИ НА САМОСТОЯТЕЛЬНУЮ ПРОРАБОТКУ

It's not failure if you enjoyed the process.

– Oprah Winfrey

Задание 6.16. В задаче 6.1 вы построили консервативный граф вызовов. Предложите комбинацию этого решения с другими компиляторными оптимизациями, чтобы убрать из этого графа рёбра заведомо недостижимых косвенных вызовов.

Задание 6.17. Улучшите алгоритм AP (см. листинг 6.7), сделав его работающим для неизолированных статических функций. Для этого вам придётся менять код в каждой точке вызова. Попробуйте разобраться, как сделать это внутри LLVM или любого другого развитого компиляторного фреймворка.

Задание 6.18. В алгоритм, изложенный в статье Каллаха-на [10], можно внести улучшение: сначала обработать все сильно связанные компоненты и только потом сделать глобальное продвижение по всему графу. При этом все решения о клонировании следует отложить до глобального продвижения. Напишите алгоритм межпроцедурного продвижения констант с учётом этого улучшения.

Задание 6.19. Реализуйте межпроцедурное продвижение диапазонов значений, обобщая решение задачи 4.24.

Задание 6.20. Реализуйте частичную девиртуализацию в отсутствие полной информации о программе. Какие решения будут вынужденно более консервативными по сравнению с алгоритмом RTA (см. листинг 6.10)?

```
unsigned fib(unsigned x) {  
    if (x < 2)  
        return 1;  
    return fib(x - 1) + fib(x - 2);  
}
```

Листинг 6.15: Исходный код для чисел Фибоначчи

Задание 6.21. Подумайте над тем, как бы вы удаляли хвостовую рекурсию в случае чисел Фибоначчи. Пример функции задан на листинге 6.15. Посмотрите, как с этим справляется ваш компилятор. Если у вас несколько компиляторов, посмотрите во все,

получите массу удовольствия (по крайней мере автор на момент написания книги был удивлён тому, какие разные трансформации претерпевает такой простой код).

Задание 6.22. При инлайн-подстановке, могут ли возникнуть проблемы с переполнением стека и, в принципе, с размером итогового стекового фрейма? Как бы вы их решали?

Задание 6.23. Одна интересная, но не рассмотренная в этом разделе оптимизация — это замена хвостового вызова на условный переход. Тогда вместо того, чтобы в каждой точке вызова функции вставлять её тело, мы вставляем прыжок на её тело, убирая, собственно, вызов функции. Спрототипируйте эту оптимизацию, что вы о ней думаете? Применима ли она в современных компиляторах?

Глава 7

Разрушение SSA

*Mistletoe killing an oak
Rats gnawing cables in two
Moths making holes in a cloak
How they must love what they do!*
– Rudyard Kipling

Итак, мы сделали много оптимизаций, описанных в предыдущих главах этой книги, и ещё больше тех, описать которые у автора не хватило места, времени или знаний. И для большинства из них мы, разумеется, использовали SSA представление. Но всё когда-нибудь заканчивается, заканчивается и процесс компиляции (хотя иногда я начинаю сомневаться в этом тезисе, где-то после четырёх часов пересборки крупного проекта). В конце же процесса компиляции необходимо построить ассемблер целевой архитектуры. В общих чертах я уже описал последовательность шагов для этого в самом начале книги, где мы обозревали вещи с птичьего полёта, см. раздел 1.2.5. Но дьявол — в деталях.

Название «разрушение SSA» для этого раздела выбрано скорее для красоты. Им мы тоже займёмся, но в первую очередь здесь мы поговорим о генерации кода. Три кита кодогенерации — это выбор инструкций, распределение регистров и планирование инструкций. Только распределение регистров требует удаления ϕ -узлов и тотального разрушения SSA. Остальные фазы разрушают скорее высокий уровень абстракции и нашу способность на нём оперировать. Выбор инструкций сохраняет SSA представление, только понижает уровень IR. Что до планирования инструкций, то планировать можно и со свойствами

SSA и без них (с разными целями и с разным уровнем итоговой реалистичности этих планов).

В разделе 1.2.3, где мы определяли HIR, было введено также понятие LIR — низкоуровневого промежуточного представления. Прежде чем начинать путешествие в генерацию кода, опишем структуру LIR более-менее формально, так как над ним (и его вариантами) мы будем работать в этом разделе.

Структура этого промежуточного представления также очень проста (сравните со структурой HIR из раздела 1.2.3) и суммируется в нескольких несложных пунктах.

1. Каждое вычисление содержит не более одной машинно-зависимой операции. Их список довольно очевиден и зависит от машины. Тот, которым мы будем далее пользоваться, отражает некий усреднённый RISC-ассемблер: арифметика, операции с памятью, различные переходы на метки.
2. Переходы на метки, которые могут быть условными или безусловными. Для простоты откажемся от регистра флагов и будем кодировать сравнение вместе с переходом.
3. Все операции производятся над виртуальными v_i или физическими r_i регистрами, размеры которых тоже машинно-зависимы. На условной нашей физической машине все регистры имеют размер 64 бита и естественные возможности для работы с младшими частями.
4. Физические регистры могут быть разбиты на несколько классов и некоторые операции могут не иметь смысла для некоторых классов. Тогда, чтобы не отделять их цифрами (например, не назначать r_{32} первым регистром в классе регистров для операций с плавающей точкой), мы будем давать им синонимы, например, f_0 будет значить то же, что и r_{32} , но, кроме того, кодировать в своём имени свой класс.
5. Разновидностью физических также являются аргументные регистры a_i .
6. Разделителем операций является точка с запятой, несколько операций могут идти на одной строке (это, как и в случае с HIR, сделано для экономии места в книге).

Для наивного факториала с листинга 6.12, LIR будет выглядеть так, как показано на листинге 7.1.

```
int fact(int) {  
BB0: BGT a0, 1, BB1;  
    JMP BB2;  
BB1: v0 = SUB a0, 1;  
    v1 = CALL fact, v0;  
    v2 = MUL v1, a0;  
    JMP BB2;  
BB2: v3 = PHI v2, 1;  
    RET v3;  
}
```

Листинг 7.1: LIR для наивного факториала

На листинге 7.1 использованы инструкции условной RISC-машины, которые, я надеюсь, интуитивно понятны. Например, BGT это «branch if greater», MUL это «multiply» и так далее.

Обратите внимание, что в принципе, на виртуальных регистрах мы можем сохранять SSA-представление (и на листинге 7.1 оно сохранено), но в будущем мы сможем его разрушить, используя явные копирования. Виртуальных регистров в программе — сколько угодно, количество физических ограничено в платформенно-специфической манере.

Задание 7.1. *Напишите LIR с виртуальными регистрами и SSA для чуть менее тривиального HIR, например, для листинга 5.21. Попробуйте отследить, как вы делаете перевод, это поможет сформировать интуицию.*

Первая задача, которую надо решить на предстоящем в этом разделе пути, теперь очевидна. Как перейти к LIR от трёхадресного кода? Для этого надо осуществить выбор инструкций.

7.1 Выбор инструкций

Everything you are comes from your choices.

– Jeff Bezos

Прежде чем переходить к правдоподобным машинным инструкциям, я предлагаю начать выбор инструкций (instruction selection) с несколько игрушечной модели. Следуя статье [9], давайте рассмотрим исключительно упрощённую форму этой задачи в постановке для стековых машин.

Определение 7.1. *Стековой машиной называется машина с одним регистром ACC и неограниченной памятью, ячейки которой пронумерованы. Она поддерживает следующие машинно-зависимые операции:*

1. LD *a* помещает данные из ячейки *a* в ACC.
2. ST *a* помещает данные из ACC в ячейку *a*.
3. OP *a* выполняет арифметическую операцию (сложение, умножение и т.д.) над ACC и ячейкой *a* и помещает результат в ACC.

В качестве примера рассмотрим вычисление на стековой машине одновременно двух выражений: $(a + b)/(b - c)$ и $(b - c) * d$. В этом вычислении мы можем переиспользовать общее подвыражение, поэтому граф для вычисления будет не деревом, а направленным ациклическим графом (directed acyclic graph, DAG), как на рисунке 7.1. Это общая особенность задач выбора инструкций: большинство из них оперируют над DAG.

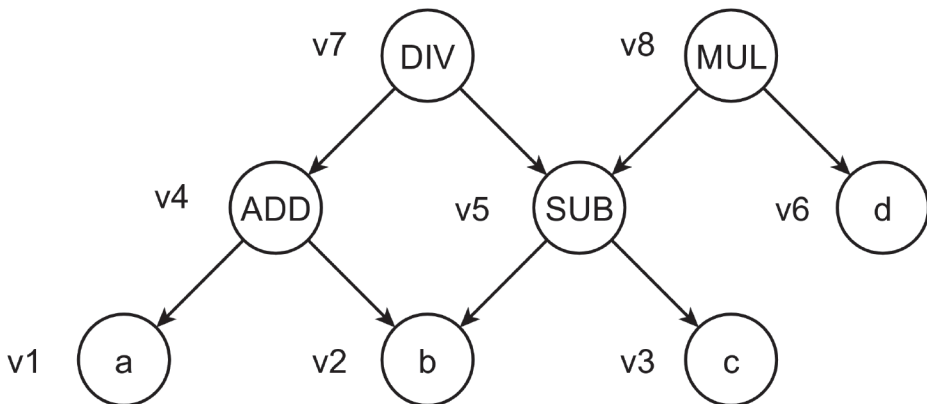


Рис. 7.1: Простейший Selection DAG
для стековой машины

Определение 7.2. *Графом выбора инструкций (Selection DAG) называется граф, в котором узлами являются инструкции LIR, а рёбра задают связь между ними по данным.*

Программа стековой машины для рисунка 7.1 приведена на листинге 7.2. Начальное значение ячеек памяти: $[v_1, v_2, v_3, v_6]$, при этом ячейки нумерованы с единицы, и v_6 лежит в ячейке с номером 4.

```
LD 2; SUB 3; ST 5;          // v5 to cell 5
MUL 4; ST 6                 // v8 to cell 6
LD 1; ADD 2; DIV 5; ST 7;   // v7 to cell 7
```

Листинг 7.2: Программа стековой машины

После работы этой программы, которая требует девять машинных инструкций и семь ячеек памяти, результаты вычисления интересующих нас выражений окажутся на стеке.

Задание 7.2. Нарисуйте на листочке ячейки памяти и исполните эту программу. Убедитесь, что в ней нет ошибок.

Теперь, когда вы потренировались, давайте подумаем, как поставить задачу EVAL в общем виде.

Определение 7.3. Проблема EVAL заключается в следующем. Заданы Selection DAG и число n . Необходимо определить существует ли программа для стековой машины для вычисления всех значений вершин с нулевой входной степенью не более, чем за n шагов.

Задание 7.3. Не читая дальше, попробуйте предложить свой алгоритм решения EVAL.

В статье [9] показано, что даже для тепличных условий стековой машины проблема EVAL NP-сложна: к ней сводится известная NP-сложная проблема SAT (и наоборот). Конкретно авторы статьи доказывают, что задачу SAT можно представить, как selection DAG, в котором, если программа, вычисляющая его за определённое количество шагов, существует, она даёт выполняющее формулу присвоение.

Задание 7.4. Прочитайте упомянутую статью и осознайте алгоритм сведения SAT к EVAL.

Подробнее про SAT и прочие NP-сложные задачи можно прочитать в замечательной работе [22], которую автор горячо рекомендует.

Разумеется, аналоги проблемы EVAL для более сложных машин с разными классами регистров и их большим количеством, будут в лучшем случае не легче, поэтому практические решения

проблемы выбора инструкций в компиляторах основаны на тех или иных приближённых алгоритмах.

7.1.1 Перевод в представление SDHIR

Как уже было сказано, простейшая стековая машина (см. определение 7.1) не очень похожа на целевые архитектуры современных компиляторов. Такие архитектуры обычно содержат большое количество операций, которые разными способами связывают разные регистры.

Первое упрощение, на которое следует пойти, чтобы сделать практически невозможную общую постановку задачи достижимой — это перевод HIR в специальное представление SDHIR, играющее роль прослойки между HIR и LIR.

Определение 7.4. *Представлением HIR на Selection DAGs является представление, в котором программа на HIR представлена иерархической последовательностью подпрограмм, таких, что никакая из этих подпрограмм в своём графе потока данных не содержит обратных (в смысле определения 5.1) и обращённых (в смысле определения 5.3) дуг. Такая подпрограмма называется вырезанным из HIR ациклическим подграфом для выбора инструкций или Selection DAG (SDAG), а само представление сокращённо называется SDHIR.*

Например, для HIR с листинга 7.3 использует одну внешнюю переменную и устанавливает две переменные, которые гипотетически могут быть использованы снаружи него.

```
BB1: x.1 = PHI(x.0, x.2);  
      tmp1 = x.1 + x.0;  
      tmp2 = x.1 - x.0;  
      x.2 = tmp1 * tmp2;  
      if (x.2 > 10) goto BB2; else goto BB1;
```

Листинг 7.3: HIR для примера перевода в SDHIR

Чтобы перевести его в SDHIR, заменим внешние источники значений на виртуальные регистры, а базовый блок на DAG. Результат можно видеть на листинге 7.4. Верхняя строчка — это комментарий для простоты понимания. Реально регистры r_0 и r_1 устанавливаются в других базовых блоках.

```

DAG1: // r0 <- PHI(x.0, x.2), r1 <- x.0
      x.1 = r0
      tmp1 = x.1 + r1;
      tmp2 = x.1 - r1;
      x.2 = tmp1 * tmp2;
      r0 <- x.2; r2 <- x.1;
      if (x.2 > 10) exec DAG2; else exec DAG1;

```

Листинг 7.4: SDHIR для листинга 7.3

Таким образом, если в теле функции встречается цикл, мы вырезаем отдельным ациклическим подграфом по данным его тело. В простом случае можно построить по одному Selection DAG для каждого базового блока. Далее ациклические подграфы сшиваются в исходную программу.

Стрелочка `<-` в SDHIR используется для присвоения модифицированной переменной. Внутри каждого Selection DAG мы наслаждаемся свойствами SSA, присвоение модифицируемым регистрам нужно для сшивания кусков.

```

int sumeven(int *x, int n) {
    int sum = 0;
    for (int i = 0; i < n; ++i)
        if ((x[i] % 2) == 0)
            sum += x[i];
    return sum;
}

```

Листинг 7.5: Пример: суммирование чётных элементов

Используем пример суммирования чётных элементов, с листинга 1.14. Я повторил его на листинге 7.5, чтобы читателю не надо было возвращаться в начало книги. После всех оптимизаций, его представление в HIR будет выглядеть как показано на листинге 7.6.

```

int sumeven(int *x, int n) {
BB0: if (n > 0) goto BB1; else goto BB3;
BB1: widen.0 = (i64) n; x.0 = x;
      goto BB2;
BB2: widen.1 = PHI(widen.0, widen.2);
      x.1 = PHI(x.0, x.2);
      sum.0 = PHI(0, sum.1);
      t0 = *x.1; t1 = t0 & 1;
      t2 = (t1 == 0) ? t0 : 0;

```

```

    sum.1 = sum.0 + t2;
    x.2 = x.1 + 4;
    widen.2 = widen.1 - 1;
    if (widen.2 == 0) goto BB3; else goto BB2;
BB3: sum = PHI(sum.1, 0);
    return sum;
}

```

Листинг 7.6: HIR для суммирования чётных элементов

Задание 7.5. *Представлен ли цикл на листинге 7.6 в каноническом виде? Вынесен ли у него прехедер? Является ли его представление LCSSA?*

Задание 7.6. *Наивный HIR для листинга 1.14 был приведён на листинге 1.15. Попробуйте оценить, сколько и каких оптимизаций из уже рассмотренных нами в предыдущих частях нужно, чтобы трансформировать его в листинг 7.6. Да, тут вам придётся немного полистать назад, но это всё-таки задача.*

В дальнейшем мы будем говорить о задачах выбора инструкций на ациклических подграфах. Если приглядеться, то и ранее, в случае стековой машины, я не рассматривал возможные циклы. Это связано с тем, что общая задача выбора инструкций становится гораздо сложнее, если мы их выбираем с учётом обратных дуг. В литературе она обычно рассматривается как совершенно абстрактная, конкретные же алгоритмы в компиляторах обычно работают над ациклическими подграфами. Как будет показано далее в разделе 7.1.4, даже для ациклического подграфа это в общем случае NP-сложная задача — есть над чем поработать.

```

DAG2: widen.1 = r0; x.1 = r1; sum.0 = r2;
      t0 = *x.1; t1 = t0 & 1;
      t2 = (t1 == 0) ? t0 : 0;
      sum.1 = sum.0 + t2;
      x.2 = x.1 + 4;
      widen.2 = widen.1 - 1;
      r0 <- widen.2; r1 <- x.2; r2 <- sum.1;
      if (widen.2 == 0) exec DAG3; else exec DAG2;

```

Листинг 7.7: SDHIR для суммирования чётных элементов

Граф с листинга 7.7 визуально представлен на рисунке 7.2

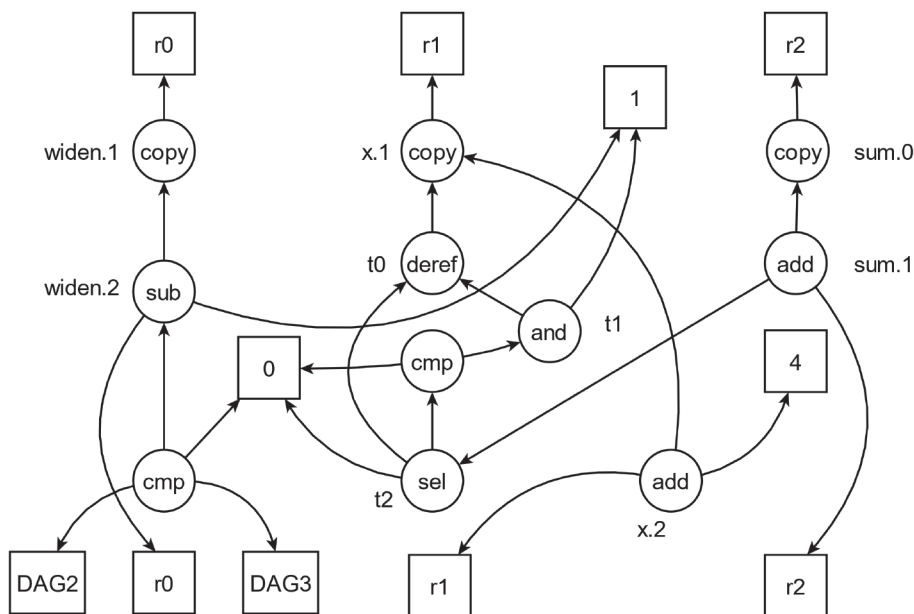


Рис. 7.2: DAG2 в SDHIR с листинга 7.7

Второе упрощение: в изложении алгоритмов я далее буду считать типы и операции легальными. Легальный тип это тот, над которым может оперировать целевая машина.

Например, если размер типа `long` равен 64 битам, а целевая машина принципиально 32-битная, то, прежде чем выбрать инструкции, необходимо разбить на части операции над переменными этого типа, если это возможно, или вообще заменить их на интринсики. Обычно интринсики далее раскрываются в вызовы библиотечных функций, которые делают эти операции возможными.

Задание 7.7. В листинге 7.7 была использована операция выбора операнда в зависимости от условия $a = x ? b : c$, которой может и не быть в целевой машине. Попробуйте легализовать DAG2 для целевой машины, в которой такой инструкции нет.

7.1.2 Легализация и выбор инструкций

В современных компиляторах используется один или несколько проходов **легализации**, то есть приведения типов и операций (а иногда ещё и отдельно векторов) к легальному виду. Алгоритмы этой легализации обычно не слишком интересные и состоят из перебо-

ра вариантов высокоуровневого представления и соотнесения их с ограничениями целевой машины. Частично это выносится на ручное описание внутри целевой машины, частично делается внутри инфраструктуры компилятора. В любом случае это скорее расчистка холста перед главным действием: собственно, выбором инструкций.

Если вас смущает обозначение `imm12`, перечитайте раздел 1.3.2, где я вводил понятие `immediate value`. Для фрагментов мы также указываем максимальный размер константы в битах, который разрешён кодировкой, отсюда, собственно, 12.

Задача выбора инструкций проиллюстрирована на рисунке 7.2 и на рисунке 7.3, если рассматривать их как поле для игры и как фишки для игры на этом поле. Смысл выбора инструкций — это переход от Selection DAG к соответствующим машинным инструкциям конкретной машины. Например, `m::sub` — это машинная инструкция `sub` для конкретной машины `m`. Каждая машинная инструкция представлена небольшим подграфом, который называется фрагментом (DAG fragment) или паттерном. Для каждой такой инструкции есть правила переписывания, поскольку она должна порождаться из платформенно-независимого представления исключительно с помощью этих правил. Скажем `m::sub` нельзя построить из сложения, а только из вычитания.

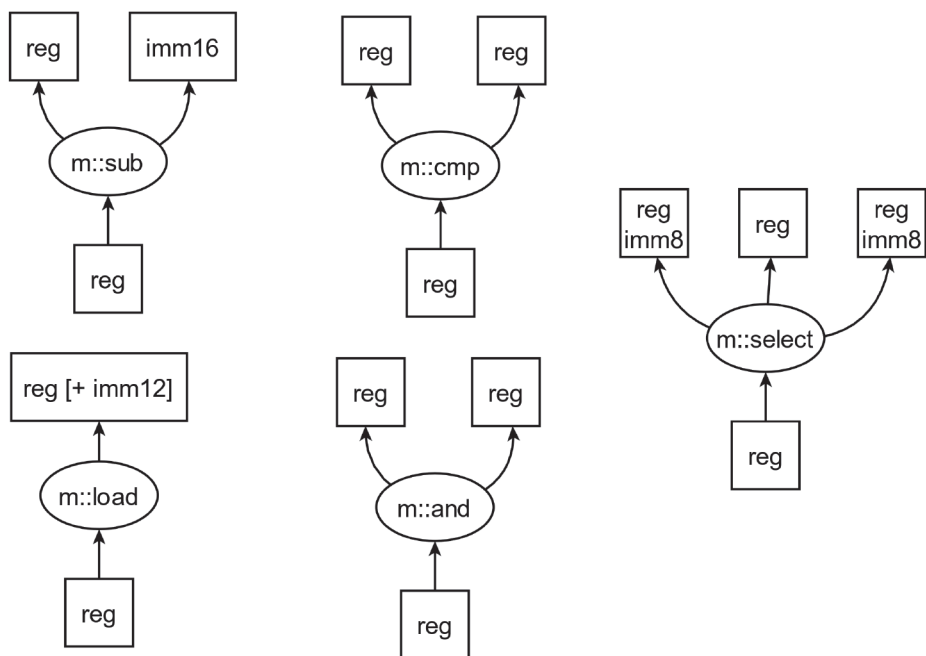


Рис. 7.3: Доступные фрагменты для выбора инструкций на рисунке 7.2

Фрагменты содержат специальные блоки-заместители (на рисунке они квадратные), у которых могут быть либо входные, либо выходные дуги (но не оба типа одновременно). Эти блоки показывают, что может быть источником для аргумента и что является результатом инструкции. При выборе инструкций блоки-заместители должны совпадать по типам. Если у какой-то машинно-зависимой операции выход — это регистр, то к этому выходу могут быть присоединены только машинные операции с регистровым же входом.

Я не буду при задании паттернов как-то отмечать коммутативность их входов, но в общем случае в описании целевой машины это, конечно, как-то учитывается. Для простоты можно принять, что все паттерны не коммутативны. Отдельный вопрос — может ли паттерн иметь много результатов, например, за одну инструкцию вычислять минимум и максимум? Этот вопрос дискутируется, но многие алгоритмы он делает существенно сложнее, поэтому пока что скажем, что нет, на уровне выбора инструкций не может. Мы можем позволить такие паттерны далее и скомбинировать в них уже выбранные узлы.

Определение 7.5. *Матчем называется отображение паттерна на подграф.*

Определение 7.6. *Покрытием (или полным покрытием, так как остальные нас не интересуют) называется набор матчей из паттернов на Selection DAG, удовлетворяющий свойству, что каждая вершина и каждая дуга покрыта один и только один раз.*

Задание 7.8. *Придумайте реалистичный набор фрагментов для покрытия графа 7.7, для операций вашей любимой архитектуры. Также постройте это покрытие вручную.*

Жадный алгоритм, который выбирает инструкции, пока может их выбрать, приведён на листинге 7.8.

```
greedy_cover(SDGraph SG, Vector<Fragment> VF) {
    if (!is_tree(SG)) error("Only trees are supported");
    auto Result = Vector<Pair<Node, Fragment>>;
    auto NNodes = SG.nodes.count();
    auto Covered = Vector<Bool>{SG.nodes.count()};
    auto NodeStack = Stack<Node>;
    NodeStack.push(SG.root());
```

```

while (!NodeStack.empty()) {
    auto N = NodeStack.pop();
    for (auto NC : N.childs())
        NodeStack.push(NC);
    if (Covered[N.number()]) continue;
    for (auto Frag : VF) {
        auto NodesUsed = Frag.matches(SG.subgraph(N));
        if (!NodesUsed.empty()) {
            for (auto NU : NodesUsed)
                Covered[SG.nodenum(NU)] = true;
            Result.emplace_back(N, Frag);
            break;
        }
    }
    if (!Covered[N.number()]) error("No pattern found");
}
return Result;
}

```

Листинг 7.8: Алгоритм GS: жадный выбор инструкций

Единственное достоинство алгоритма GS — это его простота. Он просто перебирает непокрытые узлы от корня и далее и применяет к каждому из них первый попавшийся паттерн. Если паттерн подошёл, все узлы, к которым он подошёл, помечаются как покрытые. К сожалению, единственный случай, когда он работает, это случай, когда и паттерн для выбора и Selection DAG представлены деревьями. В случае необходимости покрытия произвольного ациклического графа в нём придётся делать очень неприятный бэктрекинг (и я попрошу вас это сделать в разделе заданий).

Задание 7.9. Придумайте в качестве контрпримера ациклический граф и набор фрагментов, для которых покрытие есть, но алгоритм GS его не найдёт.

Даже если алгоритм GS нашёл покрытие, это покрытие не обязательно является оптимальным. Чтобы найти оптимальное покрытие дерева паттернами, каждый из которых также представляет собой дерево, можно использовать алгоритм, основанный на динамическом программировании и представленный в [2]. Его идея заключается в том, что, подсчитав стоимость каждого поддерева, мы можем узнать минимальную стоимость дерева, выбрав паттерн так, чтобы его стоимость и стоимость всех

поддеревьев была минимальной. Я не буду приводить этот алгоритм здесь, так как он всё-таки работает только для деревьев, а нас интересует выбор для ациклических графов. Но я оставляю его для раздела задач на самостоятельную проработку (и очень её рекомендую).

Более интересный (но тоже жадный) алгоритм, который распределяет инструкции по легализованному SDHIR приведён на листинге 7.9.

```
rewrite_cover(SDGraph SG, Vector<Fragment> VF) {
    auto Result = Vector<Pair<Node, Fragment>>;
    auto Covered = Vector<Bool>{SG.nodes.count()};
    auto NodeStack = Stack<Node>;
    for (auto R : SG.OutRegs)
        NodeStack.push(R.src());
    while (!NodeStack.empty()) {
        auto N = NodeStack.pop();
        if (Covered[N.number()]) continue;
        for (auto Frag : VF) {
            auto NodesUsed = Frag.matches(SG.subgraph(N));
            if (!NodesUsed.empty()) {
                Covered[N.number()] = true;
                for (auto NC : N.parents())
                    if (!is_constant(NC) && !is_reg(NC))
                        NodeStack.push(NC);
                Result.emplace_back(N, Frag);
                break;
            }
        }
        if (!Covered[N.number()]) error("No pattern found");
    }
    return Result;
}
```

Листинг 7.9: Алгоритм LS: жадный выбор
по легализованному DAG

Этот алгоритм начинает с выходных регистров, в которые необходимо записать значения. Метод `R.src()` выбирает инструкцию для источника значения этого регистра. Далее именно эти инструкции выбираются (по свойствам легальности это всегда возможно), а все источники их значений записываются в стек, и процесс продолжается.

В конце этого алгоритма мы имеем искомое покрытие, после чего подстановка всех фрагментов переписывает SDHIR в LIR, что и требовалось исходно.

Задание 7.10. *Напишите процедуру порождения LIR, имея легализованный SDHIR и вектор фрагментов, определяющий покрытие.*

В реалистичных компиляторах, конечно, никто не набирает вектор фрагментов. Алгоритм LS обязан сойтись из-за свойств легальности SDHIR, поэтому LIR можно начать порождать уже внутри обхода (и так, собственно, и делают). Более того, в процессе порождения LIR можно ещё и модифицировать в таргет-специфичной манере, подбирая на лету лучшие инструкции и всё в этом роде.

Задание 7.11. *Модифицируйте алгоритм LS для порождения LIR на лету по мере продвижения по SDHIR.*

Фактически именно алгоритм LS является индустриальным стандартом для выбора инструкций, и в том или ином виде он используется во всех крупных компиляторах. В последнее время набирает силу его альтернатива, основанная не на модификации HIR (связанном с построением SDHIR), а на модификации LIR.

7.1.3 Альтернативный подход: модификация LIR

Необходимость иметь представление в SDHIR, разбитое на ациклические фрагменты, серьёзно ухудшает нашу способность разбивать преобразования этого представления на пассы и теоретически мешает переиспользовать тот факт, что в каждой точке виден глобальный контекст. Вместо этого можно ввести представление обобщённого LIR, сразу сбросить в него и далее уже на нём делать все трансформации,

Определение 7.7. *Представлением GLIR на Selection DAGs является представление, в котором программа на LIR представлена в обобщённом виде, подходящем для произвольной машины. Вместо конкретных инструкций вида `m::add` такая программа может использовать обобщённые инструкции вроде `g::add`.*

На листинге 7.10 приведён фрагмент HIR, такой же как на листинге 7.3, но на этот раз мы его используем для иллюстрации перевода в GLIR.

```
BB1: x.1 = PHI(x.0, x.2);  
      tmp1 = x.1 + x.0;  
      tmp2 = x.1 - x.0;  
      x.2 = tmp1 * tmp2;  
      if (x.2 > 10) goto BB2; else goto BB1;
```

Листинг 7.10: HIR для примера перевода в GLIR

Для этого перевода достаточно механически заменить все инструкции на обобщённые инструкции LIR. Получившийся GLIR показан на листинге 7.11.

```
BB1: v1 = PHI(v0, v2);  
      v3 = G::ADD v1, v0;  
      v4 = G::SUB v1, v0;  
      v2 = G::MUL v3, v4;  
      G::BGT v2, 10, BB2;  
      G::JMP BB1;
```

Листинг 7.11: GLIR для примера с листинга 7.10

Мы сохранили ϕ -узлы, тут не придётся вставлять копирования, а потом восстанавливать их при сшивании. Кроме того, мы сохранили единство программы, у нас остались нормальные базовые блоки.

Можно заметить, что, получив такое представление в своё распоряжение, бэкэнд конкретной целевой машины вполне может заняться прямо на этом обобщённом представлении и легализацией типов, и легализацией операций.

Более того, даже алгоритм LS для этого представления будет практически таким же: его вариант для GLIR будет идти по каждому базовому блоку из конца в начало, а сами базовые блоки обходить в порядке постордера дерева доминаторов. В разделе задач вам будет предложено написать аналог алгоритма LS для GLIR.

Представление на GLIR выглядит перспективнее, чем на SDHIR. Что ещё более важно — оно открывает гипотетическую возможность сделать выбор инструкций, используя дополнительную информацию обо всей программе. Поэтому я предлагаю на нём не останавливаться, а заглянуть чуть дальше и посмотреть, что мы можем ещё, хотя бы в теории.

7.1.4 Конструкция Пробстинга

Каждый паттерн, который мы прикладываем к DAG, прикладывается грамматически, переписывая часть DAG. Это очень важное наблюдение, и надо придумать, как связать его с покрытием ациклического графа ациклическими фрагментами. В этом нам поможет одна изящная конструкция. Тодд Пробстинг (Todd Proebsting) придумал её для ещё одного доказательства NP-полноты обобщённого выбора инструкций. Он предложил построить DAG из исследуемой булевой формулы и далее выбрать по этому графу правила для её переписывания в True или False.

```
True: VARIABLE; False: VARIABLE
False: not(True); True: not(False)
True: or(True,True); True: or(True,False);
True: or(False,True); False: or(False,False);
True: and(True,True); False: and(True,False);
False: and(False,True); False: and(False,False);
```

Листинг 7.12: Конструкция Пробстинга

Если возможно таким образом выбрать для формулы инструкции, которые сводят её к True за столько шагов, сколько в ней узлов, то она выполнима.

В работе [33] даётся связь этой редукции с покрытием. Для этого вводятся специальные узлы, такие, что мы можем различать истину и ложь по каждому ребру. Истина соответствует небольшому квадратику на выходе, тогда как ложь соответствует небольшому квадратику на входе. Например, таблица истинности для конъюнкции показана на рисунке 7.4. То же самое можно построить для дизъюнкции и для отрицания.

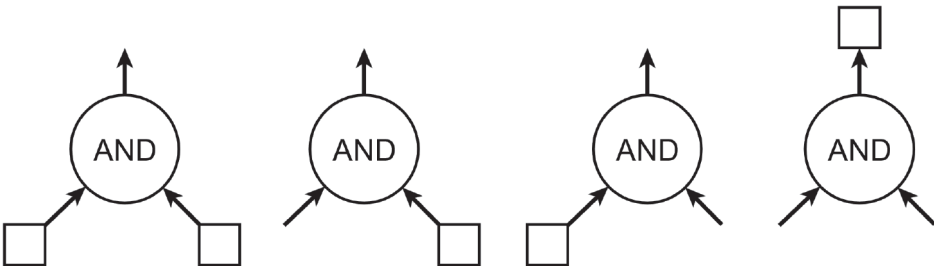


Рис. 7.4: Таблица истинности конъюнкции
в конструкции Пробстинга

Задание 7.12. Нарисуйте таблицу истинности для дизъюнкции в конструкции Пробстинга.

Теперь каждая переменная — это кружок с нулевой входной степенью. Она может матчиться в истину, если матчится с квадратиком по выходному ребру, и в ложь — если отдельно. Договоримся, что каждую переменную мы матчим либо в истину, либо в ложь по всем её рёбрам. Введём специальный кружок с нулевой выходной степенью, который будет означать, что формула удовлетворяется, если мы его матчим в истину, то есть без квадратика на входе (по каждому ребру).

$$f(a,b) = (a \wedge b) \vee (\neg b) \quad (7.1)$$

Например, замощение для формулы (7.1), которое демонстрирует, что её истинным присваиванием является $a = false, b = false$ показано на рисунке 7.5.

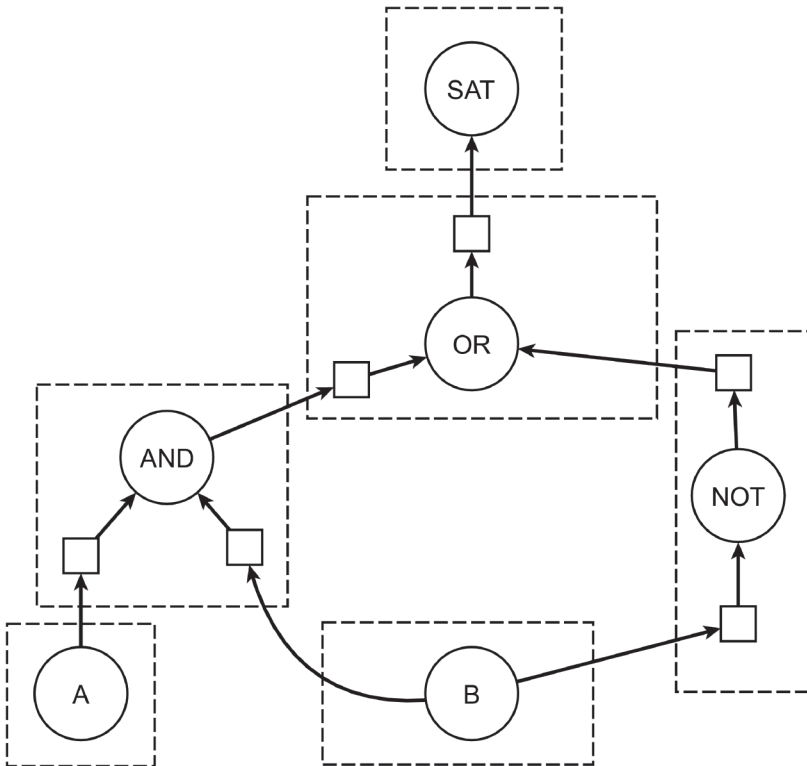


Рис. 7.5: Конструкция Пробстинга в виде замощения

Рассматривая рисунок 7.12, обратите внимание, что на каждом ребре разрешён только один квадратик, так что если его захватила формула–источник (что означает истину), то у формулы–приёмника квадратика на входе нет, что тоже означает истину. И наоборот, если квадратик отошёл к приёмнику, то его не будет у источника, что означает ложь. Таким образом, на рисунке выбраны определённые конструкции, которые, складываясь вместе в пазл, показывают выполнимость исследуемой формулы.

Задание 7.13. Выберите другие значения, например, $a = \text{true}$, $b = \text{false}$ или $a = \text{false}$, $b = \text{true}$ и попробуйте сложить пазл для них. Найдёте ли вы такое, для которого пазл не сойдётся?

Если отобразить покрытие с рисунка 7.5 как правила переписывания в системе с листинга 7.12, то получится список правил, показанный на листинге 7.13.

```
False: b; False: a;  
False: and(False,False);  
True: not(False);  
True: or(False,True);
```

Листинг 7.13: Правила переписывания для (7.1)

Эта конструкция иллюстрирует действительную сложность и важность алгоритмов выбора инструкций. Действительно, хороший алгоритм выбора инструкций вполне можно использовать как солвер для задач выполнимости.

Но, кроме того, данная конструкция показывает связь покрытия с правилами переписывания, и это понадобится нам для того, чтобы решить обратную задачу: найти какой–нибудь известный солвер, к которому можно свести выбор инструкций. Этим мы займёмся уже довольно скоро в разделе 7.1.5.

Тодд Пробстинг также является автором «закона Пробстинга» — своеобразного аналога закона Мура в компиляторах. Этот закон гласит, что только за счёт улучшения промышленных компиляторов быстродействие программ удваивается каждые 16 лет. Это даёт нам надежду, что даже когда компьютерное железо упрётся в свои естественные пределы, возможность роста останется.

7.1.5 Подход на основе PBQP

Интересным направлением атаки для решения NP-сложной задачи выбора инструкций на произвольном ациклическом графе является сведение её к задаче распределённого квадратичного булева программирования (PBQP, partitioned boolean quadratic programming). Эта задача тоже NP-сложна, но она интересна тем, что для неё есть довольно эффективные процедуры решения, которые вполне можно применять за разумное время, если рассматриваемый DAG невелик (а если строить по одному DAG на каждый базовый блок, как в разделе 7.1.1, то они обычно получаются разумных размеров).

$$f(x, c, C) = \sum_{0 \leq i \leq n} x_i^T c_i + \sum_{0 \leq i < j \leq n} x_i^T C_{ij} x_j \quad (7.2)$$

В своей минимизационной форме задача PBQP ставится как задача выбора таких векторов x_i , что при заданных c_i и заданных C_{ij} , функционал, задаваемый квадратичной формой как в уравнении (7.2), будет минимизирован. При этом есть два ограничения.

1. Элементы векторов x_i могут принимать только значения 0 или 1 (поэтому boolean programming).
2. Один и только один элемент каждого вектора x_i должен принять значение 1.

В более классической форме с ответом да или нет, PBQP ставится как задача поиска таких x_i , что функционал примет значение меньше наперёд заданного.

Разумеется, такие ограничения что-то нам говорят про x_i . Создаётся впечатление что эти вектора соответствуют выбору одной из спектра альтернатив. При этом вектор c_i оценивает (изолированно) качество сделанного выбора, а матрица C_{ij} оценивает совместимость выборов x_i и x_j . Часть элементов матрицы C_{ij} может быть установлена в *inf*, показывая, что выбор не совместим.

В построении сведения задачи выбора инструкций к PBQP, будем следовать работе [19]. Рассмотрим ещё раз SSA граф, похожий на тот, который изображён на рисунке 7.2, но сделаем его проще, чтобы не путаться в мелких деталях.

На рисунке 7.6 изображены два узла, которые нужно выбрать (они отмечены кружками), и три узла с фиксированными значениями (отмечены квадратами). Это могут быть регистры, константы, операции копирования и т. д. Вдохновляясь кон-

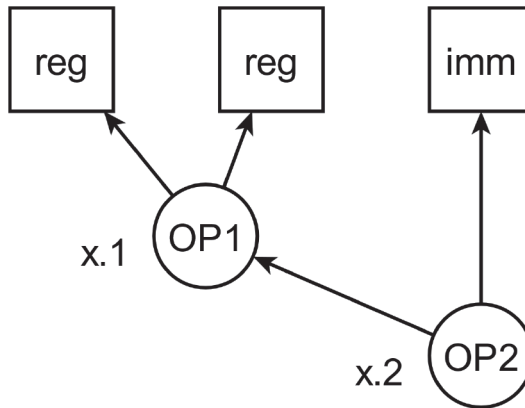


Рис. 7.6: Пример графа для сведения к RBQP

струкцией Пробстинга (см. раздел 7.1.4), представим возможные варианты выбора для этих двух узлов в виде грамматических правил, где слева находится нетерминал, соответствующий этому узлу, а справа — комбинация терминалов и нетерминалов, в которые он может раскрыться.

На листинге 7.14 показаны правила раскрытия для OP1 и каждому присвоена стоимость.

```
OP1: +[reg, OP3], 10
OP1: +[OP3, OP2], 5
OP1: +[reg, imm], 20
```

Листинг 7.14: Правила переписывания для узла OP1 с рисунка 7.6

Выбор правила для терминала OP_i как раз и соответствует буглеву вектору x_i с соответствующим ему весом.

Задание 7.14. Выпишите вектор весов c_1 для примера на листинге 7.14. Это не будет сложно. Хинт: если $x_1 = (0, 0, 1)$, то $x_1^T c_1$ будет равно 20.

Теперь давайте определимся с правилами совместимости. Если для OP_1 мы выберем третье правило, то это не срастётся с нашим актуальным графом на рисунке 7.6. Таким образом, матрица весов должна зависеть от того, насколько дёшево вывести из OP_i его зависимость OP_j .

Если возможно построить такой вывод $OP_i \rightarrow \dots \rightarrow OP_j$, то i -я строка и j -й столбец в матрице стоимости будут заполнены минимальной стоимостью такого вывода. Если этот вывод невозможен, то мы ставим там *inf*.

Тот факт, что граф для проблемы PBQP полностью соответствует SSA графу, в котором в вершинах находятся вектора, а по рёбрам — матрицы, кажется мне просто потрясающим и удивительным.

Задание 7.15. Возьмите любой реалистичный SDHIR и постройте для него сведение к PBQP.

Интересным наблюдением является тот факт, что не обязательно даже строить SDHIR: в сведении к PBQP вполне может быть циклическая структура графа, это сделает решение проблемы более сложным, но не является препятствием.

В работе [18] было исследовано влияние PBQP на производительность приложений. Для пакета SPECINT время компиляции увеличилось вдвое, а качество кода улучшилось от нуля до десяти процентов. Это не означает, что идея плохая. Просто нужно больше исследований.

7.2 Распределение регистров

*During optimization, assume an infinite set of registers;
treat register allocation as a separate problem.*

– Preston Briggs

Почему люди не пишут всё на ассемблере? В принципе, ассемблер не такой плохой язык: там есть функции, циклы, ветвления и бездна ручного контроля. Первая причина — это кроссплатформенность, но это не слишком убедительный аргумент. У нас мог бы быть вместо компилятора транслятор из ассемблера одной архитектуры в ассемблер другой архитектуры (и даже с выполнением оптимизаций). Тогда пока специализированный код не написан, можно было бы жить примерно, как сейчас.

С моей точки зрения (и с ней соглашается любой человек, написавший на ассемблере хотя бы программу размером в десять тысяч строчек кода), главной причиной, почему мы избегаем ассемблера как основного языка программирования, является тот факт, что люди очень плохи в распределении регистров. Вы пишете какие-то вычисления, при этом у вас есть конечное число

регистров. Они быстро заполняются, и поэтому важно помнить, какой регистр для чего использовался, где и в какой момент его можно использовать повторно. Можно, конечно, было бы писать на ассемблере с виртуальными регистрами, но тогда это мало отличается от C, просто менее удобно.

Кроме того, подумайте обо всех оптимизациях, которые мы уже рассмотрели, и попытайтесь представить, что было бы с каждой из них, если бы в ней мы были вынуждены думать про ограниченность ресурсов в платформенно-специфичном ключе. По этой причине уже в семидесятые годы XX века оптимизатор компилятора был отделён от его кодогенератора. Это разделение обязанностей позволило достичь в цикловых и межпроцедурных оптимизациях впечатляющих успехов: они оптимизируют всё так, как будто ресурсы вычислительной системы бесконечны.

Между тем они не бесконечны, и хорошее распределение регистров критически важно. Инструкции, работающие над регистрами, занимают меньше места в кодировке и выполняются быстрее, тогда как память сильно зависит от попаданий в кеш и может приводить к неприятным простоям конвейера. Во многих RISC системах единственная разновидность операций с памятью — это просто загрузка и сохранение. Однако даже в CISC системах, где есть, например, сложение регистрового операнда с операндом в памяти, эта операция, скорее всего, будет выполняться гораздо медленнее, чем операция регистр-регистр.

И поскольку распределение происходит совместно с финализацией смысла инструкций LIR, а никаких архитектур с поддержкой ϕ -узлов пока не известно, распределение регистров обычно предваряется разрушением SSA (иногда до этого ещё по живому SSA собирается какой-нибудь полезный анализ).

7.2.1 Разрушение SSA

Мы будем заниматься разрушением SSA, имея на входе LIR-представление на виртуальных регистрах. Поскольку регистры уже являются модифицируемыми, и сами инструкции LIR способны их модифицировать, речь на этом этапе идёт о том, чтобы просто убрать ϕ -узлы.

Прежде чем начать разрушение SSA, обозначим идеальный случай: если SSA форма только что построена, никак и никем не модифицирована, то достаточно сделать следующее. Во-первых,

определить связанную с SSA-переменной семантическую сеть, состоящую из всех использований и всех переопределений этой переменной, замкнутых рекурсивно. Мы уже сталкивались с понятием семантической сети в разделе 3.3.2. Далее можно просто назначить для всех входящих в неё переменных один виртуальный регистр.

```
int fact(int) {
BB0: BGT a0, 1, BB1;
      JMP BB2;
BB1: v0 = SUB a0, 1;
      v1 = CALL fact, v0;
      v2 = MUL v1, a0;
      JMP BB2;
BB2: v3 = PHI v2, 1;
      RET v3;
}
```

Листинг 7.15: LIR для наивного факториала ещё раз

На ещё раз продублированном мной листинге 7.15 (мы его уже видели, см., например, листинг 7.1) ситуация уже не идеальная: тут требуется вставка копирования по одной из веток и только потом переименование. LIR после этой вставки будет выглядеть как на листинге 7.16.

```
int fact(int) {
BB0: v4 = COPY 1;
      BGT a0, 1, BB1;
      JMP BB2;
BB1: v0 = SUB a0, 1;
      v1 = CALL fact, v0;
      v2 = MUL v1, a0;
      JMP BB2;
BB2: v3 = PHI v2, v4;
      RET v3;
}
```

Листинг 7.16: LIR для наивного факториала после вставки копий

Здесь семантическая сеть состоит из переменных v_2 , v_3 и v_4 . В принципе их все можно заменить на одну любую и теперь

убрать ϕ -узел. Алгоритм обнаружения семантической сети с помощью union-find метода приведён на листинге 7.17.

```
web_discovery(MachineFunction MF) {
    auto PhiWeb = MultiMap<VReg, Vector<VReg>>;
    for (auto V : MF.vregs())
        PhiWeb[V] = {V};
    for (auto P : MF.phis())
        for (auto VS : P.operands())
            union(PhiWeb[P.dest()], PhiWeb[VS]);
}
```

Листинг 7.17: Алгоритм SN: построение семантической сети

Прежде чем радостно кидаться применять этот алгоритм, посмотрите, пожалуйста, ещё раз на листинг 7.16. Вам не кажется, что код стал хуже? Смотрите: у нас не было никакой загрузки единицы в регистр, когда мы шли по ветке $B_0 \rightarrow B_1$, а сейчас оно появилось. Что, если в B_1 будет несколько выходов и один из них пойдёт не в B_2 , а в какой-то другой блок? В этом случае на этом пути у нас окажется загрузка, которая нам не нужна.

Задание 7.16. Если вы читали книгу внимательно и последовательно, не перескакивая с главы на главу, здесь у вас должно появиться ощущение, что эту булочку мы уже жевали. Не читая дальше, попробуйте сформулировать, где мы это уже встречали, и как мы там с этим боролись?

Да, вы правы, это классический случай критического ребра, которое уже мешало нам устранять избыточность, а теперь мешает разрушать SSA-представление. Если вы это не вспомнили, пожалуйста прямо сейчас посмотрите ещё раз в раздел 4.4.1, особенно внимательно перечитайте определение 4.1. Я не хочу его тут дублировать, но не хочу также вас терять прямо здесь.

Итак, прежде чем разрушать SSA-представление, нам следует (как и в случае удаления частичной избыточности) разбить критические рёбра. Затем нужно вставить копирования для переменных, чтобы вернуть SSA-представлению каноничность. LIR наивного факториала после разбиения рёбер и разрушения SSA будет выглядеть так, как показано в листинге 7.18.

```

int fact(int) {
BB0: BGT a0, 1, BB1;
    JMP BB3;
BB1: v0 = SUB a0, 1;
    v1 = CALL fact, v0;
    v4 = MUL v1, a0;
    JMP BB2;
BB3: v4 = COPY 1;
    JMP BB2;
BB2: RET v4;
}

```

Листинг 7.18: LIR для наивного факториала
после разрушения SSA

Единственная ли это проблема? Увы нет. Рассмотрим листинг 7.19. Он получен переводом ранее встречавшегося листинга 4.11 в LIR, то есть саму ситуацию создал пасс *cory propagation*, но это не принципиально, так как её могла создать и другая трансформация.

```

BB0: x0 = COPY arg0;
    y0 = COPY arg1;
    goto BB1;
BB1: x1 = PHI(x0, y1);
    y1 = PHI(y0, x1);
    BGT x1, y1, BB1;
    JMP BB3;
BB3: return x1;

```

Листинг 7.19: Использование параллельных копий

На листинге 7.19 активно используется то свойство SSA что все ϕ -операции производятся одновременно, то есть в начале B_1 происходит параллельный обмен значениями. Если вставить копии наивным способом, то их сериализация будет неправильной. В разделе задач я предложу вам придумать, как с этим бороться.

Задание 7.17. Попробуйте просто написать LIR после разрушения SSA, соответствующий листингу 7.19.

Итак, с учётом этих двух обстоятельств, SSA можно считать разрушенным и переходить, собственно, к распределению регистров.

7.2.2 Подходы, основанные на раскраске графа

Задача раскраски графа формулируется следующим образом: задан граф и ограниченное число красок. Необходимо раскрасить все вершины так, чтобы ни у каких соседних вершин не было окраски в один цвет, либо доказать, что это невозможно. Например, графы, которые можно покрасить в две краски, называются двудольными.

Приоритет в идее распределения регистров через раскраску графа принадлежит академику Ершову, который высказал эти идеи в своей работе 1966-го года [59]. Первым, кто проработал идею до состояния конкретного алгоритма был Грегори Чайтин, см. [12]. Далее Престон Бриггс существенно улучшил алгоритм Чайтина, см. [8]. В итоге весь класс алгоритмов распределения регистров через раскраску графов сейчас называется алгоритмами раскраски Чайтина–Бриггса (Chaitin–Briggs coloring). Иногда по-русски говорят: «Чейтина», но я буду стараться писать фамилию правильно, насколько это возможно. Впрочем, в русской википедии его вообще называют Хайтином, но я ни разу не слышал про алгоритм Хайтина–Бриггса в разговорах с коллегами.

Основное наблюдение является простым и в то же время красивым. Мы строим граф, основанный на концепции одно-временно активных переменных. Определение 7.8 вводит понятие интервала активности переменной, который опять-таки должен оставлять у внимательного читателя смутное чувство узнавания.

Определение 7.8. *Интервалом активности (live interval) для виртуального регистра v_i называется совокупность мест в программе, где он может быть использован, от его определения до его последнего использования.*

На самом деле мы уже встречались с анализом активных переменных для конкретной точки программы в разделе 2.3.3. Это сэкономит нам время здесь, так как мне не придётся объяснять ещё раз как его построить.

На листинге 7.20 приведён пример единичного базового блока, в котором активность переменных явно размечена.

```

BB0: v1 = a0;
      v2 = a1;
      v3 = AND v2, 0x7F;
      v4 = ADD v1, v3; // KILL v1, v3
      v5 = MUL v2, v4; // KILL v2
      v6 = DIV v4, v5; // KILL v4, v5
      goto BB1;

```

Листинг 7.20: Пример LIR для графа зависимости

Для построения графа, необходимого для распределения регистров (Ершов строил его матрицу смежности и называл её «матрицей несовместимостей»), положим, что вершинами будут виртуальные регистры, а рёбро между v_i и v_j будет присутствовать только в том случае, если они совместно активны. В этом случае граф несовместимостей (inference graph) для листинга 7.20 представлен на рисунке 7.7.

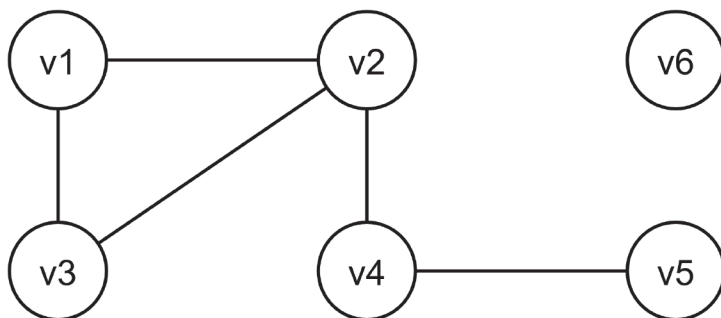


Рис. 7.7: Граф несовместимостей для листинга 7.20

Задание 7.18. Не читая дальше, оцените, во сколько минимум красок можно раскрасить граф на рисунке 7.7? Предложите раскраску, которая покажется вам разумной. Напишите LIR с физическими регистрами, соответствующий вашей раскраске.

Разумеется, граф на рисунке 7.7 содержит цикл нечётной длины и не является 2-раскрашиваемым. Назначим три краски: r_1 , r_2 , и r_3 . Тогда распределение регистров будет следующим: $v_1 \rightarrow r_1$, $v_4 \rightarrow r_1$, $v_6 \rightarrow r_6$, $v_2 \rightarrow r_2$, $v_5 \rightarrow r_2$, $v_3 \rightarrow r_3$. LIR после переименования распределённых регистров показан на листинге 7.21.

```
BB0: r1 = a0;  
      r2 = a1;  
      r3 = AND r2, 0x7F;  
      r1 = ADD r1, r3;  
      r2 = MUL r2, r1;  
      r1 = DIV r1, r2;  
      goto BB1;
```

Листинг 7.21: LIR после 3-раскраски

В общем случае граф несовместимостей может быть довольно сложен. Теперь фактическая задача распределения регистров состоит в покраске этого графа в N красок, где N — это количество доступных на конкретной платформе физических регистров. При этом две смежные вершины не должны быть покрашены в один цвет.

Относительно этой задачи сразу возникают два вопроса: что делать, если не удалось покрасить таким образом (например, N красок с гарантией не хватит для клики размером $N + 1$), и как быть с тем фактом, что задача раскраски графа в приведённых ограничениях доказанно NP-сложная?

Начнём с ответа на первый вопрос. Если регистровое давление (количество необходимых регистров в какой-то точке) превышает количество доступных физических регистров, часть виртуальных регистров требуется спилить (от английского *to spill*, что значит «перелить» то есть по смыслу: «временно вылить жидкость чтобы освободить ёмкость»).

Определение 7.9. *Спилл (spill) — это процесс временной материализации значения виртуального регистра (например, сохранение его в память). Обратный ему процесс называется восстановлением (fill).*

Восстановление регистра — это не обязательно загрузка из памяти. В некоторых случаях, например, если регистр хранил константу или простое выражение, возможна рематериализация этого значения без обращения к памяти. Но, конечно, в общем случае спилл идёт именно в память и восстанавливается именно оттуда.

Задание 7.19. *Не читая дальше, оцените, что делать с графом на рисунке 7.7, если у вас свободно только два регистра? Какие спиллы можно сделать?*

Один из возможных вариантов LIR со спиллами показан на листинге 7.22.

```
BB0: r1 = a0;
      r2 = a1;
      STORE fi#0, r2;
      r2 = AND r2, 0x7F;
      r1 = ADD r1, r2;
      r2 = LOAD fi#0;
      r2 = MUL r2, r1;
      r1 = DIV r1, r2;
      goto BB1;
```

Листинг 7.22: LIR после 2-раскраски со спиллами

Стековые слоты для спиллов (можно также говорить просто спил слоты) помечены как `fi#n`. Это аббревиатура от «frame index» т.е. у каждого спил слота есть свой индекс во фрейме функции. Они распределяются в момент, когда происходит фиксирование ABI и вычисление этих самых фреймов, что может быть несколько позже распределения регистров.

Задание 7.20. *Предложите, как на листинге 7.22 заменить работу со стеком на рематериализацию?*

Теперь можно ответить на второй вопрос. Разумеется, точно решать NP-сложную задачу в процессе компиляции не рационально — она займёт слишком много вычислительных ресурсов. Приближённое решение Чайтина–Бриггса выглядит как показано на листинге 7.23.

```
simplify(LiveIntervals LS, InferenceGraph G) {
    int N = InferenceGraph.size();
    Vector<VReg> Colorable, ForSpill;
    for (auto L : LS) {
        if (G.degree(L) < N) {
            Colorable.push(L.vreg());
            G.remove(L.vreg()); LS.remove(L);
        }
    }
    for (auto L : LS)
        ForSpill.push(L.vreg());
    return {Colorable, ForSpill};
}
```

Листинг 7.23: Алгоритм СВ: приближённая покраска графа

Алгоритм CB работает следующим образом: каждая вершина степени меньше k , где k — это число красок, точно может быть покрашена. Она удаляется из графа и кладётся в вектор `Colorable`. Процедура покраски далее извлекает вершины из этого вектора по одной в порядке обратном порядку помещения (то есть вектор `Colorable` — это своего рода стек). Перед этим все вершины, помеченные на спилл, либо спиллятся, либо рематериализуются.

Этот алгоритм довольно несовершенен, и в разделе задач вам будет предложено реализовать два его очевидных улучшения.

Дополнительно можно вводить сложные и интересные эвристики, как определяется стоимость спилла. Например, она может зависеть от того, насколько глубоко место предполагаемого спилла закопано во вложенные циклы (чем глубже, тем дороже), или руководствоваться подобными соображениями.

Обычно также в качестве дополнительной оптимизации после построения графа делается слияние интервалов (*coalescing*). Суть этой оптимизации в том, чтобы в случае, если один виртуальный регистр является точной копией другого, и их интервалы активности не пересекаются, совместить (также говорят скоалесить) их в один регистр.

Задание 7.21. *Предложите алгоритм для слияния интервалов.*

Увы, слишком агрессивное слияние интервалов может приводить к плохо раскрашиваемым графам и увеличивать количество спиллов.

Рассматривать раскрашивание Чайтина–Бриггса можно и далее, но это не слишком продуктивно, потому что этот подход на сегодня считается устаревшим, дорогим по накладным расходам и не слишком эффективным. Тем не менее, его очень полезно понимать, потому что это в любом случае «нулевой вариант», с которым можно далее всех сравнивать, и потому что он задаёт тот общий язык, на котором разговаривают прочие регистровые аллокаторы. На момент написания этой книги, алгоритм распределения регистров на основе Чайтина–Бриггса является основным в компиляторе GCC и используется там по умолчанию.

7.2.3 Линейный подход к распределению регистров

В 1997 году Массимилиано Полетто предложил алгоритм линейного сканирования, который сейчас, спустя уже почти тридцать лет, всё ещё является фактическим стандартом в оптими-

зирующих компиляторах. Достаточно сказать, что, из четырёх аллокаторов, доступных на момент написания книги в LLVM (fast, basic, greedy и pbqp), первые три основаны на идеях линейного сканирования. Изложение в этом разделе будет следовать статье [47] и идеям, почерпнутым из вариантов реализации подхода Полетто в LLVM.

Основной алгоритм распределения регистров с помощью линейного сканирования представлен на листинге 7.24. В нём предполагается, что между всеми интервалами активности установлен общий линейный порядок. Внутри базового блока такого рода порядок очевиден, внутри сложного графа потока управления его можно выстроить с использованием дерева доминаторов.

```
linear_scan(LiveIntervals LS, RegPool FreeRegs) {
    int N = FreeRegs.size();
    LiveIntervals Active = {};
    auto Coloring = Map<VReg, PhysReg>;
    for (auto L : sorted_range(LS, increasing,
                               start_point)) {
        expire_old_intervals(Active, FreeRegs, Coloring, L);
        if (Active.size() == N)
            spill_at_interval(i);
        else {
            auto PhysReg = FreePool.get();
            FreePool.remove(PhysReg);
            Coloring.add(L.vreg(), PhysReg);
            Active.add(L);
        }
    }
    return Coloring;
}
```

Листинг 7.24: Алгоритм LSRA:
распределение линейным сканированием

Алгоритм LSRA работает следующим образом: все интервалы активности сортируются по возрастанию начальной точки в упомянутом выше общем линейном порядке кода. Далее мы выбираем текущий интервал и должны объявить устаревшими все интервалы, закончившиеся ранее него. Когда интервал объявляется устаревшим, использованные для него физические регистры возвращаются в общий пул. Для этого я вынес отдельную функцию, см. листинг 7.25. Далее у нас

два варианта. Если физические регистры закончились, то мы должны сделать спилл текущего интервала. Если они не закончились (или были недавно возвращены), то мы распределяем на текущий активный интервал очередной физический регистр.

По сути LSRA — это жадный алгоритм, он работает за линейное время и у него есть простая жадная эвристика — распределять те интервалы, которые можно взять пока на них остаются регистры.

```
expire_old_intervals(/* args */) {  
    for (auto LI : sorted_range(Active, increasing,  
                                end_point)) {  
        if (LI.end_point() >= L.start_point())  
            return;  
        Active.remove(LI);  
        FreeRegs.push(Coloring[LI.vreg()]);  
    }  
}
```

Листинг 7.25: Вспомогательная функция для LSRA

Подпрограмма `expire_old_intervals` на листинге 7.25 довольно проста и носит служебный характер: по всем активным диапазонам она ищет те интервалы, которые закончились ранее, чем начался текущий, удаляет их из активных и возвращает их регистры в пул. Другую подпрограмму, `spill_at_interval`, которая делает спилл текущего интервала, если регистры закончились, вам будет предложено реализовать самостоятельно в разделе задач.

У алгоритма LSRA есть несколько возможных реализаций в настоящем компиляторном фреймворке. Например, если нас интересует максимальная скорость, можно вообще не вычислять предварительно интервалов активности, а просто для каждого базового блока идти в обратном порядке и распределять физические регистры для каждого встреченного виртуального. При проходе через точку определения регистра, он возвращается в активный пул. Если в пуле регистров не хватает, мы оставляем виртуальный не распределённым. Когда первый проход распределения закончен, для оставшихся виртуальных регистров делается слияние и далее расставляются спиллы.

Заметьте, что при таком подходе слияние уменьшает количество спиллов, а не увеличивает его как в алгоритмах, основан-

ных на раскраске графа. Этот алгоритм можно назвать LSRAF, где буква F означает Fast.

Задание 7.22. *Реализуйте алгоритм LSRAF по несколько неформальному описанию выше. Примените его и LSRA на какой-нибудь реалистичный LIR и посмотрите отличия в распределении.*

Можно также улучшить алгоритм LSRA в сторону усложнения. Например, ввести в него концепцию стоимости спилла (по оценочной частоте блока, в котором делается спилл) и заменить обычный жадный проход по интервалам на более сложный (и тоже жадный) проход, который выбирает регион, оценивает в нём количество спиллов, а далее выбирает регистры в первую очередь для регионов, где стоимость спилла максимально высока. Назовём этот подход LSRA_G, где буква G означает Greedy.

Задание 7.23. *Реализуйте алгоритм LSRA_G по несколько неформальному описанию выше. Примените его и LSRA на какой-нибудь реалистичный LIR с большим количеством вложенных циклов и оцените выгоду от дополнительной эвристики.*

Подходы, основанные на линейном сканировании очень полезны и практически применимы, но, конечно, скучноваты. Давайте отвлечёмся от них и закончим рассмотрение распределения регистров более теоретически сложными и интересными алгоритмами.

7.2.4 Подходы, основанные на RBQP и ILP

Задача RBQP уже рассматривалась в разделе 7.1.5, где она применялась для оптимального выбора машинных инструкций. Удивительно, но ровно к той же задаче в общем случае сводится и распределение регистров. Мало того, в отличие от случая выбора инструкций, где эти исследования остаются больше теоретическими, в распределении регистров у такого подхода есть конкретная мотивация.

В поле зрения компиляторных инженеров с конца девяностых годов были так называемые нерегулярные регистровые архитектуры.

Определение 7.10. *Регистровая архитектура **регулярна** в своей программной модели, если все регистры могут быть исполь-*

зованы взаимозаменяемо в каждой инструкции, и это не сказывается на размере инструкции, а также на времени её выполнения. Соответственно, **иррегулярной** называется архитектура, в которой выбор регистров влияет на время исполнения инструкции, её размер или на то и на другое.

До некоторой степени даже мейнстримные архитектуры, такие как x86 имеют свои нерегулярности, например, специальное использование в некоторых инструкциях регистра `RAH`, который существенно уменьшает длину кодировки. Во всех случаях, когда однородность регистровой структуры нарушается, в ней появляются такого рода нерегулярности, которые для некоторых процессоров могут быть существенными.

Теоретики подхода в стиле Чайтина–Бриггса пытались решить эту проблему внутри парадигмы раскраски графа. Есть даже упоминание специальной обработки для регистровых пар в работе самого Бриггса [8]. Но это до обидного плохо работало. Были идеи решать это через целочисленное линейное программирование, о них будет ниже, но они тоже плохо работали. В итоге в 2002-м году Бернард Шольц и Эрик Экштейн опубликовали статью [53], в которой предложили следующее сведение к уже знакомой нам задаче квадратичного булева программирования.

Пусть переменные x_{ij} обозначают распределение i -го виртуального регистра на j -й физический, где решение о спилле кодируется номером несуществующего физического регистра. Довольно очевидно выполняется соотношение (7.3), так как один регистр распределяется единожды.

$$\forall i \sum_{0 \leq j \leq n} x_{ij} = 1 \quad (7.3)$$

Задание 7.24. Запишите систему уравнений вида «все одновременно активные виртуальные регистры должны иметь не совпадающие физические». Предположите, что информация об интервалах активности у вас есть.

Обозначим вектором x_i вектор всех решений для изолированного виртуального регистра. У каждого такого решения есть цена c_i , которая может оказаться и бесконечной, если распределение невозможно.

Кроме того, для каждой пары виртуальных регистров x_i и x_k существует матрица их совместимости C_{ik} которая отражает ограничения на их распределение. Например, если мы допод-

лино знаем, что два регистра должны распределиться парой, их совместную стоимость можно поставить нулевой, а отдельную — бесконечной.

В итоге задача RBQP выглядит как показано в соотношении (7.4), где n — это количество виртуальных регистров.

$$f(x, c, C) = \sum_{0 \leq i \leq n} x_i^T c_i + \sum_{0 \leq i < j \leq n} x_i^T C_{ij} x_j \quad (7.4)$$

Сравните это соотношение с (7.2). Вероятно, вы не видите разницы? Это потому, что её тут нет. Отличаются только более строгие наборы ограничений.

Задача RBQP для распределения регистров может решаться не точно, а с помощью эвристики, основанной на динамическом программировании. Удивительно сложный и изощрённый алгоритм, основанный на нескольких типах редукций, можно найти в [53]. Я, пожалуй, не буду помещать его сюда.

Задание 7.25. Разберитесь самостоятельно с решением RBQP для распределения регистров с помощью эвристики, основанной на динамическом программировании.

Детальный обзор по применению RBQP для нерегулярных архитектур см. в работе [26].

Ещё одним подходом (и исторически — первым подходом, где задумались о нерегулярных архитектурах) является подход через целочисленное программирование (ILP, integer linear programming). Он был предложен в работе [24] и улучшен в [35].

Сведение распределения регистров к задаче линейного целочисленного программирования с учётом всевозможных ограничений строится довольно просто: используются, по сути, те же переменные x_{ij} только они линейризуются в единый вектор $x_{j \cdot n + i}$. Все ограничения вида (7.3) для использованных регистров записываются в том же виде, но трактуются просто как ограничения вида равенств. Если в процессе получаются ограничения вида неравенств, их можно свести к равенствам введением дополнительных переменных s . Далее решается задача линейного программирования: для системы ограничений вида $Ax + s = b$ максимизируется функционал $f(x) = x^T c$

Понятно, что матрица A получается огромной, а сама задача решается куда хуже, чем PBQP даже в простых случаях. Учёт взаимодействия регистров при таком подходе приводит к введению ещё большего количества новых переменных, и всё это превращается в ночной кошмар. Переход Шольца от целочисленного линейного к целочисленному квадратичному программированию здесь был действительно превосходной идеей, существенно уменьшающей размеры задачи.

7.3 Планирование инструкций

*It does not do to leave a live dragon
out of your calculations,
if you live near one.
– J. R. R. Tolkien*

До сих пор я рассматривал процессор, который будет исполнять скомпилированные программы, как своего рода чёрный ящик, у которого известен внешний интерфейс вроде количества и типов регистров и кодировок для инструкций. Но на самом деле мы знаем о процессорах гораздо больше. Например, для LIR, представленного на листинге 7.26, возможна тривиально простая оптимизация, если у компилятора есть информация, что инструкция `LOAD` может занять довольно много времени, если не попадёт в кеш.

```
BB0: v2 = ADD v1, 0x100;  
      v3 = MUL v2, 10  
      v4 = AND v3, 0xFF  
      v5 = LOAD v2;  
      v6 = DIV v4, v5;  
      goto BB1;
```

Листинг 7.26: LIR для простого планирования

Задание 7.26. Не читая дальше, предложите, как здесь можно оптимизировать время выполнения?

Разумеется, правильным решением здесь будет поднять загрузку настолько высоко, насколько возможно (например, поставить её сразу после сложения, как только для неё готов адрес). В этом случае, пока она будет бегать в память, арифметические устрой-

ства не будут простаивать, а будут делать умножение, потом логическую операцию и т.д.

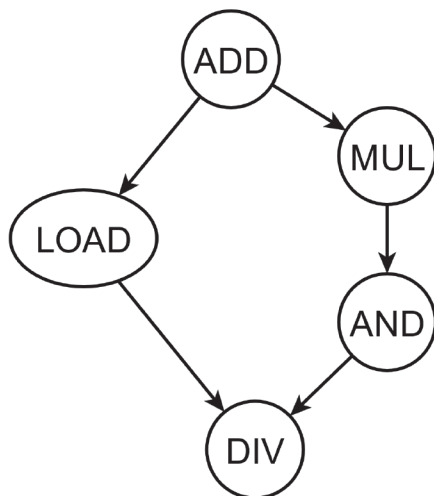


Рис. 7.8: Граф зависимостей для листинга 7.26

Граф зависимости между инструкциями представлен на рисунке 7.8. Формальное определение графа зависимостей я введу позже (см. определение 7.14). Пока что наша цель — сформировать интуицию. На рисунке отображены только прямые зависимости вида «Инструкция А читает данные которые записала инструкция В». Однако другие зависимости на виртуальных регистрах встречаются крайне редко.

Такого рода перестановка инструкций, исходя из знания об их возможных задержках выполнения, позволяет добиваться впечатляющих результатов. Например, на рисунке 7.9 показаны два конвейера исполнения внутри гипотетического процессора: один с загрузкой, оставшейся внизу, и второй с загрузкой, вынесенной вверх.

Градиентными блоками на рисунке 7.9 показано время нахождения инструкции в конвейере, время при этом идёт слева направо. Стрелками показаны зависимости между инструкциями.

Вроде бы всё хорошо. Но здесь есть три вопроса.

Во-первых, такую перестановку может выполнить сам процессор динамически. Более того, многие выполняют. Должен ли вообще о таком думать компилятор? Обычный практический ответ — да, должен. Никто не откажется от такого рода

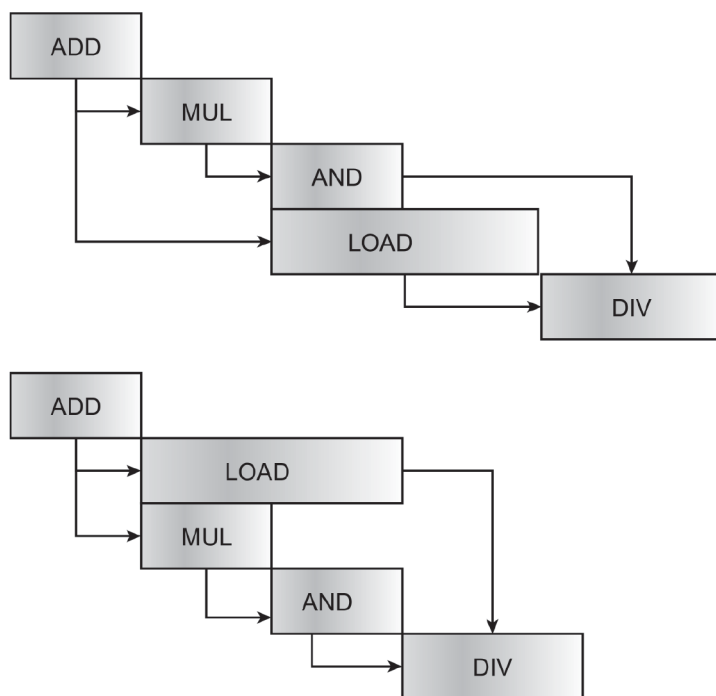


Рис. 7.9: Пайплайн с оптимизированными задержками

помощи, и, к тому же, большое окно динамического планирования есть только у больших процессоров. Если речь идёт о небольшом устройстве, таком как микроконтроллер, или о более крупном, но с большой площадью кристалла, отданной под специфические задачи, к примеру, задачи видеокарты, то планирование в компиляторе становится критически важным.

Во-вторых, неясно, откуда и как компилятор узнает детали архитектуры и, тем более, детали микроархитектуры? То есть понятно, что это мы ему расскажем, но надо как-то решить, как конкретно и с каким уровнем детализации. Должен ли, например, компилятор знать детали протокола когерентности кешей?

В-третьих, на листинге 7.26 я показал конвейеризацию на виртуальных регистрах. А что, если в посчитанную таким образом конвейеризацию включится распределитель регистров и куда-нибудь посередине посадит жирный спилл? В принципе, можно делать планирование после распределителя, но тог-

да в графе будет гораздо больше зависимостей, и компилятор будет куда сильнее стеснён в своих возможностях переместить инструкцию.

Обычно третья проблема решается просто: кое-что делают на виртуальных регистрах, надеясь, что последующее распределение регистров ничего не испортит. А всё остальное выносят на физические регистры. Разные компиляторы по-разному решают, куда и сколько процентов сил потратить. Например, GCC на момент написания книги почти всё пытается делать после распределителя, а LLVM почти всё пытается делать до.

Мы, разумеется, рассмотрим обе стадии.

7.3.1 Планирование на виртуальных регистрах

Основная идея планирования на виртуальных регистрах — это распределение графа зависимостей программы по доступным ресурсам из описания целевой машины. Мы уже примерно познакомились с графом зависимостей на рисунке 7.8 и создали о нём какую-то интуицию. Теперь настало время определить вещи формально.

Мы уже встречались с графами зависимости, когда говорили о цикловых оптимизациях в разделе 5.3.2, см. определение 5.20. Я переиспользую введённые там определения истинной зависимости и анти-зависимости, но теперь я перепишу их в терминах виртуальных регистров и поэтому к ним добавится также зависимость через выход, которая невозможна в SSA-графе.

Определение 7.11. *Истинной зависимостью по данным между инструкциями i и j , такими, что j упорядочена после i , является запись в инструкции i виртуального регистра, который читает j .*

Определение 7.12. *Анти-зависимостью по данным между инструкциями i и j , такими, что j упорядочена после i , является чтение в инструкции i виртуального регистра, который записывает j .*

Определение 7.13. *Зависимостью через выход по данным между инструкциями i и j , такими, что j упорядочена после i , является запись в инструкции i виртуального регистра, который записывает j .*

Все три ситуации (чтение после записи, запись после чтения и запись после записи) накладывают ограничения на перестановку i и j местами.

Кроме зависимости по данным существует ещё зависимость по обращениям в память. В этой книге она не рассматривается, хотя это отдельная большая сложная и интересная тема. Например, правила строгого алиасинга в таких языках как C и C++ во многом введены, чтобы компилятору было проще доказывать независимость обращений в два участка памяти, соответствующих разным типам данных. Мы будем консервативно считать, что память алиасится всегда, но на самом деле там открываются бездны. Что же, по крайней мере, я вас предупредил.

Определение 7.14. *Графом зависимости (dependency graph) по данным называется граф, в котором вершинами являются инструкции LIR, а ребро между двумя инструкциями существует, если между ними есть одна из рассмотренных выше зависимостей по данным или того же рода зависимость по памяти.*

Кроме того, для каждой виртуальной инструкции известен перечень ресурсов, необходимых для её выполнения. Ресурсом может быть арифметико-логическое устройство (ALU), предсказатель перехода, очередь загрузок и сохранений и так далее. Все ресурсы описываются вместе с описанием целевой машины.

Мы примем самую простую модель, когда для каждой инструкции просто известна её задержка на выполнение (latency) и вид ресурса, который она занимает.

Определение 7.15. *Задачей планирования является задача топологического упорядочения графа зависимостей так, чтобы минимизировать суммарную задержку выполнения всех инструкций.*

Гэри и Джонсон показали, что даже в самом простом случае эта задача NP-полна (см. работу [22]). Хочется добавить — так же NP полна, как и все остальные задачи в этой заключительной части книги. Пожалуй, ни одна область компиляторной теории так не бьёт через край NP-сложными задачами, как финальная кодогенерация. Хотя ни одна такого рода задач и не лишена. Мы, если помните, находили их даже в цикловых оптимизациях.

Лучшим из известных человечеству приближённых решений этой задачи и фактическим индустриальным стандартом является списочное планирование (list scheduling).

```
list_sched(DepGraph G, Resources R) {  
    auto RT = vector<int>(R.size(), 0);  
    for (auto V : topo_order(G)) {  
        auto RV = R.available_for(V);  
        auto RX = RV.first();  
        for (auto X : RV)  
            if (RT[X.idx()] < RT[RX.idx()])  
                RX = X;  
        RT[RX.idx()] += R.latency(V);  
    }  
}
```

Листинг 7.27: Алгоритм LSCH: списочное планирование

Алгоритм работает довольно просто и назначает ресурсы лишь единожды. Он обходит граф в топологическом порядке, ищет для каждой операции доступные ресурсы, ищет ресурс, на котором можно начать выполнение раньше прочих, и распределяет эту операцию туда.

Очевидно, это жадный алгоритм. Вследствие своей жадной природы, он работает быстро, но грубовато и неаккуратно.

7.3.2 Планирование на физических регистрах

Главной проблемой планирования после распределения регистров является гигантское количество анти-зависимостей. До распределения регистров антизависимостей практически не было, поскольку каждая инструкция по необходимости брала себе новые виртуальные регистры. После распределения регистров все инструкции начинают многократно использовать одни и те же физические регистры. Запись в тот же физический регистр после его чтения — это и есть каноническая антизависимость в терминах определения 7.12.

Поэтому планирование на основании алгоритма LSCH (см. листинг 7.27) на физических регистрах должно быть дополнительно специальной фазой, которая разрывает анти-зависимости при наличии свободных физических регистров, в которые их можно переименовать. Здесь опять проявляется вечная борьба планировщика инструкций с распределителем регистров: все те регистры, которые распределитель так экономил, планировщик теперь щедро разбрасывает.

Чтобы понять почему анти-зависимости опасны, введём концепцию критического пути в графе зависимостей.

Определение 7.16. *Критическим путём (critical path) называется самый дорогой по времени путь в графе зависимостей.*

Очевидно, что в условиях идеально параллельной машины с бесконечным количеством вычислительных устройств, суммарное время работы функции не будет меньше времени критического пути. Листинг 7.28 показывает, как плохое распределение регистров может сериализовать вычисления и насколько важно планирование после распределителя.

```
BB0: r1 = ADD r1, 0x100;
      r2 = LOAD r1;
      r1 = MUL r1, 10
      r1 = AND r1, 0xFF
      r1 = DIV r1, r2;
      goto BB1;
```

Листинг 7.28: LIR для уменьшения критического пути

Алгоритм для разрывания анти-зависимостей и сокращения критического пути представлен на листинге 7.29.

```
adbreak(DepGraph G, MachineBasicBlock MBB) {
    RegPool RP = MBB.unusedOnExit();
    RegPool Forbid = {};
    for (auto MI : MBB.reverse()) {
        auto E = G.antidep(MI);
        if (E && !RP.haveFreeReg()) {
            RP.add(Forbid);
            Forbid.clean();
        }
        if (E && RP.haveFreeReg()) {
            auto Reg = RP.pick(Forbid);
            RP.push(E.reg());
            Forbid.push(Reg);
            update_dest_rec(MI, Reg);
        }
        RP.update(MI);
    }
}
```

Листинг 7.29: Алгоритм ABD: разрыв антизависимостей

Алгоритм ABD идёт по базовому блоку задом наперёд и на каждой итерации обновляет информацию о текущих регистрах: освобождает тот, который определяет текущая инструкция, и бронирует те, которые она использует. Если в графе для этой инструкции есть антизависимость, он берёт свободный регистр, чтобы её разорвать.

При этом он поддерживает в множестве `Forbid` те регистры, которые нельзя использовать для разрыва антизависимости, даже если они свободны: это те регистры, которые мы уже использовали. Это нужно для того, чтобы не влететь в неприятную ситуацию, когда цепочка антизависимостей вида $A \rightarrow A \rightarrow A \rightarrow A$ разрывается, но всюду используется один и тот же свободный регистр B . В итоге получается $B \rightarrow B \rightarrow B \rightarrow A$, что, согласимся не намного лучше. Мы бы хотели что-то вроде $B \rightarrow C \rightarrow D \rightarrow A$.

Если регистры заканчиваются, мы отпускаем те, которые мы отмечали как запрещённые, обратно в свободные, потому что лучше порвать как-то, чем не порвать никак. После исполнения алгоритма ABD, LIR может выглядеть, как на листинге 7.30.

```
BB0: r3 = ADD r1, 0x100;  
      r2 = LOAD r3;  
      r4 = MUL r3, 10  
      r5 = AND r4, 0xFF  
      r6 = DIV r5, r2;  
      goto BB1;
```

Листинг 7.30: LIR после разрыва анти-зависимостей

Далее естественным шагом будет всё-таки спланировать всё, что осталось возможным спланировать с учётом новой реальности и старых аппаратных ресурсов. Интересно, что именно для фазы после регаллока нам открывается один из путей не столь очевидного улучшения алгоритма LSCH. Речь идёт о замене обычного топологического порядка на так называемый приоритетный топологический порядок. Приоритетный топологический порядок учитывает высоту узла в графе зависимостей и сначала берёт самые высокие узлы. В среднем это позволяет сократить критический путь.

Задание 7.27. *Напишите реализацию приоритетного топологического порядка для фазы алгоритма LSCH после регаллока.*

7.4 Задачи на самостоятельную проработку

*Probably the biggest problem I had was
I didn't understand the importance of scheduling.*

– Anne Donovan

Задание 7.28. Придумайте алгоритм для генерации ациклических графов, состоящих из заранее заданного набора фрагментов. Основная сложность этой задачи — добиться равномерного распределения вероятности получения каждого графа.

Задание 7.29. Попробуйте воспользоваться результатами проблемы 7.28 для сшивания нескольких ациклических графов и генерации HIR. Что, по-вашему, проще: написать генератор произвольного HIR с такими же комбинаторными свойствами или поднять HIR обратно из сшитых ациклических графов?

Задание 7.30. Алгоритм GS на листинге 7.8 может не находить существующее покрытие, если либо покрываемый граф, либо возможные паттерны представляют собой что-то сложнее деревьев. Введите для него наивный бэктрекинг, чтобы в таких случаях он возвращался в ближайшую точку, где можно изменить ранее принятое решение о покрытии, и пробовал ещё раз. Оцените асимптотику наивного алгоритма с учётом бэктрекинга.

Задание 7.31. Разберитесь с алгоритмом выбора инструкций, основанном на динамическом программировании, из работы [2] и попробуйте ответить на вопрос, почему он работает только для деревьев. Что сломается, если те же уравнения динамического программирования выписать и решить для произвольных ациклических графов?

Задание 7.32. Напишите аналог алгоритма LS, который работает не на SDHIR, а на GLIR представлении.

Задание 7.33. Придумайте способ вставлять копии в случае параллельного копирования, как на листинге 7.19. Проверьте себя по книге [49], где изложен алгоритм.

Задание 7.34. Реализуйте подпрограмму `spill_at_interval` для алгоритма LSRA (см. листинг 7.24).

Задание 7.35. В алгоритме 7.23 после удаления очередного интервала в первом цикле у нас понижается степень как минимум

одной из вершин. Придумайте, как это использовать для улучшения качества раскраски.

Задание 7.36. *В алгоритме 7.23, если мы ушли в спиллы, то заспиллим все вершины. В оригинальном алгоритме Чайтин, конечно же, это учел. Нам нужно заспиллить ровно столько вершин, чтобы появилась хотя бы одна со степенью меньше N . Дальше мы снова должны перейти в первый цикл и назначить ей цвет. Сделайте такую доработку в алгоритм.*

Заключение

*Starting strong is good.
Finishing strong is epic.*

– Robin Sharma

Если вы добрались до этого заключения, прочитав всю книгу, а не просто заглянули в конец, я вас поздравляю и надеюсь, оно того стоило.

Эта книга не является исчерпывающей. Она выросла из мими-курса, который я читал свежим стажёрам в компиляторной команде с простой и понятной целью: чтобы они как можно скорее освоили основы, включились в практическую работу над компилятором и стали приносить пользу. Вы можете предположить (и не ошибётесь), что я оставил за кадром массу интересной теории, которая занимает тысячестраничные монографии. Даже просто список не использованной мной литературы, которую я мог бы использовать, если бы я его записал, получился бы весьма внушительным. В частности, в эту книгу не вошли интереснейшие алгоритмы векторизации (это была бы плюс одна глава). Я ничего не написал о полиэдральных и аффинных трансформациях циклов (что составило бы ещё главу). В этой книге также ничего не сказано про анализ зависимостей и анализ алиасов (пожалуй, это была бы ещё одна глава). Возможно, эти главы появятся в следующих изданиях. Мне бы также хотелось бы гораздо больше сказать про оптимизации, основанные на машинном обучении, но тут, пожалуй, пришлось бы писать ещё одну книгу.

Я помню, когда я только начинал работать над компилятором, это был довольно ранний GCC (что-то вроде GCC 4.3), и я перешёл от работы над библиотеками к работе над компилятором. Поскольку тогда компиляторы были новой для меня областью, я взялся читать гигантский фундаментальный труд Стивена Муч-

ника [43]. И, что меня больше всего поражало, так это разительное отличие того, что было написано там, и того, что я видел в коде на самом деле. В итоге Мучник (а также Купер [16], Ахо и Ульман [37] и многие другие) мне, конечно, очень пригодились, и я много на них ссылаюсь. Но у меня очень долго двойлось в глазах: теория была отдельно, а то, что я делал по работе, имело с ней совсем мало общего.

В этой книге я постарался убрать это двоение в глазах. Структура книги и вошедший в неё материал имитируют работу настоящего компилятора и (за исключением некоторых эстетических добавлений вроде конструкции Пробстинга) существенно укоренены в реальности. При этом я старался не закладываться на конкретные версии конкретных компиляторов, чтобы не повторить судьбу замечательной книги Скеппстеда [54], где он много полезного сказал про компиляторы с опорой на LLVM 6.0.1, но сама цифра при версии компилятора стала выглядеть смешно уже через год после публикации.

В заключение я хотел бы выразить громадную признательность тем, кто помогал мне в подготовке этой книги, в частности моим коллегам Александру Усу, Антону Сидоренко, Владиславу Белову, Сергею Качкову и Анатолию Паршинцеву и многим другим моим коллегам и студентам за многочисленные советы по улучшению отдельных глав и алгоритмов. Отдельное спасибо хочется сказать Виктору Владиславлеву, без которого компиляторная карьера автора просто не началась бы и значит этой книги тоже бы не было.

Разумеется, я не ожидаю, что все читатели этой книги будут искать себе работу в практической компиляторной инженерии: там нет столько мест и это очень специфическая область, иногда максимально далёкая от высокой теории. Вместо этого я ожидаю, что практикующий программист, лучше понимающий новую и интересную для себя область компиляторных алгоритмов, будет с большим пониманием относиться к тому, что его компилятор делает и как он это делает. С другой стороны, если эта книга вызовет поток новых интересных патчей в тот же GCC, LLVM и прочие открытые и свободные компиляторные проекты, это очень порадует автора.

Я также верю, что компиляторные технологии и компиляторные алгоритмы могут быть полезны даже в максимально далёких от компилятора областях: генерации тестов, безопасности компьютерных систем, проектировании аппаратуры и даже в нейросетях и квантовых вычислениях. Усложняющийся мир требует

усложняющихся и специализированных языков его описания. Везде, где есть описание на высокоуровневом языке, которое нужно эффективно выразить в более низкоуровневом представлении, найдётся место и для оптимизирующей компиляции на пути от точки А до точки Б.

Но это, конечно, уже совсем другая история.

Предметный указатель

- ϕ -узел, 83
- ABI, 37
- Anti dependency, 254
- As If Rule, 17
- Assembler, 14, 34
- AST, 21, 24
- Back edge, 144
- Basic block, 51
- Basic I. V., 159
- Binary relation, 56
- Call graph, 190
- Canonical loop, 155
- CFG, 53
- Coalescing, 241
- Compiler, 14, 15
- Compiler backend, 18
- Compiler frontend, 18
- Compiler Heuristics, 185
- Complete cover, 222
- Critical edge, 127
- Critical path, 252
- DAG, 77, 149, 215
- DAG fragment, 222
- Dead code elimination, 108
- Def-Join Graph, 93
- Dependency graph, 250
- Dependent I. V., 167
- DF-graph, 91
- Displacement, 40
- Dominance frontier, 90
- Dominator tree, 89
- Dynamic programming, 23

- Frame index, 240
- FRG, 131
- Full unroll, 172
- GCC, 14
- Generalized I. V., 159
- Gimple, 26
- GLIR, 225
- Grammar, 21
- HIR, 25
- I. V. Optimizations, 168
- ILP, 246
- immediate, 40
- Inference graph, 238
- Inline, 206
- Instruction selection, 32, 214
- IR, 25
- Irregular architecture, 245
- ISA, 36
- Join, 60
- Join edges, 94
- latency, 251
- Lattice, 59
- Lattices, 56
- LCSSA, 156
- Legalization, 220
- Lexical analysis, 18
- LICM, 174
- Linker, 15
- LIR, 25
- List scheduling, 252
- Live interval, 237
- Live variable, 72
- LLVM IR, 26
- Loop distribution, 180
- Loop exit block, 154
- Loop exiting block, 154
- Loop flattening, 183
- Loop fusion, 180
- Loop header, 151
- Loop interchange, 182
- Loop invariant, 174
- Loop latch, 154
- Loop peeling, 176
- Loop preheader, 151
- Loop splitting, 178
- Loop Strength Reduction, 168
- Loop tiling, 182
- Loop tree, 162

- Loop Unroll, 170
- Loop unswitching, 185
- LTO, 44
- MachineIR, 26
- Match, 222
- meet, 60
- MLIR, 25
- MOP, 65
- Natural loop, 144
- Object code, 34
- Optimization pass, 30
- Outline, 206
- Output dependency, 250
- P. O. Set, 59
- Parsing, 21 PBQP, 230
- Pragma Directive, 186
- Preprocessing, 18
- Pruned SSA, 96
- Reaching Definitions, 67
- Reducibility, 148
- Relaxation, 45
- Reroll, 172
- Retreating edge, 147
- Reverse post order, 88
- RTL, 26
- SCEV, 165
- Scheduling, 33
- SDHIR, 217
- Selection DAG, 215
- Semantic analyzer, 18
- Semantic network, 96, 109, 234
- spill, 239
- SROA, 102
- SSA, 9, 29, 81, 82
- SSA anti dependency, 177
- SSA Graph, 110
- SSA true dependency, 177
- Tail recursion, 203
- Target Hook, 170
- Terminator instruction, 52
- TLS, 46
- Token, 18
- Toolchain, 14
- True dependency, 250
- Unix Shell, 30
- Value Numbering, 76
- Variable dependency graph, 177

- VRP, 119
- Дерево доминаторов, 90
- Закон Пробстинга, 229
- Квази-инвариант, 176
- Неподвижная точка, 62
- Непосредственное доминирование, 89
- Отношение доминирования, 87
- Передаточная функция, 61
- Проблема EVAL, 216
- Проблема подстановки, 206
- Рефлексивность, 58
- Семантическая ошибка, 24
- Симметричность, 58
- Стековая машина, 215
- Строгое доминирование, 91
- Терминатор, 29
- Транзитивность, 58
- Трёхадресный код, 27

Литература

- [1] Aho, Alfred V, John E Hopcroft, and Jeffrey D Ullman. 1974. «The Design and Analysis of Computer Algorithms.» Addison–Wesley Series in Computer Science and Information Processing.
- [2] Aho, Alfred V, and Stephen C Johnson. 1976. «Optimal Code Generation for Expression Trees.» *Journal of the ACM (JACM)* 23 (3): 488–501.
- [3] Allen, Frances E., and John Cocke. 1976. «A Program Data Flow Analysis Procedure.» *Communications of the ACM* 19 (3): 137.
- [4] Alpern, Bowen, Mark N Wegman, and F Kenneth Zadeck. 1988. «Detecting Equality of Variables in Programs.» In *Proceedings of the 15th Acm Sigplan–Sigact Symposium on Principles of Programming Languages*, 1–11.
- [5] Bacon, David F, and Peter F Sweeney. 1996. «Fast Static Analysis of C++ Virtual Function Calls.» In *Proceedings of the 11th Acm Sigplan Conference on Object–Oriented Programming, Systems, Languages, and Applications*, 324–41.
- [6] Briggs, Preston, and Keith D Cooper. 1994. «Effective Partial Redundancy Elimination.» *ACM SIGPLAN Notices* 29 (6): 159–70.
- [7] Briggs, Preston, Keith D Cooper, Timothy J Harvey, and L Taylor Simpson. 1998. «Practical Improvements to the Construction and Destruction of Static Single Assignment Form.» *Software: Practice and Experience* 28 (8): 859–81.

- [8] Briggs, Preston, Keith D Cooper, and Linda Torczon. 1994. «Improvements to Graph Coloring Register Allocation.» *ACM Transactions on Programming Languages and Systems (TOPLAS)* 16 (3): 428–55.
- [9] Bruno, John, and Ravi Sethi. 1976. «Code Generation for a One-Register Machine.» *Journal of the ACM (JACM)* 23 (3): 502–10.
- [10] Callahan, David, Keith D Cooper, Ken Kennedy, and Linda Torczon. 1986. «Interprocedural Constant Propagation.» *ACM SIGPLAN Notices* 21 (7): 152–61.
- [11] Carter, Larry, Jeanne Ferrante, and Clark Thomborson. 2003. «Folklore Confirmed: Reducible Flow Graphs Are Exponentially Larger.» In *Proceedings of the 30th Acm Sigplan–Sigact Symposium on Principles of Programming Languages*, 106–14.
- [12] Chaitin, Gregory J, Marc A Auslander, Ashok K Chandra, John Cocke, Martin E Hopkins, and Peter W Markstein. 1981. «Register Allocation via Coloring.» *Computer Languages* 6 (1): 47–57.
- [13] Choi, Jong-Deok, Ron Cytron, and Jeanne Ferrante. 1991. «Automatic Construction of Sparse Data Flow Evaluation Graphs.» In *Proceedings of the 18th Acm Sigplan–Sigact Symposium on Principles of Programming Languages*, 55–66.
- [14] Cocke, John, and Ken Kennedy. 1977. «An Algorithm for Reduction of Operator Strength.» *Communications of the ACM* 20 (11): 850–56.
- [15] Cooper, Keith D, Timothy J Harvey, and Ken Kennedy. 2001. «A Simple, Fast Dominance Algorithm.» *Software Practice & Experience* 4 (1–10): 1–8.
- [16] Cooper, Keith D, and Linda Torczon. 2011. *Engineering a Compiler*. Elsevier.
- [17] Cytron, Ron, Jeanne Ferrante, Barry K Rosen, Mark N Wegman, and F Kenneth Zadeck. 1991. «Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.»

ACM Transactions on Programming Languages and Systems (TOPLAS) 13 (4): 451–90.

- [18] Ebner, Dietmar, Florian Brandner, Bernhard Scholz, Andreas Krall, Peter Wiedermann, and Albrecht Kadlec. 2008. «Generalized Instruction Selection Using Ssa-Graphs.» In Proceedings of the 2008 Acm Sigplan–Sigbed Conference on Languages, Compilers, and Tools for Embedded Systems, 31–40.
- [19] Eckstein, Erik, Oliver König, and Bernhard Scholz. 2003. «Code Instruction Selection Based on Ssa-Graphs.» In International Workshop on Software and Compilers for Embedded Systems, 49–65. Springer.
- [20] Ershov, Andrei P. 1958. «On Programming of Arithmetic Operations.» Communications of the ACM 1 (8): 3–6.
- [21] Friedl, Jeffrey EF. 2006. Mastering Regular Expressions. " O'Reilly Media, Inc."
- [22] Garey, Michael R, and David S Johnson. 1979. Computers and Intractability. Vol. 174. freeman San Francisco.
- [23] Gerlek, Michael P. 1993. «Detecting Induction Variables Using Ssa Form.» Dept. Of CSE, Oregon Graduate Institute OGI–CES Technical Report, 93–014.
- [24] Goodwin, David W, and Kent D Wilken. 1996. «Optimal and Near-Optimal Global Register Allocation Using 0–1 Integer Programming.» Software: Practice and Experience 26 (8): 929–65.
- [25] Grosser, Tobias, Hongbin Zheng, Raghesh Aloor, Andreas Simbürger, Armin Größlinger, and Louis-Noël Pouchet. 2011. «Polyhedral Optimization in Llvm.» In Proceedings of the First International Workshop on Polyhedral Compilation Techniques (Impact), 2011:1.
- [26] Hames, Lang, and Bernhard Scholz. 2006. «Nearly Optimal Register Allocation with Pbqp.» In Joint Modular Languages Conference, 346–61. Springer.
- [27] Kallmeyer, Laura. 2010. Parsing Beyond Context-Free Grammars. Springer Science & Business Media.

- [28] Kam, John B, and Jeffrey D Ullman. 1977. «Monotone Data Flow Analysis Frameworks.» *Acta Informatica* 7 (3): 305–17.
- [29] Kavi, Litong, and Krishna M Song. 2002. «A Technique for Variable Dependence Driven Loop Peeling.» In *Fifth International Conference on Algorithms and Architectures for Parallel Processing*, 23–25.
- [30] Kennedy, Ken, and Kathryn S McKinley. 1993. «Maximizing Loop Parallelism and Improving Data Locality via Loop Fusion and Distribution.» In *International Workshop on Languages and Compilers for Parallel Computing*, 301–20. Springer.
- [31] Kennedy, Robert, Sun Chan, Shin–Ming Liu, Raymond Lo, Peng Tu, and Fred Chow. 1999. «Partial Redundancy Elimination in Ssa Form.» *ACM Transactions on Programming Languages and Systems (TOPLAS)* 21 (3): 627–76.
- [32] Kildall, Gary A. 1973. «A Unified Approach to Global Program Optimization.» In *Proceedings of the 1st Annual Acm Sigact–Siplan Symposium on Principles of Programming Languages*, 194–206.
- [33] Koes, David Ryan, and Seth Copen Goldstein. 2008. «Near–Optimal Instruction Selection on Dags.» In *Proceedings of the 6th Annual Ieee/Acm International Symposium on Code Generation and Optimization*, 45–54.
- [34] Komondoor, Raghavan, and Susan Horwitz. 2000. «Semantics–Preserving Procedure Extraction.» In *Proceedings of the 27th Acm Sigplan–Sigact Symposium on Principles of Programming Languages*, 155–69.
- [35] Kong, Timothy, and Kent D Wilken. 1998. «Precise Register Allocation for Irregular Architectures.» In *Proceedings. 31st Annual Acm/Ieee International Symposium on Microarchitecture*, 297–307. IEEE.
- [36] Kongstad, Mårten. 2004. *An Implementation of Global Value Numbering in the Gnu Compiler Collection with Performance Measurements*. Citeseer.

- [37] Lam, Monica, Ravi Sethi, Jeffrey D Ullman, and Alfred Aho. 2006. «Compilers: Principles, Techniques, and Tools.» Pearson Education.
- [38] Lengauer, Thomas, and Robert Endre Tarjan. 1979. «A Fast Algorithm for Finding Dominators in a Flowgraph.» *ACM Transactions on Programming Languages and Systems (TOPLAS)* 1 (1): 121–41.
- [39] Levine, John R. 1999. *Linkers and Loaders*. Morgan Kaufmann Publishers Inc.
- [40] Lóki, Gábor, Ákos Kiss, Judit Jász, and Árpád Beszédes. 2004. «Code Factoring in Gcc.» In *Proceedings of the 2004 Gcc Developers' Summit*, 79–84.
- [41] Lokuciejewski, Paul, Fatih Gedikli, and Peter Marwedel. 2009. «Accelerating Wcet-Driven Optimizations by the Invariant Path Paradigm: A Case Study of Loop Unswitching.» In *Proceedings of Th 12th International Workshop on Software and Compilers for Embedded Systems*, 11–20.
- [42] Morel, Etienne, and Claude Renvoise. 1979. «Global Optimization by Suppression of Partial Redundancies.» *Communications of the ACM* 22 (2): 96–103.
- [43] Muchnick, Steven. 1997. *Advanced Compiler Design and Implementation*. Morgan kaufmann.
- [44] Namolaru, Mircea. 2006. «Devirtualization in GCC.» In *Proceedings of the GCC Developers' Summit*, 125–33. Citeseer.
- [45] Newham, Cameron. 2005. *Learning the Bash Shell: Unix Shell Programming*. " O'Reilly Media, Inc."
- [46] Pemmaraju, Sriram, and Steven Skiena. 2003. *Computational Discrete Mathematics: Combinatorics and Graph Theory with Mathematica*. Cambridge university press.
- [47] Poletto, Massimiliano, and Vivek Sarkar. 1999. «Linear Scan Register Allocation.» *ACM Transactions on Programming Languages and Systems (TOPLAS)* 21 (5): 895–913.

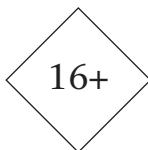
- [48] Pop, Sébastien. 2003. «Analysis of Induction Variables Using Chains of Recurrences: Extensions.» PhD thesis, Citeseer.
- [49] Rastello, Fabrice, and Florent Bouchez Tichadou. 2022. SSA-Based Compiler Design. Springer Nature.
- [50] Reif, John H, and Harry R Lewis. 1977. «Symbolic Evaluation and the Global Value Graph.» In Proceedings of the 4th Acm Sigact-Sigplan Symposium on Principles of Programming Languages, 104–18.
- [51] Rosen, Barry K, Mark N Wegman, and F Kenneth Zadeck. 1988. «Global Value Numbers and Redundant Computations.» In Proceedings of the 15th Acm Sigplan-Sigact Symposium on Principles of Programming Languages, 12–27.
- [52] Scheifler, Robert W. 1977. «An Analysis of Inline Substitution for a Structured Programming Language.» Communications of the ACM 20 (9): 647–54.
- [53] Scholz, Bernhard, and Erik Eckstein. 2002. «Register Allocation for Irregular Architectures.» In Proceedings of the Joint Conference on Languages, Compilers and Tools for Embedded Systems: Software and Compilers for Embedded Systems, 139–48.
- [54] Skeppstedt, J. 2012. An Introduction to the Theory of Optimizing Compilers. Skeppberg.
- [55] Song, Ray. 2023. «The dark side of RISC-V linker relaxation.» <https://maskray.me/blog/2021-03-14-the-dark-side-of-riscv-linker-relaxation>.
- [56] Van Bulck, Jo, Daniel Moghimi, Michael Schwarz, Moritz Lippi, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. 2020. «LVI: Hijacking Transient Execution Through Microarchitectural Load Value Injection.» In 2020 Ieee Symposium on Security and Privacy (Sp), 54–72. IEEE.
- [57] Vladimirov, Konstantin Igorevich, and Ilya Vitalyevich Andreev. 2022. «Оптимизация разбиения структур для векторного оптимизатора в графическом компиляторе Intel.»

Современные информационные технологии и ИТ-образование 18 (2): 249–55.

- [58] Wegbreit, Ben. 1975. «Property Extraction in Well-Founded Property Sets.» IEEE Transactions on Software Engineering, no. 3: 270–85.
- [59] Yershov, Andrei P. 1966. «ALPHA—an Automatic Programming System of High Efficiency.» Journal of the ACM (JACM) 13 (1): 17–24.
- [60] Younger, Daniel H. 1967. «Recognition and Parsing of Context-Free Languages in Time N^3 .» Information and Control 10 (2): 189–208.
- [61] Зубков, Сергей. 2022. Assembler. Для Dos, Windows и Unix. Litres.

*Научно-популярное издание
Танымал ғылыми басылым*

Серия «Программирование для всех»



Константин Владимиров
ОПТИМИЗИРУЮЩИЕ КОМПИЛЯТОРЫ.
Структура и алгоритмы

Ответственный редактор *А. Ходякова*

Менеджер проекта *В. Живина*

Научный редактор *А. Сидоренко*

Технический редактор *Н. Чернышева*

Подписано в печать 13.09.2024. Формат 70×100/16. Усл. печ. л. 14.

Печать офсетная. Гарнитура SonetSerif. Бумага офсетная.

Тираж 5000 экз. Заказ №

Произведено в Российской Федерации

Изготовлено в 2024 г.

Оригинал-макет подготовлен редакцией «Времена»,
импринт «Альфа»

Изготовитель: ООО «Издательство АСТ»

129085, Российская Федерация, г. Москва, Звездный бульвар, д. 21, стр. 1,
комн. 705, пом. I, этаж 7

Наш сайт: WWW.AST.RU

E-mail: ask@ast.ru

Общероссийский классификатор продукции ОК–034–2014 (КПЕС 2008);
58.11.1 — книги, брошюры печатные

«Баспа Аста» деген ООО

129085, г. Мәскеу, Жұлдызды гүлзар, д. 21, 1 құрылым, 705 бөлме,
пом. 1, 7-қабат

Біздің электрондық мекенжайымыз : www.ast.ru

E-mail: ask@ast.ru

Интернет-магазин: www.book24.kz Интернет-дүкен: www.book24.kz

Импортер в Республику Казахстан и Представитель по приему претензий
в Республике Казахстан — ТОО РДЦ Алматы, г. Алматы.

Қазақстан Республикасына импорттаушы және Қазақстан Республикасында
наразылықтарды қабылдау бойынша өкіл — «РДЦ-Алматы» ЖШС, Алматы
қ., Домбровский көш., 3«а», Б литері офис 1. Тел.: 8(727) 2 51 59 90,91 ,
факс: 8 (727) 251 59 92 ішкі 107; E-mail: RDC-Almaty@eksmo.kz, www.book24.kz

Тауар белгісі: «АСТ» Өндірілген жылы: 2024

Өнімнің жарамдылық мерзімі шектелмеген.

Сертификация қарастырылмаған

КОНСТАНТИН ВЛАДИМИРОВ — IT-специалист с более чем 20-летним стажем, преподаватель кафедры микропроцессорных технологий в МФТИ, постоянный спикер технических конференций по темам C++, компиляторов и высокопроизводительных вычислений, автор популярного YouTube-канала @tilir.



«ОПТИМИЗИРУЮЩИЕ КОМПИЛЯТОРЫ» —

настольная книга специалиста, который решил не просто укрепить свои знания, но и вывести навыки на новый уровень. Вместе с Константином Владимировым вы разберете теорию оптимизирующей компиляции — все те сложные преобразования, которые происходят с текстом программы на его пути к исполняемому файлу, узнаете, что такое тулчейны и каким этапам трансформации подвергается программа до того, как будет впервые запущена, а также закрепите полученные знания, выполняя задания.

ИСКЛЮЧИТЕЛЬНО НАУЧНО И ПРОФЕССИОНАЛЬНО!

книги для любого настроения здесь



ИЗДАТЕЛЬСКАЯ ГРУППА АСТ

www.ast.ru | www.book24.ru

vk.com/izdatelstvoast
ok.ru/izdatelstvoast

ISBN 978-5-17-167965-1



9 785171 679651